
SCIE1500

Analytical Methods for Scientists

Course Reader

The University of Western Australia

Version 0.8

This reader may be refined in response to student comments. Please post feedback in the **Course Reader Discussion Board** in the LMS.

Narrative Arc: From Observation to Decision

- | | | |
|----------------|--------------------------|---------------|
| Act I | Understanding Systems | (Weeks 1–5) |
| Act II | Measuring Accumulation | (Weeks 6–7) |
| Act III | Predicting Interactions | (Weeks 8–10) |
| Act IV | Making Optimal Decisions | (Weeks 11–12) |

Contents

How to Use This Reader	9
Prologue: Mathematical Foundations (Week 0)	11
0.1 Reading Greek Letters and Mathematical Symbols	11
0.2 Number Sets and Algebra	12
0.2.1 Order of Operations (BODMAS)	13
0.2.2 Exponents and Radicals	13
0.2.3 Algebraic Expressions	14
0.3 Equations and Inequalities	14
0.4 Scientific Notation and Units	15
0.5 Function Notation	15
0.6 Interval Notation	16
0.7 Python Lab: A Guided Walkthrough	17
0.7.1 Running Your First Commands	17
0.7.2 Variables, <code>print()</code> , and Syntax Essentials	17
0.7.3 Data Types and Operators	18
0.7.4 Getting Help	19
0.7.5 NumPy: Arrays, <code>linspace</code> , and 2-D Arrays	19
0.7.6 Plotting with Matplotlib	20
0.8 Why This Matters: The Scientific Method	21
0.9 Week 0 Summary	22
0.10 Before Week 1: Self-Check	22
0.11 Exercises	22
0.12 Greek Alphabet Quick Reference	24
I Understanding Systems	25
1 Functions and Scientific Language (Week 1)	27
1.1 The Challenge: Ocean Plastic Pollution	27
1.2 Functions: Definitions and Properties	28
1.2.1 The Vertical Line Test	29
1.2.2 Domain and Range	29
1.3 Linear Functions	30
1.4 Quadratic Functions	31
1.5 Function Symmetry	33
1.6 Domain Restrictions in Scientific Modelling	33
1.7 Python Lab: A Guided Walkthrough	34
1.7.1 Part A — Function Basics and Rates of Change	34
1.7.2 Part B — Interpreting Slope Across Decades	35
1.7.3 Part C — Australian Policy Application	36
1.8 Why This Matters: The Reisser Depth-Integration Model	37
1.9 Week 1 Summary	37
1.10 Before Week 2: Self-Check	37
1.11 Exercises	38

2	Exponential and Logarithmic Functions (Week 2)	41
2.1	Why Exponential Growth Matters	41
2.2	The Exponential Function	41
2.3	The Number e : Two Roads to the Same Constant	42
2.3.1	Definition 1: Continuous Compounding	42
2.3.2	Definition 2: Slope Equals Height	43
2.4	The Graph of $\exp(x)$	44
2.5	Growth and Decay Models	45
2.6	The Logarithm Function	45
2.7	Solving Exponential and Logarithmic Equations	46
2.8	The 70/ r Rule	47
2.9	Geometric Sequences	47
2.10	Geometric Series	48
2.11	Population Dynamics Parameters	48
2.12	Application: Probability and Risk	49
2.13	Python Lab: A Guided Walkthrough	49
2.13.1	Part A — Building the Three Models	50
2.13.2	Part B — Doubling Time and the Rule of 70	51
2.13.3	Part C — New Scenarios: Invasive Species and Carbon Dating	51
2.14	Week 2 Summary	52
2.15	Before Week 3: Self-Check	53
2.16	Exercises	53
3	Logistic Functions and Bounded Growth (Week 3)	55
3.1	Why Growth Has Limits	55
3.2	Arithmetic Sequences: Completing Last Week's Pair	55
3.3	The Logistic Function	56
3.4	The Schaefer Growth Model	58
3.5	Maximum Sustainable Yield: It's Just the Vertex	59
3.5.1	Equilibrium Stock Levels for a Given Harvest	60
3.6	Connecting Models: Two Lenses on the Same Phenomenon	61
3.7	Function Composition	62
3.8	Inverse Functions	62
3.9	Python Lab: A Guided Walkthrough	63
3.9.1	Part A — The Schaefer Model	63
3.9.2	Part B — Sustainability Analysis	64
3.9.3	Part C — Policy Recommendation	65
3.10	Why This Matters: Preparing for Optimisation	65
3.11	Week 3 Summary	65
3.12	Before Week 4: Self-Check	66
3.13	Exercises	66
4	Limits, Continuity, and Derivatives (Week 4)	69
4.1	Why Calculus? Two 17th-Century Problems	69
4.2	From Total Change to Instantaneous Change	69
4.3	Limits: The Foundation of Calculus	70
4.3.1	The Easy Case: Direct Substitution	71
4.3.2	The $0/0$ Indeterminate Form	71
4.3.3	Limits at Infinity	72
4.3.4	One-Sided Limits	72
4.3.5	Limits in Nature	72
4.4	Continuity: When Functions Behave Well	73

4.5	The Derivative: From Secant to Tangent	74
4.6	Basic Differentiation Rules	75
4.7	Tangent Lines	76
4.8	Application: Instantaneous Rates in Science	77
4.9	When Derivatives Don't Exist	78
4.10	Python Lab: A Guided Walkthrough	78
4.10.1	Part A — Derivatives as Rates of Change	79
4.10.2	Part B — Finding the Optimal Temperature	80
4.10.3	Part C — Practical Constraints	80
4.10.4	Part D — Symbolic Differentiation with SymPy	81
4.11	Why This Matters: From 1687 to Your Toolbox	81
4.12	Week 4 Summary	82
4.13	Before Week 5: Self-Check	82
4.14	Exercises	83
5	Differentiation Techniques and Optimisation (Week 5)	85
5.1	Why Optimisation Matters	85
5.2	Advanced Differentiation Rules	86
5.2.1	The Product Rule	86
5.2.2	The Quotient Rule	87
5.2.3	The Chain Rule	87
5.3	Derivatives of Exponential and Logarithmic Functions	88
5.3.1	Revisiting Euler's Number	88
5.3.2	The Natural Logarithm	89
5.4	Optimisation: Finding Maxima and Minima	89
5.5	Local vs. Global Optima, and Inflection Points	91
5.6	Gradient Descent and Newton's Method	91
5.7	Constrained Optimisation	92
5.8	Economic Optimisation: The Bioeconomic Fishery Model	92
5.9	Application: Profit Maximisation Word Problems	94
5.10	Python Lab: A Guided Walkthrough	94
5.10.1	Part A — Problem Setup	95
5.10.2	Part B — Optimal Dimensions	95
5.10.3	Part C — Sensitivity Analysis	95
5.10.4	Part D — Symbolic Differentiation with SymPy	96
5.11	Week 5 Summary	96
5.12	Before Week 6: Self-Check	97
5.13	Exercises	97
II	Measuring Accumulation	99
6	Introduction to Integration (Week 6)	101
6.1	The Challenge: From Rates to Totals	101
6.2	Antiderivatives: Reversing Differentiation	101
6.3	Basic Integration Rules	102
6.4	Rules That Do <i>Not</i> Hold for Integration	103
6.5	Finding Specific Antiderivatives: Initial Conditions	104
6.6	Limiting Values: Where Systems End Up	104
6.7	Science Context: Land Degradation and Rehabilitation	105
6.8	Preview: The Definite Integral and Area	106
6.9	Python: SymPy for Symbolic Integration	107

6.10	Python Lab: A Guided Walkthrough	107
6.10.1	Part A — Rates vs. Totals	108
6.10.2	Part B — Antiderivatives	108
6.10.3	Part C — Project Valuation	109
6.10.4	Part D — Symbolic Integration with SymPy	110
6.11	Week 6 Summary	110
6.12	Before Week 7: Self-Check	110
6.13	Exercises	111
7	Definite Integrals and Applications (Week 7)	113
7.1	From Indefinite to Definite: Completing Calculus	113
7.2	The Fundamental Theorem of Calculus	113
7.3	Properties of Definite Integrals	115
7.4	Geometric Interpretation: Area and Signed Area	115
7.5	Area Between Two Curves	116
7.6	Average Value of a Function	116
7.7	Sequences and Series: Bringing It Together	117
7.8	The Malthusian Trap: Arithmetic vs. Geometric Growth	117
7.9	Consumer and Producer Surplus	118
7.10	Riemann Sums: Connecting Series to Integrals	120
7.11	Python Lab: A Guided Walkthrough	120
7.11.1	Part A — Finding Equilibrium	121
7.11.2	Part B — Surplus, Graphically	121
7.11.3	Part C — Surplus via Integration	122
7.11.4	Part D — Symbolic Integration with SymPy	122
7.12	Week 7 Summary	123
7.13	Before Week 8: Self-Check	123
7.14	Exercises	123
III	Predicting Interactions	125
8	Predator–Prey Dynamics (Week 8)	127
8.1	From Single Populations to Interacting Systems	127
8.2	State Space: The Geometry of Behaviour	127
8.3	The Lotka–Volterra Model	128
8.4	Finding Equilibrium (Fixed) Points	129
8.5	Phase Portraits and Direction Fields	130
8.5.1	Nullclines	131
8.6	Qualitative Behaviour: Time Series	131
8.7	Limitations and Extensions	132
8.7.1	Adding Carrying Capacity	133
8.7.2	Australian Application: Feral Cats and Small Marsupials	133
8.8	Connection to Scientific Method	134
8.9	Python Lab: A Guided Walkthrough	134
8.9.1	Part A — Equilibrium and Nullclines	135
8.9.2	Part B — Phase Plane: Turning Algebra into Movement	135
8.9.3	Part C — Longer Trajectories and a Management Scenario	136
8.10	Week 8 Summary	137
8.11	Before Week 9: Self-Check	137
8.12	Exercises	137

9	Probability Foundations (Week 9)	139
9.1	From Deterministic to Probabilistic Thinking	139
9.2	Random Experiments, Sample Spaces, and Events	139
9.3	Properties and Rules of Probability	140
9.4	Exam-Style Application: Coin Tosses	141
9.5	Conditional Probability	142
9.6	Bayes' Theorem	143
9.7	Independent Events	144
9.8	Connection to Disease Spread	145
9.9	Tree Diagrams for Sequential Events	145
9.10	The Binomial Distribution	146
9.11	Hypothesis Testing: A Preview	147
9.12	Python Lab: A Guided Walkthrough	147
9.12.1	Part A — Bayes' Theorem	147
9.12.2	Part B — A Negative Result Answers a Different Question	148
9.12.3	Part C — Sensitivity Analysis: Same Test, Different Meaning	148
9.13	Week 9 Summary	149
9.14	Before Week 10: Self-Check	149
9.15	Exercises	149
10	Random Variables and Hypothesis Testing (Week 10)	151
10.1	Random Variables: Building the Idea	151
10.2	Expected Value and Variance	152
10.3	Named Discrete Distributions	153
10.4	The Normal Distribution and Z-Scores	154
10.5	Hypothesis Testing	155
10.6	Type I and Type II Errors	156
10.7	Statistical Power and Sample Size	157
10.8	Python Lab: A Guided Walkthrough	158
10.8.1	Q1 — Hypothesis Testing	158
10.8.2	Q2 — Optimisation (Review, Week 5)	159
10.8.3	Q3 — Integration (Review, Week 7)	159
10.8.4	Bonus: Separable Differential Equations (Optional)	160
10.9	Week 10 Summary	160
10.10	Before Week 11: Self-Check	161
10.11	Exercises	161
IV	Making Optimal Decisions	163
11	Trigonometric Functions and Periodic Models (Week 11)	165
11.1	Why Trigonometry for Scientists?	165
11.2	The Geometry of Circles: Degrees and Radians	166
11.3	The Unit Circle: Defining Trigonometric Functions	167
11.4	Signs in Quadrants: The ASTC Rule	168
11.5	Graphs of Trigonometric Functions	168
11.6	Modelling Periodic Phenomena: Parameter Fitting	169
11.7	Seasonal Temperature Modelling	170
11.8	Trigonometric Identities	171
11.9	From One Wave to Many: Fourier's Insight	172
11.10	Application: Reading the Heart's Rhythm	173
11.11	Connection to Decision-Making	174

11.12	Python Lab: A Guided Walkthrough	174
11.12.1	Part 1 — Perth Temperature Warm-Up	174
11.12.2	Part 2 — Solve Your Assigned Scenario Card	175
11.12.3	Part 3 — Verify the Next Group's Work	175
11.13	Week 11 Summary	176
11.14	Before Week 12: Self-Check	176
11.15	Exercises	177
12	Simultaneous Equations and Linear Programming (Week 12)	179
12.1	Why Optimisation Under Constraints?	179
12.2	Simultaneous Linear Equations	180
12.3	Market Equilibrium: Supply and Demand	180
12.4	Consumer and Producer Surplus (Revisited)	181
12.5	Linear Programming: The Framework	182
12.6	Solving LP Graphically: Sustainable Campus Dining	183
12.7	LP Solution Algorithm and Special Cases	185
12.8	Applications Beyond Diet Optimisation	185
12.9	Connection to Previous Weeks	186
12.10	Python Lab: A Guided Walkthrough	186
12.10.1	Part 1–3 — Warmup Scenario (Everyone)	186
12.10.2	Part 4 — Your Assigned LP Problem	186
12.10.3	Part 5 — Class Discussion and Instructor Confirmation	187
12.10.4	Debrief	188
12.11	Week 12 Summary	188
12.12	Before the Exam: Self-Check	188
12.13	Exercises	189
A	Getting Started with Python and Jupyter Notebook	191
A.1	Introduction	191
A.2	Setting Up Your Environment	191
A.2.1	Two Ways to Run Jupyter Notebook	191
A.2.2	Installing Miniconda	192
A.2.3	Using Google Colab	192
A.2.4	Organising Your Files	192
A.3	The Jupyter Notebook Environment	192
A.4	Quick Recap: Python Basics	192
A.5	Importing Modules and Key Packages	193
A.5.1	math: Constants and Functions	193
A.5.2	numpy: Vectorised Arrays	193
A.5.3	matplotlib: Plotting	193
A.5.4	Finding Help	194
A.6	Defining Your Own Functions	194
A.7	Working with Data: A Quick Preview	194
A.8	Common Mistakes and How to Avoid Them	195
A.9	Summary of Key Commands: Python vs. R	195
A.10	Resources for Further Learning	195
B	Formula Reference Sheet	197
C	Common Mathematical Constants	199
	Index	200

How to Use This Reader

Every chapter in this reader is built from the same small set of coloured boxes. Learning to recognise them at a glance — before Week 0 starts — makes the rest of the book much faster to navigate.

Box type	What it's for
 Definition	A precise statement of a term or concept. Learn these; exam questions often ask you to state one directly.
 Example	A fully worked problem, start to finish. Follow the working line by line before attempting the exercises.
 Key Formula	A boxed formula worth memorising or bookmarking. Every one of these also appears in Appendix B, the Formula Reference Sheet. (Exercise answer keys use the same grey style as Examples, so they are easy to cover with a hand while you work.)
 Science Connection	Historical or real-world context — who discovered this, or where it shows up in research. Not examinable in itself, but often explains <i>why</i> a method works the way it does.
 Try it in Python	Points to a numbered code snippet from that week's companion notebook — W3-CS02 means <i>Week 3, Code Snippet 02</i> . Run these yourself — reading the reference isn't a substitute.
 Common Trap	Highlights common algebraic, conceptual, or procedural errors to avoid before they cost marks.
 Intuition & Notation	Plain-English explanations of mathematical symbols (“how to read it out loud”) and intuitive concept hooks.
 Activity	A fill-in-the-blank task from the lab notebook, reproduced here for reference or offline review.

A few other conventions used throughout:

- Each chapter opens with a **WEEK N** tag and a one-line *theme*.
- Every chapter ends the same way: a **Summary** (all key formulas in one box), a **Self-Check** list, and a set of numbered **Exercises** with a boxed answer key — use the Self-Check before attempting the exercises, and the exercises before checking the answers.

- Formulas are cross-referenced, not repeated in full: if a later chapter reuses an earlier result, it says so (“from Week 3”) rather than re-deriving it.
- Appendix A is a Python and Jupyter reference that complements the Week 0 pre-lab notebook; Appendix B is a formula reference sheet; Appendix C is a quick reference to common mathematical constants; the **Index** at the very back lists key terms with the page each is defined or introduced on.

Prologue: Mathematical Foundations

Theme: “The Scientist’s Toolkit — Preparing for Quantitative Analysis”

Week 0 is a diagnostic and review week. Its purpose is to confirm you have the prerequisite mathematical vocabulary — and the Python vocabulary — before the course proper begins. No new exam content is introduced here, but fluency in these ideas underpins every subsequent week.

A Fresh Start: Not a Test of Whether You “Are a Maths Person”

If maths has felt difficult or distant for a while, start here slowly. This chapter is not asking you to remember everything at once; it rebuilds a small set of tools that you will reuse throughout the course. At each section, first read the idea in words, then follow one worked example, then try the matching Python activity. Getting stuck is useful information: it shows you exactly which tool to revisit.

Learning outcomes. By the end of this week you should be able to:

1. Identify and work with real number sets ($\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R}$);
2. apply BODMAS/order of operations correctly in scientific calculations;
3. manipulate exponents, radicals, and algebraic expressions;
4. solve linear and quadratic equations and linear inequalities;
5. use scientific notation and perform unit conversions;
6. interpret and apply function notation $f(x)$;
7. write interval notation for domains and ranges; and
8. use Python for basic calculations, arrays, and plotting.

0.1 Reading Greek Letters and Mathematical Symbols

Mathematical symbols are labels, not secret instructions. Their meaning comes from the definition beside an equation, and the same letter can mean something different in another model. Read an equation as a sentence: identify what changes, then ask what each symbol represents *in that model*.

Symbol	Name	Typical role in this reader
α	alpha	rate or model parameter; prey birth rate in Week 8
β	beta	coefficient or interaction rate; predation rate in Week 8
γ	gamma	rate or model parameter; predator death rate in Week 8
λ	lambda	parameter or multiplier; predator efficiency in Week 8
Δ	delta	a change or difference
μ, σ	mu, sigma	mean and spread; used in statistics
π	pi	the circle constant, approximately 3.14159

Symbol	Read it as	Meaning
$\approx$	approximately equals	close to, but not exactly, equal to
$\propto$	is proportional to	changes in a constant ratio with
$\frac{dH}{dt}$	rate of change of H with time	how quickly H changes as time passes
H_0	H nought	starting value of H
H^*	H star	special value, often an equilibrium
$\sum$	sum	add a sequence of quantities

Asking for Help About a Symbol

Name the equation and give the symbol context. For example: “*In the Lotka–Volterra predator equation $\frac{dP}{dt} = \lambda HP - \gamma P$, what does γ mean biologically, and what are its units?*” Do not ask only “*What is gamma?*”: γ has no universal meaning on its own.

0.2 Number Sets and Algebra

Mathematics uses symbols as shorthand for precise statements. Don’t let Greek letters or set notation intimidate you — every symbol is just a standardized abbreviation for a sentence you already understand.

Reading Number Set Notation

When you see a chain like $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$, read it out loud as: “*Natural counting numbers are contained inside integers, which are contained inside rational fractions, which are contained inside all real numbers.*”

Set-builder notation $\{x \in \mathbb{Z} : x > 0\}$ is read out loud as: “*The set of all integers x such that x is strictly greater than zero.*” The colon ($:$) or vertical bar ($|$) always translates to “**such that**”.

Definition: Standard Number Sets

$\mathbb{N} = \{1, 2, 3, \dots\}$ (natural numbers), $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$ (integers), $\mathbb{Q} = \{p/q : p, q \in \mathbb{Z}, q \neq 0\}$ (rationals), $\mathbb{R}$ = all real numbers, with

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}.$$

Numbers such as π , $\sqrt{2}$, e are **irrational**: real but not rational.

$\mathbb{R}$: all real numbers

$\mathbb{Q}$: fractions and terminating or repeating decimals

$\mathbb{Z}$: $\dots, -2, -1, 0, 1, 2, \dots$

$\mathbb{N}$: $1, 2, 3, \dots$

Each inner set is part of every set around it.

Set-builder notation: $\{x : x > 0\}$ reads “the set of all x such that $x > 0$ ”; $\{x \in \mathbb{Z} : x > 0\} = \{1, 2, 3, \dots\}$; $\emptyset$ is the empty set.

0.2.1 Order of Operations (BODMAS)

BODMAS

Brackets → **O**rders (powers/roots) → **D**ivision/**M**ultiplication (left to right) → **A**ddition/**S**ubtraction (left to right).

Example: Applying BODMAS

$2^3 + 4 \times 5 - 10 \div 2 = 8 + 20 - 5 = 23$. Note $3 + 4 \times 2 = 11$, whereas $(3 + 4) \times 2 = 14$ — the brackets change the answer.

Python respects the same precedence, so mathematical expressions translate directly into code.

Try it in Python

W0-CS01 — *BODMAS – Order of Operations*: verify operator precedence with a runnable example.

W0-CS12 — *Comparison and Logical Operators*: combine BODMAS with **and/or** to evaluate conditions.

```
print(3 + 4 * 2)           # 11
print((3 + 4) * 2)         # 14
print(2**3 + 4*5 - 10/2)   # 23.0
```

0.2.2 Exponents and Radicals

Powers and roots can look like a collection of unrelated rules. They are all shorthand for repeated multiplication or its reverse; when unsure, expand a small numerical example rather than relying on memory alone.

Rule	Formula	Example
Product	$a^m \cdot a^n = a^{m+n}$	$2^3 \cdot 2^4 = 2^7 = 128$
Quotient	$a^m / a^n = a^{m-n}$	$5^6 / 5^2 = 5^4 = 625$
Power	$(a^m)^n = a^{mn}$	$(3^2)^4 = 3^8 = 6561$
Negative exponent	$a^{-n} = 1/a^n$	$2^{-3} = 1/8$
Fractional exponent	$a^{1/n} = \sqrt[n]{a}$	$8^{1/3} = 2$
General fractional	$a^{m/n} = \sqrt[n]{a^m}$	$4^{3/2} = 8$

Radical rules mirror these: $\sqrt{ab} = \sqrt{a}\sqrt{b}$, $\sqrt{a/b} = \sqrt{a}/\sqrt{b}$, $\sqrt{a^2} = |a|$.

Try it in Python

W0-CS02 — *Exponentiation*: compare ******, **math.pow**, and **math.sqrt**.

W0-CS11 — *Math Module Constants and Functions*: use **math.pi**, **math.e**, **math.log**.

```
import math
print(2 ** 3)           # 8
print(math.sqrt(16))    # 4.0
print(27 ** (1/3))      # 3.0 (cube root)
```

```
print(2 ** -3)           # 0.125
```

0.2.3 Algebraic Expressions

The Universal Expansion Error

A very common trap for students is writing $(a + b)^2 = a^2 + b^2$. Exponentiation does **not** distribute over addition! Always write out the squared term in full and FOIL (First, Outer, Inner, Last):

$$(a + b)^2 = (a + b)(a + b) = a^2 + ab + ba + b^2 = a^2 + 2ab + b^2.$$

Combine like terms: $3x^2 + 5x - 2x^2 + 7 - 3x = x^2 + 2x + 7$. Expand: $(a + b)(c + d) = ac + ad + bc + bd$.

Factoring Patterns

$$a^2 - b^2 = (a + b)(a - b), \quad a^2 \pm 2ab + b^2 = (a \pm b)^2.$$

Example: Rational expression

$\frac{x^2 - 4}{x + 2} = \frac{(x + 2)(x - 2)}{x + 2} = x - 2$ for $x \neq -2$ — always note the restriction removed by cancelling.

0.3 Equations and Inequalities

An equation asks which value makes two expressions equal. An inequality asks which whole range of values makes a comparison true. In both cases, your aim is to isolate the unknown while doing the same operation to both sides.

Linear: $ax + b = c \Rightarrow x = (c - b)/a$.

Quadratic: $ax^2 + bx + c = 0$, solved by factoring or

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

Key Formulas

$$\text{Quadratic formula: } x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

The discriminant $\Delta = b^2 - 4ac$ determines the number of real roots:

Discriminant	Meaning
$\Delta > 0$	two distinct real solutions
$\Delta = 0$	one repeated real solution
$\Delta < 0$	no real solutions

Example: Solving a quadratic

$$2x^2 + 3x - 5 = 0 \text{ with } a = 2, b = 3, c = -5: x = \frac{-3 \pm \sqrt{9 + 40}}{4} = \frac{-3 \pm 7}{4}, \text{ so } x = 1 \text{ or } x = -2.5.$$

Linear inequalities follow the same rules as equations, *except* multiplying or dividing by a negative number reverses the inequality.

Why Inequalities Flip on the Number Line

Think of inequality reversal geometrically on the number line. We know $2 < 5$ (2 is to the left of 5). If we multiply both sides by -1 , we get -2 and -5 . But -2 is to the *right* of -5 , so $-2 > -5$. Whenever you multiply or divide by a negative quantity, the relative positions on the number line flip!

Example: Inequality reversal

$$-2x + 5 > 11 \Rightarrow -2x > 6 \Rightarrow x < -3 \text{ (dividing by } -2 \text{ flips } > \text{ to } < \text{)}.$$

0.4 Scientific Notation and Units

Scientific notation is a way to make scale visible, not a new kind of number. Before calculating, write down the unit you start with and the unit you need; this catches many errors before they spread through the arithmetic.

Large and small numbers are written $a \times 10^n$ with $1 \leq |a| < 10$, e.g. $381,000,000 = 3.81 \times 10^8$ and $0.00032 = 3.2 \times 10^{-4}$.

Example: Unit conversion

$$381,000,000 \text{ tonnes} = 381,000,000 / 1,000,000 = 381 \text{ million metric tonnes (MMT)}.$$

Try it in Python

W0-CS03 — *Scientific Notation*: convert between `1.5e6` and decimal form; apply unit conversions.

W0-CS13 — *Floor Division and Modulus*: see how `//` and `%` arise in unit and time calculations.

```
tonnes = 381_000_000
mmt = tonnes / 1_000_000
print(f"{mmt} MMT")      # 381.0 MMT
print(17 // 5, 17 % 5)    # 3 2 (since 17 = 5*3 + 2)
```

0.5 Function Notation

Function notation is the bridge from arithmetic to scientific modelling. Read $f(x)$ as “the output produced by the rule f when the input is x ”; the next chapter will build whole models from this simple input-output idea.

Function Notation vs. Multiplication

The notation $f(x)$ is **never** f multiplied by x . The letter f is the *name* of the mathematical machine or rule, and the variable inside parentheses (x) is the *input variable* passed into that machine. Reading $f(x)$ as “ f times x ” will cause severe errors when evaluating compositions or derivatives later.

Definition: Function notation

$f(x)$ denotes the output of function f for input x — **not** f multiplied by x . Domain = set of valid inputs; range = set of possible outputs. The vertical line test determines whether a graph represents a function.

Example: Evaluating f

If $f(x) = 2x + 3$: $f(5) = 13$, $f(-2) = -1$, $f(a) = 2a + 3$, $f(x + 1) = 2x + 5$.

Python functions mirror this notation directly: a mathematical rule $f(x) = 2x + 3$ becomes

```
def f(x): return 2*x + 3.
```

Try it in Python

W0-CS04 — *Defining and Evaluating Functions*: encode $f(x) = 2x + 3$ exactly as written on paper.

W0-CS05 — *Quadratic Function*: evaluate $f(x) = 3x^2 - 2x + 1$ at several points.

```
def f(x):
    return 2*x + 3

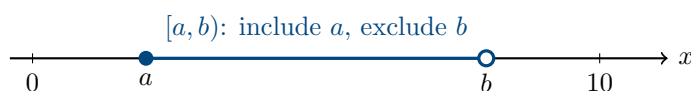
print(f(5), f(-2), f(0))    # 13 -1 3
```

0.6 Interval Notation

Interval notation is a compact way to report a range of allowed values. Read from left to right: identify the endpoints, then use the bracket shapes to decide whether each endpoint is included.

Interval	Notation	Meaning
Closed	$[a, b]$	$a \leq x \leq b$
Open	(a, b)	$a < x < b$
Half-open	$[a, b)$, $(a, b]$	one endpoint included
Infinite	$[a, \infty)$, $(-\infty, b]$	unbounded on one side
All reals	$(-\infty, \infty)$	every $x \in \mathbb{R}$

Square brackets include the endpoint ($\leq, \geq$); round brackets exclude it ($<, >$); ∞ always takes a round bracket.



0.7 Python Lab: A Guided Walkthrough

The Week 0 lab (`Week00_Lab_STUDENT.ipynb`) is a self-paced, 60–90 minute pre-lab notebook that takes you from your very first Python command through to arrays and plots. This section summarises its content and reproduces its seven “Your Turn” activities as exercises; the fully worked companion reference is Appendix A, [Getting Started with Python and Jupyter Notebook](#).

Treat Python as a checking partner, not as a replacement for reasoning. Predict what a line of code should do, run it, then explain any difference between your prediction and the output. This habit will make later mathematical modelling much easier to debug.

0.7.1 Running Your First Commands

A Jupyter notebook is built from **cells**: code cells (which you run with **Shift+Enter**) and markdown cells (formatted text, like this one). Python can be used immediately as a calculator:

```
2 + 2      # 4
10 - 3     # 7
4 * 7      # 28
100 / 8     # 12.5
3 ** 2     # 9 (power, not 3x2)
```

Note that Python uses `*` (not $\times$) for multiplication, `/` (not $\div$) for division, and `**` (not a caret) for powers.

Activity: 1 — Calculator warm-up

Write and run Python instructions to evaluate:

$$(a) (12.5 - 3.1)(3.2 - 2.7) \quad (b) \frac{17.5 - 8.1}{11 + 23}.$$

```
# (a)
a = -----
print(a)

# (b)
b = -----
print(b)
```

0.7.2 Variables, `print()`, and Syntax Essentials

Store a result with `=` so it can be reused:

```
temperature_f = 68
temperature_c = (temperature_f - 32) * (5/9)
print(temperature_c)
```

Variable Naming Rules

Names must start with a letter or underscore (not a digit); they are **case-sensitive** (`Temperature` $\neq$ `temperature`); descriptive names such as `temperature_celsius` are preferred over single letters.

`print()` is a function: input goes inside parentheses, exactly as $f(x)$ takes input inside its own. Omitting the parentheses (`print 2+2`) is a syntax error — and reading error messages, which point to the offending line, is a core programming skill.

Two further rules matter throughout the course:

- **Comments** start with `#` and are ignored by Python — use them to document your reasoning.
- **Indentation** groups code inside `if/else`, loops, and function definitions (Python uses whitespace where Java or R would use `{}`).

```
x = 10
if x > 5:
    print("x is greater than 5")    # indented: inside the if-block
else:
    print("x is 5 or less")
```

Activity: 2 — Spot the bugs

The snippet below has two bugs (hint: Python keywords are lowercase). Find and fix them.

```
x = 2
y = 3
If x > y:
    print("x is greater than y.")
Else:
    print("x is less than or equal to y.")
```

Your corrected version:

```
x = 2
y = 3
---- x > y:
    print("x is greater than y.")
----:
    print("x is less than or equal to y.")
```

0.7.3 Data Types and Operators

Type	Python name	Examples
Whole numbers	<code>int</code>	1, 42, -7
Decimal numbers	<code>float</code>	3.14, -0.5
Text	<code>str</code>	"hello", 'SCIE1500'
True/False	<code>bool</code>	True, False

Use `type(x)` to check a value's type. A **list** such as `[0, 68, 97, 112, 134]` stores several values in order; `len()` gives its length and `temperatures[0]` its first element — **Python counts from 0**.

Category	Operators	Notes
Arithmetic	<code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>**</code> <code>//</code> <code>%</code>	<code>//</code> floor division, <code>%</code> remainder
Comparison	<code>==</code> <code>!=</code> <code>></code> <code><</code> <code>>=</code> <code><=</code>	returns <code>True/False</code>
Logical	<code>and</code> <code>or</code> <code>not</code>	combine boolean conditions

Precedence follows the same BODMAS rule as §0.2.1: parentheses $\rightarrow$ powers $\rightarrow$ $\times, \div$ $\rightarrow$ $+, -$.

Activity: 3 — Predict then Check

Predict the result, then run the code to check:

- `(2 < 3) and (5 > 7)` Prediction: _____ Actual: _____
- `"bread" == "food"` Prediction: _____ Actual: _____

Activity: 4 — Order of operations

Evaluate by hand, then verify in Python:

$$4^2 - \frac{19 + 5}{3} \times 0.1.$$

By hand: _____

```
answer = -----
print(answer)
```

0.7.4 Getting Help

`help(print)` opens Python's built-in documentation for any function. Reading these pages is a skill that improves with practice; web resources such as W3Schools, Real Python, and Stack Overflow are also fair game throughout the semester (see Appendix A, §9 for a curated list).

0.7.5 NumPy: Arrays, linspace, and 2-D Arrays

So far, each calculation has used one number at a time. Arrays let you apply the same rule to a whole collection of measurements, which is the step from calculator work to working with scientific data.

`numpy` lets a formula be applied to **every element of an array at once** — no loop required:

Try it in Python

W0-CS06 import setup **W0-CS07** arrays & `linspace` **W0-CS08** vectorised arithmetic **W0-CS14** 2D arrays & transpose (previews Week 12).

```
import numpy as np
Ftemps = np.array([0, 68, 97, 112, 134])
Ctemps = (Ftemps - 32) * (5/9)           # applies to every
      element at once
print(np.round(Ctemps, 2))
```

`np.linspace(start, stop, n)` generates n evenly spaced values including both endpoints, e.g. `np.linspace(0, 10, 5)` $\rightarrow$ `[0, 2.5, 5, 7.5, 10]`.

Activity: 5 — linspace practice

Use `np.linspace()` to create: (1) 4 equally spaced values between 1 and 3 inclusive; (2) 6 equally spaced values between 0 and 25 inclusive.

```
# (1) 4 equally spaced values between 1 and 3
a = np.linspace(_____)

# (2) 6 equally spaced values between 0 and 25
b = np.linspace(_____)
```

A **2-D array** behaves like a small spreadsheet. For Jack's shopping data (quantities and prices of 4 items):

```
quantity = np.array([2, 5, 3, 1])
price = np.array([3.50, 1.20, 4.00, 15.00])
expenditure = quantity * price          # element-wise
      multiplication
table = np.array([quantity, price, expenditure])
print(table.transpose())                # items as rows, variables
      as columns
```

0.7.6 Plotting with Matplotlib

Plots turn a table of numbers into a pattern you can inspect. Before writing code, decide what belongs on the horizontal axis (the input or explanatory variable) and what belongs on the vertical axis (the measured response).

```
import matplotlib.pyplot as plt
plt.plot(Ftemps, Ctemps, '--ro')      # dashed red line, circle
      markers
plt.title("Temperature Conversion")
plt.xlabel("Fahrenheit ($^\circ$F)")
plt.ylabel("Celsius ($^\circ$C)")
plt.grid()
plt.show()
```

Format Strings

'--ro': dashed line, red, circle markers. Other combinations: 'b-' (solid blue), 'g^' (green triangles), 'ks' (black squares).

Try it in Python

W0-CS09 — line plots **W0-CS10** — scatter plots.

Activity: 6 — Scatter plot

A researcher recorded leaf weight and leaf area:

Leaf weight (kg)	Leaf area (m ²)
1.51	0.17
1.07	0.12
4.74	0.55
0.58	0.07
3.53	0.35
2.34	0.29
6.72	0.80

Create a scatter plot of leaf area (y-axis) against leaf weight (x-axis), with a title, axis labels, and grid.

```
lweight = np.array([_____])
larea   = np.array([_____])

plt._____
plt.title(_____)
plt.xlabel(_____)
plt.ylabel(_____)
plt.grid()
plt.show()
```

Activity: 7 — Plot a parabola

Plot $y = -0.4x^2 + 0.5x + 12$ for $x \in [-8, 8]$: create x -values with `np.linspace(-8, 8, 100)`, compute y , then plot with a title, axis labels, and grid.

```
# x in [-8, 8]
x = np.linspace(start=-8, stop=8, num=100)

# y values
y = _____

# plot
_____
_____
_____
```

0.8 Why This Matters: The Scientific Method

Ocean Plastic and the Scientific Method

Observation → Hypothesis → Mathematical Model → Test against data → Refine.
Plastic production grew from 2 million tonnes in 1950 to 381 million tonnes in 2015 — over 190×, and clearly not linear. Linear, quadratic, and (from Week 2) exponential functions are the mathematical hypotheses scientists use to describe growth like this; the rest of this course develops the toolkit to build, analyse, and test them.

0.9 Week 0 Summary

Key Formulas

Exponent product/quotient	$a^m a^n = a^{m+n}, a^m / a^n = a^{m-n}$
Fractional exponent	$a^{1/n} = \sqrt[n]{a}$
Quadratic formula	$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
Closed/open interval	$[a, b] : a \leq x \leq b; (a, b) : a < x < b$
Function notation	$f(x) = \text{"}f \text{ evaluated at } x\text{"}$

0.10 Before Week 1: Self-Check

Before moving on, make sure you can:

- classify a number by its number set ($\mathbb{N}$, $\mathbb{Z}$, $\mathbb{Q}$, or $\mathbb{R}$);
- apply BODMAS to evaluate a multi-step expression;
- use the exponent and radical rules to simplify an expression;
- solve a linear equation and a quadratic equation (by factoring and by formula);
- solve a linear inequality, remembering to flip it when dividing by a negative;
- convert a number to and from scientific notation, and perform a unit conversion;
- evaluate a function such as $f(x) = 3x^2 - 2$ at a given input;
- write a domain or range in correct interval notation; and
- run basic Python calculations, build a `numpy` array, and produce a labelled `matplotlib` plot.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §0.7 — before Week 1 begins.

0.11 Exercises

These exercises mirror the topics and difficulty of the Week 0 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test. Answers are collected at the end of the section; full worked solutions follow the same steps as the examples earlier in this prologue.

E1. Order of operations. Evaluate:

$$5 + 3 \times 2^3 - 12 \div 4.$$

E2. Exponent rules. Simplify each expression.

- (a) $\frac{x^4 \cdot x^6}{x^3}$
- (b) $(3^2)^3$
- (c) $16^{3/4}$
- (d) 4^{-3}

E3. Algebraic manipulation.

- (a) Expand: $(2x + 5)(x - 3)$
- (b) Factor: $x^2 - 25$

(c) Simplify: $\frac{x^2 - 16}{x - 4}$

E4. Solving equations. Solve each equation for x .

- (a) $6x - 5 = 2x + 11$
- (b) $x^2 - 9x + 20 = 0$
- (c) $3x^2 - x - 4 = 0$

E5. Inequalities. Solve and express in interval notation.

- (a) $2x + 7 > 17$
- (b) $-3x + 4 \leq 19$
- (c) $-2 < 3x + 1 \leq 13$

E6. Scientific notation and units.

- (a) Express 6,340,000 in scientific notation.
- (b) Express 0.000091 in scientific notation.
- (c) Calculate $(4 \times 10^3) \times (5 \times 10^6)$.
- (d) Convert 8,500,000 tonnes to million metric tonnes (MMT).

E7. Function notation. Given $f(x) = 2x^2 + 3x - 1$ and $g(x) = 4x - 7$, evaluate:

- (a) $f(3)$
- (b) $g(-2)$
- (c) $f(0) + g(0)$
- (d) $f(a)$ (leave in terms of a)

E8. Interval notation. Write the correct interval notation for each description.

- (a) All real numbers greater than 2.
- (b) All real numbers from -5 (included) to 4 (excluded).
- (c) All real numbers less than or equal to -1 .
- (d) All real numbers from -6 to 6 , inclusive.
- (e) All negative real numbers.

E9. Python basics.

- (a) What does `print(3 + 4 * 2)` output?
- (b) What does `print(5 ** 2)` output?
- (c) What does `print(22 // 7)` output?
- (d) What is wrong with `if x > 5 print("big")`?
- (e) What does `np.linspace(2, 12, 6)` produce?

E10. Number sets. State the smallest set each number belongs to: $\sqrt{49}$, -8 , $\frac{2}{7}$, and $\sqrt{2}$.

E11. Scientific notation and prefixes. A water sample contains 0.00000042 g of a contaminant. Express this mass in scientific notation and in nanograms (ng), given that 1 ng = 10^{-9} g.

E12. Python calculation and plot. The temperatures $[10, 15, 20, 25]$ are measured in degrees Celsius. Write Python code using `numpy` to convert them to kelvin using $K = C + 273.15$, then make a labelled plot of temperature (K) against measurement number.

Answers

E1: 26.
E2: (a) x^7 (b) 729 (c) 8 (d) $1/64$.
E3: (a) $2x^2 - x - 15$ (b) $(x+5)(x-5)$ (c) $x+4$ ($x \neq 4$).
E4: (a) $x=4$ (b) $x=4$ or $x=5$ (c) $x=4/3$ or $x=-1$.
E5: (a) $(5, \infty)$ (b) $[-5, \infty)$ (c) $(-1, 4]$.
E6: (a) 6.34×10^6 (b) 9.1×10^{-5} (c) 2×10^{10} (d) 8.5 MMT.
E7: (a) 26 (b) -15 (c) -8 (d) $2a^2 + 3a - 1$.
E8: (a) $(2, \infty)$ (b) $[-5, 4)$ (c) $(-\infty, -1]$ (d) $[-6, 6]$ (e) $(-\infty, 0)$.
E9: (a) 11 (b) 25 (c) 3 (d) missing colon: `if x > 5:` (e) `[2, 4, 6, 8, 10, 12]`.
E10: $\sqrt{49} = 7 \in \mathbb{N}$; $-8 \in \mathbb{Z}$; $2/7 \in \mathbb{Q}$; $\sqrt{2} \in \mathbb{R} \setminus \mathbb{Q}$.
E11: 4.2×10^{-7} g; 420 ng.
E12: For example: `C = np.array([10, 15, 20, 25]); K = C + 273.15; plot K against np.arange(1, 5) with labelled axes.`

0.12 Greek Alphabet Quick Reference

The table below gives the standard mathematical forms of all 24 Greek letters. Capitals that look like ordinary Roman letters are still Greek capitals in context. A letter is only a label: its meaning is set by the model or definition where it appears.

Lower	Upper	Name	Lower	Upper	Name
α	A	alpha	ν	N	nu
β	B	beta	ξ	Ξ	xi
γ	Γ	gamma	$\omicron$	O	omicron
δ	Δ	delta	π	Π	pi
ϵ	E	epsilon	ρ	P	rho
ζ	Z	zeta	σ	Σ	sigma
η	H	eta	τ	T	tau
θ	Θ	theta	υ	Υ	upsilon
ι	I	iota	ϕ	Φ	phi
κ	K	kappa	χ	X	chi
λ	Λ	lambda	ψ	Ψ	psi
μ	M	mu	ω	Ω	omega

You may also encounter alternate mathematical forms: ε (epsilon), ϑ (theta), φ (phi), ϱ (rho), and ς (final sigma). They are not different letters.

Part I

Understanding Systems

Chapter 1

Functions and Scientific Language

WEEK 1

Theme: “Functions as the Language of Scientific Models”

“The first task of any analytical scientist is to describe what they observe. Functions give us that language.”

This chapter uses ocean plastic pollution — one of the defining environmental data problems of our time — as a running example for what a function is, how to read one, and how to build one from real data. By the end you should be able to define a function, find its domain and range, work confidently with linear and quadratic functions, recognise even/odd symmetry, and translate a scientific scenario into working Python code.

1.1 The Challenge: Ocean Plastic Pollution

Oceans underpin life on Earth: they produce over half of the planet’s oxygen, regulate roughly 70% of its surface climate, support a US ocean economy worth over \$282 billion in goods and services, and more than 3 billion people rely on seafood as a primary protein source. That makes what is happening to them a quantitative science problem, not only an environmental one.

The Scale of the Problem

200×	more plastic produced today than in 1950
4.8–12.7 million tonnes	entered the oceans in 2010 alone
6	major plastic garbage patches now exist in ocean gyres
80%	of ocean plastic originates from land-based sources
346 million tonnes/yr	from packaging alone — the single largest sector (Geyer et al. 2017)

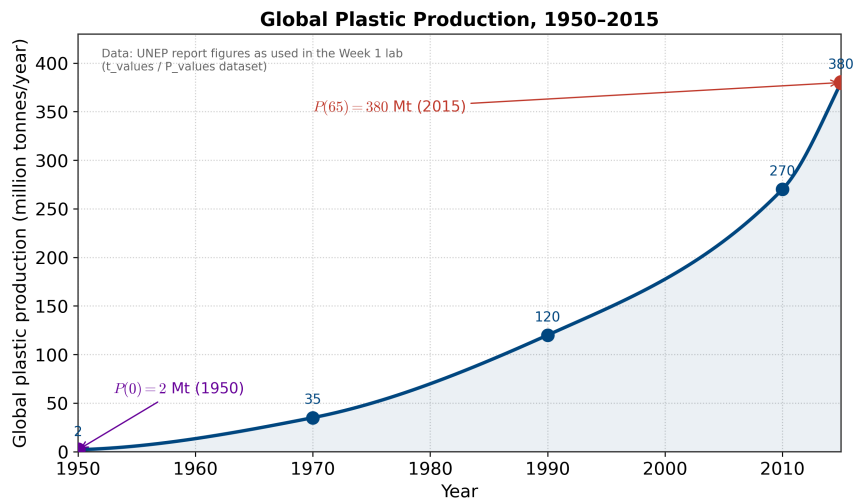
If current trends continue, projections suggest there could be more plastic than fish, by weight, in our oceans by 2050.

As analytical scientists, our job isn’t just to be alarmed — it’s to **quantify, model, and inform decisions**. Reisser et al. (2013) illustrate this well: they sampled ocean plastic around Australia using surface net tows, but wind-driven mixing pushes buoyant plastic below the surface, so a raw surface count underestimates the true amount. Their solution was a *function* that converts a measured surface concentration into a depth-integrated estimate:

$$C_i = \frac{C_s}{1 - e^{-d \cdot w_b \cdot A_0^{-1}}},$$

where C_s is the measured surface concentration, d is sampling depth, w_b is the buoyant velocity of the plastic, and A_0 is a near-surface turbulence coefficient. Applying it moved

their estimate from 4,256 pieces/km² (surface only) to 8,966 pieces/km² (depth-integrated) — more than double. **That is the power of functions:** the same data, correctly modelled, tells a very different story. We return to this model in §1.8.



Global plastic production climbed from about 2 million tonnes in 1950 to roughly 380 million tonnes in 2015. Over this chapter and the next, you will build the vocabulary — functions, domains, slopes, vertices — needed to describe a curve like this precisely, and (in Week 2) to identify what shape it actually is.

1.2 Functions: Definitions and Properties

Think of a function as an automated scientific instrument: you feed in an input variable x (such as time, sample depth, or temperature), and the instrument processes it to produce exactly one output value $y = f(x)$ (such as population density or concentration). If a single input could produce two different outputs at the same time, the instrument would be unpredictable — and mathematically, it would fail to be a function.

Domain vs. Range: The Input Diet and Output Spectrum

Think of the **domain** as the allowable “input diet” that your mathematical machine can process without breaking (e.g., you cannot divide by zero or take the square root of a negative mass). Think of the **range** as the full spectrum of actual output results that the machine can produce.

Definition: Function

A **function** $f : A \rightarrow B$ assigns to every element $x \in A$ exactly one element $y = f(x) \in B$. The set A is the **domain**; the set of actual outputs $\{f(x) : x \in A\}$ is the **range**.

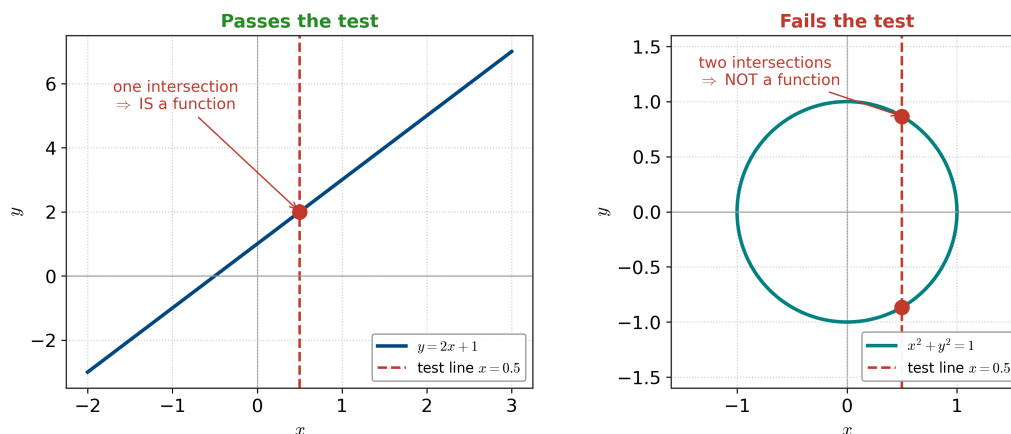
We say y is a function of x if each value of x gives *only one* value of y .

Example: Function or not?

$y = 2x + 1$ **is** a function: there is a single y value for any given x value, e.g. $x = 2$ always gives $y = 5$. But $x^2 + y^2 = 1$ (a circle) is **not**: at $x = 0$, $y = \pm 1$ — two outputs for one input.

1.2.1 The Vertical Line Test

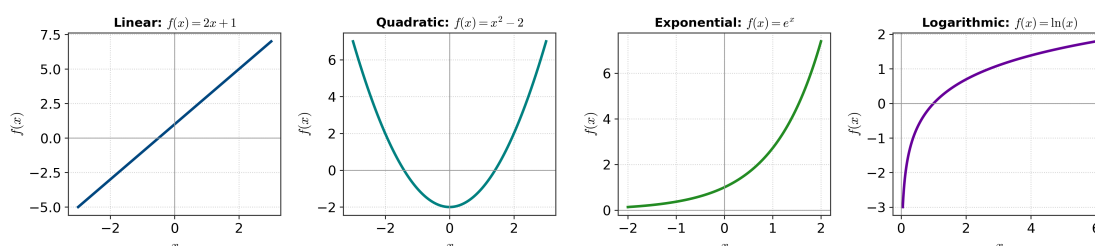
A graph represents a function if and only if **no vertical line intersects the curve more than once**. If a vertical line hits the graph twice, that x -value produces two y -values, which violates the definition of a function.



1.2.2 Domain and Range

- **Domain D :** the set of all valid input values for which f can be computed.
- **Range R :** the set of all output values f can actually produce.

Function	Domain	Range
$f(x) = x^2$	$\{x \in \mathbb{R}\}$	$\{y \in \mathbb{R} : y \geq 0\}$
$f(x) = \sqrt{x}$	$\{x \in \mathbb{R} : x \geq 0\}$	$\{y \in \mathbb{R} : y \geq 0\}$
$f(x) = 2^x$	$\{x \in \mathbb{R}\}$	$\{y \in \mathbb{R} : y > 0\}$



Example: Plastic production as a function

Let $P(t)$ be global plastic production (million tonnes), t years since 1950. From the data, $P(0) = 2$ and $P(65) = 380$, so the average rate of change over the full period is

$$\frac{P(65) - P(0)}{65 - 0} = \frac{380 - 2}{65} \approx 5.82 \text{ million tonnes per year.}$$

This turns a real dataset into inputs, outputs, and an interpretable rate — and it already hints at a subtlety we return to below: the **mathematical** domain of a formula and the **physical** domain of the real system are not always the same thing.

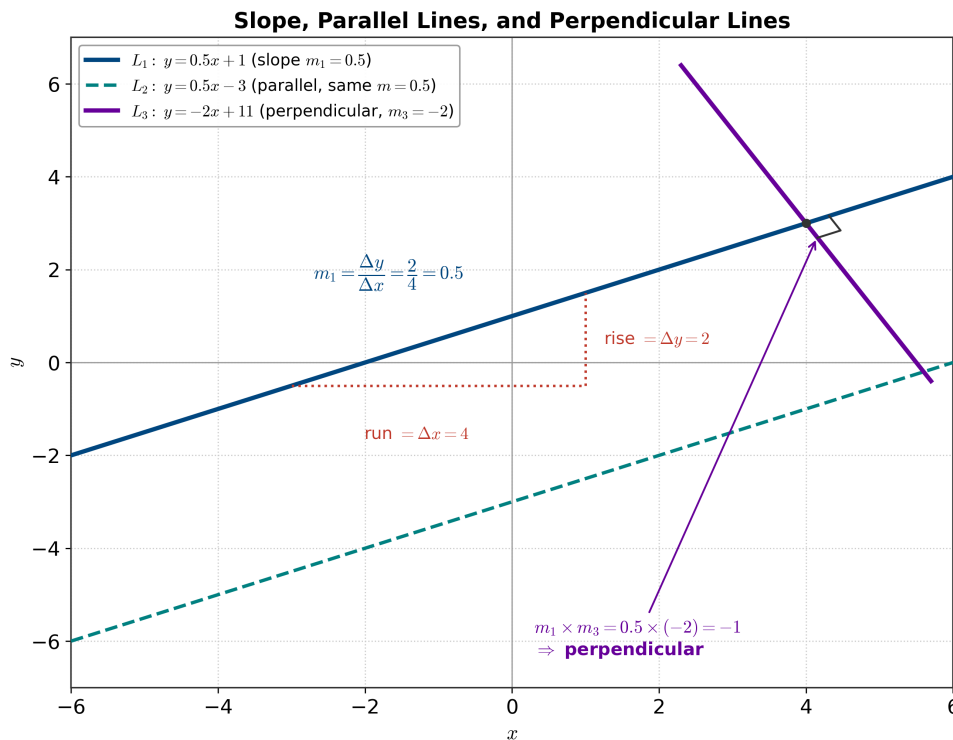
1.3 Linear Functions

Before using the formula, identify what changes, what it changes with, and the units of one step in x . That unit-by-unit change is the slope.

$$f(x) = mx + c$$

- m = slope = rate of change = $\Delta y / \Delta x$; a positive slope means the output increases as x increases, negative means it decreases.
- c = y -intercept: the modelled value when $x = 0$.
- Domain $D = \{x \in \mathbb{R}\}$, range $R = \{y \in \mathbb{R}\}$; the graph is a straight line.

In scientific modelling, slope carries **units**: if production is in million tonnes and time in years, slope has units of million tonnes per year. Lines with equal slope are **parallel**; a larger $|m|$ gives a steeper line; changing c shifts the line up or down without changing its steepness. Two lines are **perpendicular** when the product of their slopes is exactly -1 : $m_1 \times m_2 = -1$, equivalently $m_2 = -1/m_1$.



Example: Ocean plastic depth model

Suppose plastic concentration decreases linearly with depth: $C(d) = 8 - 0.4d$ (g/m³). Then $m = -0.4$ means 0.4 g/m³ reduction per metre depth, and $C = 0$ at $d = 20$ m.

Example: Why linear growth fails for plastic production

Model early production as $P(t) = 2 + 0.5t$ (million tonnes, $t = 0$ in 1950). This gives $P(0) = 2$ and $P(20) = 12$ — but actual 1970 production was 35 million tonnes. A linear model is easy to interpret, but simplicity is not enough once the data clearly bends away from a straight line.

Translating a function into code. Moving from $f(x) = mx + c$ on paper to Python changes two things. First, the *names*: the notebook writes the same line as $f(x) = ax + b$, so **a** is the slope and **b** the intercept — the letters are conventions, not part of the mathematics (compare §0.1). Second, the *input*: instead of one number x , we hand the function a whole **numpy** array built with `np.linspace()`, and the formula `a * x + b` is applied to every element at once. This element-by-element evaluation is called **vectorisation**; it replaces the explicit loop you would write in most other languages, and it is the pattern behind almost every plot in this reader: build an x -array, apply the formula, plot the result.

Try it in Python

W1-CS02 — *Defining a Linear Function*: encode $f(x) = ax + b$ and evaluate it at many points with **numpy**.

W1-CS10 — *Plotting Multiple Linear Functions*: compare several slopes on one figure.

W1-CS14 — *Evaluating a Function at Multiple Points*: vectorised evaluation with arrays.

```
def linear_fun(a, b, x):
    return a * x + b

xvals = np.linspace(-4, 4, 9)
yivals = linear_fun(a=2, b=5, x=xvals)    # y = 2x + 5
print(yivals)
# Output: [-3. -1.  1.  3.  5.  7.  9. 11. 13.]
```

1.4 Quadratic Functions

Linear models assume that growth or decay continues at the exact same constant speed forever (m). But real scientific processes hit peaks, change direction, or slow down — such as a thrown sampling projectile reaching its maximum height, or a crop yield peaking before fertilizer toxicity sets in. To capture curves with a single turning point (a vertex), we escalate from linear models to **quadratic functions**.

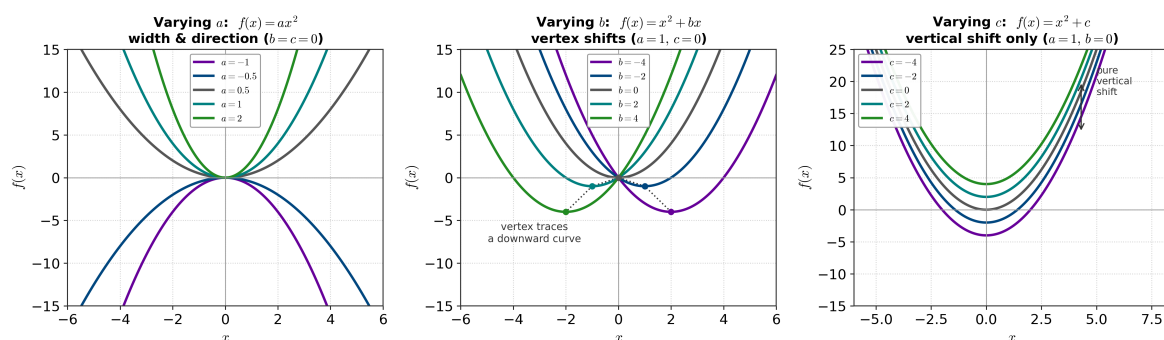
$$f(x) = ax^2 + bx + c, \quad a \neq 0.$$

- Parabola opens upward if $a > 0$, downward if $a < 0$.
- Vertex (turning point) at $x^* = -b/(2a)$, giving $y^* = f(x^*)$; this is also the axis of symmetry.
- Roots (x-intercepts) via the quadratic formula: $x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$.
- y -intercept is c (the value at $x = 0$).
- Domain $D = \{x \in \mathbb{R}\}$; range depends on a — if $a > 0$, $R = \{y : y \geq y^*\}$, if $a < 0$, $R = \{y : y \leq y^*\}$.

Key Formulas

$$\text{Vertex: } x^* = -\frac{b}{2a}; \quad \text{Quadratic formula: } x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

The Quadratic Family: How a , b , and c Shape the Parabola



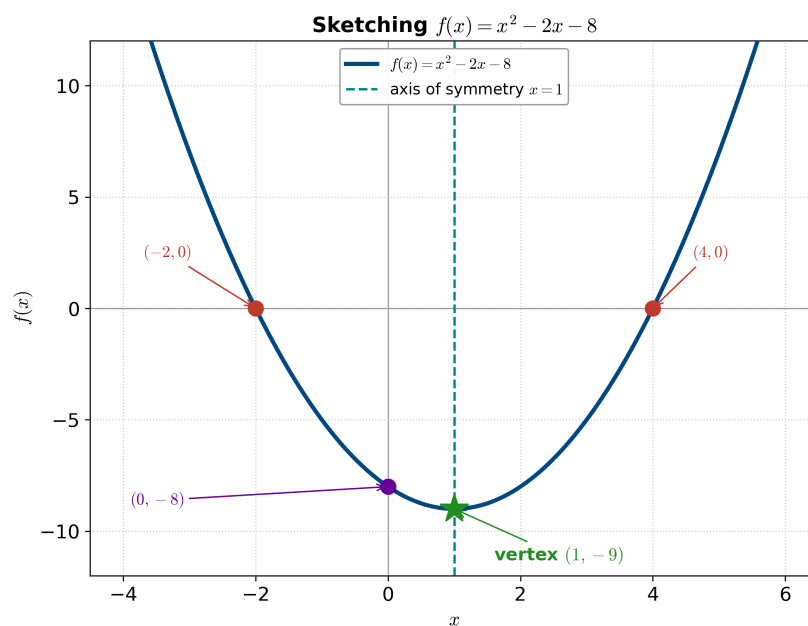
Sketching a quadratic: find the x -intercepts (set $y = 0$), the y -intercept (set $x = 0$), and the vertex; then use the sign of a to decide which way it opens. If two x -intercepts exist, their midpoint is the vertex's x -coordinate.

Example: Sketching a Parabola

Sketch $f(x) = x^2 - 2x - 8$.

1. **x -intercepts:** $0 = (x - 4)(x + 2)$, so $x = 4$ or $x = -2$.
2. **y -intercept:** $y = -8$.
3. **Vertex:** $x = -\frac{-2}{2(1)} = 1$, $y = 1 - 2 - 8 = -9$, giving $(1, -9)$.

The vertex is $(1, -9)$, the axis of symmetry is $x = 1$, and the parabola opens upward since $a = 1 > 0$.



The **discriminant** $\Delta = b^2 - 4ac$ is a quick diagnostic of how many real roots exist, without fully solving the equation:

Discriminant	Meaning
$\Delta > 0$	two distinct real roots
$\Delta = 0$	one repeated real root
$\Delta < 0$	no real roots

Scientific meaning. Quadratic functions describe *accelerating or decelerating change* — cleanup efficiency that peaks then declines, fish growth that accelerates then slows (foreshadowing the Schaefer model in Week 3), or plastic accumulation growing at an increasing rate. In these settings, the key scientific question is often not just “what is the output?” but “where is the turning point, and what does it mean?”

Try it in Python

W1-CS03 — *Defining a Quadratic Function*: encode $f(x) = ax^2 + bx + c$.

W1-CS11 — *Plotting Quadratic Functions*: visualise $y = x^2 - 2x - 8$ and others.

W1-CS12 — *Finding Quadratic Vertex*: compute the vertex algebraically and confirm graphically.

W1-CS13 — *Quadratic Formula*: solve for the roots in Python, including the no-real-roots case.

```
def quad_fun(x, a, b, c):
    return a * (x**2) + b * x + c

def find_vertex(a, b, c):
    x_v = -b / (2 * a)
    y_v = a * x_v**2 + b * x_v + c
    return (x_v, y_v)

print(find_vertex(a=1, b=-2, c=-8))    # (1.0, -9.0)
```

1.5 Function Symmetry

Symmetry is a quick visual shortcut: it lets you predict one side of a graph from the other and check whether an algebraic answer is plausible.

Definition: Even and odd functions

A function is **even** if $f(-x) = f(x)$ for all x in the domain — its graph is symmetric about the y -axis (e.g. $f(x) = x^2$). A function is **odd** if $f(-x) = -f(x)$ for all x — its graph is symmetric about the origin (e.g. $f(x) = x^3$).

Example: Testing symmetry

$f(x) = x^4 - 2x^2 + 3$: $f(-x) = x^4 - 2x^2 + 3 = f(x)$ — **even**. A polynomial with only even powers of x (including the constant term, x^0) is always even. Compare $g(x) = x^3 - 4x$: $g(-x) = -x^3 + 4x = -g(x)$ — **odd**.

Recognising symmetry can simplify analysis and helps identify a function’s type directly from a graph, without needing its formula.

1.6 Domain Restrictions in Scientific Modelling

Getting the domain right is essential for valid science: a model that predicts a negative concentration or an undefined value is scientifically meaningless, whatever the algebra says.

Expression type	Restriction	Example
$1/g(x)$	$g(x) \neq 0$	$\frac{1}{x-2}$: $x \neq 2$
$\sqrt{g(x)}$	$g(x) \geq 0$	$\sqrt{x-3}$: $x \geq 3$
$\ln(g(x))$	$g(x) > 0$	$\ln(x+1)$: $x > -1$

Example: Combined restriction

For $f(x) = \sqrt{x-1}$: the square root needs $x-1 \geq 0$, so the domain is $D = \{x \in \mathbb{R} : x \geq 1\} = [1, \infty)$.

Beyond algebra, **physical** constraints often narrow the domain further: time $t \geq 0$ (nothing before an experiment starts), population $P \geq 0$, concentration $C \geq 0$, or a proportion $0 \leq p \leq 1$. The Jambeck et al. (2015) country-level waste data is a good example — coastal population, waste-generation rate, and mismanaged fraction are all inherently non-negative, so any model built from them inherits that restriction whether or not the bare algebra requires it.

1.7 Python Lab: A Guided Walkthrough

The Week 1 lab (`Week01_Lab_STUDENT.ipynb`) works through three linked parts using the UNEP global plastics dataset and an Australian coastal-waste policy scenario. This section summarises it and reproduces its activities with space to work through them.

1.7.1 Part A — Function Basics and Rates of Change

The lab loads five data points for global plastic production $P(t)$, t in years since 1950:

```
t_values = np.array([0, 20, 40, 60, 65])    # years since 1950
P_values = np.array([2, 35, 120, 270, 380])  # million tonnes
```

The **average rate of change** of P from $t = a$ to $t = b$ is $\frac{P(b) - P(a)}{b - a}$ — the slope of the line joining $(a, P(a))$ and $(b, P(b))$, with units million tonnes per year:

```
avg_rate = (P_values[-1] - P_values[0]) / (t_values[-1] - t_values[0])
print("Average rate of change:", avg_rate, "Mt/yr")
```

Activity: A.2 — Domain and range

Working from the data above, answer as a group:

1. What is the **mathematical** domain of $P(t)$ (all t for which the formula could be evaluated)?
2. What is the **physical** domain (values of t that make sense for this real scenario)?
3. What is the range of P , based on the data given?

```
# 1. Mathematical domain:
# -----

# 2. Physical domain:
# -----
```

```
# 3. Range (from the data):
# -----
```

1.7.2 Part B — Interpreting Slope Across Decades

The overall 1950–2015 average rate hides a lot of detail. Does production grow at the same pace every decade? A loop computes the rate for each of four periods automatically:

```
periods = [(0, 20, '1950-1970'), (20, 40, '1970-1990'),
           (40, 60, '1990-2010'), (60, 65, '2010-2015')]

for t1, t2, label in periods:
    P1 = P_values[t_values == t1][0]
    P2 = P_values[t_values == t2][0]
    rate = (P2 - P1) / (t2 - t1)
    print(f"{label}: {rate:.2f} Mt/yr")
```

Activity: B.1 — Is the rate constant?

1. Is the rate of change constant across decades? What does this tell you about the shape of $P(t)$?
2. The 2010–2015 rate looks lower than 1990–2010 — can you trust that comparison directly? (*Hint: think about the length of each period.*)

```
# 1. Constant rate? What does this imply about the shape of P
# (t)?
# -----

# 2. Why might comparing 2010-2015 to 1990-2010 directly be
# misleading?
# -----
```

Activity: B.2 — Plot and interpret

Plot $P(t)$ against year (scatter points joined by a line). Does the graph look linear or curved? If you extended it to 2030, what shape would you expect, and why?

```
fig, ax = plt.subplots()
ax.scatter(1950 + t_values, P_values, color='steelblue',
           zorder=5)
ax.plot(_____)
ax.set_xlabel(_____)
ax.set_ylabel(_____)
ax.set_title(_____)
plt.show()
```

1.7.3 Part C — Australian Policy Application

Australia's coastal population in 2010 was about 20 million, each person generating roughly 50 kg of plastic waste per year, with 2% mismanaged (i.e. escaping proper disposal). The 2010 baseline, in tonnes/year, is

$$\text{waste}_{2010} = \frac{20,000,000 \times 50 \times 0.02}{1000} = 20,000 \text{ tonnes/year.}$$

If the coastal population grows 1.5% per year, population at time t (years after 2010) is $20,000,000 \times 1.015^t$, giving the function

$$W(t) = \frac{(20,000,000 \times 1.015^t) \times 50 \times 0.02}{1000} \quad (\text{tonnes/year}).$$

Activity: C.1 — Write and test $W(t)$

Complete the function below so that $W(0)$ reproduces the 2010 baseline of 20,000 tonnes/year.

```
def W(t):
    'Annual mismanaged plastic waste (tonnes/year), t years
    after 2010.'
    population_t = _____
    waste = _____
    return waste

print(W(0))      # should match the 2010 baseline
print(W(10))     # projection for 2020
print(W(20))     # projection for 2030
```

The government proposes cutting the mismanagement rate from 2% to 0.5% by 2030. Before running any numbers, discuss: will this fully offset 1.5%/year population growth, or only partially?

Activity: C.2 — Test the policy scenario

Define $W_{\text{policy}}(t)$, identical to $W(t)$ except for the mismanagement rate, then compare both at 2030 ($t = 20$) against the 2010 baseline.

```
def W_policy(t):
    'Annual mismanaged waste (tonnes/year) under the 0.5%
    policy.'
    population_t = _____
    policy_rate = 0.005
    waste = _____
    return waste

print("BAU 2030: ", W(20))
print("Policy 2030:", W_policy(20))
print("2010 baseline:", W(0))
```

Group conclusion: does the policy fully offset population growth, or only partially? What does this imply for mismanagement-reduction policies on their own?

Try it in Python

W1-CS04 — *Piecewise Function with if-else*: useful for modelling regulation thresholds that kick in above/below some value.

W1-CS07–W1-CS09 — loading the real global-plastics CSV with **pandas**, adding derived columns, and plotting the trend.

```
def h(x):
    if (x >= 0) and (x < 4):
        return 9
    elif (x >= 4) and (x < 10):
        return 1 + 2*x
    else:
        return 21
```

1.8 Why This Matters: The Reisser Depth-Integration Model

Revisiting the Reisser Model

Recall from §1.1: $C_i = C_s / (1 - e^{-d \cdot w_b \cdot A_0^{-1}})$. Three questions worth sitting with:

1. If plastics are less buoyant (smaller w_b), does a surface sample underestimate total plastic by *more* or *less*? (*Hint: what happens to the denominator as w_b shrinks?*)
2. If there is less near-surface turbulence (smaller A_0), is there more plastic in the ocean than surface measurements alone would suggest? (*Hint: less mixing means plastic stays nearer the surface — which way does C_i/C_s move?*)
3. How does ocean wind fit into this story? (*Hint: wind drives turbulence A_0 and affects how buoyant particles are distributed with depth.*)

This is exactly the kind of question a function lets you ask precisely: change one input, reason about the output, before you ever touch data.

1.9 Week 1 Summary

Key Formulas

Function definition	each x gives exactly one y
Linear function	$f(x) = mx + c$
Quadratic function	$f(x) = ax^2 + bx + c$
Quadratic vertex	$x = -b/(2a)$
Quadratic formula	$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
Even function	$f(-x) = f(x)$
Odd function	$f(-x) = -f(x)$

1.10 Before Week 2: Self-Check

Before moving on, make sure you can:

- state the definition of a function and explain what the vertical line test checks;
- find the domain and range of a function from its formula or its graph;
- identify the slope and intercept of a linear function and interpret the slope's units;
- find the vertex, roots, axis of symmetry, and concavity of a quadratic function;
- use the discriminant to predict how many real roots a quadratic has, before solving it;
- test whether a function is even, odd, or neither;
- distinguish a function's mathematical domain from the physical domain of the system it models; and
- define a Python function for a linear or quadratic model and evaluate it at an array of points.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §1.7 — before Week 2 begins.

1.11 Exercises

These exercises mirror the topics and difficulty of the Week 1 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Function evaluation. For $g(x) = -2x^2 + 7x - 3$, find $g(0)$, $g(3)$, and $g(-1)$.

E2. Domain — square root. Find the domain of $f(x) = \sqrt{3x - 9}$.

E3. Domain — combined restrictions. Find the domain of $f(x) = \frac{\sqrt{x+5}}{x-2}$.

E4. Linear function — interpretation. A tank drains according to $V(t) = 250 - 15t$ (litres), t in minutes.

- Identify the slope and y -intercept, and interpret each in context.
- At what time does the tank empty?

E5. Quadratic vertex. Find the vertex of $y = -3x^2 + 12x - 5$, and state whether it is a maximum or minimum.

E6. Quadratic roots. Use the quadratic formula to solve $2x^2 + 3x - 9 = 0$.

E7. Concavity and range. For $y = 4x^2 - 16x + 11$, state whether the parabola opens upward or downward, and give its range.

E8. Even and odd functions. Classify each as even, odd, or neither.

- $f(x) = x^3 - 4x$
- $g(x) = 2x^4 + 3x^2 - 1$

E9. Interval notation.

- Write the domain of $f(x) = \sqrt{x-5}$ in interval notation.
- Write the range of $y = (x-2)^2 + 3$ in interval notation.

E10. Applied optimisation. A café's daily profit is modelled by $\pi(x) = -4x^2 + 64x - 100$ (dollars), where x is the number of staff rostered on. Find the number of staff that maximises profit, and the maximum profit.

E11. Identifying function type from context. A bacterial colony triples in size every 4 hours. Which type of function — linear, quadratic, or exponential — best describes this growth, and why?

E12. Mathematical and physical domains. A ball is launched from a 1 m platform and its height is modelled by $h(t) = -5t^2 + 20t + 1$, where h is in metres and t is in seconds. State the mathematical domain, then give a sensible physical domain and range for this model.

Answers

E1: $g(0) = -3$, $g(3) = 0$, $g(-1) = -12$.

E2: $x \geq 3$, i.e. $[3, \infty)$.

E3: $x \geq -5$ and $x \neq 2$, i.e. $[-5, 2) \cup (2, \infty)$.

E4: (a) slope = -15 L/min; intercept = 250 L. (b) $t = 250/15 \approx 16.7$ min.

E5: Vertex $(2, 7)$; a maximum, since $a = -3 < 0$.

E6: $x = 3/2$ or $x = -3$.

E7: Opens upward ($a = 4 > 0$); range $[-5, \infty)$.

E8: (a) odd (b) even.

E9: (a) $[5, \infty)$ (b) $[3, \infty)$.

E10: $x = 8$ staff; maximum profit = \$156.

E11: Exponential — multiplied by a constant factor (3) each period, not a constant amount.

E12: Mathematical domain: all real t . Physical domain: $0 \leq t \leq 2 + \sqrt{105}/5 \approx 4.05$ s; physical range: $0 \leq h \leq 21$ m.

Chapter 2

Exponential and Logarithmic Functions

WEEK 2

Theme: “Modeling Unbounded Growth and Decay”

“Exponential growth is not intuitive — but once you see it in data, you cannot unsee it.”

2.1 Why Exponential Growth Matters

Without limits on space, food, or other resources, populations grow **geometrically**: each unit of time multiplies, rather than adds to, what came before. If a bacterium replicates every 20 minutes, after h hours the population is $P(h) = P(0) \cdot 2^{3h}$.

Time	Population
1 hour	8 bacteria
3 hours	512 bacteria
10 hours	1,073,741,824 bacteria (≈ 1.07 billion)

If that same bacterium divided every 19 minutes instead of 20, after 10 hours it would reach 3.2 billion — nearly triple, from a barely-noticeable change in rate.

Key Insight

The compounding effect means small differences in growth rate make huge differences over time. This is why exponential growth defeats human intuition: we are used to things that add up, not things that multiply.

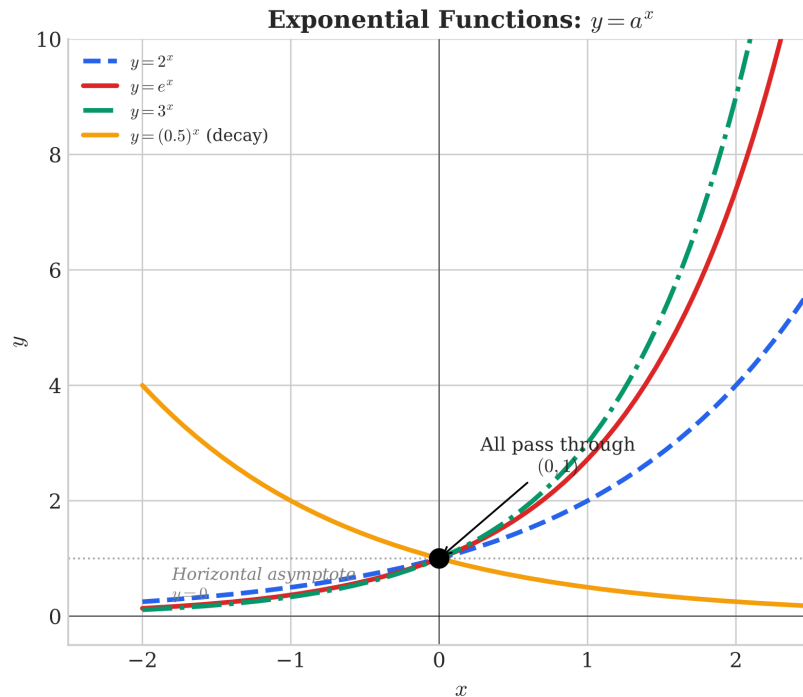
2.2 The Exponential Function

In linear growth, a fixed amount is added to the quantity in each time step ($+m$). In exponential growth, the current quantity is multiplied by a fixed factor in each time step ($\times r$). Because the amount added depends directly on the current size, the bigger the population gets, the faster it grows!

Definition: Exponential function

$f(x) = a \cdot b^x$, where $a, b > 0$ and $b \neq 1$. The base b controls the rate: $b > 1$ gives growth, $0 < b < 1$ gives decay.

Function	Behaviour
$f(x) = 2^x$	doubling with each unit increase in x
$f(x) = 3^x$	tripling with each unit increase in x
$f(x) = (0.5)^x$	halving with each unit increase (decay)



Every curve above passes through $(0, 1)$, since $a^0 = 1$ regardless of the base — only the steepness differs. A larger base climbs faster for $x > 0$; a base between 0 and 1 decays instead of growing.

Try it in Python

W2-CS01 — *Setup: Import Libraries*: standard imports used throughout this chapter.

W2-CS04 — *Exponential Growth vs Decay*: plot b^x for $b > 1$ and $0 < b < 1$ side by side.

W2-CS15 — *Population Growth Rates Comparison*: linear vs exponential trajectories at a glance.

2.3 The Number e : Two Roads to the Same Constant

You do not need to memorise both derivations immediately. The practical point is that e is the base used when change is continuous; the calculus reason becomes useful in Weeks 4–6.

Every exponential function can be rewritten in base e , since $b = e^{\ln b}$. Why single out this particular base? Two independent lines of reasoning arrive at the same number.

2.3.1 Definition 1: Continuous Compounding

Suppose you invest \$1 at 100% annual interest. How much do you have after one year? It depends on how often the interest compounds.

Compounding	Formula	Value
Annual ($n = 1$)	$(1 + 1/1)^1$	\$2.0000
Semi-annual ($n = 2$)	$(1 + 1/2)^2$	\$2.2500
Quarterly ($n = 4$)	$(1 + 1/4)^4$	\$2.4414
Monthly ($n = 12$)	$(1 + 1/12)^{12}$	\$2.6130
Daily ($n = 365$)	$(1 + 1/365)^{365}$	\$2.7146
Continuous ($n \rightarrow \infty$)	$\lim_{n \rightarrow \infty} (1 + 1/n)^n$	\$2.71828...

As compounding gets more frequent, the total keeps increasing — but it converges rather than growing without bound.

Euler's Number — Compound Interest Definition

$$e = \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n}\right)^n \approx 2.71828\dots$$

This is **continuous compounding**: interest added infinitely often, in infinitely small amounts. Compounding n times per year at rate r gives $A = P(1 + r/n)^n$, and as $n \rightarrow \infty$ this becomes $A(t) = A_0 e^{rt}$.

2.3.2 Definition 2: Slope Equals Height

Recall from Week 1 that slope measures how steeply a curve rises. For a curve, the slope changes from point to point, and can be approximated at any x by

$$\text{slope at } x \approx \frac{f(x+h) - f(x)}{h}$$

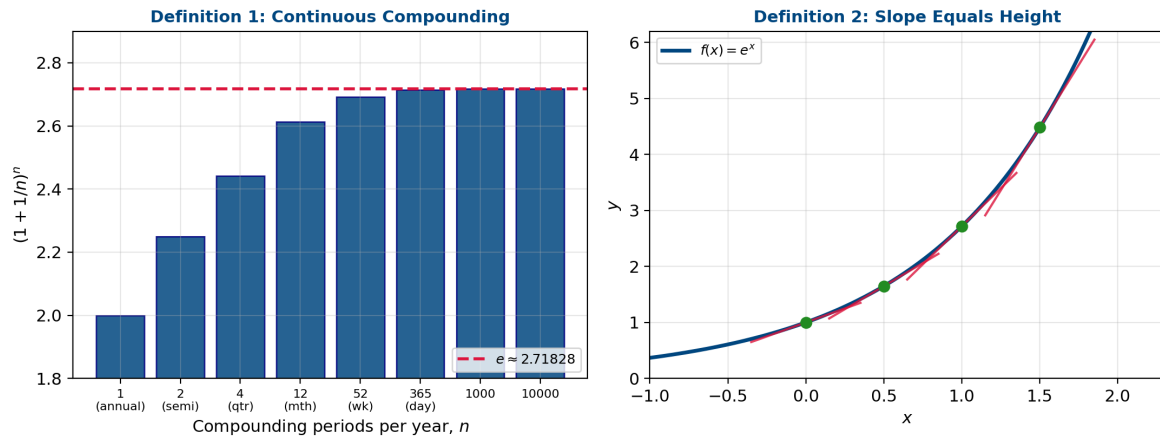
for small h . Computing this at $x = 0$ for different exponential bases:

h	$(2^h - 1)/h$	$(e^h - 1)/h$	$(3^h - 1)/h$
0.1	0.7177	1.0517	1.1612
0.01	0.6956	1.0050	1.1047
0.001	0.6934	1.0005	1.0992
0.0001	0.6932	1.00005	1.0987

Base 2 gives a slope at $x = 0$ below 1; base 3 gives a slope above 1. Somewhere between them sits a base whose slope is *exactly* 1 — and that base is e . Checking further points confirms something remarkable: for $f(x) = e^x$, the slope at $x = 1$ is $e^1 \approx 2.718$, at $x = 2$ it's $e^2 \approx 7.389$ — the slope always equals the function's own height.

Euler's Number — Slope Definition

e is the unique base for which the slope of b^x at every point equals the function's value there. In other words, e^x grows at a rate exactly proportional to its current size; more generally, every function of the form Ce^x has this property.



This is why e appears throughout science: populations grow at a rate proportional to their size, radioactive decay occurs at a rate proportional to what remains, and continuously compounded interest grows proportionally to the balance — in each case, e shows up as the natural base.

Looking Ahead

In Week 5 you'll meet **derivatives**, which give “slope at a point” a formal name and precise definition. The property above becomes the elegant statement $\frac{d}{dx}[e^x] = e^x$ — making e the most important base in calculus.

Try it in Python

W2-CS02 — *The Number e : Limit Definition*: watch $(1 + 1/n)^n$ approach e as n grows.

W2-CS03 — *Why e ? Derivative Investigation*: numerically verify that $\frac{d}{dx}e^x = e^x$.

2.4 The Graph of $\exp(x)$

We often write $\exp(x)$ for e^x .

Property	Value/description
Domain	$\{x \in \mathbb{R}\}$
Range	$\{y \in \mathbb{R} : y > 0\}$
Passes through	$(0, 1)$, since $e^0 = 1$
Monotonicity	always increasing
Horizontal asymptote	$y = 0$ as $x \rightarrow -\infty$

Exponent laws (same as Week 0, applied to base e):

$$e^x \cdot e^y = e^{x+y}, \quad \frac{e^x}{e^y} = e^{x-y}, \quad (e^x)^y = e^{xy}, \quad e^{-x} = \frac{1}{e^x}, \quad e^0 = 1.$$

2.5 Growth and Decay Models

Definition: Exponential growth/decay model

$$P(t) = P_0 e^{kt}$$

where $P_0 = P(0)$ is the initial value and k is the growth/decay rate constant: $k > 0$ gives growth, $k < 0$ gives decay.

Example: Bacteria population growth

A bacteria population starts at 400 and grows as $b(t) = 400e^{1.12567t}$ (t in hours).

(1) **Population after 3 hours:**

$$b(3) = 400e^{1.12567 \times 3} = 400e^{3.377} \approx 11,713 \text{ bacteria.}$$

(2) **Time to double** ($b(t) = 800$):

$$400e^{1.12567t} = 800 \Rightarrow e^{1.12567t} = 2 \Rightarrow t = \frac{\ln 2}{1.12567} = \frac{0.693}{1.12567} \approx 0.616 \text{ hours.}$$

Example: Acacia tree growth — Devaranavadgi et al. 2013

Height-age growth of *Acacia catechu* is modelled as $h(t) = 5.966 \exp\left(-\frac{3.0001}{t + 0.3501}\right)$ metres, t in years.

(1) **Maximum height:** as $t \rightarrow \infty$, the exponent $\rightarrow 0$, so $h_{\max} = 5.966$ m.

(2) **Initial height:** $h(0) = 5.966 \exp(-3.0001/0.3501) = 5.966 e^{-8.57} \approx 0.0004$ m.

(3) **Time to reach half the final height** ($h(t) = 2.983$): solving numerically gives $t \approx 3.978$ years.

Try it in Python

W2-CS05 — *Bacteria Growth Example*: project a population growing at a fixed continuous rate.

W2-CS13 — *Acacia Tree Growth Model*: a real ecology example used in the Week 2 lab.

2.6 The Logarithm Function

Exponential functions answer the question: “If time t passes, how large does the population grow?” Logarithms simply reverse the question: “If we want to reach a specific population target, how much time t must elapse?” Logarithms are the ultimate tool for “un-sticking” variables trapped in exponents.

The Logarithm Addition Fallacy

Logarithms convert multiplication inside into addition outside: $\ln(A \cdot B) = \ln A + \ln B$. However, $\ln(A + B)$ **cannot** be expanded! A very common student trap is writing $\ln(A + B) = \ln A + \ln B$. Logarithms do **not** distribute across sums.

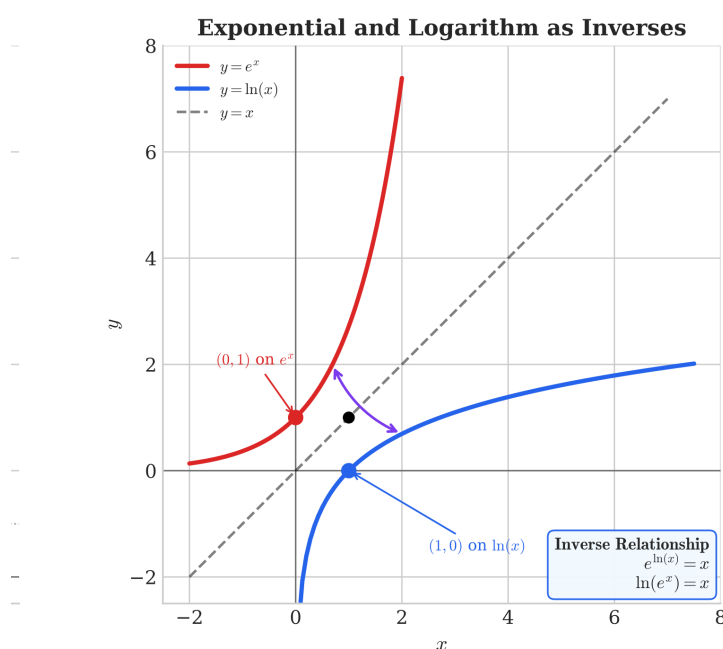
Since $\exp$ is one-to-one, it has an inverse: the **natural logarithm**, $\ln(x)$.

Definition: Natural logarithm

$$y = e^x \iff x = \ln(y).$$

Equivalently, $y = \log_a x \iff a^y = x$ for a general base a , with $\ln x = \log_e x$.

Property	Value/description
Domain	$\{x \in \mathbb{R} : x > 0\}$
Range	$\{y \in \mathbb{R}\}$
Passes through	$(1, 0)$, since $\ln(1) = 0$
Vertical asymptote	$x = 0$



The graph of $\ln(x)$ is the reflection of e^x across the line $y = x$ — the same graphical signature of an inverse pair you'll meet again in §3.8. Since exp and ln undo each other,

$$e^{\ln x} = x, \quad \ln(e^x) = x.$$

Logarithm laws:

$$\ln(AB) = \ln A + \ln B, \quad \ln(A/B) = \ln A - \ln B, \quad \ln(A^n) = n \ln A, \quad \ln 1 = 0, \quad \ln e = 1.$$

Example: Simplifying logarithmic expressions

(a) $\ln(e^x) + \ln(e^{3x}) = x + 3x = 4x$. (b) $e^{\ln(x^2)} = x^2$. (c) $e^{\ln(\sqrt{x})} = \sqrt{x}$. (d) $\exp(\ln(x^2 + x + 2)) = x^2 + x + 2$.

2.7 Solving Exponential and Logarithmic Equations

The strategy is always the same: use the inverse relationship between exp and ln to move the unknown out of the exponent.

Example: Solving for x

- (a) $e^{2x} = 1 \Rightarrow 2x = \ln 1 = 0 \Rightarrow x = 0.$
 (b) $e^{x^2} = 4 \Rightarrow x^2 = \ln 4 \Rightarrow x = \pm\sqrt{\ln 4} = \pm\sqrt{1.386} \approx \pm 1.177.$
 (c) $\ln(x^2) = 4 \Rightarrow x^2 = e^4 \Rightarrow x = \pm e^2 \approx \pm 7.389.$
 (d) $\ln(x^2) = 8 \Rightarrow x^2 = e^8 \Rightarrow x = \pm e^4 \approx \pm 54.60.$

Try it in Python

W2-CS06 — *Logarithm as Inverse of Exponential*: confirm $\log_b(b^x) = x$ visually.

W2-CS07 — *Solving Exponential Equations*: use logs to solve for an unknown exponent.

2.8 The 70/r Rule

Doubling Time

The doubling time of a population growing continuously at rate r is $T = \frac{\ln 2}{r}$. When r is a percentage, this is approximately $T \approx \frac{70}{r\%}$, since $\ln 2 \approx 0.693 \approx 69.3/100$.

Derivation: solve $P_0 e^{rT} = 2P_0$ for T :

$$e^{rT} = 2 \Rightarrow rT = \ln 2 \approx 0.693 \Rightarrow T = \frac{0.693}{r}.$$

For $r = 0.05$ (5%): $T = 0.693/0.05 = 13.9 \approx 70/5 = 14$ years. The same reasoning gives a **tripling time** of $\ln 3/r \approx 110/r\%$.

Try it in Python

W2-CS08 — *The 70/r Rule for Doubling Time*: verify the rule of thumb against exact log calculations.

2.9 Geometric Sequences

Definition: Geometric sequence

A sequence $\{a_1, a_2, a_3, \dots\}$ is **geometric** if $a_k = a \cdot r^{k-1}$ for a constant **common ratio** $r \neq 0$.

Condition	Behaviour
$r > 1$	increasing sequence
$0 < r < 1$	decreasing sequence
$r = 1$	constant sequence

A geometric sequence is a **discrete sampling** of an exponential function: $a_n = a \cdot r^{n-1} \leftrightarrow f(x) = a \cdot r^{x-1}$, with the sequence values lying exactly on the curve at integer points. (Its additive counterpart, the arithmetic sequence, appears in Week 3.)

Example: Compound interest

An investment of \$100,000 earns 5% monthly interest.

(a) **After 3 years (36 months):**

$$a_{36} = 100,000 \times (1.05)^{36} = \$579,181.60.$$

(b) **Time to double:**

$$100,000(1.05)^k = 200,000 \Rightarrow (1.05)^k = 2 \Rightarrow k = \frac{\ln 2}{\ln 1.05} = \frac{0.693}{0.0488} \approx 14.2 \text{ months},$$

so it takes **15 months** (the first whole month by which it has doubled).

Example: Petroleum usage

Global petroleum use grows at 7% annually; usage was 3.95 million barrels/day in 2022. Ten years later (2032):

$$P_{11} = 3.95 \times (1.07)^{10} \approx 7.77 \text{ million barrels/day}.$$

Try it in Python

W2-CS09 — *Geometric Sequence Visualization*: plot the discrete-time analogue of exponential growth.

W2-CS11 — *Compound Interest Calculator*: apply geometric growth to finance.

2.10 Geometric Series

A sequence is a list of values. A series is what you get when you add values from that list. First decide whether a question asks for one term or for a total; the formulas and the interpretation are different.

Let $S_n = a + ar + ar^2 + \dots + ar^{n-1}$. Multiplying by r and subtracting gives $S_n(1-r) = a(1-r^n)$, so:

Geometric Series

$$S_n = \frac{a(1-r^n)}{1-r} \quad (r \neq 1); \quad S_\infty = \lim_{n \rightarrow \infty} S_n = \frac{a}{1-r} \quad \text{if } |r| < 1,$$

where a is the first term, r is the common ratio, and n is the number of terms summed.

Try it in Python

W2-CS10 — *Geometric Series Sum*: compute $\sum ar^n$ in closed form and term by term.

2.11 Population Dynamics Parameters

From population studies (especially Andrewartha, 1970), several growth measures are connected through e :

Symbol	Name	Relation
r_m	Intrinsic rate of increase	—
λ	Finite rate of increase	$\lambda = e^{r_m}$
R_0	Net reproductive rate	$R_0 = e^{r_m T}$
T	Generation time	—

Example: Rice weevil

For $r_m = 0.76$ per week and $T = 6.2$ weeks: $\lambda = e^{0.76} = 2.14$ (the population multiplies by 2.14 each week), and $R_0 = e^{0.76 \times 6.2} = e^{4.71} = 111$ (each individual produces about 111 offspring per generation).

2.12 Application: Probability and Risk

Example: Disease risk

Suppose the probability p of contracting a disease depends on risk factor x as $p = \frac{1}{1 + e^{-3x}}$.

- At $x = 0$: $p = \frac{1}{2}$.
- p is always increasing in x ; as $x \rightarrow -\infty$, $p \rightarrow 0$; as $x \rightarrow +\infty$, $p \rightarrow 1$.

Preview

This S-shaped function is a **logistic function** — the exact shape studied in depth next week, where it models bounded population growth rather than probability.

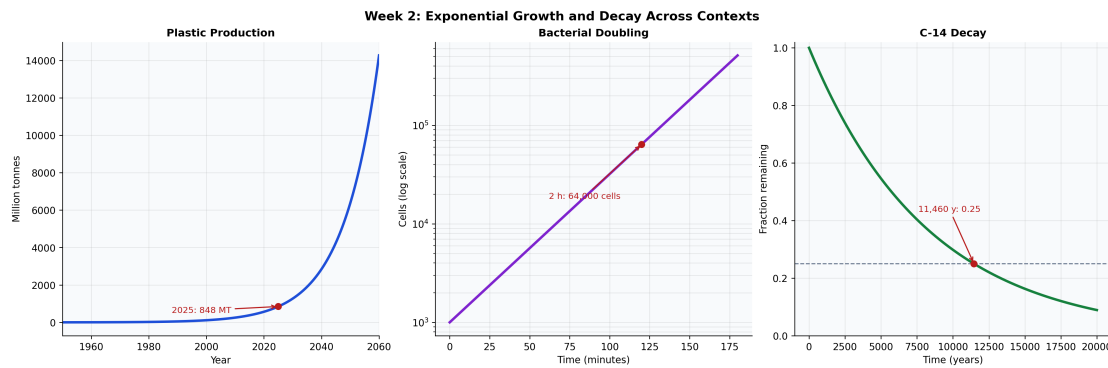
Try it in Python

W2-CS12 — *Probability Function (Logistic-type)*: a non-linear example used to model risk.

W2-CS14 — *Function Composition Demo*: how $f(g(x))$ behaves for exponential and log pairs.

2.13 Python Lab: A Guided Walkthrough

The Week 2 lab (`Week02.Lab_STUDENT.ipynb`), “*Doubling Times — From Plastic to Pandemics*,” builds exponential models for three scenarios (plastic production, bacterial growth, radioactive decay), tests the Rule of 70 against the exact doubling-time formula, then applies logarithmic solving to two further scenarios: an invasive carp population and forensic carbon dating.



2.13.1 Part A — Building the Three Models

All three scenarios share the same structure, $P(t) = P_0 e^{kt}$, but k is supplied in different forms. **Plastic production** grew at a discrete 8.4% per year from $P_0 = 2$ million tonnes (1950); converting the discrete rate to a continuous one uses $k = \ln(1 + r)$:

```
P0_plastic = 2          # million tonnes in 1950
r = 0.084               # 8.4% per year, discrete

k_plastic = np.log(1 + r) # k = ln(1+r) approx 0.0807
def P_plastic(t):
    return P0_plastic * np.exp(k_plastic * t)
```

Bacterial growth is instead specified by a doubling time, giving two equivalent forms:

$$N(t) = N_0 \cdot 2^{t/T_{\text{double}}} \quad \text{or} \quad N(t) = N_0 e^{kt}, \quad k = \frac{\ln 2}{T_{\text{double}}}.$$

```
N0, T_double = 1000, 20 # cells; minutes
k_bact = np.log(2) / T_double
def N_v1(t): return N0 * 2**(t / T_double)
def N_v2(t): return N0 * np.exp(k_bact * t) # identical values
```

Carbon-14 decay uses the same law with a negative rate, written via the **decay constant** $\lambda = \ln 2 / T_{\text{half}} > 0$:

```
A0_c14, T_half = 100, 5730 # % remaining; years
lambda_c14 = np.log(2) / T_half
def A_c14(t):
    return A0_c14 * np.exp(-lambda_c14 * t) # negative sign:
    decay
```

Activity: A — Discuss as a group

1. What does $k \approx 0.0807$ mean in plain English for the plastic model?
2. Why is the exponent negative for decay but positive for growth, even though both use the same $P_0 e^{kt}$ structure?
3. Confirm: after 2 half-lives of carbon-14, exactly 25% remains.

```
# 1. -----
# 2. -----
```

```
# 3. -----
```

2.13.2 Part B — Doubling Time and the Rule of 70

Every exponential model has an exact doubling time, $T = \ln 2/k$ (§2.8). The code below checks how well the quick $70/r$ approximation matches that exact value for the plastic-production model, using its 8.4% annual rate:

```
r_pct = 8.4
T_exact = np.log(2) / k_plastic
T_rule70 = 70 / r_pct
error = abs(T_exact - T_rule70) / T_exact * 100
```

Activity: B.1 — Rule of 70 accuracy

1. At what growth rates is the Rule of 70 most accurate? When does it break down?
2. For bacteria doubling every 20 minutes, what is the *annual* equivalent percentage growth rate? (1 year $\approx 525,600$ minutes.)

```
# 1. -----
# 2. -----
```

Until now the question has been “how much at time t ?” — substituting t into the model. Logarithms let us flip the question: *when* does the population reach a given target C ? Starting from $P_0 e^{kt} = C$,

$$t = \frac{\ln(C/P_0)}{k}.$$

Activity: B.2 — When does plastic production reach 1 billion and 10 billion tonnes?

Use the formula above with $C = 1000$ and $C = 10,000$ (million tonnes) and the plastic model’s k . (These projections assume the 8.4% growth rate holds indefinitely — discuss with your group whether that is realistic.)

```
t_1bn = -----
t_10bn = -----
print(t_1bn, t_10bn)
```

2.13.3 Part C — New Scenarios: Invasive Species and Carbon Dating

The same forward/inverse pairing — substitute for t , or take logs to solve for t — now transfers to two new applied scenarios.

Invasive carp. A carp population in a river system is modelled by $P(t) = 50 e^{0.15t}$, with t in months. Management options become sharply more limited once the population passes certain thresholds:

```
P0_carp, k_carp = 50, 0.15
```

```

limit = 1000    # eradication becomes infeasible above this

def carp(t):    return P0_carp * np.exp(k_carp * t)
def t_to_reach(target): return np.log(target / P0_carp) / k_carp

```

Activity: C.1 — How long until intervention windows close?

Find how long it takes the carp population to reach 500, 1000, and 5000 fish. Which of these thresholds marks the last realistic point for cost-effective eradication, and why?

```

t_500    = -----
t_1000   = -----
t_5000   = -----

```

Carbon dating. A fossil is found to retain 25% of its original carbon-14. Using the decay constant $\lambda = \ln 2 / T_{\text{half}}$ from Part A, solving $A_0 e^{-\lambda t} = 0.25 A_0$ for t gives the fossil's age:

$$t_{\text{fossil}} = \frac{-\ln(0.25)}{\lambda}.$$

Activity: C.2 — How old is the fossil?

Compute t_{fossil} , and express your answer both in years and as a number of half-lives (it should come out to a clean multiple of T_{half} — explain why 25% remaining gives such a clean answer).

```

t_fossil = -----
half_lives = t_fossil / T_half

```

Key Insight

Every exponential model answers two kinds of question with the *same* formula, used two ways: substitute for t to find the value; take logarithms to find the time. This forward/inverse pairing — $P_0 e^{kt}$ versus $t = \ln(C/P_0)/k$ — is exactly the exp/ln inverse relationship from earlier in this chapter, now put to work across growth (plastic, bacteria, carp) and decay (carbon-14) alike.

2.14 Week 2 Summary

Key Formulas

Euler's number (limit)	$e = \lim_{n \rightarrow \infty} (1 + 1/n)^n \approx 2.71828$
Euler's number (slope)	unique base where slope of e^x equals value e^x
Exponential growth/decay	$P(t) = P_0 e^{kt}$
Logarithm definition	$y = \ln x \iff e^y = x$
Log of product/quotient/power	$\ln(ab) = \ln a + \ln b$; $\ln(a/b) = \ln a - \ln b$; $\ln(a^n) = n \ln a$
Doubling time	$T = \frac{\ln 2}{k} \approx \frac{70}{r\%}$
Geometric sequence	$a_n = a_1 r^{n-1}$
Geometric series	$S_n = \frac{a(1 - r^n)}{1 - r}$; $S_\infty = \frac{a}{1 - r}$ if $ r < 1$
Finite rate / net reproductive rate	$\lambda = e^{r_m}$; $R_0 = e^{r_m T}$

2.15 Before Week 3: Self-Check

Before moving on, make sure you can:

- explain, in your own words, why compounding more frequently converges to e rather than growing without bound;
- state both definitions of e (compound-interest limit and slope-equals-height) and why they agree;
- write a growth or decay scenario in the form $P(t) = P_0 e^{kt}$, choosing the sign of k correctly;
- convert between a discrete percentage rate and a continuous rate constant, $k = \ln(1+r)$;
- apply the logarithm laws to simplify expressions and solve equations for an unknown exponent;
- find a doubling (or tripling) time exactly, and approximate it with the $70/r$ (or $110/r$) rule;
- recognise a geometric sequence as a discrete sampling of an exponential function, and sum a geometric series.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §2.13 — before Week 3 begins, where exponential growth meets its first real constraint: a carrying capacity.

2.16 Exercises

These exercises mirror the topics and difficulty of the Week 2 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Identifying exponential functions. Which of $f(x) = x^4$, $g(x) = 5^x$, $h(x) = 4x + 7$, $j(x) = \sqrt[3]{x}$ is exponential, and why?

E2. Evaluating expressions. Evaluate e^0 , $\ln(1)$, and $\ln(e^7)$.

E3. Domain of the logarithm. State the domain of $g(x) = \ln(x - 4)$.

E4. Exponential laws. Simplify $\frac{e^{3x} \cdot e^{-x}}{e^x}$.

E5. Logarithm laws. Simplify $\ln(x^3) - \ln(x) + \ln(2)$.

E6. Solving an exponential equation. Solve $e^{4x} = 9$ for x .

E7. Solving a logarithmic equation. Solve $\ln(x^2) = 6$ for x .

E8. Half-life. A radioactive isotope decays as $A(t) = A_0 e^{-0.02t}$ (t in years). Find its half-life.

E9. Rate constant from data. A population is 300 at $t = 0$ and 900 at $t = 5$ years. Assuming $P(t) = P_0 e^{kt}$, find k .

E10. Geometric sequence. Find the 10th term of the geometric sequence 5, 15, 45, 135, ...

E11. Geometric series to infinity. Find the sum to infinity of $20 + 10 + 5 + 2.5 + \dots$

E12. Compound interest. \$40,000 is invested at 3% monthly interest. Find the balance after 18 months, and the number of months to double.

Answers

E1: $g(x) = 5^x$ — variable in the exponent, constant base; the others are power/linear/root functions.

E2: $e^0 = 1$; $\ln(1) = 0$; $\ln(e^7) = 7$.

E3: $\{x \in \mathbb{R} : x > 4\}$.

E4: $e^{3x} \cdot e^{-x} / e^x = e^{3x-x-x} = e^x$.

E5: $\ln(x^3) - \ln(x) + \ln(2) = \ln(x^3/x) + \ln 2 = \ln(x^2) + \ln 2 = \ln(2x^2)$.

E6: $4x = \ln 9 \Rightarrow x = \ln(9)/4 \approx 0.549$.

E7: $x^2 = e^6 \Rightarrow x = \pm e^3 \approx \pm 20.09$.

E8: $T = \ln 2 / 0.02 \approx 34.7$ years.

E9: $900 = 300e^{5k} \Rightarrow k = \ln(3)/5 \approx 0.220$.

E10: $r = 3$; $a_{10} = 5 \times 3^9 = 98,415$.

E11: $a = 20$, $r = 0.5$; $S_\infty = 20/(1 - 0.5) = 40$.

E12: $a_{18} = 40,000 \times (1.03)^{18} \approx \$68,148$; doubling: $k = \ln 2 / \ln(1.03) \approx 23.4$, so 24 months.

Chapter 3

Logistic Functions and Bounded Growth

WEEK 3

Theme: “When Growth Has Limits”

“Every system has a ceiling. The logistic curve is nature’s way of reminding us that limits are real.”

3.1 Why Growth Has Limits

Exponential models assume infinite space, unlimited food, and zero competition — causing populations to explode toward infinity ($N(t) \rightarrow \infty$). But no habitat on Earth is infinite. As resources become scarce, carrying capacity (K) creates a natural ceiling. When environmental limitations slow down unchecked exponential growth, the curve bends into an S-shape. This is **bounded growth**.

In Week 2 you met exponential growth, $P(t) = P_0 e^{kt}$, which captures the *early* stages of growth beautifully — bacteria in a petri dish, plastic production with abundant resources. But can any real population grow exponentially forever? Clearly not:

System	Limiting factor
Bacteria in a petri dish	nutrients exhausted, waste accumulates
Fish in the ocean	food competition, predation, habitat space
Human population	resources, space, Earth’s carrying capacity
Market for a product	saturation, competing products

Every real growth system eventually meets **compensating factors** that slow, then halt, its growth. The exponential model is only valid while a population is small relative to available resources.

For a renewable resource like fish, it helps to separate two ideas: the **stock** S (total biomass at a point in time) and the **growth** or **flow** G (the change in stock per unit time). The question this chapter answers is: *how does the flow depend on the level of the stock?*

3.2 Arithmetic Sequences: Completing Last Week’s Pair

Last week you met the **geometric sequence** — the discrete cousin of exponential growth, where each term is found by *multiplying* by a constant ratio. Its natural partner is the **arithmetic sequence**: the discrete cousin of linear change, where each term is found by *adding* a constant amount.

Definition: Arithmetic sequence

A sequence $\{a_1, a_2, a_3, \dots\}$ is **arithmetic** if consecutive terms differ by a constant **common difference** d :

$$a_n = a_1 + (n - 1)d.$$

An arithmetic sequence is a **discrete sampling** of a linear function: comparing $a_n = a_1 + (n - 1)d$ with $f(x) = dx + (a_1 - d)$ shows the sequence values lie exactly on a line of slope $m = d$ at integer points.

Example: Fish stock under constant decline

Suppose fish stock declines linearly: $S_n = 12000 - 500(n - 1)$ tonnes in year n . Then $S_1 = 12000$ tonnes, common difference $d = -500$ tonnes/year, and stock reaches zero when $12000 - 500(n - 1) = 0$, i.e. $n = 25$.

Arithmetic Series

$$S_n = \frac{n}{2}(a_1 + a_n) = \frac{n}{2}[2a_1 + (n - 1)d],$$

where a_1 is the first term, a_n is the n -th term, d is the common difference, and n is the number of terms summed.

Linear change is the *simplest* possible model of a changing stock, and sometimes it's genuinely all you need. But most bounded biological growth starts fast, like exponential growth, then *bends* as it nears a limit. Describing that bend is exactly the job of the logistic function, built next.

Try it in Python

W3-CS07 — *Arithmetic Sequence Visualization*: plot a sequence as discrete points alongside its underlying linear function.

```
def arithmetic_term(a1, d, n):
    return a1 + (n - 1) * d

def arithmetic_sum(a1, d, n):
    return n / 2 * (2 * a1 + (n - 1) * d)
```

3.3 The Logistic Function

Read this model in layers: K tells you the ceiling, $P(0)$ tells you where you begin, and α tells you how quickly the system moves between them.

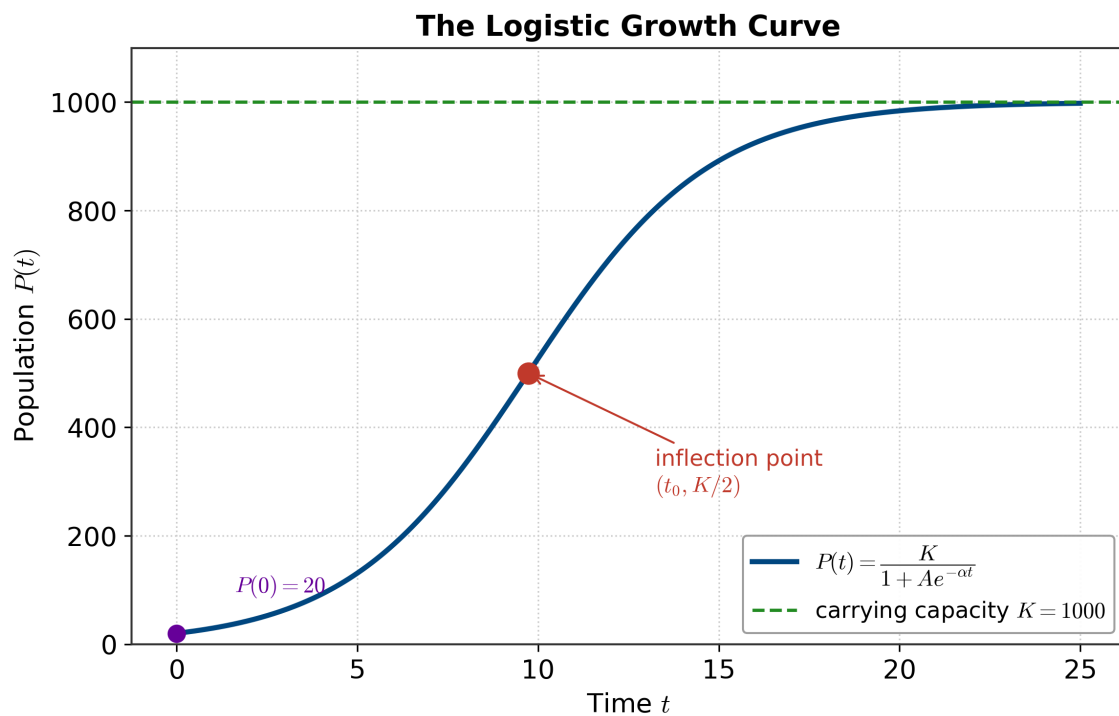
If fish grew at a constant rate g regardless of population size, $G(S) = g \cdot S$, stock would grow exponentially: $S(t) = S_0 e^{gt}$. But as stock grows, competition for resources increases, so the actual growth rate must decline as the population nears the environment's **carrying capacity** K .

Definition: Logistic function

$$P(t) = \frac{K}{1 + Ae^{-\alpha t}}$$

where K is the **carrying capacity**, α is the **intrinsic growth rate**, and $A = \frac{K - P(0)}{P(0)} = \frac{K}{P(0)} - 1$ depends on the initial condition. An equivalent form uses an inflection-time parameter: $P(t) = \frac{L}{1 + e^{-k(t-t_0)}}$, where $L = K$, $k = \alpha$, and t_0 is the time at the inflection point.

Property	Value/description
Domain	$\{t \in \mathbb{R}\}$
Range	$\{P \in \mathbb{R} : 0 < P < K\}$
Initial value	$P(0) = K/(1 + A)$
As $t \rightarrow \infty$	$P(t) \rightarrow K$ (horizontal asymptote)
As $t \rightarrow -\infty$	$P(t) \rightarrow 0$ (horizontal asymptote)
Inflection point	at $P = K/2$, where growth rate is maximum
Shape	S-curve (sigmoid)

**Example: Population with carrying capacity**

A fish population follows $P(t) = \frac{15000}{1 + 120e^{-0.15t}}$ (tonnes, t in years).

(1) **Carrying capacity:** $K = 15000$ tonnes.

(2) **Initial population:** $P(0) = \frac{15000}{1 + 120} = \frac{15000}{121} \approx 124$ tonnes.

(3) **Time to reach 50% of K** ($P = 7500$):

$$7500 = \frac{15000}{1 + 120e^{-0.15t}} \Rightarrow 1 + 120e^{-0.15t} = 2 \Rightarrow e^{-0.15t} = \frac{1}{120} \Rightarrow t = \frac{\ln(120)}{0.15} \approx 32 \text{ years.}$$

That time is exactly the inflection point — the moment the population stops speeding up and begins to slow.

Try it in Python

W3-CS01 — *Logistic Growth Curve*: plot the S-curve and mark the inflection point at $K/2$.

W3-CS05 — *Fitting a Logistic Curve to Real Data*: estimate K , A , α from observed populations with `scipy.optimize.curve_fit`.

```
def logistic(t, K, A, alpha):
    return K / (1 + A * np.exp(-alpha * t))

t = np.linspace(0, 80, 500)
P = logistic(t, K=15000, A=120, alpha=0.15)
t_inflection = np.log(120) / 0.15 # P = K/2 here
```

3.4 The Schaefer Growth Model

The **Schaefer (1957) model** focuses directly on the *growth function* $G(S)$ rather than the trajectory $P(t)$ — particularly useful for fisheries management.

Definition: Schaefer (1957) model

$$G(S) = gS \left(1 - \frac{S}{K}\right)$$

where $G(S)$ is the growth (flow) of fish biomass, g is the intrinsic growth rate, S the current stock, and K the carrying capacity.

The term $(1 - S/K)$ is the key innovation — a **compensating factor**:

Stock level	Compensating factor	Effect
$S \ll K$	$(1 - S/K) \approx 1$	growth near intrinsic rate
$S = K/2$	$(1 - S/K) = 0.5$	maximum total growth
$S = K$	$(1 - S/K) = 0$	zero growth (at capacity)
$S > K$	$(1 - S/K) < 0$	negative growth (decline)

The **actual growth rate** $G(S)/S = g(1 - S/K)$ declines *linearly* from g to 0 as stock moves from 0 to K . The *total* flow $G(S)$, however, rises then falls — a parabola.

Reading the growth curve. Notice what the Schaefer graph does *not* show: time. The horizontal axis is the stock S (tonnes in the water) and the vertical axis is the growth $G(S)$ (tonnes added per year), so every point answers the question “if the stock were S right now, how fast would it be growing?” Growth is zero at $S = 0$ (nothing to reproduce) and again at $S = K$ (no spare capacity), and it is positive in between, so the curve is an inverted parabola

with a single peak. The location of that peak is the stock level at which the resource replaces itself fastest — and, as the next section shows, it is the most a fishery can take out year after year without the stock shrinking. Keeping “stock on the axis, time nowhere” in mind will pay off again in Week 8, where the phase plane uses the same convention for two populations at once.

Example: Fish Stock Dynamics

For $g = 0.1$ and $K = 12,000$ tonnes:

Stock S (t)	$1 - S/K$	$G(S)$ (t/yr)	Growth rate $G(S)/S$
2,000	0.833	167	8.33%
4,000	0.667	267	6.67%
6,000	0.500	300	5.00%
8,000	0.333	267	3.33%
10,000	0.167	167	1.67%
12,000	0.000	0	0.00%

Growth is maximised at $S = 6000$ tonnes — exactly $K/2$. Beyond $K/2$, the compensating factor dominates and total growth *falls* even though the stock is larger.

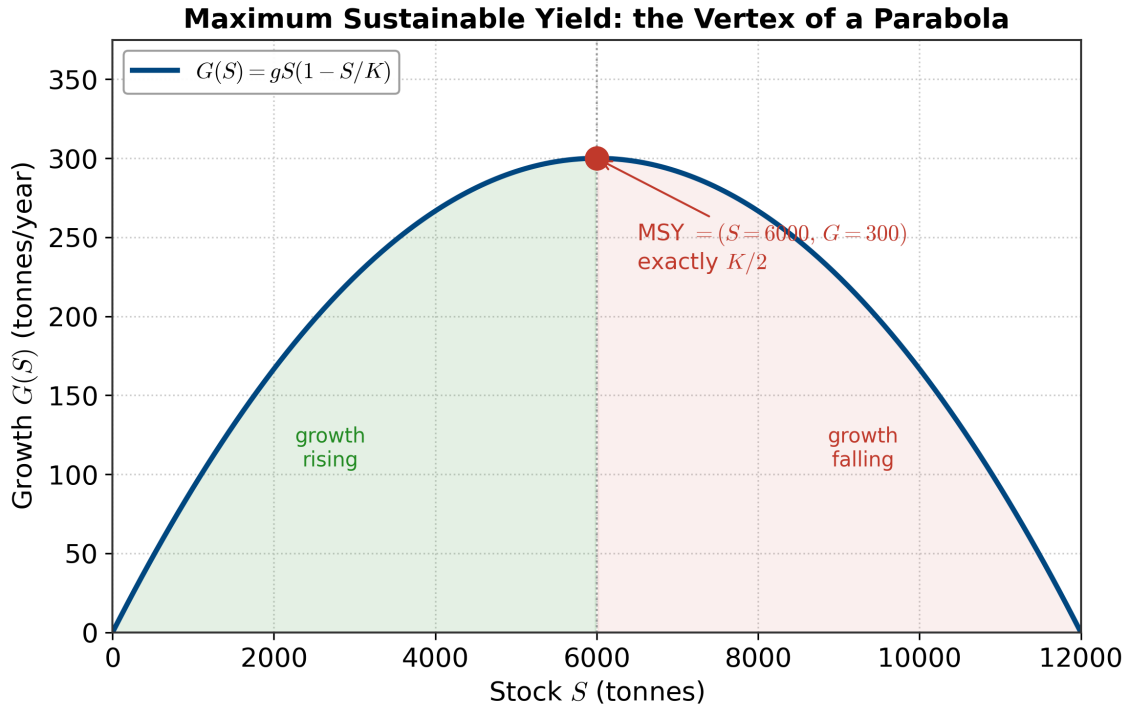
Try it in Python

W3-CS02 — *Schaefer Fish Growth Model*: plot $G(S) = gS(1 - S/K)$, locate MSY, and see the actual growth rate decline linearly.

```
def schaefer(S, g, K):
    return g * S * (1 - S / K)
```

3.5 Maximum Sustainable Yield: It's Just the Vertex

Maximum Sustainable Yield (MSY) is the largest harvest that can be taken from a renewable resource indefinitely without depleting the stock. At MSY, harvest equals growth, and stock remains constant year after year.



Expand the Schaefer model: $G(S) = gS - \frac{g}{K}S^2$ — a quadratic in S with $a = -g/K$, $b = g$, $c = 0$. Since $a < 0$, the parabola opens downward, so it has a single highest point — a vertex, found the ordinary way:

$$S_{MSY} = -\frac{b}{2a} = \frac{-g}{2(-g/K)} = \frac{K}{2}.$$

This is the same vertex formula from Week 1, applied to the growth curve — not a new formula to memorise. Substituting back gives the height of the peak:

$$G_{MSY} = g \cdot \frac{K}{2} \cdot \frac{1}{2} = \frac{gK}{4}.$$

Key Formulas

$$S_{MSY} = \frac{K}{2}; \quad G_{MSY} = \frac{gK}{4}.$$

Example: MSY calculation

For $g = 0.1$, $K = 12,000$ tonnes: $S_{MSY} = 6,000$ tonnes, $G_{MSY} = 300$ tonnes/year. By maintaining stock at 6,000 tonnes, we can sustainably harvest 300 tonnes every year.

3.5.1 Equilibrium Stock Levels for a Given Harvest

MSY tells you the single *best* stock level. But for a fixed harvest H below MSY, there are generally **two** stock levels at which the fishery is in equilibrium (growth exactly replaces the harvest): set $G(S) = H$,

$$gS - \frac{g}{K}S^2 = H \implies \frac{g}{K}S^2 - gS + H = 0,$$

and solve with the quadratic formula. The **lower** root S_1 is an **unstable** collapse threshold: just below it, growth can't keep up with harvest, and stock spirals down. The **upper** root S_2

is a **stable** management target: nearby stock levels drift back toward it. If $H > G_{MSY}$, the discriminant is negative — **no equilibrium exists at all**, and the fishery collapses however much stock you start with.

Current stock vs. S_{MSY}	Management action
$S < S_{MSY}$	reduce harvest, let stock recover
$S = S_{MSY}$	harvest exactly G_{MSY} to maintain steady state
$S > S_{MSY}$	can temporarily harvest more, moving toward MSY

Why the Safe Side Matters

Sitting slightly above $K/2$ is often wiser than sitting exactly on it. A given harvest can be sustained at two different stock levels — S_1 below $K/2$, S_2 above — but S_2 is the safer place to be: larger stocks are cheaper to catch, and a stock on the high side has more room to absorb a shock (a bad season, a disease outbreak) without collapsing. A stock on the low side risks being pushed into a downward spiral. MSY tells you the *biggest* sustainable catch — it doesn't always tell you the *safest* place to sit.

Try it in Python

W3-CS04 — *Sustainable Yield Analysis Table*: growth and sustainable yield across stock levels.

W3-CS06 — *Interactive MSY Exploration*: vary g and K , see how MSY shifts (MSY doubles if you double either g or K ; $S_{MSY} = K/2$ always, regardless of g).

3.6 Connecting Models: Two Lenses on the Same Phenomenon

The Schaefer model can also be written as a differential equation:

$$\frac{dS}{dt} = gS \left(1 - \frac{S}{K} \right).$$

When $S \ll K$, this simplifies to $\frac{dS}{dt} \approx gS$ — exponential growth re-emerges, exactly as expected early on. The solution to the full logistic differential equation is

$$S(t) = \frac{K}{1 + Ae^{-gt}}, \quad A = \frac{K - S_0}{S_0}.$$

Two Views on One Phenomenon

Schaefer is the *flow* view — the natural language of fisheries *management*, since it speaks in terms of what's safe to harvest. **Logistic** is the *trajectory* view — the natural language of *forecasting*, since it tells you where the population will be at a given time. They describe the same S-shaped phenomenon, viewed two ways.

Try it in Python

W3-CS03 — *Exponential vs Logistic Growth Comparison*: plot both curves on the same axes to see why bounded models matter for real populations.

3.7 Function Composition

Composition chains modelling steps forward; an inverse works backward from an observed output. Both are ways of asking how information moves through a model.

Reading Function Composition Notation

The composition $(f \circ g)(x) = f(g(x))$ reads out loud as: “*f of g of x.*” It means you pass your input x into the inner function g first, take the resulting output $g(x)$, and feed that directly as the input into the outer function f .

Definition: Composition

Given functions f and g , the **composition** $f \circ g$ is $(f \circ g)(x) = f(g(x))$ — read as “first apply g , then apply f .” Order matters: generally $f \circ g \neq g \circ f$. Domain matters too: $g(x)$ must lie in the domain of f .

Example: Composing functions

Let $f(x) = e^x$ and $g(x) = 2x + 1$. Then $(f \circ g)(x) = e^{2x+1}$, while $(g \circ f)(x) = 2e^x + 1$ — different functions. And $(f \circ g)(0) = e^1 = e$.

Example: Building a risk model

Exposure depends on distance, $\text{Exposure} = g(\text{Distance})$; risk depends on exposure, $\text{Risk} = f(\text{Exposure})$. Chained together, $\text{Risk} = (f \circ g)(\text{Distance})$. For instance, if $f(x) = \frac{1}{1 + e^{-x}}$ (a sigmoid response) and $g(x) = 3x$ (a linear exposure model), then $(f \circ g)(x) = \frac{1}{1 + e^{-3x}}$ — built by composing exactly this way. Complex scientific models are routinely assembled from simple, understandable parts.

3.8 Inverse Functions

Definition: Inverse function

If f is one-to-one, its **inverse** f^{-1} satisfies $f^{-1}(f(x)) = x$ and $f(f^{-1}(x)) = x$. To find it: write $y = f(x)$, solve for x in terms of y , then swap x and y . Geometrically, f^{-1} is the reflection of f across the line $y = x$.

Example: Finding inverses

- (a) For $f(x) = 3 + \frac{1}{4}x$: $f^{-1}(x) = 4x - 12$.
 (b) For $f(x) = e^{2x}$: $f^{-1}(x) = \frac{1}{2} \ln(x)$.

For the logistic function $P = \frac{K}{1 + Ae^{-\alpha t}}$, solving for t gives

$$t = \frac{1}{\alpha} \ln \left(\frac{AP}{K - P} \right),$$

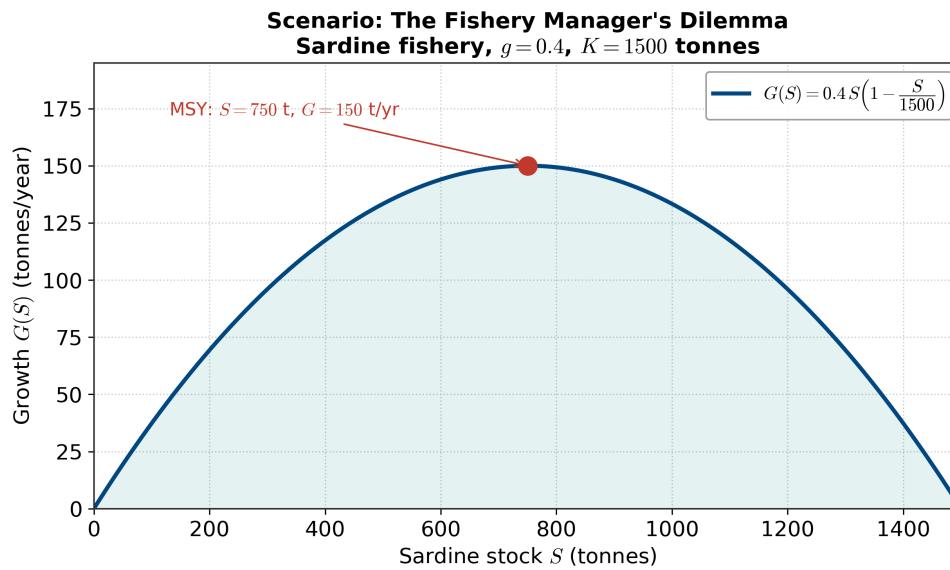
which turns a *forecasting* formula (given t , find P) into a *timing* formula (given a target P , find t).

Try it in Python

W3-CS08 — *Function Composition and Inverse Visualization*: see $f \circ g$ and f^{-1} plotted alongside f , with the $y = x$ reflection line.

3.9 Python Lab: A Guided Walkthrough

The Week 3 lab (Week03_Lab_STUDENT.ipynb), “*The Fishery Manager’s Dilemma*,” puts you in the role of scientific advisor to a state fisheries authority managing a sardine fishery with $g = 0.4$ and $K = 1500$ tonnes.



3.9.1 Part A — The Schaefer Model

$$G(S) = 0.4S \left(1 - \frac{S}{1500}\right) = 0.4S - \frac{0.4}{1500}S^2.$$

This is a downward-opening parabola: growth is zero at $S = 0$ and $S = K$, and peaks at $S^* = K/2$.

```
g = 0.4
K = 1500

def G(S):
    return g * S * (1 - S / K)

for S_level in [400, 750, 1200]:
    print(f"G({S_level}) = {G(S_level):.1f} tonnes/year")

S_msy = K / 2
G_msy = G(S_msy)    # 750 tonnes, 150 tonnes/year
```

Activity: A — Discuss as a group

1. Why does growth peak at $S = K/2$ rather than near the maximum stock level?
2. What does $G(K) = 0$ represent ecologically?
3. At $S = 1200$ t (above $K/2$), is growth increasing or decreasing as stock rises further?

```
# 1. -----
# 2. -----
# 3. -----
```

3.9.2 Part B — Sustainability Analysis

A harvest H is **sustainable** when growth replaces what is removed: $G(S) \geq H$. Setting $G(S) = H$ gives the equilibrium stock levels — states where stock stays constant — via $\frac{g}{K}S^2 - gS + H = 0$.

```
S_current = 1200
H = 120    # current harvest, tonnes/year

G_current = G(S_current)          # G(1200) = 96.0 t/yr
print("Sustainable?", G_current >= H)  # False: 96 < 120
```

Activity: B.2 — Find the equilibrium stock levels

Complete the quadratic-formula calculation for $H = 120$ t/yr.

```
a_coef = g / K
b_coef = -g
c_coef = H

disc = b_coef**2 - 4 * a_coef * c_coef
S1 = ----- # lower root (unstable)
S2 = ----- # upper root (stable)
print(S1, S2)  # should give roughly 414.6 and 1085.4
```

With $H = 120$, the equilibria are $S_1 \approx 414.6$ t (unstable collapse threshold) and $S_2 \approx 1085.4$ t (stable management target). The current stock, 1200 t, sits *above* S_2 — growth (96 t/yr) is less than harvest (120 t/yr), so stock will drift back down toward the stable target of 1085 t, well clear of the collapse threshold.

Activity: B.3 — Raise the harvest to 140 t/yr

Repeat the calculation with $H_{\text{proposed}} = 140$. What happens to the discriminant?

```
H_proposed = 140
c_new = H_proposed
disc_new = b_coef**2 - 4 * a_coef * c_new
# If disc_new < 0: no sustainable equilibrium exists at all.
```

With $H_{\text{proposed}} = 140$: $S_1 \approx 556.4$ t (collapse threshold *rises*) and $S_2 \approx 943.6$ t (stable

target *falls*) — both equilibria move toward the vertex as harvest approaches MSY = 150 t/yr.

Activity: Discuss as a group

1. The current stock (1200 t) is above S_2 for $H = 120$. What happens to the stock over time?
2. Increasing harvest to 140 t/yr raises the collapse threshold. By how many tonnes?
3. The MSY is 150 t/yr. Why must harvest never exceed it?

3.9.3 Part C — Policy Recommendation

Activity: Write a 4–5 sentence recommendation to the Fisheries Authority

Address: is the current 120 t/yr harvest sustainable? Should the proposed 140 t/yr increase be approved? What harvest level do you recommend for long-term stability?

RECOMMENDATION TO THE FISHERIES AUTHORITY:

Key Insight

There are always *two* equilibria for a sustainable harvest — one stable (recovery target), one unstable (collapse threshold). A higher harvest brings these closer together, until they meet exactly at MSY.

3.10 Why This Matters: Preparing for Optimisation

Future topic	Connection
Weeks 4–5: Derivatives	finding MSY using $dG/dS = 0$
Week 5: Optimisation	maximising sustainable yield directly
Week 8: Predator–Prey	Lotka–Volterra extends this to interacting populations
Week 12: Linear Programming	resource allocation under constraints

The Schaefer model shows how quadratic structure leads naturally to optimisation problems with a clear maximum — exactly the kind of problem derivatives are built to solve.

3.11 Week 3 Summary

Key Formulas

Arithmetic sequence	$a_n = a_1 + (n - 1)d$
Arithmetic series	$S_n = \frac{n}{2}(a_1 + a_n)$
Logistic function	$P(t) = \frac{K}{1 + Ae^{-\alpha t}}$
Parameter A	$A = \frac{K}{P(0)} - 1$
Schaefer growth model	$G(S) = gS \left(1 - \frac{S}{K}\right)$
MSY stock level	$S_{MSY} = \frac{K}{2}$
MSY harvest	$G_{MSY} = \frac{gK}{4}$
Function composition	$(f \circ g)(x) = f(g(x))$
Inverse relationship	$f^{-1}(f(x)) = x$

3.12 Before Week 4: Self-Check

Before moving on, make sure you can:

- explain why exponential growth cannot continue indefinitely in a real system;
- find the n th term and the sum of an arithmetic sequence;
- state the carrying capacity, initial value, and inflection point of a logistic function;
- apply the Schaefer model to compute growth $G(S)$ at a given stock level;
- find S_{MSY} and G_{MSY} from g and K , and explain why $S_{MSY} = K/2$ is just a vertex;
- solve for the two equilibrium stock levels at a given harvest, and identify which is stable;
- compose two functions in either order, and explain why order matters;
- find the inverse of a function algebraically and interpret it graphically as a reflection.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §3.9 — before Week 4 begins.

3.13 Exercises

These exercises mirror the topics and difficulty of the Week 3 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Logistic basics. For $P(t) = \frac{8000}{1 + 50e^{-0.2t}}$, state the carrying capacity K and the initial population $P(0)$.

E2. Logistic time calculation. Using the function from E1, find t when the population reaches 50% of carrying capacity.

E3. MSY calculation. A fishery has $g = 0.12$ per year and $K = 18,000$ tonnes. Find S_{MSY} and G_{MSY} .

E4. Schaefer growth calculation. Using the parameters from E3, find $G(5000)$.

E5. Schaefer growth rate. Using the parameters from E3, find the actual growth rate (as a percentage) at $S = 5000$.

E6. Arithmetic sequence. Find the 15th term of the sequence 7, 11, 15, 19, ...

E7. Arithmetic series. Find the sum of the first 25 terms of the sequence in E6.

E8. Function composition. For $f(x) = \sqrt{x}$ and $g(x) = 3x - 2$, find $(f \circ g)(x)$ and evaluate $(f \circ g)(2)$.

E9. Composition order. Using the same f and g as E8, find $(g \circ f)(x)$.

E10. Inverse function. Find the inverse of $f(x) = \frac{x-7}{2}$.

E11. Inverse of the logistic function. Starting from $P = \frac{K}{1 + Ae^{-\alpha t}}$, derive an expression for t in terms of P , K , A , and α .

E12. Equilibrium analysis. A fishery has $g = 0.3$ per year, $K = 2000$ tonnes, and a proposed harvest $H = 90$ tonnes/year. Is H below, at, or above MSY? If sustainable, find both equilibrium stock levels.

Answers

E1: $K = 8000$; $P(0) = 8000/51 \approx 156.9$.

E2: $t = \ln(50)/0.2 \approx 19.6$.

E3: $S_{MSY} = 9000$ t; $G_{MSY} = 540$ tonnes/year.

E4: $G(5000) \approx 433.3$ tonnes/year.

E5: $\approx 8.67\%$.

E6: $a_{15} = 63$.

E7: $a_{25} = 103$; $S_{25} = 1375$.

E8: $(f \circ g)(x) = \sqrt{3x-2}$; $(f \circ g)(2) = 2$.

E9: $(g \circ f)(x) = 3\sqrt{x} - 2$.

E10: $f^{-1}(x) = 2x + 7$.

E11: $t = \frac{1}{\alpha} \ln\left(\frac{AP}{K-P}\right)$.

E12: $MSY = 150$ t/yr, so $H = 90$ is below MSY (sustainable); $S_1 \approx 367.5$ t, $S_2 \approx 1632.5$ t.

Chapter 4

Limits, Continuity, and Derivatives

WEEK 4

Theme: “Measuring Instantaneous Change”

“The analytical scientist doesn’t just ask ‘how much?’ but ‘how fast is it changing?’ Derivatives give us that answer.”

4.1 Why Calculus? Two 17th-Century Problems

Welcome to Calculus! In Weeks 1–3, we analysed static snapshots and average rates of change over broad intervals ($\Delta y/\Delta x$). Calculus is the mathematics of continuous change and smooth curves. Chapter 4 addresses one fundamental question: *How fast is a scientific system changing at this exact, fleeting millisecond?*

Calculus was invented to solve two urgent problems of the 1660s–1680s. Kepler had mapped planetary orbits, but nobody could calculate the continuously changing speed of a planet with static geometry — so Newton built a new mathematics of motion ($v = ds/dt$), eventually proving that a single inverse-square force law explains all three of Kepler’s laws. At the same time, mathematicians needed a systematic way to find areas under curves; Leibniz and Newton independently developed the **integral** to sum infinitesimally small slices, and connected it to the derivative through what we now call the Fundamental Theorem of Calculus (Week 7).

The Logical Chain

This chapter builds three ideas in a helpful order: **Limits** (“what value is f heading towards?”) → **Continuity** (“does f actually arrive there?”) → **Derivatives** (“how fast is f changing?”). You cannot state what continuity means without limits, and differentiability implies continuity, so we develop limits and continuity before derivatives.

4.2 From Total Change to Instantaneous Change

In Weeks 1–3 you used functions to describe how systems behave over time. This week shifts the question: not just *how much* has changed, but **how fast is it changing right now?**

Domain	Question	What we need
Environmental science	How fast is plastic production accelerating this year?	Instantaneous rate
Microbiology	At what rate is a colony growing at this moment?	Instantaneous growth rate
Nuclear physics	How quickly is a sample decaying right now?	Instantaneous decay rate
Fisheries	At what stock level is fish growth maximised?	Rate of change zero

Average rates, from Week 2, are not enough here. Suppose a bacterial colony grows as $N(t) = 1000e^{0.0347t}$ (N cells, t minutes). The **average** growth rate from $t = 0$ to $t = 60$ is

$$\frac{N(60) - N(0)}{60 - 0} = \frac{8017 - 1000}{60} \approx 116.95 \text{ cells/min},$$

but this hides what's happening *at* $t = 60$ specifically. Answering that requires shrinking the interval toward zero width — and that leads directly to **limits**.

4.3 Limits: The Foundation of Calculus

Reading Limit Notation Out Loud

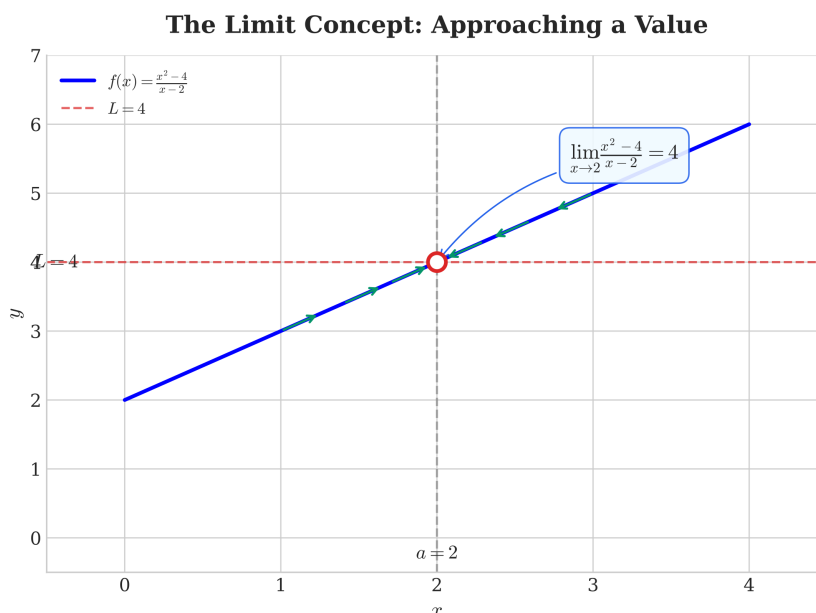
When you see the symbol $\lim_{x \rightarrow c} f(x) = L$, read it out loud as: “*The limit of $f(x)$ as x approaches c is equal to L .*”

It describes the target value that $f(x)$ gets closer and closer to as x gets infinitely close to c on the x -axis. Crucially, a limit cares only about what happens *near* c , never requiring x to actually equal c !

Before we can define a derivative or a definite integral, we need a precise way to describe what a function does as its input creeps arbitrarily close to some value — without necessarily ever reaching it. That idea is the limit.

Definition: Limit

$\lim_{x \rightarrow a} f(x) = L$ means that as x gets arbitrarily close to a (but $x \neq a$), $f(x)$ gets arbitrarily close to L .



4.3.1 The Easy Case: Direct Substitution

Most limits are straightforward — just plug in the number:

Limit	Substitution	Value
$\lim_{x \rightarrow 3} (2x + 1)$	$2(3) + 1$	7
$\lim_{x \rightarrow 4} \sqrt{x}$	$\sqrt{4}$	2
$\lim_{x \rightarrow 1} (x^2 - 3x + 5)$	$1 - 3 + 5$	3

This works whenever f is **continuous** at the target point: $\lim_{x \rightarrow a} f(x) = f(a)$ (formalised in §4.4). **Try plugging in first.** If you get a number, you're done; if you get $\frac{0}{0}$, factor and cancel — see below.

4.3.2 The 0/0 Indeterminate Form

Consider $\frac{x^2 - 4}{x - 2}$ at $x = 2$: direct substitution gives $\frac{2^2 - 4}{2 - 2} = \frac{0}{0}$, which is undefined as arithmetic and an **indeterminate form** in a limit. It does not determine the limit on its own. But watch what happens as x *approaches* 2:

x	$\frac{x^2 - 4}{x - 2}$	x	$\frac{x^2 - 4}{x - 2}$
2.1	4.1	1.9	3.9
2.01	4.01	1.99	3.99
2.001	4.001	1.999	3.999

From both sides, the values approach **4** — that is the limit, even though the function itself is undefined at $x = 2$.

Strategy: factor and cancel.

$$\lim_{x \rightarrow 2} \frac{x^2 - 4}{x - 2} = \lim_{x \rightarrow 2} \frac{(x + 2)(x - 2)}{x - 2} = \lim_{x \rightarrow 2} (x + 2) = 4.$$

Example: Factoring to resolve 0/0

$$\begin{aligned} \text{(a)} \quad \lim_{x \rightarrow 3} \frac{x^2 - 9}{x - 3} &= \lim_{x \rightarrow 3} \frac{(x+3)(x-3)}{x-3} = \lim_{x \rightarrow 3} (x+3) = 6. \\ \text{(b)} \quad \lim_{x \rightarrow -1} \frac{x^2 + 3x + 2}{x + 1} &= \lim_{x \rightarrow -1} \frac{(x+1)(x+2)}{x+1} = \lim_{x \rightarrow -1} (x+2) = 1. \end{aligned}$$

When a cubic numerator doesn't factor by inspection, **polynomial long division** finds the hidden factor — and this arises naturally when computing instantaneous rates from first principles.

Example: Long division in a rate calculation

A bacterial count follows $N(t) = t^3 - t$. The instantaneous rate of change at $t = 2$, from first principles, is

$$\lim_{t \rightarrow 2} \frac{N(t) - N(2)}{t - 2} = \lim_{t \rightarrow 2} \frac{(t^3 - t) - 6}{t - 2} = \lim_{t \rightarrow 2} \frac{t^3 - t - 6}{t - 2}.$$

Direct substitution gives $\frac{0}{0}$. Dividing $t^3 - t - 6$ by $(t - 2)$ gives $t^3 - t - 6 = (t - 2)(t^2 + 2t + 3)$, so

$$\lim_{t \rightarrow 2} (t^2 + 2t + 3) = 4 + 4 + 3 = 11 \text{ cells per minute.}$$

4.3.3 Limits at Infinity

This is not asking for a value at a very large number. It asks for the long-run behaviour: which terms still matter after the smaller terms have become negligible?

What happens as x grows without bound? **Strategy: divide every term by the highest power of x .**

$$\lim_{x \rightarrow \infty} \frac{2x^2 + 3x - 1}{4x^2 + 5} = \lim_{x \rightarrow \infty} \frac{2 + \frac{3}{x} - \frac{1}{x^2}}{4 + \frac{5}{x^2}} = \frac{2 + 0 - 0}{4 + 0} = \frac{1}{2},$$

using the key principle $\lim_{x \rightarrow \infty} \frac{1}{x^n} = 0$ for any $n > 0$.

Example: Limit at infinity

$$\lim_{x \rightarrow \infty} \frac{3x^3 + x - 1}{2x^3 + 4} = \lim_{x \rightarrow \infty} \frac{3 + \frac{1}{x^2} - \frac{1}{x^3}}{2 + \frac{4}{x^3}} = \frac{3}{2}.$$

4.3.4 One-Sided Limits

Sometimes the left and right approaches disagree. For $f(x) = |x|/x$:

$$\lim_{x \rightarrow 0^+} \frac{|x|}{x} = \lim_{x \rightarrow 0^+} \frac{x}{x} = 1, \quad \lim_{x \rightarrow 0^-} \frac{|x|}{x} = \lim_{x \rightarrow 0^-} \frac{-x}{x} = -1.$$

Since the one-sided limits disagree, $\lim_{x \rightarrow 0} |x|/x$ **does not exist**. The two-sided limit exists only when both one-sided limits agree.

4.3.5 Limits in Nature

The 0/0 pattern is not a mathematical curiosity — it shows up whenever a scientific quantity is itself defined as a limit.

Enzyme Kinetics

The rate of an enzyme-catalysed reaction follows the Michaelis–Menten equation $v = \frac{V_{\max}[S]}{K_m + [S]}$. The **catalytic efficiency** is the initial slope of this saturation curve, $\lim_{[S] \rightarrow 0} v/[S] = V_{\max}/K_m$ — direct substitution gives $0/0$, but the limit resolves cleanly.

Per-Capita Population Growth

The growth rate per individual, over a shrinking interval,, $\frac{N(t+h) - N(t)}{h N(t)}$, has limit $\lim_{h \rightarrow 0} \frac{N(t+h) - N(t)}{h N(t)} = \frac{1}{N} \frac{dN}{dt} = r$ as $h \rightarrow 0$ — the intrinsic rate of increase r_m from Week 2, now defined precisely. This is the foundation of the logistic and Schaefer models.

Try it in Python

W4-CS01 — *Standard Imports for Week 4*: run this first.

W4-CS02 — *Visualizing Limits: Secant Lines Approaching Tangent*: watch slopes converge as $h \rightarrow 0$.

W4-CS03 — *Computing Limits Numerically*: tabulate $f(x)$ for $x \rightarrow a$ from both sides.

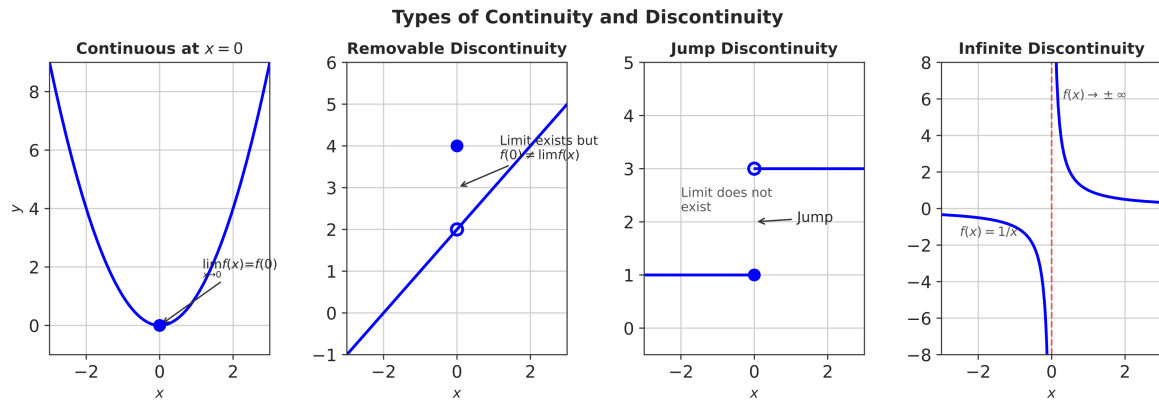
4.4 Continuity: When Functions Behave Well

A function can have a well-defined limit at a point and still not be continuous there — continuity is the stronger, more intuitive requirement that the function’s actual value at that point matches what the limit predicts.

Definition: Continuity

f is **continuous at** $x = a$ if all three hold: (1) $f(a)$ is defined; (2) $\lim_{x \rightarrow a} f(x)$ exists; (3) $\lim_{x \rightarrow a} f(x) = f(a)$. Intuitively: you can draw the graph through a without lifting your pen.

Type	Description	Example
Removable	limit exists, but $f(a)$ missing/different	$\frac{x^2 - 4}{x - 2}$ at $x = 2$
Jump	one-sided limits exist but differ	step functions
Infinite	function approaches $\pm\infty$	$\frac{1}{x}$ at $x = 0$



Many scientific quantities change smoothly, so continuity is often the default modelling assumption. The Schaefer model $G(S) = gS(1 - S/K)$ is continuous for $S \geq 0$: fish growth changes smoothly with stock level, with no sudden jumps. If a model predicts a jump where the real system is smooth, that's a signal the model needs revision.

Quantities That Are *Not* Continuous

Not everything changes smoothly — genuine discontinuities arise when a system switches state abruptly. An **action potential** jumps a neuron's membrane voltage from -70 to $+40$ mV almost instantaneously. At a **phase transition** (e.g. water boiling at 100°C), latent heat is absorbed with no temperature rise. **Fishing quotas** are often step functions: zero below a minimum stock, fixed above it. **Coral bleaching** can shift reef cover from over 50% to near zero as a catastrophic bifurcation rather than a gradual decline. Recognising which quantities are discontinuous helps you choose the right model and interpret its limits correctly.

Try it in Python

W4-CS10 — *Continuity Check with Piecewise Functions*: detect jumps and removable discontinuities.

4.5 The Derivative: From Secant to Tangent

Reading Derivative Notation

The derivative is written as $\frac{dy}{dx}$ or $f'(x)$. Read $\frac{dy}{dx}$ out loud as: “*The derivative of y with respect to x.*”

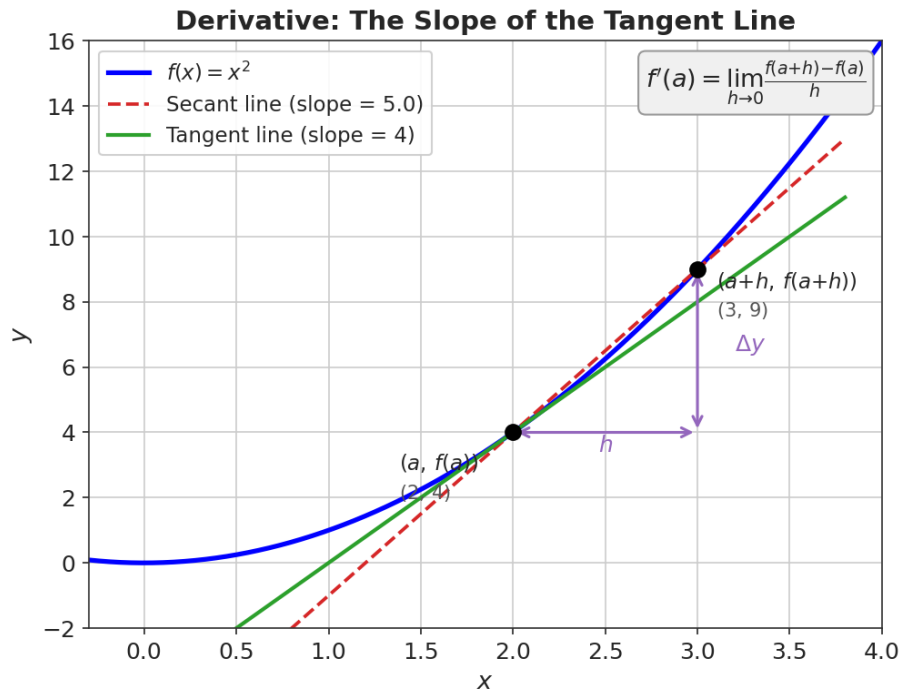
Crucial conceptual note: $\frac{dy}{dx}$ is **not** a fraction of two separate numbers being divided! It is a single, unified mathematical operator symbol representing the instantaneous rate of change (the slope of the tangent line).

The **average rate of change** of f between x and $x + h$ is the slope of the **secant line** through both points, $\frac{f(x+h) - f(x)}{h}$. As $h \rightarrow 0$, the secant rotates into the **tangent line**, and its slope becomes the **derivative**.

Definition: Derivative from first principles

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

Notation: $f'(x)$, $\frac{df}{dx}$, $\frac{dy}{dx}$, and y' are used interchangeably.

**Example: Derivative of x^2 from first principles**

$$\begin{aligned} f'(x) &= \lim_{h \rightarrow 0} \frac{(x+h)^2 - x^2}{h} = \lim_{h \rightarrow 0} \frac{x^2 + 2xh + h^2 - x^2}{h} = \lim_{h \rightarrow 0} \frac{h(2x + h)}{h} \\ &= \lim_{h \rightarrow 0} (2x + h) = 2x. \end{aligned}$$

So $\frac{d}{dx}[x^2] = 2x$: at any point x , the instantaneous rate of change is $2x$. At $x = 3$, the tangent's slope is $2(3) = 6$.

Try it in Python

W4-CS04 — *Numerical Differentiation with Cubic Splines*: a practical way to estimate derivatives from data.

W4-CS12 — *Finite Difference Methods Comparison*: forward, backward, and central differences side by side.

4.6 Basic Differentiation Rules

Computing derivatives from first principles every time is slow. In practice, we use rules.

The Power Rule

$$\frac{d}{dx}[x^n] = nx^{n-1} \quad \text{for any real } n.$$

Function	Derivative	Note
x^3	$3x^2$	power comes down, exponent drops by 1
x^5	$5x^4$	
$x^{1/2} = \sqrt{x}$	$\frac{1}{2}x^{-1/2} = \frac{1}{2\sqrt{x}}$	works for fractional powers
$x^{-1} = 1/x$	$-x^{-2} = -1/x^2$	works for negative powers

Example: Power rule

(a) $f(x) = x^7 \Rightarrow f'(x) = 7x^6$. (b) $g(x) = 1/x^3 = x^{-3} \Rightarrow g'(x) = -3x^{-4} = -3/x^4$.

Constant rule: $\frac{d}{dx}[c] = 0$ — constants don't change, so their rate of change is zero.

Constant multiple rule: $\frac{d}{dx}[cf(x)] = c \cdot f'(x)$ — constant multipliers pull straight out.

Example: Constant multiple

$$f(x) = 5x^4 \Rightarrow f'(x) = 5 \cdot 4x^3 = 20x^3.$$

Sum/difference rule: $\frac{d}{dx}[f(x) \pm g(x)] = f'(x) \pm g'(x)$ — differentiate term by term.

Example: Sum rule

(a) $y = 3x^4 - 2x^3 + 5x - 7 \Rightarrow y' = 12x^3 - 6x^2 + 5$.

(b) $f(x) = 4x^3 - \frac{3}{x} + 2\sqrt{x} - 7$. Rewrite as $4x^3 - 3x^{-1} + 2x^{1/2} - 7$, so

$$f'(x) = 12x^2 - 3(-1)x^{-2} + 2 \cdot \frac{1}{2}x^{-1/2} - 0 = 12x^2 + \frac{3}{x^2} + \frac{1}{\sqrt{x}}.$$

Try it in Python

W4-CS08 — *Power Rule Verification*: numerically check $\frac{d}{dx}x^n = nx^{n-1}$.

W4-CS09 — *Derivative of Exponential Functions*: verify $\frac{d}{dx}e^x = e^x$ and $\frac{d}{dx}b^x = b^x \ln b$ — a preview of Week 5.

4.7 Tangent Lines

At the point $(a, f(a))$, the tangent line has slope $m = f'(a)$ and equation

$$y - f(a) = f'(a)(x - a).$$

Example: Equation of a tangent line

Find the tangent to $y = x^2 - 3x + 1$ at $x = 2$.

(1) Point: $y(2) = 4 - 6 + 1 = -1$, so $(2, -1)$. (2) Derivative: $y' = 2x - 3$. (3) Slope at $x = 2$: $y'(2) = 1$. (4) Tangent: $y - (-1) = 1(x - 2)$, so $y = x - 3$.

Try it in Python

W4-CS05 — *Tangent Line Visualization*: plot $f(x)$ with its tangent at a chosen point.

W4-CS11 — *Tangent Line for the Lab Exercise*: $y = 3x^2 - 4x + 5$, used directly in this week's lab.

4.8 Application: Instantaneous Rates in Science

Example: Bacterial growth rate

For $N(t) = 1000e^{0.0347t}$, this week's rule set can't yet differentiate e^{kt} directly (that's Week 5), but the instantaneous rate at $t = 60$ can be estimated by shrinking the interval:

Interval	Average rate (cells/min)
[60, 120]	936.1
[60, 70]	332.4
[60, 61]	278.6
[60, 60.1]	278.9
[60, 60.01]	278.6

The rate approaches about **279 cells/minute**, so $N'(60) \approx 279$.

Example: Radioactive decay rate

Iodine-131 decays as $A(t) = A_0e^{-0.0866t}$ (half-life ≈ 8 days). $A'(t)$ is the rate of decay — negative, since the amount is decreasing.

Example: Schaefer model: locating MSY with calculus

Recall the Schaefer growth model from Week 3, $G(S) = gS(1 - S/K) = gS - \frac{g}{K}S^2$ — a polynomial in S , so it differentiates directly:

$$G'(S) = g - \frac{2g}{K}S.$$

Setting $G'(S^*) = 0$: $g - \frac{2g}{K}S^* = 0 \Rightarrow S^* = \frac{K}{2}$. This confirms the Week 3 graphical result — MSY occurs at half the carrying capacity — now derived directly with calculus, instead of the vertex-formula shortcut.

Try it in Python

W4-CS06 — *Bacterial Growth Rate Approximation*: estimate dN/dt from population data.

W4-CS07 — *Schaefer Model Derivative Verification*: differentiate the fishery model

used in lecture.

4.9 When Derivatives Don't Exist

A function fails to be differentiable at a point where:

1. **it isn't continuous** there (e.g. a jump discontinuity);
2. **there's a corner or cusp** (e.g. $f(x) = |x|$ at $x = 0$: slope -1 from the left, $+1$ from the right — no single tangent line exists);
3. **there's a vertical tangent** (e.g. $f(x) = x^{1/3}$ at $x = 0$).

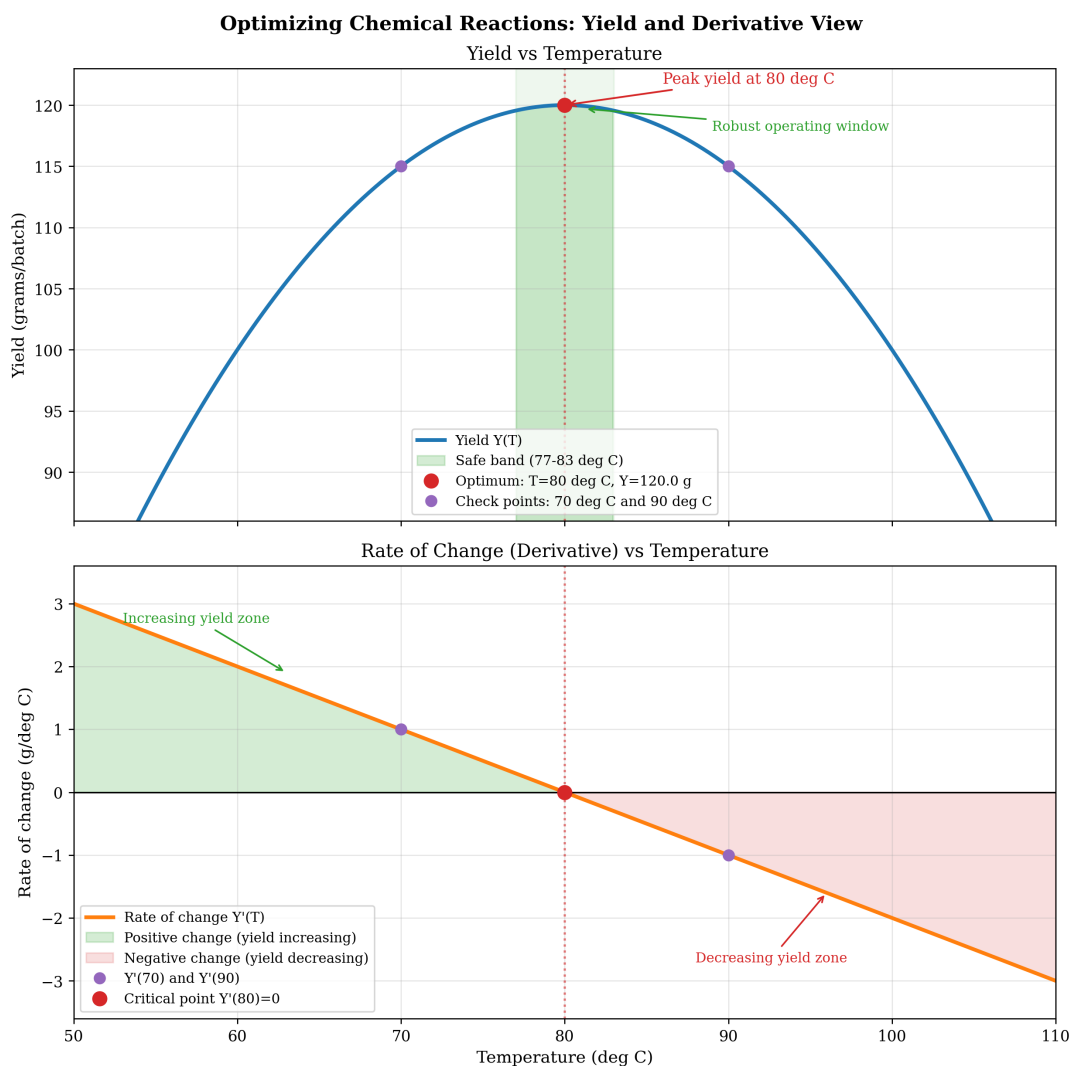
Key Relationship

Differentiable $\Rightarrow$ continuous (a smooth curve cannot have a hole or jump). But continuous $\nRightarrow$ differentiable — continuity is necessary, not sufficient.

4.10 Python Lab: A Guided Walkthrough

The Week 4 lab (`Week04_Lab_STUDENT.ipynb`), “*Optimizing Chemical Reactions*,” asks you to help a pharmaceutical company maximise drug yield. Yield depends on reaction temperature as

$$Y(T) = -0.05T^2 + 8T - 200 \quad (\text{g/batch, } 50 \leq T \leq 110^\circ\text{C}).$$



4.10.1 Part A — Derivatives as Rates of Change

The derivative $Y'(T) = dY/dT$ tells us how yield changes per degree of temperature — the **marginal yield**. Applying the power rule to $Y(T) = -0.05T^2 + 8T - 200$:

$$Y'(T) = -0.1T + 8.$$

```
def Y(T):
    'Drug yield (grams/batch).'
```

$$\text{return } -0.05 * T^2 + 8 * T - 200$$

```
def dY(T):
    'Marginal yield (grams per batch per degC).'
```

$$\text{return } -0.1 * T + 8$$

Activity: A.1 — Interpret the marginal yield

1. At $T = 70^\circ\text{C}$, $Y'(70) = ?$ g/ $^\circ\text{C}$. What does a positive value mean here?
2. At $T = 90^\circ\text{C}$, $Y'(90) = ?$ g/ $^\circ\text{C}$. Should the company increase or decrease temperature to gain yield?
3. What does it mean when $Y'(T) = 0$?

```
# 1. -----
# 2. -----
# 3. -----
```

4.10.2 Part B — Finding the Optimal Temperature

A **critical point** occurs where $Y'(T) = 0$ — a candidate for a maximum or minimum. Solving $-0.1T + 8 = 0$ gives $T_{\text{opt}} = 80^\circ\text{C}$. The **second derivative** $Y''(T)$ confirms which: $Y'' < 0$ (concave down) means a maximum; $Y'' > 0$ (concave up) means a minimum. Here $Y''(T) = -0.1$ is a negative constant everywhere, so the reaction is always concave down and $T_{\text{opt}} = 80^\circ\text{C}$ is confirmed as a maximum.

```
def d2Y(T):
    'Second derivative (curvature).'
```

$$\text{return } -0.1 \quad \# \text{ constant, negative everywhere } \rightarrow \text{ always concave down}$$

```
T_opt = 8 / 0.1      # solve -0.1T + 8 = 0  -> T = 80
Y_opt = Y(T_opt)
```

Activity: B — Test the optimum

Predict what happens to yield if the setpoint moves from 80°C to 90°C , then use the sign of $Y'(90)$ to check your prediction. Explain why the marked point is a maximum *within the safe operating range* — not merely the highest point drawn on the graph.

```
# Prediction: -----
# Y'(90) check: -----
# Why a maximum within [50, 110]: -----
```

4.10.3 Part C — Practical Constraints

Real reactors hold temperature within $\pm 3^\circ\text{C}$ of the setpoint, so at $T_{\text{opt}} = 80^\circ\text{C}$ the vessel actually runs between 77°C and 83°C . A rival manufacturer runs an older, unoptimised process at a fixed 70°C . Comparing $Y(70)$ against Y_{opt} , together with an energy cost of $\$0.50$ per degree above 50°C and a sale price of $\$5.00$ per gram, turns the calculus result into a profit figure.

```
energy_cost_per_deg = 0.50      # $/degC above 50degC per batch
price_per_gram      = 5.00      # $/gram of drug

for T_s in [70, T_opt]:
    energy = (T_s - 50) * energy_cost_per_deg
    revenue = Y(T_s) * price_per_gram
    profit = revenue - energy
```

Activity: C — Cost-benefit recommendation

1. How much additional profit per batch does the optimal temperature generate versus 70°C?
2. If the reactor drifts to 77°C instead of 80°C, how much yield is lost?
3. Write a 2-sentence recommendation to the production manager.

```
# 1. -----
# 2. -----
# Recommendation: -----
```

4.10.4 Part D — Symbolic Differentiation with SymPy

Throughout this lab you found $Y'(T)$ *by hand*, applying the power rule term by term. Python’s **SymPy** library performs the same algebra automatically — not just evaluating a formula at numbers (all **numpy** ever does), but manipulating the symbols themselves, exactly as you would on paper. This is **symbolic computation**, and it’s the tool you’ll reach for whenever you need to differentiate or integrate in Python for the rest of this unit.

```
import sympy as sp

T_sym = sp.symbols('T')
Y_expr = -0.05*T_sym**2 + 8*T_sym - 200

dY_expr = sp.diff(Y_expr, T_sym)           # first derivative,
symbolic
d2Y_expr = sp.diff(Y_expr, T_sym, 2)       # second derivative,
symbolic

critical_points = sp.solve(sp.Eq(dY_expr, 0), T_sym) # same as
Part B
```

`sp.diff()` performs symbolic differentiation using the same power rule, product rule, and chain rule you use on paper, rather than approximating from numbers the way **numpy** does. SymPy can also `solve()` equations exactly and simplify algebra — and, as you’ll see in Weeks 6 and 7, `integrate()` expressions symbolically too.

4.11 Why This Matters: From 1687 to Your Toolbox

17th-century problem	The math tool	Application in your field
Instantaneous speed of a planet	Derivative $f'(x)$	Growth rates, decay rates, MSY
Behaviour approaching a boundary	Limit $\lim_{x \rightarrow a}$	Catalytic efficiency, per-capita r
Where is a trajectory highest?	$f' = 0$ at a maximum	Schaefer model: $S^* = K/2$
Does a quantity change smoothly?	Continuity	Modelling biological thresholds

Calculus Today: Two Modern Examples

At Rothamsted — the world's longest-running agricultural experiment — wheat yield responds to nitrogen fertiliser with diminishing returns, $Y(N) = Y_{\max}(1 - e^{-kN})$. Setting dY/dN equal to the fertiliser-to-wheat price ratio finds the *economically* optimal dose: the point where one more kilogram of nitrogen costs more than the extra grain it buys. And every time a neural network learns from its mistakes — ChatGPT, AlphaFold, image recognition — it is using derivatives: **gradient descent** nudges each internal weight in the direction that reduces error fastest, and the **chain rule** (Week 5) is what makes it possible to compute that direction efficiently across millions of connected weights, a technique called backpropagation.

Newton and Leibniz built these tools to solve the most urgent scientific problems of their century. Three hundred and fifty years later, the same tools let you model ecosystems, optimise outcomes, and find thresholds in yours.

4.12 Week 4 Summary

Key Formulas

Limit	$\lim_{x \rightarrow a} f(x) = L$ means $f(x) \rightarrow L$ as $x \rightarrow a$
Continuity	f continuous at a iff $\lim_{x \rightarrow a} f(x) = f(a)$
Derivative (first principles)	$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}$
Power rule	$\frac{d}{dx}[x^n] = nx^{n-1}$
Constant rule	$\frac{d}{dx}[c] = 0$
Constant multiple	$\frac{d}{dx}[cf] = cf'$
Sum/difference	$\frac{d}{dx}[f \pm g] = f' \pm g'$
Tangent line	$y = f(a) + f'(a)(x - a)$

4.13 Before Week 5: Self-Check

Before moving on, make sure you can:

- evaluate a limit by direct substitution, and recognise when you instead need to factor and cancel a 0/0 form;
- evaluate a limit at infinity by dividing through by the highest power;
- state the three conditions for continuity at a point, and classify a discontinuity as removable, jump, or infinite;
- write down and interpret the limit definition of the derivative, and compute a simple derivative from first principles;
- apply the power, constant, constant-multiple, and sum/difference rules fluently;
- find the equation of a tangent line at a given point;
- explain why differentiability implies continuity but not the reverse.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §4.9 — before Week 5 begins, where the product, quotient, and chain rules complete your differentiation toolkit.

4.14 Exercises

These exercises mirror the topics and difficulty of the Week 4 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Limit by factoring. Evaluate $\lim_{x \rightarrow 5} \frac{x^2 - 25}{x - 5}$.

E2. Limit by factoring (harder). Evaluate $\lim_{x \rightarrow -2} \frac{x^2 - x - 6}{x + 2}$.

E3. Limit at infinity. Evaluate $\lim_{x \rightarrow \infty} \frac{5x^2 - 2}{3x^2 + x + 1}$.

E4. Continuity. State the three conditions required for f to be continuous at $x = a$.

E5. Power rule. Differentiate $f(x) = x^6$.

E6. Power rule with negative exponent. Differentiate $g(x) = \frac{1}{x^2}$.

E7. Sum rule. Differentiate $y = 2x^5 - 3x^2 + 7x - 4$.

E8. Derivative from first principles. Use the limit definition to find $f'(x)$ for $f(x) = x^2 + 3x$.

E9. Mixed derivative. Differentiate $f(x) = 6x^4 - \frac{2}{x} + 3\sqrt{x}$.

E10. Tangent line. Find the equation of the tangent to $y = x^2 + 2x - 1$ at $x = 1$.

E11. Average vs. instantaneous rate. A population follows $N(t) = 300 + 15t + t^2$ cells. Find (a) the average growth rate from $t = 1$ to $t = 4$, and (b) the instantaneous growth rate at $t = 1$.

E12. Schaefer derivative. For $G(S) = gS(1 - S/K)$ with $g = 0.2$ and $K = 6000$, find $G'(S)$ and the stock level S^* that maximises growth.

Answers

E1: $(x + 5)(x - 5)/(x - 5) \rightarrow x + 5$, so the limit is 10.

E2: $(x - 3)(x + 2)/(x + 2) \rightarrow x - 3$, so the limit is -5 .

E3: divide by x^2 : limit is $5/3$.

E4: $f(a)$ defined; $\lim_{x \rightarrow a} f(x)$ exists; $\lim_{x \rightarrow a} f(x) = f(a)$.

E5: $f'(x) = 6x^5$.

E6: $g(x) = x^{-2} \Rightarrow g'(x) = -2x^{-3} = -2/x^3$.

E7: $y' = 10x^4 - 6x + 7$.

E8: $f'(x) = \lim_{h \rightarrow 0} \frac{(x+h)^2 + 3(x+h) - x^2 - 3x}{h} = \lim_{h \rightarrow 0} (2x + h + 3) = 2x + 3$.

E9: rewrite $6x^4 - 2x^{-1} + 3x^{1/2}$; $f'(x) = 24x^3 + 2/x^2 + \frac{3}{2\sqrt{x}}$.

E10: $y(1) = 2$; $y' = 2x + 2$, $y'(1) = 4$; tangent: $y = 4x - 2$.

E11: (a) avg rate $= (376 - 316)/3 = 20$ cells/hr; (b) $N'(t) = 15 + 2t$, so $N'(1) = 17$ cells/hr.

E12: $G'(S) = 0.2 - \frac{0.4}{6000}S = 0.2 - \frac{S}{15000}$; $S^* = K/2 = 3000$.

Chapter 5

Differentiation Techniques and Optimisation

WEEK 5

Theme: “Finding the Best Outcome”

“The analytical scientist doesn’t just measure change — they find the best possible outcome. Optimisation is the capstone of calculus.”

5.1 Why Optimisation Matters

Week 4 gave you the derivative: the instantaneous rate of change $f'(x)$. This week asks a deeper question — **at what point does a system achieve its best possible value?** — and the answer often begins with a single observation about smooth curves: at a differentiable interior maximum or minimum, the curve momentarily levels off, so its tangent line is horizontal and its slope is zero, $f'(x) = 0$. Endpoints and points where a derivative is undefined must also be checked. It is the capstone of Act I: the last tool needed to *understand* a system before Act II turns to measuring what accumulates in it.

Domain	Question	Optimisation goal
Fisheries	What stock level maximises long-term profit?	Maximum Yield
Agriculture	How much fertiliser maximises net revenue?	Economic Profit maximisation
Medicine	What drug dosage maximises therapeutic effect?	Optimal dose
Engineering	What dimensions minimise material cost?	Cost minimisation
Livestock	When should animals be sold to maximise profit?	Optimal timing

These share a common structure: find the input x^* that maximises (or minimises) an objective function $f(x)$. The key tool is unchanged from Week 4: **a differentiable interior maximum or minimum has derivative zero**. What’s new this week is the toolkit needed to differentiate the more complex functions — products, quotients, compositions — that real objective functions are built from.

From MSY to MEY: The Economic Lens

In Week 3 you found the Maximum Sustainable Yield for the Schaefer model, $G(S) = gS(1 - S/K)$, at $S_{MSY} = K/2$. But MSY ignores *cost*: harvesting at low stock levels requires more effort, and thus more cost, than at high stock levels. **Maximum**

Economic Yield (MEY) accounts for this by maximising $\text{Profit}(S) = \text{Revenue}(S) - \text{Cost}(S)$ instead of growth alone — the full derivation is in §5.7.

5.2 Advanced Differentiation Rules

Before differentiating, identify the outermost operation: addition, multiplication, division, or one function inside another. That choice tells you which rule to start with.

Memory Anchors for Advanced Derivative Rules

- **Product Rule** $\frac{d}{dx}[u \cdot v] = u'v + uv'$: “Derivative of the first times the second, plus the first times the derivative of the second.”
- **Quotient Rule** $\frac{d}{dx}\left[\frac{u}{v}\right] = \frac{u'v - uv'}{v^2}$: “Low d-high minus high d-low, all over the square of what’s below.”
- **Chain Rule** $\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$: “Derivative of the outside function (leaving the inside untouched) times the derivative of the inside function.”

Forgetting the Chain Rule’s Inner Derivative

When differentiating composite functions like $y = (3x^2 + 1)^5$, students frequently write $5(3x^2 + 1)^4$ and stop. You **must** remember to multiply by the derivative of the inner function $(6x)$, yielding the complete derivative $5(3x^2 + 1)^4 \cdot (6x) = 30x(3x^2 + 1)^4$.

5.2.1 The Product Rule

Differentiating a sum is easy — just add the derivatives. A product isn’t so forgiving: the derivative of $f \cdot g$ is *not* $f' \cdot g'$, and the product rule tells you what it actually is.

Product Rule

$$\frac{d}{dx}[f(x) \cdot g(x)] = f'(x)g(x) + f(x)g'(x).$$

Memory aid: “first times derivative of second, plus second times derivative of first.”

Example: Product rule

(a) $y = x^2e^x$. With $f = x^2$, $g = e^x$:

$$y' = (2x)(e^x) + (x^2)(e^x) = e^x(2x + x^2) = xe^x(2 + x).$$

(b) $y = (3x + 1)(x^2 - 4)$:

$$y' = (3)(x^2 - 4) + (3x + 1)(2x) = 3x^2 - 12 + 6x^2 + 2x = 9x^2 + 2x - 12,$$

which matches differentiating the expanded form $3x^3 + x^2 - 12x - 4$ directly — a useful check.

5.2.2 The Quotient Rule

A quotient f/g could in principle be handled with the product and chain rules together, but the combination is used often enough to be worth its own compact, memorisable form.

Quotient Rule

$$\frac{d}{dx} \left[\frac{f(x)}{g(x)} \right] = \frac{f'(x)g(x) - f(x)g'(x)}{[g(x)]^2}, \quad g(x) \neq 0.$$

Memory aid: “low d-high minus high d-low, over low squared.”

Example: Quotient rule

(a) $y = \frac{x^2 + 1}{x - 3}$:

$$y' = \frac{(2x)(x - 3) - (x^2 + 1)(1)}{(x - 3)^2} = \frac{x^2 - 6x - 1}{(x - 3)^2}.$$

(b) $y = \frac{e^x}{x^2}$:

$$y' = \frac{(e^x)(x^2) - (e^x)(2x)}{x^4} = \frac{e^x(x - 2)}{x^3}.$$

5.2.3 The Chain Rule

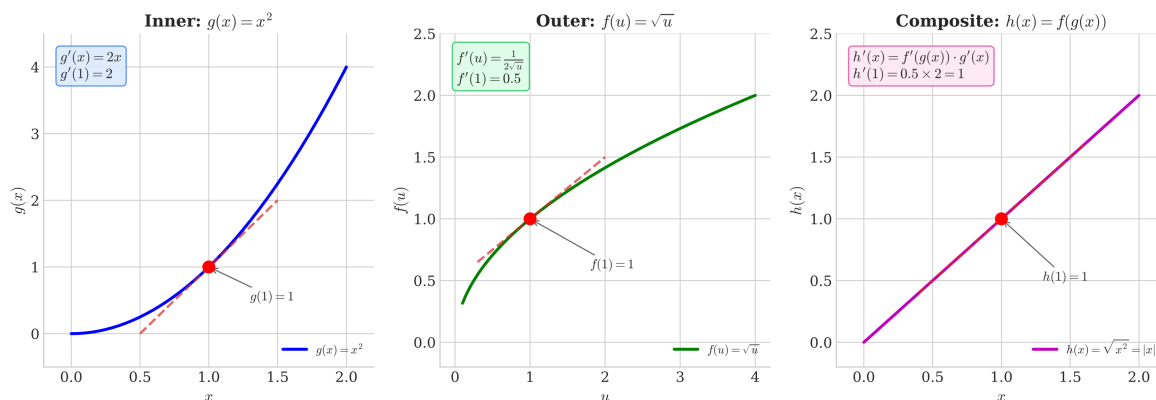
The chain rule handles **composite functions** — functions of functions.

Chain Rule

If $y = f(g(x))$, then $\frac{dy}{dx} = f'(g(x)) \cdot g'(x)$. In Leibniz notation, with $y = f(u)$ and $u = g(x)$:

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}.$$

Memory aid: “derivative of outer (at the inner value) times derivative of inner.”



Example: Chain rule

- (a) $y = (2x + 3)^5$. Let $u = 2x + 3$, so $y = u^5$: $\frac{dy}{du} = 5u^4$, $\frac{du}{dx} = 2$, so $\frac{dy}{dx} = 10(2x + 3)^4$.
- (b) $y = e^{3x^2 - x}$. Let $u = 3x^2 - x$, so $y = e^u$: $\frac{dy}{du} = e^u$, $\frac{du}{dx} = 6x - 1$, so $\frac{dy}{dx} = e^{3x^2 - x}(6x - 1)$.
- (c) $y = \ln(x^2 + 1)$. Let $u = x^2 + 1$, so $y = \ln u$: $\frac{dy}{du} = \frac{1}{u}$, $\frac{du}{dx} = 2x$, so $\frac{dy}{dx} = \frac{2x}{x^2 + 1}$.

Try it in Python

W5-CS01 — *Setting Up SymPy for Calculus*: run this first.

W5-CS02 — *Basic Differentiation*: SymPy `.diff()` for polynomials.

W5-CS03 — *Product Rule Verification*: confirm $(fg)' = f'g + fg'$ symbolically.

W5-CS04 — *Chain Rule with Composite Functions*: differentiate $f(g(x))$ step by step.

5.3 Derivatives of Exponential and Logarithmic Functions

5.3.1 Revisiting Euler's Number

In Week 2 you met e two ways: as the compound-interest limit $e = \lim_{n \rightarrow \infty} (1 + 1/n)^n$, and numerically, by observing that the slope of e^x always equalled its height. With derivatives now in hand, that second observation can be *proved*.

Derivative of e^x

$$\frac{d}{dx}[e^x] = e^x.$$

Example: Proof from first principles

Using the Week 4 limit definition:

$$\frac{d}{dx}[e^x] = \lim_{h \rightarrow 0} \frac{e^{x+h} - e^x}{h} = \lim_{h \rightarrow 0} \frac{e^x e^h - e^x}{h} = e^x \lim_{h \rightarrow 0} \frac{e^h - 1}{h},$$

factoring out e^x since it doesn't depend on h . Numerically, $\lim_{h \rightarrow 0} (e^h - 1)/h = 1$ exactly (this is precisely the Week 2 “slope equals height” observation), so

$$\frac{d}{dx}[e^x] = e^x \cdot 1 = e^x. \quad \blacksquare$$

For a general base a , the same argument gives $\frac{d}{dx}[a^x] = a^x \lim_{h \rightarrow 0} \frac{a^h - 1}{h}$, and it turns out $\lim_{h \rightarrow 0} (a^h - 1)/h = \ln a$:

Base a	$\lim_{h \rightarrow 0} (a^h - 1)/h$	Derivative of a^x
2	$0.693 = \ln 2$	$0.693 \cdot 2^x$
e	$1.000 = \ln e$	$1 \cdot e^x = e^x$
3	$1.099 = \ln 3$	$1.099 \cdot 3^x$

General Exponential Rule

$$\frac{d}{dx}[a^x] = a^x \ln a.$$

Since $\ln e = 1$, base e is the only base needing no extra constant factor — which is exactly why it's called the **natural** base.

5.3.2 The Natural Logarithm**Derivative of $\ln x$**

$$\frac{d}{dx}[\ln x] = \frac{1}{x}.$$

Proof sketch (implicit differentiation): if $y = \ln x$, then $e^y = x$. Differentiating both sides with respect to x : $e^y \cdot y' = 1$, so $y' = 1/e^y = 1/x$. Combined with the chain rule, $\frac{d}{dx}[\ln(g(x))] = \frac{g'(x)}{g(x)}$.

Function	Derivative
e^x	e^x
e^{kx}	ke^{kx}
$e^{g(x)}$	$g'(x)e^{g(x)}$
a^x	$a^x \ln a$
$\ln(g(x))$	$g'(x)/g(x)$

Example: Exponential and log derivatives

(a) $y = 5e^{-0.2x} \Rightarrow y' = 5(-0.2)e^{-0.2x} = -e^{-0.2x}.$

(b) $y = 3 \cdot 2^x \Rightarrow y' = 3 \cdot 2^x \ln 2 \approx 2.079 \cdot 2^x.$

(c) $y = \ln(3x^2 + 2) \Rightarrow y' = \frac{6x}{3x^2 + 2}.$

(d) $y = \frac{3}{4}x^4 + \frac{1}{4}x^2 - 3x + \ln x \Rightarrow y' = 3x^3 + \frac{1}{2}x - 3 + \frac{1}{x}.$

5.4 Optimisation: Finding Maxima and Minima

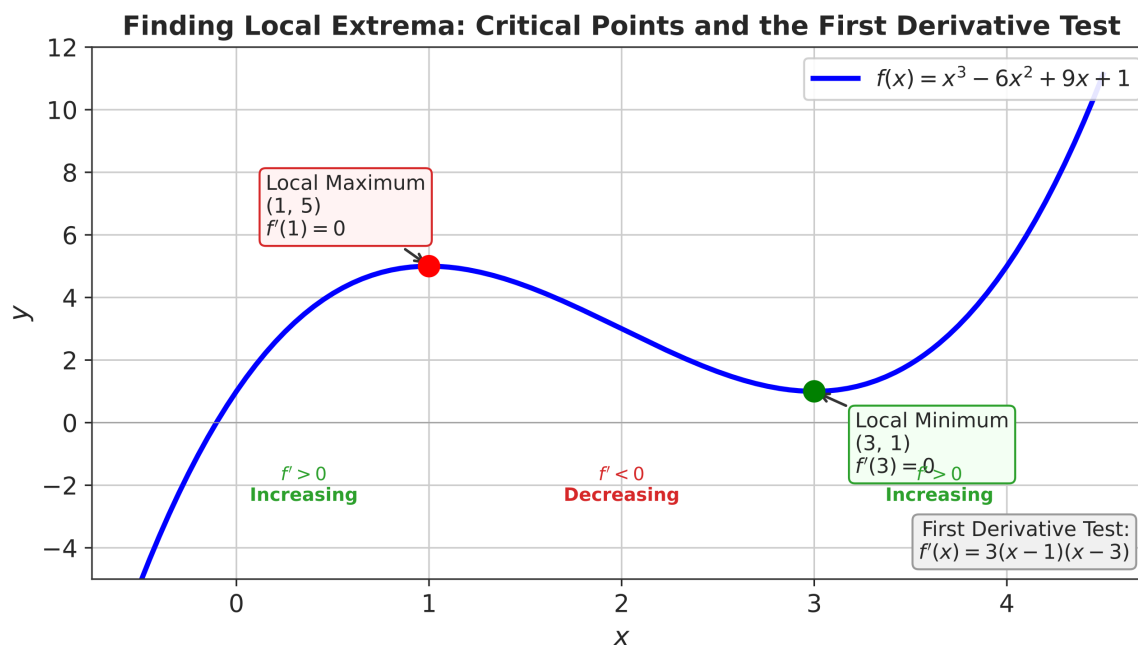
An optimisation answer is incomplete until you give the input, the resulting best value, and a sentence explaining both in the units of the scenario.

At a local maximum or minimum, the derivative equals zero. These are **critical points** (or stationary points).

Optimisation Algorithm

1. Find $f'(x)$.
2. Solve $f'(x) = 0$ for critical points x^* .
3. Classify each with the second derivative test.
4. Evaluate $f(x^*)$ to find the optimal value.

$f''(x_0)$	Conclusion	Graph shape
> 0	local minimum	concave up
< 0	local maximum	concave down
$= 0$	inconclusive — may be an inflection point	test further



Example: Local extrema of a cubic

Find all local extrema of $y = x^3 - 6x^2 + 9x + 1$.

(1) $y' = 3x^2 - 12x + 9 = 3(x-1)(x-3)$. (2) $y' = 0$ at $x = 1$ or $x = 3$. (3) $y'' = 6x - 12$: $y''(1) = -6 < 0$ (local max); $y''(3) = 6 > 0$ (local min). (4) $y(1) = 5$, $y(3) = 1$.

Who Discovered the Extremum Rule?

Fermat noticed in the 1640s — before calculus was formalised — that maxima and minima occur where a curve's “difference vanishes,” essentially $f'(x) = 0$. Newton and Leibniz built full calculus around this in the 1680s. Euler systematised the notation in the 1740s and extended optimisation to entire curves (the calculus of variations). Lagrange, in 1788, invented **Lagrange multipliers** for optimising under constraints — the general version of the substitution method in §5.6.

Try it in Python

W5-CS05 — *Solving $f'(x) = 0$ for Critical Points*: locate optima symbolically.

W5-CS06 — *Second Derivative Test*: classify each critical point as max, min, or inflection.

W5-CS11 — *Plotting Functions and Derivatives*: see f , f' , and f'' on one figure.

5.5 Local vs. Global Optima, and Inflection Points

Local maximum: $f(x^*) \geq f(x)$ for x near x^* . **Global maximum:** $f(x^*) \geq f(x)$ for all x in the domain. Every global optimum is local, but not every local optimum is global.

Finding a Global Optimum on $[a, b]$

Evaluate f at every critical point *and* at the endpoints a, b . The largest value is the global maximum; the smallest is the global minimum. A boundary point can beat every critical point.

An **inflection point** is where the curve switches from concave up to concave down (or vice versa): $f''(x) = 0$ *and* the sign of f'' changes there.

Warning

$f'(x) = 0$ and $f''(x) = 0$ together do *not* guarantee an extremum — it may be an inflection point. Example: $f(x) = x^3$ at $x = 0$ has $f'(0) = f''(0) = 0$, but f'' changes sign (negative to positive) there, so $x = 0$ is an inflection point, not a max or min.

Inflection points mark the steepest rate of change — important in growth models, where the curve switches from accelerating to decelerating (recall the logistic inflection point at $K/2$, Week 3).

5.6 Gradient Descent and Newton's Method

When $f'(x) = 0$ can't be solved algebraically, iterative numerical methods take over.

Gradient descent: imagine standing on a hilly landscape in fog — you can't see the bottom, but you can feel the slope underfoot. Take a small step downhill (opposite the gradient) and repeat:

$$x_{n+1} = x_n - \alpha f'(x_n),$$

where $\alpha > 0$ is the **step size** (learning rate). This is the same calculus used inside modern machine learning.

Newton's method uses the second derivative for faster, smarter steps:

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}.$$

Example: Newton's method on a rugged landscape

Molecular conformational energy is often multi-modal — a bond rotation sweeps through several stable and unstable configurations. Take $f(x) = x^4 - 6x^2 + x + 8$ (a reaction coordinate), with $f'(x) = 4x^3 - 12x + 1$ (no closed-form root) and $f''(x) = 12x^2 - 12$. Starting from $x_0 = 2.0$:

n	x_n	$f'(x_n)$	$f''(x_n)$	x_{n+1}
0	2.000	9.00	36.00	1.750
1	1.750	1.44	24.75	1.692
2	1.692	0.07	22.35	1.689

Three iterations converge to 3 d.p. But $x \approx 1.689$ is only a *local* minimum ($f \approx 0.71$); the true global minimum sits near $x \approx -1.78$ ($f \approx -2.75$). Starting from $x_0 = 2.0$,

Newton's method never finds it — it converges to whichever minimum is nearest the starting point.

	Gradient descent	Newton's method
Uses	f' only	f' and f''
Convergence	linear (many steps)	quadratic (few steps)
Cost per step	low	higher

Caution

Both methods converge to the *nearest* local optimum, not necessarily the global one. Starting-point choice matters whenever a function has multiple local optima.

5.7 Constrained Optimisation

Many real problems come with a **constraint**. The method: (1) write the objective function; (2) write the constraint equation; (3) use the constraint to eliminate one variable; (4) optimise the resulting single-variable function.

Example: Classic fencing problem

A farmer has 1000 m of fencing to enclose a rectangular paddock along a river (no fencing needed on the river side). What dimensions maximise the enclosed area?

Let x = width (perpendicular to river), y = length (parallel to river).

1. **Objective:** maximise $A = xy$.
2. **Constraint:** $2x + y = 1000$, so $y = 1000 - 2x$.
3. **Substitute:**

$$A(x) = x(1000 - 2x) = 1000x - 2x^2.$$

4. **Optimise:** $A'(x) = 1000 - 4x = 0 \Rightarrow x = 250$; $A''(x) = -4 < 0$ confirms a maximum.

So $x = 250$ m, $y = 500$ m, and maximum area $A = 250 \times 500 = 125,000$ m².

Try it in Python

W5-CS09 — *Constrained Optimization: Fencing Problem*: the classic max-area-with-fixed-perimeter setup.

W5-CS10 — *Profit Maximization: Livestock Problem*: full walkthrough (§5.9).

W5-CS12 — *Parametric Optimization*: watch the optimum shift as a parameter sweeps.

5.8 Economic Optimisation: The Bioeconomic Fishery Model

The bioeconomic model integrates biology, technology, and economics into a single profit function.

Component	Formula
Biology (Schaefer growth)	$G(S) = gS(1 - S/K)$
Technology (harvest)	$H = eES \Rightarrow E = H/(eS)$, where e is catchability, E is effort
Revenue	$TR = p \cdot H$
Cost	$TC = c \cdot E$
Profit	$\pi = TR - TC$

At steady state, harvest equals growth ($H = G(S)$), so effort is $E(S) = G(S)/(eS) = g(1 - S/K)/e$, and

$$\pi(S) = pG(S) - \frac{cg}{e} \left(1 - \frac{S}{K}\right) = g \left(1 - \frac{S}{K}\right) \left(pS - \frac{c}{e}\right)$$

after factoring out the common term.

Example: MEY calculation

For $K = 12,000$, $g = 0.10$, $e = 0.001$, $c = 4500$, $p = 3000$, expanding gives

$$\pi(S) = -0.025S^2 + 337.5S - 450,000.$$

Setting $\pi'(S) = -0.05S + 337.5 = 0$ gives $S^* = 6750$ tonnes, and $\pi''(S) = -0.05 < 0$ confirms a maximum.

Strategy	Stock level	Harvest	Profit
MSY	$K/2 = 6000$ t	300 t	\$675,000
MEY	6750 t	295.3 t	\$689,062

MEY maintains a *higher* stock than MSY — lower fishing cost and higher profit, despite a slightly smaller harvest.

From MEY to the Supply Curve

Setting $\pi'(S) = 0$ gives the optimal stock $S^*(p)$ as a function of *price*. Substituting back into $G(S)$ gives $H^*(p) = G(S^*(p))$ — the industry's **supply curve**: how much fish is optimally supplied at each market price. As p rises, S^* falls and H^* rises (up to a point) — the familiar upward-sloping supply curve, derived here from biology and calculus rather than assumed.

Activity: Student challenge — sonar shifts the supply curve

Modern vessels use sonar to locate fish schools, increasing the catchability coefficient e . Discuss: (1) if e increases, what happens to the cost of catching a given harvest? (2) Does the supply curve shift left or right? (3) What happens to the MEY stock level S^* ? (4) Is this good or bad for the fish population? (*Hint: a higher e means each boat catches more per trip, so cost per tonne falls.*)

- # 1. -----
- # 2. -----
- # 3. -----
- # 4. -----

Try it in Python

W5-CS07 — *Schaefer Model: Finding MSY*: Maximum Sustainable Yield via $dG/dS = 0$.

W5-CS08 — *Bioeconomic Model: Finding MEY*: Maximum Economic Yield with cost included.

5.9 Application: Profit Maximisation Word Problems

Many exam problems follow this pattern: (1) identify the variable and time frame; (2) build the objective function, typically profit = revenue – cost; (3) set up the component functions; (4) differentiate and solve $\pi' = 0$; (5) interpret the result in context.

Example: Livestock fattening

A farmer is fattening 100 deer, currently 50 kg each, gaining weight at $2 - 0.1t$ kg/day (t = days from now). Feed costs \$0.50/animal/day; market price is \$5.00/kg. When should the farmer sell?

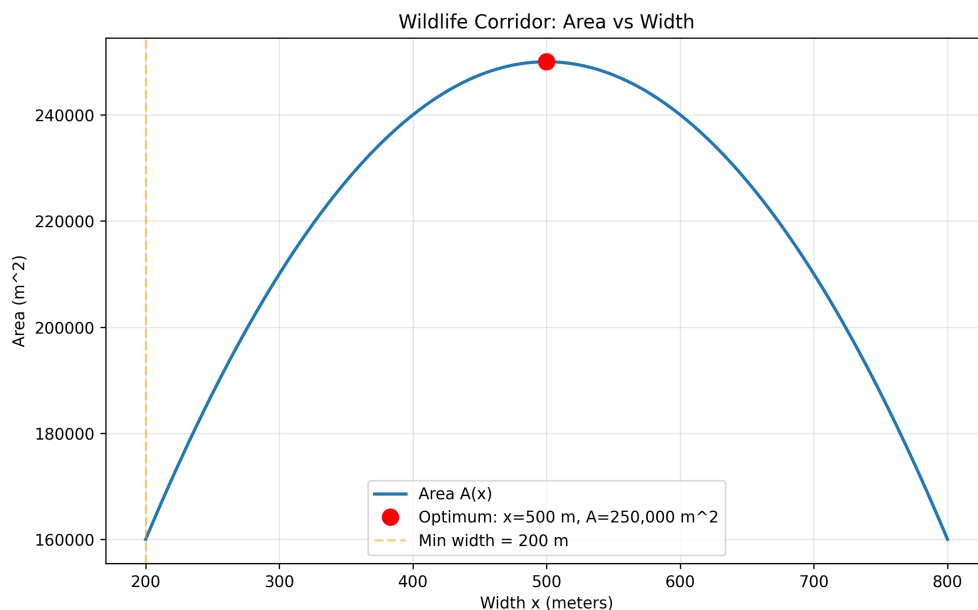
Weight per deer at time t is $W(t) = 50 + 2t - 0.05t^2$ (accumulating the daily gain rate — an integral, previewed here and formalised in Week 6). Then

$$TR(t) = 500 W(t) = 25,000 + 1000t - 25t^2, \quad TC(t) = 50t,$$

so $\pi(t) = 25,000 + 950t - 25t^2$. Setting $\pi'(t) = 950 - 50t = 0$ gives $t^* = 19$ days, and $\pi''(t) = -50 < 0$ confirms a maximum. At $t = 19$: $W(19) = 69.95$ kg, revenue \$34,975, cost \$950, **maximum profit \$34,025**.

5.10 Python Lab: A Guided Walkthrough

The Week 5 lab (`Week05_Lab.STUDENT.ipynb`), “*Designing an Optimal Wildlife Corridor*,” asks you to help the Department of Biodiversity design a rectangular corridor connecting two nature reserves using 2000 m of fencing, with a minimum width of 200 m to limit edge effects.



5.10.1 Part A — Problem Setup

Let x = width, y = length. **Constraint:** $2x + 2y = 2000 \Rightarrow y = 1000 - x$. **Objective:** maximise $A(x) = x(1000 - x)$. **Domain:** $x \in [200, 800]$ — the lower bound is the minimum-width rule, and the upper bound ensures $y \geq 200$ by the same rule applied to the other side.

```
def A(x):
    'Area of corridor (m^2) as a function of width x.'
    return x * (1000 - x)      # after substituting y = 1000 - x
```

Activity: A — Why this domain?

1. Why is the feasible domain $[200, 800]$ and not $[0, 1000]$?
2. What happens to the corridor if $x = 1000$? Is this physically meaningful?

```
# 1. -----
# 2. -----
```

5.10.2 Part B — Optimal Dimensions

$A(x) = 1000x - x^2 \Rightarrow A'(x) = 1000 - 2x$. Setting $A'(x) = 0$ gives $x = 500$ m, and $A''(x) = -2 < 0$ confirms a maximum — a 500×500 m square, with area $250,000 \text{ m}^2$, comfortably inside the feasible domain $[200, 800]$.

```
def dA(x):
    'Marginal area (m^2/m) -- rate of area gain per metre of added width.'
    return 1000 - 2 * x

x_crit = 1000 / 2
feasible = 200 <= x_crit <= 800
```

Activity: B — Interpret the optimum

1. The optimal corridor is a square ($x = y = 500$ m). Why does a square maximise area here?
2. $A'(300) = ?$ What does its sign tell you about area at $x = 300$?
3. $A'(700) = ?$ What does this tell you?

```
# 1. -----
# 2. -----
# 3. -----
```

5.10.3 Part C — Sensitivity Analysis

The optimum stays a square whenever fencing changes: for perimeter budget P , $x^* = P/4$ always. With $P = 1800$ m instead of 2000 m, $x^* = 450$ m and area drops accordingly — quantifying exactly how much a smaller budget costs the design.

```
def A_budget(x, perim):
```

```

    'Area given a perimeter budget.'
    y = perim / 2 - x
    return x * y

perim_new = 1800
x_opt_new = perim_new / 4    # optimum is still a square

```

Activity: C — Group recommendation

Write a 3–4 sentence recommendation to the Department, covering: optimal dimensions under both budgets; how much area is lost per 100 m of fencing removed; and whether a 250 m minimum-width requirement would change the design.

RECOMMENDATION:

5.10.4 Part D — Symbolic Differentiation with SymPy

In Part B you found $A'(x)$ by hand. SymPy performs the same algebra automatically, manipulating the expression symbolically rather than only evaluating it at numbers the way `numpy` does.

```

import sympy as sp

x_sym = sp.symbols('x')
A_expr = x_sym * (1000 - x_sym)
dA_expr = sp.diff(A_expr, x_sym)
critical_points = sp.solve(sp.Eq(dA_expr, 0), x_sym)    # same as
Part B

```

5.11 Week 5 Summary

Key Formulas

Product rule	$(fg)' = f'g + fg'$
Quotient rule	$(f/g)' = (f'g - fg')/g^2$
Chain rule	$\frac{d}{dx}[f(g(x))] = f'(g(x)) \cdot g'(x)$
Exponential	$\frac{d}{dx}[e^x] = e^x; \quad \frac{d}{dx}[e^{kx}] = ke^{kx}; \quad \frac{d}{dx}[a^x] = a^x \ln a$
Logarithm	$\frac{d}{dx}[\ln x] = 1/x; \quad \frac{d}{dx}[\ln(g(x))] = g'(x)/g(x)$
Critical points	solve $f'(x) = 0$
Second derivative test	$f'' > 0$ min; $f'' < 0$ max; $f'' = 0$ inconclusive
Global optimum on $[a, b]$	compare critical points <i>and</i> endpoints
Gradient descent	$x_{n+1} = x_n - \alpha f'(x_n)$
Newton's method	$x_{n+1} = x_n - f'(x_n)/f''(x_n)$
Bioeconomic profit	$\pi(S) = g(1 - S/K)(pS - c/e)$

5.12 Before Week 6: Self-Check

Before moving on, make sure you can:

- apply the product, quotient, and chain rules fluently, including in combination;
- differentiate exponential and logarithmic functions, with and without the chain rule;
- find and classify critical points using the second derivative test;
- distinguish a local optimum from a global one, and find a global optimum on a closed interval;
- recognise an inflection point and explain why $f' = f'' = 0$ doesn't guarantee an extremum;
- set up and solve a constrained optimisation problem by substitution;
- build a profit function from a word problem and optimise it.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §5.10 — before Week 6 begins, where the reverse process, integration, turns rates back into totals.

5.13 Exercises

These exercises mirror the topics and difficulty of the Week 5 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Product rule. Differentiate $y = x^3 e^x$.

E2. Quotient rule. Differentiate $y = \frac{x^2 - 1}{x + 2}$.

E3. Chain rule. Differentiate $y = (4x - 1)^3$.

E4. Chain rule with exponential. Differentiate $y = e^{5x^2+x}$.

E5. Chain rule with logarithm. Differentiate $y = \ln(5x^3 - 2)$.

E6. Mixed derivative. Differentiate $y = \frac{2}{5}x^5 - 4x + \ln(3x)$.

E7. Critical points and classification. Find and classify all critical points of $f(x) = x^3 - 3x^2 - 9x + 5$.

E8. Global optimum on an interval. Find the global maximum of $f(x) = x^3 - 3x$ on $[-2, 2]$.

E9. Inflection point. Show that $f(x) = x^3 - 6x^2$ has an inflection point, and find it.

E10. Constrained optimisation. A rectangular garden against a wall (no fencing needed on that side) uses 80 m of fencing for the other three sides. Find the dimensions that maximise area.

E11. Gradient descent. Starting at $x_0 = 6$ with step size $\alpha = 0.2$, find x_1 for minimising $f(x) = (x - 1)^2$ (note $f'(x) = 2(x - 1)$).

E12. Profit optimisation. A firm's profit is $\pi(t) = 20,000 + 800t - 20t^2$ (\$, t in days). Find the optimal t and the maximum profit.

Answers

E1: $y' = 3x^2e^x + x^3e^x = x^2e^x(3 + x)$.

E2: $y' = \frac{(2x)(x+2) - (x^2-1)(1)}{(x+2)^2} = \frac{x^2 + 4x + 1}{(x+2)^2}$.

E3: $y' = 12(4x-1)^2$.

E4: $y' = (10x+1)e^{5x^2+x}$.

E5: $y' = \frac{15x^2}{5x^3-2}$.

E6: $y' = 2x^4 - 4 + \frac{1}{x}$.

E7: $f' = 3x^2 - 6x - 9 = 3(x-3)(x+1)$; $f'' = 6x - 6$: $x = -1$ local max, $x = 3$ local min.

E8: $f' = 3x^2 - 3 = 0 \Rightarrow x = \pm 1$; $f(-2) = -2$, $f(-1) = 2$, $f(1) = -2$, $f(2) = 2$: global max is 2.

E9: $f'' = 6x - 12 = 0 \Rightarrow x = 2$; f'' changes sign there, so $(2, -16)$ is an inflection point.

E10: $2x + y = 80 \Rightarrow A = 80x - 2x^2$; $A' = 80 - 4x = 0 \Rightarrow x = 20$, $y = 40$; max area 800 m^2 .

E11: $f'(6) = 2(6-1) = 10$; $x_1 = 6 - 0.2 \times 10 = 4$.

E12: $\pi'(t) = 800 - 40t = 0 \Rightarrow t^* = 20$ days; $\pi(20) = 20,000 + 16,000 - 8000 = \$28,000$.

Part II

Measuring Accumulation

Chapter 6

Introduction to Integration

WEEK 6

Theme: “From Rates to Totals”

“Differentiation tells us how fast things change; integration tells us how much has accumulated. Together, they complete the calculus story.”

6.1 The Challenge: From Rates to Totals

Welcome to Act II (Measuring Accumulation)! In Act I (Weeks 1–5) we started with a total quantity and calculated its instantaneous rate of change (differentiation). This week we reverse the camera angle: given an observed rate of flow, rate of growth, or rate of degradation, how do we calculate the total cumulative amount over time? This inverse operation is **integration**, and it is what Act II is about.

Domain	Given rate	Need total
Carbon science	CO ₂ absorption rate (t/yr)	Total carbon sequestered
Medicine	White blood cell change rate (cells/hr)	Total lymphocyte count
Economics	Marginal cost (\$/unit)	Total cost
Hydrology	Flow rate (m ³ /s)	Total water volume
Agriculture	Yield response rate (t/ha per kg fertiliser)	Total crop yield

Integration is the tool for these questions. If differentiation answers “how fast?”, integration answers “how much?” — they are **inverse operations**:

Differentiation: $F(x) \xrightarrow{d/dx} f(x)$, Integration: $f(x) \xrightarrow{\int} F(x) + C$.

6.2 Antiderivatives: Reversing Differentiation

Reading Integral Notation Out Loud

The indefinite integral is written as $\int f(x) dx = F(x) + C$. Read it out loud as: “*The indefinite integral of $f(x)$ with respect to x equals $F(x)$ plus C .*”

The symbol $\int$ is an elongated letter ‘S’, standing for “Sum”. The dx tells you which variable is being integrated. The constant $+C$ is the **constant of integration**, accounting for any initial baseline quantity that was flattened to zero during differentiation.

The Forgotten $+C$

Because the derivative of any constant is zero ($\frac{d}{dx}[5] = 0$), infinitely many functions share the exact same derivative. When finding an indefinite integral, always include $+C$. Omitting $+C$ loses the initial condition of your scientific system!

Differentiation turns a total into a rate — position into velocity, cost into marginal cost. Antidifferentiation asks the reverse question: given a rate, can we recover the total it came from?

Definition: Antiderivative

$F(x)$ is an **antiderivative** of $f(x)$ if $F'(x) = f(x)$.

Example: Finding an antiderivative

What function, differentiated, gives $2x$? Since $\frac{d}{dx}[x^2] = 2x$, $F(x) = x^2$ works. But so does $x^2 + 5$, $x^2 - 17$, or $x^2 + \pi$ — *any* constant added to x^2 still differentiates to $2x$, since $\frac{d}{dx}[c] = 0$.

Indefinite Integral

If $F(x)$ is one antiderivative of $f(x)$, every antiderivative has the form $F(x) + C$ for some constant C . We write

$$\int f(x) dx = F(x) + C,$$

where C is the **constant of integration**. Any two antiderivatives of the same function differ only by a constant.

Two Notations — One War of Priority

The $\int f(x) dx$ notation you are using was introduced by **Gottfried Wilhelm Leibniz** in the 1670s, alongside his $\frac{dy}{dx}$ for derivatives — both chosen to make the mechanics of calculus almost self-evident on the page, which is why they are still the standard today. Working independently across the English Channel, **Isaac Newton** developed an equivalent theory a few years earlier using his own “fluxion” notation, but published it later. When both men’s supporters realised in the 1700s that the two theories were essentially the same, a bitter, decades-long priority dispute broke out over who truly deserved credit — a dispute so damaging that British mathematicians, loyal to Newton’s notation out of national pride, cut themselves off from Continental advances in calculus for the best part of a century. The episode is a cautionary tale: notation is not neutral. A well-designed symbol, like Leibniz’s $\int \cdots dx$, can make a hard idea easier to think with, decades or centuries after the argument over who invented it has been forgotten.

6.3 Basic Integration Rules

Integration reverses the power rule: add one to the exponent, then divide by the new exponent. As a check, differentiate your answer and recover the original integrand.

Power Rule for Integration

$$\int x^n dx = \frac{x^{n+1}}{n+1} + C, \quad n \neq -1.$$

Check by differentiating: $\frac{d}{dx} \left[\frac{x^{n+1}}{n+1} \right] = \frac{(n+1)x^n}{n+1} = x^n. \checkmark$

$f(x)$	$\int f(x) dx$	Check
x^2	$x^3/3 + C$	$\frac{d}{dx}[x^3/3] = x^2 \checkmark$
x^3	$x^4/4 + C$	$\frac{d}{dx}[x^4/4] = x^3 \checkmark$
x^{-2}	$-1/x + C$	$\frac{d}{dx}[-x^{-1}] = x^{-2} \checkmark$
$\sqrt{x} = x^{1/2}$	$\frac{2}{3}x^{3/2} + C$	$\frac{d}{dx}[\frac{2}{3}x^{3/2}] = x^{1/2} \checkmark$

Constant rule: $\int k dx = kx + C.$ **Sum rule:** $\int [f(x) + g(x)] dx = \int f dx + \int g dx.$

Constant multiple: $\int k f(x) dx = k \int f dx.$

Example: Combining the rules

$$\int (3x^2 - 7x + 4) dx = 3 \cdot \frac{x^3}{3} - 7 \cdot \frac{x^2}{2} + 4x + C = x^3 - \frac{7x^2}{2} + 4x + C.$$

Special Integrals

$$\int e^x dx = e^x + C, \quad \int e^{kx} dx = \frac{1}{k}e^{kx} + C, \quad \int \frac{1}{x} dx = \ln|x| + C \quad (x \neq 0).$$

Example: Exponential and logarithmic integrals

$$\int \left(e^{2x} + \frac{3}{x} \right) dx = \frac{1}{2}e^{2x} + 3 \ln|x| + C.$$

Try it in Python

W6-CS01 — *Setup and Imports:* run this first.

W6-CS02 — *Basic Indefinite Integration:* SymPy `integrate()` for polynomials.

W6-CS03 — *Exponential and Logarithmic Integration:* $\int e^x dx$, $\int 1/x dx$.

6.4 Rules That Do *Not* Hold for Integration**Warning**

Unlike differentiation, integration does not distribute over products, quotients, or powers:

$$\int fg dx \neq \left(\int f dx \right) \left(\int g dx \right), \quad \int \frac{f}{g} dx \neq \frac{\int f dx}{\int g dx}, \quad \int f^n dx \neq \left(\int f dx \right)^n.$$

Example: Why these fail

$\int x \cdot x \, dx = \int x^2 \, dx = \frac{x^3}{3} + C$, but $(\int x \, dx)(\int x \, dx) = \frac{x^2}{2} \cdot \frac{x^2}{2} = \frac{x^4}{4}$ — completely different. Each factor in a product must be integrated together, not separately (the reverse of this problem, the *product rule*, is why: Week 5).

6.5 Finding Specific Antiderivatives: Initial Conditions

The constant C is pinned down once we know the function's value at one point — an **initial condition**.

Solving an Initial Value Problem

1. Find the indefinite integral $F(x) + C$.
2. Use the initial condition $F(x_0) = y_0$ to solve for C .
3. Write the specific antiderivative.

Example: Soap bubble radius

A soap bubble's radius changes at $r'(t) = -3t^2 + 6t$ (cm/s), with $r(0) = 2$. Find $r(t)$.
(1) $r(t) = \int (-3t^2 + 6t) \, dt = -t^3 + 3t^2 + C$. **(2)** $r(0) = 2 \Rightarrow C = 2$. **(3)** $r(t) = -t^3 + 3t^2 + 2$.

Follow-up: maximum radius occurs where $r'(t) = 0$: $-3t^2 + 6t = 0 \Rightarrow t = 0$ or $t = 2$.
 At $t = 2$: $r(2) = -8 + 12 + 2 = 6$ cm (maximum).

6.6 Limiting Values: Where Systems End Up

A powerful feature of exponential-decay initial value problems: the solution always approaches a **finite limiting value** as $t \rightarrow \infty$.

The General Pattern

If $F'(t) = ae^{-kt}$ with $k > 0$ and $F(0) = F_0$, then

$$F(t) = \underbrace{\left(F_0 + \frac{a}{k}\right)}_L - \frac{a}{k} e^{-kt}.$$

As $t \rightarrow \infty$, $e^{-kt} \rightarrow 0$, so $\lim_{t \rightarrow \infty} F(t) = L = F_0 + a/k$. The system starts at F_0 , changes rapidly at first, then settles toward the limiting value L — readable directly from the formula as the constant term once the exponential has vanished.

Example: Carbon sequestration

A reforestation project absorbs CO_2 at a declining rate, $C'(t) = 50e^{-0.02t}$ (t/yr), $C(0) = 0$.

(1) $C(t) = \frac{50}{-0.02} e^{-0.02t} + K = -2500e^{-0.02t} + K$. **(2)** $C(0) = 0 \Rightarrow K = 2500$, so $C(t) = 2500(1 - e^{-0.02t})$. **(3)** Limiting value: $\lim_{t \rightarrow \infty} C(t) = 2500$ tonnes.

The forest absorbs CO₂ quickly at first, then more slowly, with total sequestration capped at 2500 tonnes.

Example: Lymphocyte recovery

After chemotherapy, white blood cell count changes at $L'(t) = 200e^{-0.1t}$ (cells/day), $L(0) = 5000$. Following the same three steps: $L(t) = -2000e^{-0.1t} + C$; $L(0) = 5000 \Rightarrow C = 7000$; so $L(t) = 7000 - 2000e^{-0.1t}$, with limiting count $\lim_{t \rightarrow \infty} L(t) = 7000$ cells. A clinician can use this limiting value to anticipate when recovery is substantially complete.

	Carbon (sequestration)	Lymphocyte (recovery)
Rate	$50e^{-0.02t}$	$200e^{-0.1t}$
Initial value	$C(0) = 0$	$L(0) = 5000$
Solution	$2500(1 - e^{-0.02t})$	$7000 - 2000e^{-0.1t}$
Limiting value	2500 tonnes	7000 cells

Every exponential-decay IVP has this shape: the limiting value L is the constant term; the exponential term Be^{-kt} is “how far the system still has to go.”

Try it in Python

W6-CS05 — *Initial Value Problems*: solve for the constant of integration from a known point.

W6-CS06 — *Finding Maximum After Integration*: combine integration with optimisation.

W6-CS11 — *Exponential Decay Integration*: $\int e^{-kx} dx$ in environmental contexts.

6.7 Science Context: Land Degradation and Rehabilitation

According to the IPBES 2018 report, over **75% of Earth’s land area** is substantially degraded, with a possible rise to 95% by 2050, directly affecting 1.5 billion people and costing roughly 10% of global GDP annually. Major drivers include urbanisation, contamination, agriculture (salinisation, nutrient depletion, erosion), and mining.

Example: Exponential decay in element leaching

Heavy-metal concentration in rehabilitated soil decays according to $C'(t) = -0.15C(t)$, with initial concentration 50 mg/kg. The solution to this differential equation is $C(t) = C_0e^{-0.15t} = 50e^{-0.15t}$ — verify by differentiating: $C'(t) = 50(-0.15)e^{-0.15t} = -0.15C(t)$. ✓

Example: Agricultural nitrogen response

Crop yield changes with nitrogen fertiliser at $y'(x) = 0.015 - 0.0001x$ (t/ha per kg/ha), with $y(0) = 1.2$ t/ha.

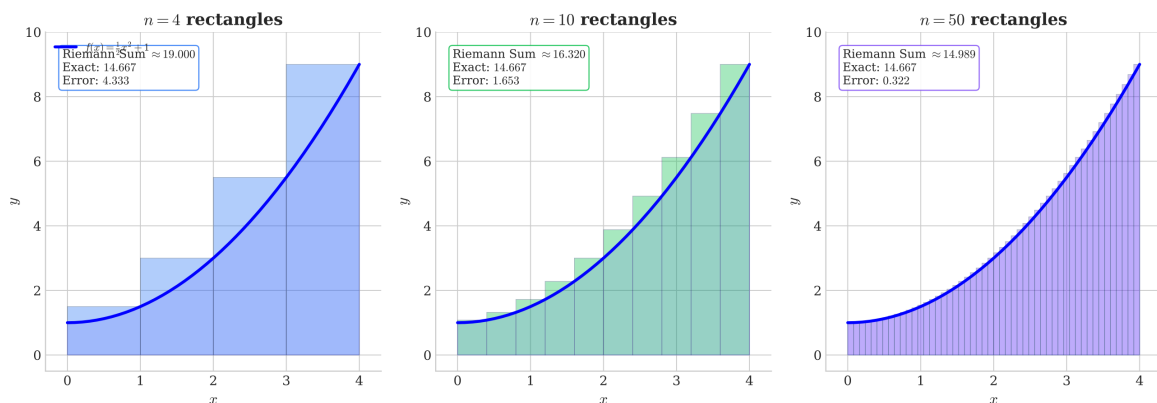
(1) $y(x) = \int (0.015 - 0.0001x) dx = 0.015x - 0.00005x^2 + C$. (2) $y(0) = 1.2 \Rightarrow C = 1.2$, so $y(x) = 1.2 + 0.015x - 0.00005x^2$. (3) Maximum yield where $y'(x) = 0$: $x = 150$ kg/ha, giving $y(150) = 1.2 + 2.25 - 1.125 = 2.325$ t/ha — the same optimisation pattern from Week 5, now built from an integrated rate.

Try it in Python

- W6-CS09** — *Area Between Two Curves*: compute $\int (f - g) dx$ over an interval.
W6-CS10 — *Nitrogen Fertiliser Response*: an applied case used in lecture.
W6-CS12 — *SymPy Plotting of Trajectories*: visualise solutions to integration problems.

6.8 Preview: The Definite Integral and Area

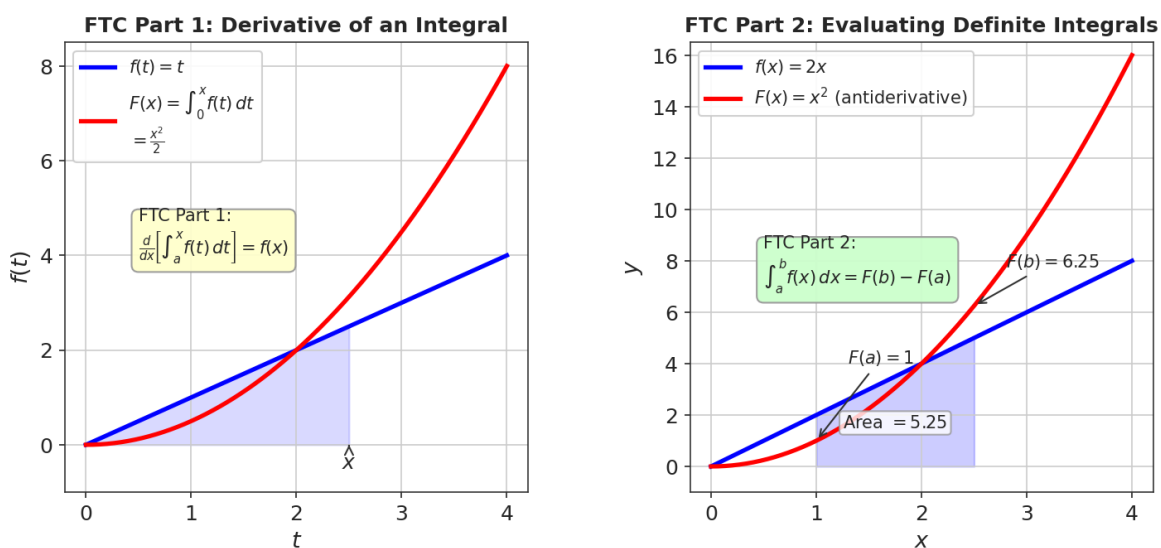
If an object travels at constant velocity v without changing direction, distance is $s = vt$ — the area of a rectangle under the velocity–time graph. If velocity varies, integrating velocity gives displacement; distance is the area under the speed curve, $\int |v(t)| dt$.



Approximating that area with n rectangles gives a **Riemann sum**; as $n \rightarrow \infty$, the approximation converges to the exact area, the **definite integral**:

$$\int_a^b f(x) dx = \lim_{n \rightarrow \infty} \sum_{i=1}^n f(x_i^*) \Delta x,$$

representing the signed area between $f(x)$ and the x -axis from $x = a$ to $x = b$.



The **Fundamental Theorem of Calculus** connects this back to antiderivatives:

$$\int_a^b f(x) dx = F(b) - F(a), \quad \text{where } F' = f$$

— the entire machinery of this chapter, evaluated at two points instead of left as $+C$. We explore this fully in Week 7.

Try it in Python

W6-CS04 — *Definite Integrals*: evaluate $\int_a^b f(x) dx$ exactly with SymPy.
W6-CS07 — *Plotting Areas Under Curves*: shade $\int_a^b f(x) dx$ on a graph.
W6-CS08 — *Plotting Curved Areas*: visualise non-rectangular regions.

6.9 Python: SymPy for Symbolic Integration

SymPy computes antiderivatives exactly — the same results you’d get by hand — and is useful for checking your work.

```
import sympy as sp
x = sp.Symbol('x')

print(sp.integrate(x**3, x))      # x**4/4
print(sp.integrate(sp.exp(2*x), x)) # exp(2*x)/2
print(sp.integrate(1/x, x))      # log(x)
```

Important

SymPy omits $+C$ — add the constant of integration yourself when writing the general antiderivative. SymPy also writes $\log(x)$ for $\ln|x|$; always translate back to maths notation in your solutions.

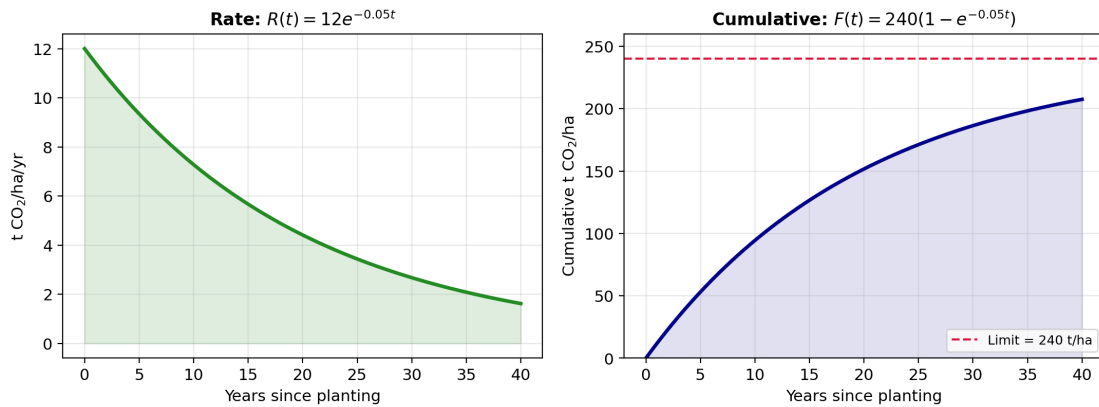
The golden rule of integration: **differentiate your answer and check you get the original function back**. This works by hand and in SymPy alike:

```
f = 3*x**2 - 7*x + 4
F = sp.integrate(f, x)
print(sp.simplify(sp.diff(F, x) - f) == 0) # True
```

6.10 Python Lab: A Guided Walkthrough

The Week 6 lab (`Week06_Lab_STUDENT.ipynb`), “*Carbon Sequestration and Integration*,” models a 500-ha reforestation project in the Wheatbelt, where the sequestration rate declines as trees mature:

$$R(t) = 12e^{-0.05t} \quad (\text{t CO}_2/\text{ha/yr, } t = 0 \text{ to } 40 \text{ years}).$$



6.10.1 Part A — Rates vs. Totals

$R(t)$ is the instantaneous rate; the total carbon captured from $t = 0$ to $t = T$ is the definite integral $F(T) = \int_0^T R(t) dt$ — but without an antiderivative yet, Part A approximates $F(5)$ numerically using the trapezoidal rule.

```
from numpy import trapezoid # or numpy.trapz on older NumPy
t_fine = np.linspace(0, 5, 1000)
total_5yr = trapezoid(R(t_fine), t_fine)
```

Activity: A — Interpret the rate curve

Describe the shape of $R(t)$ over 40 years: is the rate increasing or decreasing? Why does this make biological sense for a maturing forest?

6.10.2 Part B — Antiderivatives

Using $\int e^{kt} dt = \frac{1}{k}e^{kt} + C$: $F(t) = \int 12e^{-0.05t} dt = \frac{12}{-0.05}e^{-0.05t} + C = -240e^{-0.05t} + C$.
Applying $F(0) = 0$ (no carbon captured at $t = 0$): $C = 240$, so

$$F(t) = 240(1 - e^{-0.05t}).$$

```
def F(t):
    'Cumulative CO2 captured (t CO2/ha) from year 0 to year t.'
    return 240 * (1 - np.exp(-0.05 * t))
```

Activity: Rate-to-total checkpoint

Explain why $R(t)$ can decrease while $F(t)$ continues to increase. Use the shape of $F(t)$ to explain what its horizontal asymptote means for long-term carbon planning. Before calculating revenue, predict whether the first or last five years contribute more carbon credits, then justify your prediction.


```
# -----
```

6.10.3 Part C — Project Valuation

The project covers 500 ha; carbon credits sell at \$25/tonne CO₂.

```
area_ha = 500
price_per_tonne = 25
total_carbon = area_ha * F(40)
total_revenue = total_carbon * price_per_tonne
```

Activity: C.3 — When is half the 40-year total reached?

Solve $F(t) = \frac{1}{2}F(40)$ for t by rearranging $240(1 - e^{-0.05t}) = \text{half_target}$.

```
half_target = F(40) / 2
t_half = ----- # rearrange F(t) =
                half_target for t
print(f"Half the 40-year total is reached at year {t_half:.1f}
      ")
```

Because $F(t)$ is concave (rate $R(t)$ is always positive but decreasing), half the *carbon* is banked well *before* the halfway year, $t = 20$ — most of the sequestration happens early, while the trees are young and growing fastest.

Activity: C.4 — Annual revenue stream

Compute one year's worth of carbon at a time, $F(t) - F(t-1)$ for $t = 1, \dots, 40$, convert to revenue, and plot as a bar chart.

```
years = np.arange(1, 41)
annual_carbon_per_ha = np.array([F(t) - F(t-1) for t in years
                                ])
annual_revenue = annual_carbon_per_ha * area_ha *
                 price_per_tonne
# plt.bar(years, annual_revenue / 1e3) ...
```

How does year-1 revenue compare to year-40 revenue, and why — referring back to the shape of $R(t)$?

Activity: C — Group discussion

1. Why does the sequestration rate decrease over time? What biological process does this reflect?
2. The long-run limit of $F(t)$ is 240 t CO₂/ha. What does this mean for an infinite project?
3. Should landholders sell carbon credits annually or wait? What does the revenue profile suggest?

DISCUSSION:

6.10.4 Part D — Symbolic Integration with SymPy

In Part B you found $F(t)$ by hand, using $\int e^{kt} dt = \frac{1}{k}e^{kt} + C$. SymPy's `sp.integrate()` finds antiderivatives algebraically — the same symbolic-computation idea as `sp.diff()` from Weeks 4–5, running in reverse.

```
import sympy as sp

t_sym = sp.symbols('t')
R_expr = 12 * sp.exp(-0.05 * t_sym)

F_indef = sp.integrate(R_expr, t_sym)    # omits +C; matches Part B
with C=240
```

`sp.integrate()` finds the antiderivative algebraically, the reverse of `sp.diff()`, rather than approximating an area numerically the way `numpy`'s trapezoidal rule did in Part A. As always, SymPy omits the constant of integration — you still add C and solve for it using an initial condition yourself.

6.11 Week 6 Summary

Key Integration Rules

Constant	$\int k dx = kx + C$
Power rule	$\int x^n dx = \frac{x^{n+1}}{n+1} + C \quad (n \neq -1)$
Reciprocal	$\int \frac{1}{x} dx = \ln x + C$
Exponential	$\int e^x dx = e^x + C; \quad \int e^{kx} dx = \frac{1}{k}e^{kx} + C$
Sum/constant multiple	$\int [f + g] dx = \int f dx + \int g dx; \quad \int kf dx = k \int f dx$
IVP limiting value	$F'(t) = ae^{-kt}, F(0) = F_0 \Rightarrow \lim_{t \rightarrow \infty} F(t) = F_0 + a/k$
Definite integral (preview)	$\int_a^b f dx = F(b) - F(a)$

6.12 Before Week 7: Self-Check

Before moving on, make sure you can:

- explain why an antiderivative is only defined up to a constant C ;
- apply the power, constant, sum, and constant-multiple rules for integration;
- integrate exponential and $1/x$ functions correctly, including with a coefficient in the exponent;
- explain why integration doesn't distribute over products, quotients, or powers;
- solve an initial value problem to find a specific antiderivative;
- recognise the $F_0 + a/k$ pattern for exponential-decay IVPs and read off the limiting value directly;

- describe, in outline, how a Riemann sum converges to a definite integral, and state the Fundamental Theorem of Calculus.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §6.10 — before Week 7 begins, where the definite integral, the Fundamental Theorem, and their applications get the full treatment only previewed here.

6.13 Exercises

These exercises mirror the topics and difficulty of the Week 6 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Power rule. Find $\int (4x^3 - 2x + 5) dx$.

E2. Negative exponent. Find $\int \frac{3}{x^4} dx$.

E3. Fractional exponent. Find $\int \sqrt[3]{x} dx$.

E4. Exponential with coefficient. Find $\int e^{-0.4x} dx$.

E5. Logarithmic. Find $\int \frac{5}{x} dx$.

E6. Mixed functions. Find $\int \left(4e^{2x} - \frac{2}{x} + 3x^2 \right) dx$.

E7. Initial value problem. Solve $f'(x) = 6x^2 - 4x$, $f(0) = 3$, for $f(x)$.

E8. Exponential IVP. Solve $g'(t) = 10e^{-0.5t}$, $g(0) = 1$, for $g(t)$, and state $\lim_{t \rightarrow \infty} g(t)$.

E9. Physical application. A tank drains at rate $V'(t) = -4t + 20$ (litres/min), $V(0) = 300$ litres. Find $V(t)$ and the time at which the tank is empty.

E10. Agricultural application. Yield changes with fertiliser at $y'(x) = 0.02 - 0.0002x$ (t/ha per kg/ha), $y(0) = 1.5$ t/ha. Find $y(x)$ and the fertiliser rate that maximises yield.

E11. Limiting value. A pollutant concentration follows $C'(t) = -0.08 C(t)$ with $C(0) = 40$ mg/L. Find $C(t)$ and $\lim_{t \rightarrow \infty} C(t)$.

E12. SymPy interpretation. A student runs `sp.integrate(4*x**3 - 6/x, x)`. Write the expected SymPy output, then the complete antiderivative in standard mathematical notation.

Answers

E1: $x^4 - x^2 + 5x + C$.

E2: $\int 3x^{-4} dx = -x^{-3} + C = -1/x^3 + C$.

E3: $\int x^{1/3} dx = \frac{3}{4}x^{4/3} + C$.

E4: $-2.5e^{-0.4x} + C$.

E5: $5 \ln |x| + C$.

E6: $2e^{2x} - 2 \ln |x| + x^3 + C$.

E7: $f(x) = 2x^3 - 2x^2 + 3$.

E8: $g(t) = -20e^{-0.5t} + C$; $g(0) = 1 \Rightarrow C = 21$; $g(t) = 21 - 20e^{-0.5t}$; limit = 21.

E9: $V(t) = -2t^2 + 20t + 300$; empty when $V(t) = 0 \Rightarrow t \approx -8.23$ or 18.23 ; feasible root $t \approx 18.23$ min.

E10: $y(x) = 1.5 + 0.02x - 0.0001x^2$; $y' = 0 \Rightarrow x = 100$ kg/ha maximises yield.

E11: $C(t) = 40e^{-0.08t}$; $\lim_{t \rightarrow \infty} C(t) = 0$.

E12: SymPy: `x**4 - 6*log(x)`; standard notation: $x^4 - 6 \ln |x| + C$.

Chapter 7

Definite Integrals and Applications

WEEK 7

Theme: “Measuring Total Accumulation”

“The definite integral answers the question: what is the total? From pollution to population, from profit to land area, integration transforms rates into accumulated totals.”

7.1 From Indefinite to Definite: Completing Calculus

An indefinite integral yields a general family of formulas containing $+C$. A **definite integral** attaches specific numerical boundary limits $[a, b]$, converting the expression into a single, concrete number representing the net accumulated total (or area under the curve) between $x = a$ and $x = b$.

Week 6 found **antiderivatives**: $\int f(x) dx = F(x) + C$, a whole family of functions. This week answers a more concrete question: **what is the total accumulated quantity between two specific points?**

Indefinite integral	Definite integral
$\int f(x) dx = F(x) + C$	$\int_a^b f(x) dx = \text{a number}$
a family of functions	one specific value
includes $+C$	no constant of integration

Definition: Definite integral

$$\int_a^b f(x) dx,$$

where a is the lower limit, b the upper limit, and $f(x)$ the integrand.

7.2 The Fundamental Theorem of Calculus

Reading the Fundamental Theorem of Calculus

When evaluating $\int_a^b f(x) dx = F(b) - F(a)$, read it out loud as: “The definite integral of $f(x)$ from a to b is equal to the antiderivative F evaluated at the upper limit b minus the antiderivative F evaluated at the lower limit a .”

Notice that $+C$ cancels out completely: $(F(b) + C) - (F(a) + C) = F(b) - F(a)$.

Differentiation and integration look like opposite operations, built for different purposes — yet the single most important result in calculus is that they are, in a precise sense, inverses of each other. Part 1 makes that precise for a function built by accumulating area.

FTC Part 1 — Differentiating an Integral

If f is continuous on $[a, b]$, then $F(x) = \int_a^x f(t) dt$ is differentiable, and

$$\frac{d}{dx} \left[\int_a^x f(t) dt \right] = f(x).$$

The derivative of the “area so far” function equals the original function.

FTC Part 2 — Evaluation

If $F'(x) = f(x)$ (any antiderivative of f), then

$$\int_a^b f(x) dx = F(b) - F(a) = \left[F(x) \right]_a^b.$$

Find any antiderivative, evaluate it at the limits, and subtract.

Example: Why the $+C$ vanishes

Let $G(x) = F(x) + C$ be any antiderivative of f . Then

$$G(b) - G(a) = [F(b) + C] - [F(a) + C] = F(b) - F(a);$$

the constants always cancel. This is why definite integrals give a unique number, and why you never write $+C$ after evaluating one.

Newton and Leibniz

The FTC was independently discovered twice. **Newton** (England, c. 1666) developed his “method of fluxions” during the plague years, using integration to compute planetary orbits, but didn’t publish for decades. **Leibniz** (Germany, c. 1675–1684) developed the same ideas independently and invented the notation $\int$ and dx still used today. Their simultaneous discovery sparked one of history’s bitterest priority disputes — yet both formulations are correct and equivalent, and Leibniz’s notation won out for its computational convenience. The FTC’s real significance: it proved differentiation and integration are inverse operations, unifying centuries of separate mathematical work.

Example: Computing definite integrals

$$(a) \int_1^3 x^2 dx = \left[\frac{x^3}{3} \right]_1^3 = \frac{27}{3} - \frac{1}{3} = \frac{26}{3} \approx 8.67.$$

$$(b) \int_0^2 e^x dx = [e^x]_0^2 = e^2 - 1 \approx 6.39.$$

$$(c) \int_1^4 \frac{1}{x} dx = [\ln |x|]_1^4 = \ln 4 - \ln 1 = \ln 4 \approx 1.386.$$

$$(d) \int_0^1 (3x^2 + 2x - 1) dx = [x^3 + x^2 - x]_0^1 = (1 + 1 - 1) - 0 = 1.$$

Try it in Python

W7-CS01 — *Basic Setup and Imports*: load the libraries used throughout this week.

W7-CS02 — *Definite Integral with SymPy*: evaluate definite integrals symbolically.

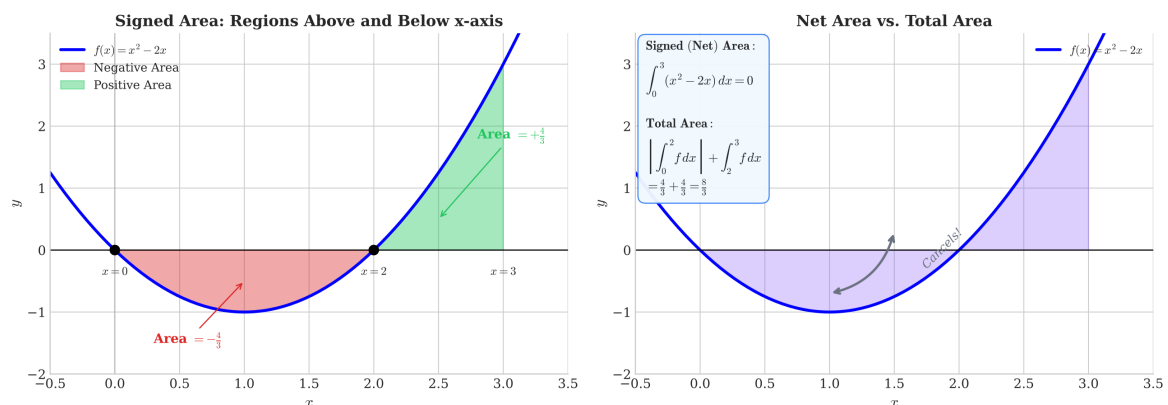
7.3 Properties of Definite Integrals

These are not extra formulas to deploy at random. Use them to split a difficult interval, reverse an inconvenient direction, or simplify an integral before calculating.

Property	Statement
Additivity	$\int_a^c f \, dx = \int_a^b f \, dx + \int_b^c f \, dx$
Reversing limits	$\int_a^b f \, dx = -\int_b^a f \, dx$
Zero-width interval	$\int_a^a f \, dx = 0$
Constant multiple	$\int_a^b k f \, dx = k \int_a^b f \, dx$
Sum/difference	$\int_a^b [f \pm g] \, dx = \int_a^b f \, dx \pm \int_a^b g \, dx$

7.4 Geometric Interpretation: Area and Signed Area

When $f(x) \geq 0$ on $[a, b]$, the definite integral equals the **area** between the curve and the x -axis. When $f(x)$ takes both signs, the integral gives **signed area**: regions above the axis count positively, regions below count negatively.



Example: Signed vs. total area

For $f(x) = x^2 - 2x$ on $[0, 3]$: since $f(x) = x(x - 2)$, the function is negative on $(0, 2)$ and positive on $(2, 3)$.

Signed (net) area: $\int_0^3 (x^2 - 2x) \, dx = \left[\frac{x^3}{3} - x^2 \right]_0^3 = (9 - 9) - 0 = 0$.

Total (unsigned) area: $\left| \int_0^2 (x^2 - 2x) \, dx \right| + \int_2^3 (x^2 - 2x) \, dx = \left| -\frac{4}{3} \right| + \frac{4}{3} = \frac{8}{3}$.

The signed area cancels to zero even though the curve clearly bounds nonzero regions — always check which quantity a question is actually asking for.

Try it in Python

W7-CS03 — *Visualizing Area Under a Curve*: plot a function and shade the region under it.

7.5 Area Between Two Curves

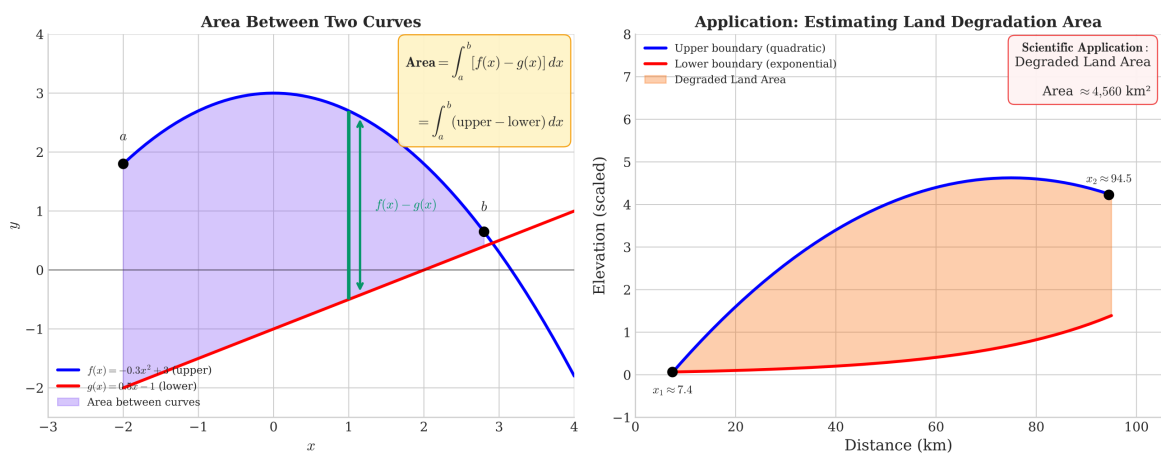
A single definite integral gives the area under one curve. Comparing two competing models, or two boundaries, means asking how much area sits *between* them instead.

Area Between Curves

If $f(x) \geq g(x)$ on $[a, b]$,

$$\text{Area} = \int_a^b [f(x) - g(x)] dx = \int_a^b (\text{upper} - \text{lower}) dx,$$

where f is the upper curve, g is the lower curve, and $[a, b]$ is the interval of integration. Find intersection points first by solving $f(x) = g(x)$.



Example: Degraded land area

A degraded land patch is bounded below by $y = e^{0.0471x}$ and above by $y = -19.96 + 2.999x - 0.02x^2$, intersecting at approximately $x = 7.4$ and $x = 94.5$ (km). Then

$$\text{Area} = \int_{7.4}^{94.5} [(-19.96 + 2.999x - 0.02x^2) - e^{0.0471x}] dx \approx 4560 \text{ km}^2,$$

evaluated with SymPy since the bounds aren't round numbers. Such calculations underpin rehabilitation cost estimates, vegetation-coverage planning, and soil-loss assessments.

Try it in Python

W7-CS04 — *Area Between Two Curves*: calculate and visualise area between two curves.

W7-CS10 — *Degraded Land Example*: compute the area of a degraded-land region bounded by two curves.

7.6 Average Value of a Function

For a finite list of numbers, an average is just a sum divided by a count. A continuously varying function has no list to sum — the definite integral steps in to play that role instead.

Average Value

$$\bar{f} = \frac{1}{b-a} \int_a^b f(x) dx,$$

where f is the function being averaged and $[a, b]$ is the interval. This is the height of a rectangle with width $b - a$ that has the same area as the region under $f(x)$.

Example: Average pollution level

Pollution concentration follows $C(t) = 50e^{-0.1t}$ (ppm) over the first 10 hours.

$$\bar{C} = \frac{1}{10} \int_0^{10} 50e^{-0.1t} dt = \frac{1}{10} \left[\frac{50}{-0.1} e^{-0.1t} \right]_0^{10} = \frac{1}{10} [500 - 500e^{-1}] \approx 31.6 \text{ ppm}.$$

Try it in Python

W7-CS09 — *Average Value of a Function*: calculate and visualise the average value of a function.

7.7 Sequences and Series: Bringing It Together

Just as integration sums infinitely many infinitesimally small pieces, **series** sum discrete terms. You met both families already — geometric sequences in Week 2 (§2.9–2.10) and arithmetic sequences in Week 3 — so here's a compact recap before putting them to work.

Type	n th term	Sum S_n
Arithmetic (Week 3)	$a_n = a_1 + (n-1)d$	$\frac{n}{2}(a_1 + a_n)$
Geometric (Week 2)	$a_n = a_1 r^{n-1}$	$a_1 \frac{1-r^n}{1-r}$ ($S_\infty = \frac{a_1}{1-r}$ if $ r < 1$)

Example: Counting terms in a range

For $a_i = 3 + 5(i-1) = 5i - 2$, how many terms satisfy $10 \leq a_i \leq 150$?

1. **Smallest i :** $5i - 2 \geq 10 \Rightarrow i \geq 2.4 \Rightarrow i_{\min} = 3$.
2. **Largest i :** $5i - 2 \leq 150 \Rightarrow i \leq 30.4 \Rightarrow i_{\max} = 30$.
3. **Count:** $30 - 3 + 1 = 28$ terms.

Try it in Python

W7-CS05 — *Arithmetic Sequence Generator*: generate arithmetic sequences in Python.

W7-CS06 — *Geometric Sequence Generator*: generate geometric sequences and compute their sums.

W7-CS07 — *Counting Terms in a Range*: find how many terms of a sequence fall within a given range.

7.8 The Malthusian Trap: Arithmetic vs. Geometric Growth

Thomas Robert Malthus (1798) observed that **food production** tends to grow **arithmetically** (linearly), $F_n = F_1 + (n-1)d$, while **population** tends to grow **geometrically** (ex-

ponentially), $P_n = P_1 r^{n-1}$. Since geometric growth eventually outpaces arithmetic growth no matter how large the arithmetic step, population inevitably overtakes food supply — the **Malthusian Crisis**.

Example: Numerical illustration

Both start at 100 (million); food capacity grows by 5/year (arithmetic), population grows at 3.5%/year (geometric).

Year	Food capacity	Population
1	100	100
10	145	136
20	195	192
50	345	540
71	450	1111

Population overtakes food capacity in year 22 in this scenario — the crossover always happens eventually, since geometric growth compounds while arithmetic growth only accumulates.

Modern Perspective

Malthus didn't foresee agricultural technology and the Green Revolution, or the demographic transition (birth rates declining with development) — so many regions escaped the “trap.” The underlying mathematical relationship, though, remains fundamental to understanding sustainability limits: any resource that grows arithmetically will eventually be outpaced by any demand that grows geometrically, regardless of the specific rates involved.

Try it in Python

W7-CS08 — *Malthusian Trap Simulation*: simulate and visualise the crisis point.

7.9 Consumer and Producer Surplus

Integration has a powerful application in economics: measuring the **welfare** that buyers and sellers gain from a competitive market. In a free market, price adjusts until quantity demanded equals quantity supplied — the **equilibrium** (Q^*, P^*). Not every consumer values the good at exactly P^* : some would have paid more, and get a bargain. Some producers could supply below P^* , and earn a bonus. Integration measures these gains precisely.

Consumer and Producer Surplus

The inverse demand function $D(Q)$ gives the maximum price consumers will pay for the Q -th unit; the inverse supply function $S(Q)$ gives the minimum price producers need to supply it.

$$\text{CS} = \int_0^{Q^*} D(Q) dQ - P^* Q^*, \quad \text{PS} = P^* Q^* - \int_0^{Q^*} S(Q) dQ.$$

Geometrically: CS is the area between the demand curve and the horizontal line at P^* ; PS is the area between that line and the supply curve, both from $Q = 0$ to $Q = Q^*$.

Example: Computing CS and PS

Market: demand $P = 50 - Q$, supply $P = 10 + Q$.

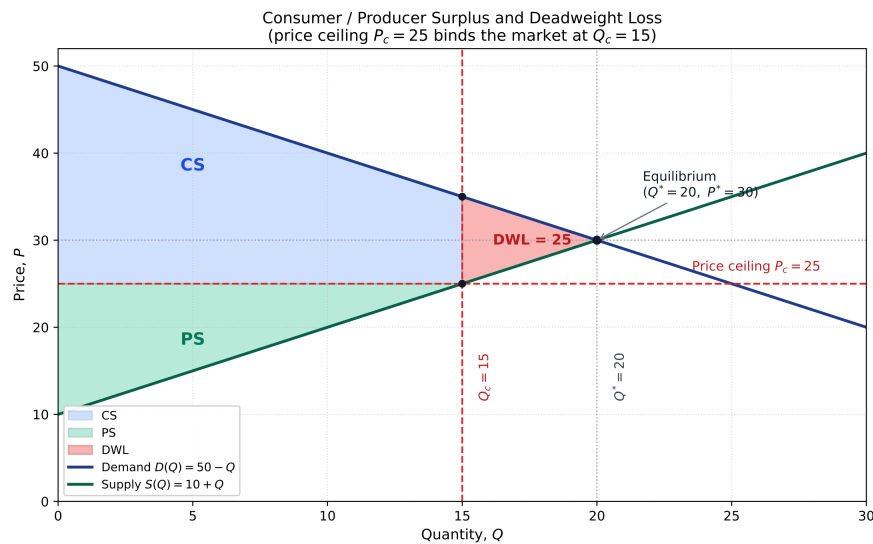
1. **Equilibrium:** $50 - Q = 10 + Q \Rightarrow Q^* = 20, P^* = 30$.

2. **Consumer surplus:** $CS = \int_0^{20} (50 - Q) dQ - 30(20) = \left[50Q - \frac{Q^2}{2} \right]_0^{20} - 600 = (1000 - 200) - 600 = 200$.

3. **Producer surplus:** $PS = 30(20) - \int_0^{20} (10 + Q) dQ = 600 - \left[10Q + \frac{Q^2}{2} \right]_0^{20} = 600 - 400 = 200$.

4. **Total welfare:** $CS + PS = 400$.

Shortcut for linear curves: when demand and supply are straight lines, CS and PS are right triangles, $CS = \frac{1}{2}Q^*(D(0) - P^*) = \frac{1}{2}(20)(20) = 200$ and $PS = \frac{1}{2}Q^*(P^* - S(0)) = \frac{1}{2}(20)(20) = 200$ — matching the integral exactly, and much faster once you recognise the shape.

**Deadweight Loss**

When a market is distorted — by a price ceiling, tax, or monopoly — some mutually beneficial transactions no longer occur. The lost welfare is **deadweight loss (DWL)**: the triangle between supply and demand over the range of foregone transactions.

Example: Deadweight loss from a price ceiling

Continuing the market above ($Q^* = 20, P^* = 30$), suppose the government imposes a price ceiling $P_c = 25$. Quantity demanded is $D^{-1}(25) = 25$, quantity supplied is $S^{-1}(25) = 15$; the market is supply-constrained, so only $Q_c = 15$ trades. At $Q = 15$: demand price $D(15) = 35$, supply price $S(15) = 25$ — these units would have benefited both sides by \$10 each, but the transactions from $Q = 15$ to $Q = 20$ don't happen.

$$DWL = \frac{1}{2}(Q^* - Q_c)(D(Q_c) - S(Q_c)) = \frac{1}{2}(5)(10) = 25.$$

Try it in Python

W7-CS11 — *Consumer and Producer Surplus Preview*: visualise and set up CS/PS calculations (full treatment continues in Week 12).

7.10 Riemann Sums: Connecting Series to Integrals

The definite integral is defined as the limit of **Riemann sums** — discrete approximations built from rectangles, previewed visually in Week 6.

Left Riemann Sum

Divide $[a, b]$ into n equal subintervals of width $\Delta x = (b - a)/n$, with $x_i = a + i\Delta x$. Then

$$L_n = \sum_{i=0}^{n-1} f(x_i) \Delta x, \quad \int_a^b f(x) dx = \lim_{n \rightarrow \infty} L_n.$$

Example: Left Riemann sum

Approximate $\int_0^2 x^2 dx$ with $n = 4$: $\Delta x = 0.5$, left endpoints $x = 0, 0.5, 1.0, 1.5$.

$$L_4 = 0.5(0 + 0.25 + 1 + 2.25) = 0.5(3.5) = 1.75.$$

The exact value is $8/3 \approx 2.67$; the left sum *underestimates* here because x^2 is increasing on $[0, 2]$, so each rectangle's left edge sits below the curve.

Series (discrete sums) become integrals (continuous sums) as the number of terms grows without bound and each term's width shrinks to zero — the same limiting idea that built the derivative from a secant line in Week 4, now applied to accumulation instead of slope.

Try it in Python

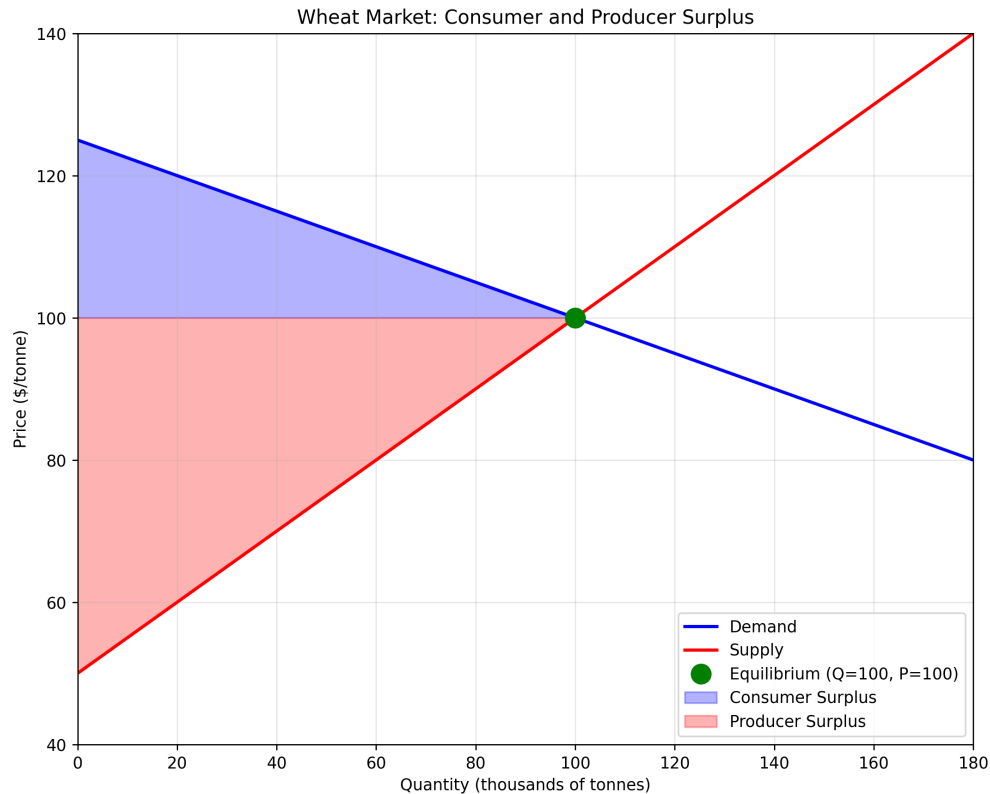
W7-CS12 — *Riemann Sum Visualization*: watch Riemann sums approximate the definite integral as n grows.

7.11 Python Lab: A Guided Walkthrough

The Week 7 lab (`Week07_Lab_STUDENT.ipynb`), “*Consumer and Producer Surplus*,” applies definite integration to the Australian domestic wheat market:

$$Q_d = 500 - 4P \quad (\text{demand}), \quad Q_s = -100 + 2P \quad (\text{supply})$$

where Q is thousand tonnes/year and P is dollars/tonne.



7.11.1 Part A — Finding Equilibrium

Rewriting in inverse form, $P = (500 - Q)/4$ (demand) and $P = (Q + 100)/2$ (supply), the lab solves for equilibrium both algebraically (setting $Q_d = Q_s$) and numerically, by writing both curves as linear equations in (Q, P) and solving with `numpy.linalg.solve`.

```
def demand_P(Q): return (500 - Q) / 4
def supply_P(Q): return (Q + 100) / 2

A = np.array([[1, 4], [1, -2]]) # Q + 4P = 500; Q - 2P = -100
b = np.array([500, -100])
Q_eq, P_eq = np.linalg.solve(A, b)
```

Activity: A — Verify equilibrium by hand

Set $Q_d = Q_s$ and solve for P^* , then find Q^* . Confirm your answer matches the code's output.

```
Qd = Qs
500 - 4P = -100 + 2P
```

```
-----
P* = ___, Q* = ___
```

7.11.2 Part B — Surplus, Graphically

CS is the triangle above P^* and below the demand curve; PS is the triangle below P^* and above the supply curve — both shaded directly on the supply–demand plot using `fill_between`.

7.11.3 Part C — Surplus via Integration

$$CS = \int_0^{Q^*} D(Q) dQ - P^*Q^*, \quad PS = P^*Q^* - \int_0^{Q^*} S(Q) dQ,$$

computed first with `numpy.trapezoid` (numerical) and then checked against the triangle-area shortcut from §7.9.

```
Q_fine = np.linspace(0, Q_eq, 10000)
CS = trapezoid(demand_P(Q_fine), Q_fine) - P_eq * Q_eq
PS = P_eq * Q_eq - trapezoid(supply_P(Q_fine), Q_fine)
```

7.11.4 Part D — Symbolic Integration with SymPy

Part C used `numpy.trapezoid` to *numerically approximate* the area under each curve. SymPy instead finds the *exact* area algebraically — the same symbolic-computation idea as `sp.diff()` in Weeks 4–5 and `sp.integrate()` in Week 6.

```
import sympy as sp

Q_sym = sp.symbols('Q', nonnegative=True)
demand_expr = (500 - Q_sym) / 4
supply_expr = (Q_sym + 100) / 2

area_under_demand = sp.integrate(
    demand_expr, (Q_sym, 0, Q_eq))
area_under_supply = sp.integrate(
    supply_expr, (Q_sym, 0, Q_eq))
```

Because demand and supply are both straight lines here, `trapezoid` was already exact — a line segment has no curvature, so the trapezoidal rule has zero error, and Parts C and D agree exactly. For a *curved* function, like the exponential sequestration rate in Week 6, `trapezoid` only approximates, and `sp.integrate()` is the only way to get the exact value.

Activity: D — Group discussion

1. Which group — buyers or sellers — benefits more from this wheat market? What does the ratio PS/CS tell you?
2. What would happen to CS and PS if the government imposed a price floor of \$120/tonne?
3. Why is total surplus maximised at the competitive equilibrium?

DISCUSSION:

7.12 Week 7 Summary

Key Formulas

Definite integral (FTC)	$\int_a^b f(x) dx = F(b) - F(a)$, where $F' = f$
Area under a curve	$\int_a^b f(x) dx$ when $f(x) \geq 0$
Area between curves	$\int_a^b [f(x) - g(x)] dx$
Average value	$\bar{f} = \frac{1}{b-a} \int_a^b f(x) dx$
Arithmetic sequence/series	$a_n = a_1 + (n-1)d$; $S_n = \frac{n}{2}(a_1 + a_n)$
Geometric sequence/series	$a_n = a_1 r^{n-1}$; $S_n = a_1 \frac{1-r^n}{1-r}$; $S_\infty = \frac{a_1}{1-r}$
Consumer/producer surplus	$CS = \int_0^{Q^*} D dQ - P^* Q^*$; $PS = P^* Q^* - \int_0^{Q^*} S dQ$
Deadweight loss	$DWL = \frac{1}{2}(Q^* - Q_c)(D(Q_c) - S(Q_c))$
Left Riemann sum	$L_n = \sum_{i=0}^{n-1} f(x_i) \Delta x \rightarrow \int_a^b f dx$ as $n \rightarrow \infty$

7.13 Before Week 8: Self-Check

Before moving on, make sure you can:

- state and apply both parts of the Fundamental Theorem of Calculus;
- evaluate definite integrals of polynomial, exponential, and logarithmic functions;
- distinguish signed (net) area from total (unsigned) area, and know which one a question is asking for;
- set up and evaluate an area-between-curves integral, finding intersection points first;
- compute and interpret the average value of a function;
- recognise arithmetic vs. geometric growth and explain why geometric growth eventually overtakes arithmetic growth (the Malthusian trap);
- compute consumer surplus, producer surplus, and deadweight loss from linear supply and demand functions;
- explain how a Riemann sum converges to a definite integral as $n \rightarrow \infty$.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §7.11 — before Week 8 begins, where these accumulation tools turn to modelling interacting populations.

7.14 Exercises

These exercises mirror the topics and difficulty of the Week 7 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Basic definite integral. Evaluate $\int_2^5 x^2 dx$.

E2. Exponential. Evaluate $\int_0^3 e^{2x} dx$.

E3. Logarithmic. Evaluate $\int_2^6 \frac{1}{x} dx$.

E4. Mixed integrand. Evaluate $\int_{-1}^2 (4x^3 - 3x^2 + 1) dx$.

E5. Signed area. For $f(x) = x^2 - 4x$ on $[0, 5]$, find the net (signed) area and the total (unsigned) area.

E6. Area between curves. Find the area enclosed between $y = x^2$ and $y = 2x + 3$.

E7. Average value. Find the average value of $f(x) = 3x^2 + 1$ on $[0, 4]$.

E8. Arithmetic vs. geometric. A resource starts at 200 units. Option A grows arithmetically by 8 units/year; Option B grows geometrically at 4%/year. After how many whole years does Option B first exceed Option A?

E9. Consumer and producer surplus. For demand $P = 80 - 2Q$ and supply $P = 20 + 2Q$, find Q^* , P^* , CS, and PS.

E10. Deadweight loss. For the market in E9, a price ceiling of $P_c = 50$ is imposed. Find Q_c and the deadweight loss.

E11. Left Riemann sum. Use a left Riemann sum with $n = 4$ to approximate $\int_1^5 (2x + 1) dx$, then compare to the exact value.

E12. Counting terms. For $a_i = 7 + 4(i - 1)$, how many terms satisfy $15 \leq a_i \leq 200$?

Answers

E1: $[x^3/3]_2^5 = 125/3 - 8/3 = 117/3 = 39$.

E2: $[\frac{1}{2}e^{2x}]_0^3 = \frac{1}{2}(e^6 - 1) \approx 201.2$.

E3: $[\ln x]_2^6 = \ln 6 - \ln 2 = \ln 3 \approx 1.099$.

E4: $[x^4 - x^3 + x]_{-1}^2 = (16 - 8 + 2) - (1 + 1 - 1) = 10 - 1 = 9$.

E5: roots at $x = 0, 4$; net $= \int_0^5 (x^2 - 4x) dx = \frac{125}{3} - 50 = -\frac{25}{3}$; total $= \frac{32}{3} + \frac{7}{3} = 13$.

E6: intersect at $x = -1, 3$; Area $= \int_{-1}^3 [(2x + 3) - x^2] dx = [x^2 + 3x - \frac{x^3}{3}]_{-1}^3 = 9 - (-\frac{5}{3}) = \frac{32}{3}$.

E7: $\bar{f} = \frac{1}{4} \int_0^4 (3x^2 + 1) dx = \frac{1}{4} [x^3 + x]_0^4 = \frac{1}{4}(68) = 17$.

E8: $A_n = 200 + 8n$; $B_n = 200(1.04)^n$; $A_1 = B_1 = 208$, and $B_n > A_n$ first at $n = 2$ ($A_2=216$, $B_2=216.32$).

E9: $80 - 2Q = 20 + 2Q \Rightarrow Q^* = 15$, $P^* = 50$; CS $= \frac{1}{2}(15)(30) = 225$; PS $= \frac{1}{2}(15)(30) = 225$.

E10: $Q_d=15$, $Q_s=15$ at $P_c=50=P^*$: no distortion since $P_c = P^*$; DWL $= 0$.

E11: $\Delta x=1$, left pts 1, 2, 3, 4: $L_4=1(3+5+7+9)=24$; exact $= [x^2 + x]_1^5 = 30 - 2 = 28$.

E12: $a_i = 4i + 3$; $i \geq 3$ for $a_i \geq 15$; $i \leq 49.25 \Rightarrow i_{\max} = 49$; count $= 49 - 3 + 1 = 47$.

Part III

Predicting Interactions

Chapter 8

Predator–Prey Dynamics

WEEK 8

Theme: “When Populations Interact”

“In nature, no species exists in isolation. The lynx pursues the hare, populations rise and fall in perpetual dance. Mathematics reveals the hidden choreography.”

8.1 From Single Populations to Interacting Systems

Welcome to Act III (Predicting Interactions)! In Acts I and II, we studied single variables isolated in time (a single population, a single pollutant). In Act III, we step into real ecological complexity where multiple species interact continuously. When predators (such as foxes) eat prey (such as rabbits), the prey density dictates predator survival, while predator numbers suppress prey growth.

Model	Equation	Behaviour
Exponential (Week 2)	$dN/dt = rN$	unlimited growth/decay
Logistic (Week 3)	$dN/dt = rN(1 - N/K)$	bounded growth to K
Schaefer (Week 3)	$G(S) = gS(1 - S/K)$	sustainable-yield framework

Every model so far treats a population in isolation. In reality, species interact: predators depend on prey, and prey are consumed by predators. This week asks: how do we model **two populations that influence each other** — lynx and snowshoe hare, wolves and deer, feral cats and numbats, even fishers and the fish stocks they exploit?

What changes when equations are coupled. Mathematically, one thing changes and it changes everything. In $dN/dt = rN$ the right-hand side involves only N , so the equation can be solved with the integration tools of Weeks 6–7 and written down as a formula, $N(t) = N_0 e^{rt}$. In a coupled system the rate of change of species A depends on the current size of species B and *vice versa*, so neither equation can be integrated on its own — and, for the model you meet next, no formula for $H(t)$ or $P(t)$ exists at all. That is not a failure; it is the reason this chapter switches tools. Instead of solving, we *analyse*: find the states where nothing changes (equilibria), read the direction of motion from the signs of the derivatives (the phase plane), and let the computer step the system forward in time (numerical simulation). Each of these uses only algebra and derivatives you already have.

8.2 State Space: The Geometry of Behaviour

A phase plane does not show time on either axis. Read it as a map of possible population combinations; time is represented by movement along a path.

For a single population, the state at any time is a point on the number line, $[0, \infty)$, and a **trajectory** shows how it moves along that line over time. For **two** interacting populations

— prey H and predator P — the state is a point in the plane, $(H, P) \in \mathbb{R}^+ \times \mathbb{R}^+$. As time passes, the system traces a trajectory through this 2D **state space** (or **phase plane**) instead of along a line.

An Everyday State-Space Analogy

The Lorenz–Zeeman model of a dog’s emotional state uses two axes: **rage** (fang exposure) and **fear** (ear position). A calm dog starts at $(1, 1)$; startled by a child, fear rises to $(1, 3)$; cornered, rage rises to $(3, 3)$. As the child flees, the state evolves again. The trajectory through this 2D space tells the whole story — exactly the idea we now apply to two populations instead of two emotions.

8.3 The Lotka–Volterra Model

Treat each equation as a balance sheet: identify what adds to the population, what subtracts from it, then ask when those two contributions cancel.



Alfred Lotka (1880–1949)



Vito Volterra (1860–1940)

Two Discoveries — One Equation

Alfred Lotka, an American mathematician and physical chemist, was studying oscillating chemical reactions when he published the coupled equations in his 1925 book *Elements of Physical Biology*, framing them as a general model for any two-species interaction. **Vito Volterra**, an eminent Italian mathematician, arrived at the same equations in 1926 through a personal connection: his future son-in-law, marine biologist Umberto D’Ancona, had been puzzling over Adriatic Sea fisheries data. During WWI, large-scale commercial fishing had virtually ceased; when it resumed, fishermen reported a striking rise in the *proportion* of predatory fish — sharks, rays, skates — in their catches. Volterra’s answer: reducing fishing (a shared predator on both species) releases prey fish from human pressure, prey abundance rises, and predatory fish thrive on the surplus. Two people, two countries, two completely different motivating problems, the same equations — one of the strongest signals in science that a mathematical structure captures something genuinely real.

Definition: Lotka–Volterra equations

With H = prey population and P = predator population,

$$\frac{dH}{dt} = \alpha H - \beta HP, \quad \frac{dP}{dt} = \lambda HP - \gamma P.$$

Parameter	Symbol	Meaning	Units
Prey birth rate	α	natural prey reproduction (no predation)	per time
Predation rate	β	rate at which encounters kill prey	per predator per time
Predator efficiency	λ	rate at which eating prey yields predator births	per prey per time
Predator death rate	γ	natural predator mortality (no food)	per time

Prey equation: $\frac{dH}{dt} = \underbrace{\alpha H}_{\text{births}} - \underbrace{\beta HP}_{\text{deaths from predation}}$. Without predators ($P = 0$), prey grows exponentially at rate α (Week 2). The βHP term is a **mass-action** effect: more encounters when both populations are large.

Predator equation: $\frac{dP}{dt} = \underbrace{\lambda HP}_{\text{births from feeding}} - \underbrace{\gamma P}_{\text{natural deaths}}$. Without prey ($H = 0$), predators decay exponentially at rate γ . Predator births depend entirely on successful hunting.

Predator Efficiency

$$\epsilon = \frac{\lambda}{\beta}$$

where λ is the predator birth rate per prey-predator encounter and β is the predation rate per encounter; ϵ is the number of new predators born per prey killed. If $\lambda = 0.0005$ and $\beta = 0.005$, then $\epsilon = 0.1 = 10\%$: for every 10 prey consumed, 1 new predator is born.

Try it in Python

W8-CS01 — *Setting Up Lotka–Volterra Parameters*: define symbols and biological parameters.

W8-CS02 — *Defining the ODE System*: write the predator–prey equations for `odeint`.

W8-CS03 — *Solving the ODEs*: integrate over time to get population trajectories.

W8-CS09 — *Verifying Equilibrium Points*: confirm fixed points algebraically and numerically.

8.4 Finding Equilibrium (Fixed) Points

Left alone, a predator-prey system rarely sits still — but at a handful of special population combinations, both species' growth rates happen to cancel out exactly, and the system stops changing entirely.

Definition: Equilibrium / fixed point

A state where both populations remain constant: $dH/dt = 0$ and $dP/dt = 0$.

From the prey equation, $H(\alpha - \beta P) = 0$ gives $H = 0$ or $P = \alpha/\beta$. From the predator equation, $P(\lambda H - \gamma) = 0$ gives $P = 0$ or $H = \gamma/\lambda$. Combining these conditions yields exactly

two equilibria:

$$(H^*, P^*) = (0, 0) \quad (\text{extinction}), \quad (H^*, P^*) = \left(\frac{\gamma}{\lambda}, \frac{\alpha}{\beta} \right) \quad (\text{coexistence}).$$

Example: Computing the coexistence equilibrium

For $\alpha = 0.15$, $\beta = 0.005$, $\lambda = 0.0005$, $\gamma = 0.10$:

$$H^* = \frac{\gamma}{\lambda} = \frac{0.10}{0.0005} = 200, \quad P^* = \frac{\alpha}{\beta} = \frac{0.15}{0.005} = 30.$$

Verify: $dH/dt = 0.15(200) - 0.005(200)(30) = 30 - 30 = 0 \checkmark$; $dP/dt = 0.0005(200)(30) - 0.10(30) = 3 - 3 = 0 \checkmark$.

8.5 Phase Portraits and Direction Fields

A **phase portrait** shows how the system evolves from any starting state: fixed points, trajectories, and a **direction field** of arrows showing instantaneous motion. At any point (H, P) , the direction of motion is the vector $(dH/dt, dP/dt)$, and the trajectory's slope is

$$\frac{dP}{dH} = \frac{dP/dt}{dH/dt} = \frac{P(\lambda H - \gamma)}{H(\alpha - \beta P)}.$$

Region	H vs. γ/λ	P vs. α/β	dH/dt	dP/dt	Direction
I	$H > \gamma/\lambda$	$P < \alpha/\beta$	+	+	NE
II	$H > \gamma/\lambda$	$P > \alpha/\beta$	−	+	NW
III	$H < \gamma/\lambda$	$P > \alpha/\beta$	−	−	SW
IV	$H < \gamma/\lambda$	$P < \alpha/\beta$	+	−	SE

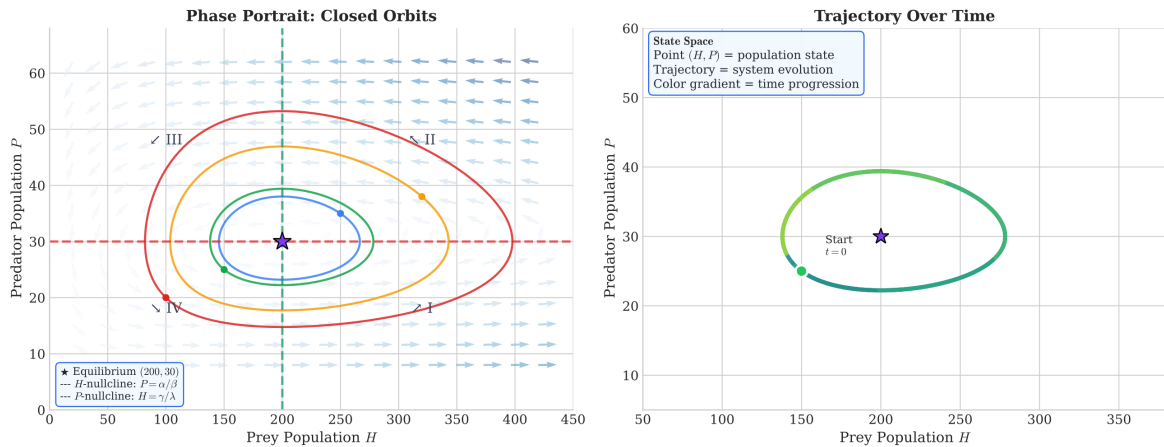
This four-region pattern always produces a **counterclockwise** flow around the coexistence equilibrium. For the basic model, trajectories form **closed orbits**: sustained periodic oscillations that never settle down, with different initial conditions producing different orbit sizes.

How the Computer Draws a Trajectory: Euler's Method

A direction-field arrow tells you where the system is heading *right now*. The simplest simulation just follows that arrow for a short time step Δt and then asks again:

$$H_{\text{next}} = H + \Delta t \cdot \frac{dH}{dt}, \quad P_{\text{next}} = P + \Delta t \cdot \frac{dP}{dt},$$

with the derivatives evaluated at the current (H, P) . This is the Week 4 definition of the derivative read backwards — $\Delta H \approx (dH/dt) \Delta t$ — and the running total of steps is a Riemann sum (Week 7) of the rate. The lab uses exactly this recipe (**Euler's method**); library solvers such as `odeint` use cleverer step rules, but the idea is the same: the phase plane shows the arrows, the simulation walks along them.



8.5.1 Nullclines

An equilibrium is a point where *both* populations stop changing at once. Nullclines get there in two steps: find where each population individually stops changing, then look for where those two conditions overlap.

Definition: Nullcline

A curve along which *one* derivative equals zero — from Latin *nullus*, “none.”

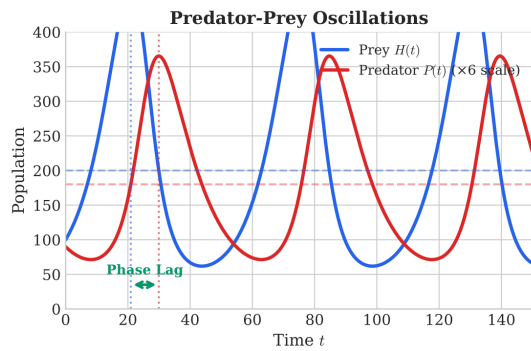
The non-trivial ***H*-nullcline** is the horizontal line $P = \alpha/\beta$: along it, prey are momentarily neither increasing nor decreasing, so trajectories cross it *vertically*. The non-trivial ***P*-nullcline** is the vertical line $H = \gamma/\lambda$: along it, the predator is momentarily stationary, so trajectories cross it *horizontally*. Their intersection is exactly the coexistence equilibrium — nullclines pinpoint equilibria geometrically and partition the plane into the four regions above.

Try it in Python

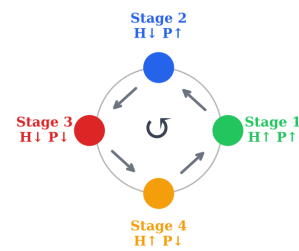
- W8-CS04** — *Plotting Time Series*: show how prey and predator populations oscillate over time.
- W8-CS05** — *Creating a Phase Portrait (Basic)*: plot trajectories in the H - P plane.
- W8-CS06** — *Adding Arrows to Phase Portrait*: indicate direction of flow.
- W8-CS07** — *Building a Vector Field*: sample $(\dot{H}, \dot{P})$ across the phase plane.
- W8-CS08** — *Complete Phase Portrait*: combine vector field with multiple trajectories.

8.6 Qualitative Behaviour: Time Series

A phase portrait shows the path in (H, P) space; a **time series** shows each population against time t directly. For basic Lotka–Volterra, both populations oscillate periodically, and **prey peaks lead predator peaks**: prey increase $\rightarrow$ more food $\rightarrow$ predators increase $\rightarrow$ more predation $\rightarrow$ prey decrease $\rightarrow$ less food $\rightarrow$ predators decrease $\rightarrow$ cycle repeats. The phase lag is approximately a quarter of the period.



The Predator-Prey Cycle



The Hudson Bay Data

Historical fur-trading records show approximately 10-year cycles for Canadian lynx and snowshoe hare — remarkably consistent with Lotka–Volterra predictions. Records from 1845–1935 reveal hare populations oscillating between roughly 2000 and 150,000 individuals per 100 km² (a 75-fold amplitude), lynx between roughly 500 and 8000 per 100 km² (16-fold), with a 9–11 year cycle period and lynx peaks following hare peaks by 1–2 years — matching the model’s quarter-cycle phase-lag prediction. Real data also shows irregular cycle lengths, varying amplitudes, and spatial synchronisation the basic model ignores.

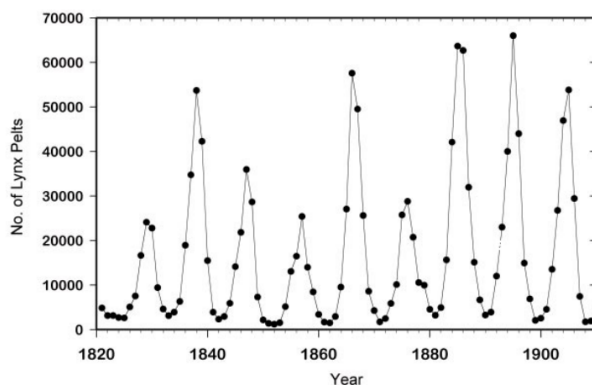


Figure 1. Canada lynx fur returns from the Northern Department of the Hudson's Bay Company from 1821 to 1910. The Northern Department occupied most of western Canada. The cycle for these data averages 9.6 years. Data are from Elton and Nicholson (1942). Photo: Mark O'Donoghue.

8.7 Limitations and Extensions

The basic model assumes: exponential prey growth without predators (ignores carrying capacity); unlimited predator appetite (no satiation); only two species (ignores food-web complexity); no age structure, migration, randomness, or spatial structure. Real ecosystems are dramatically more tangled than any two-species model:

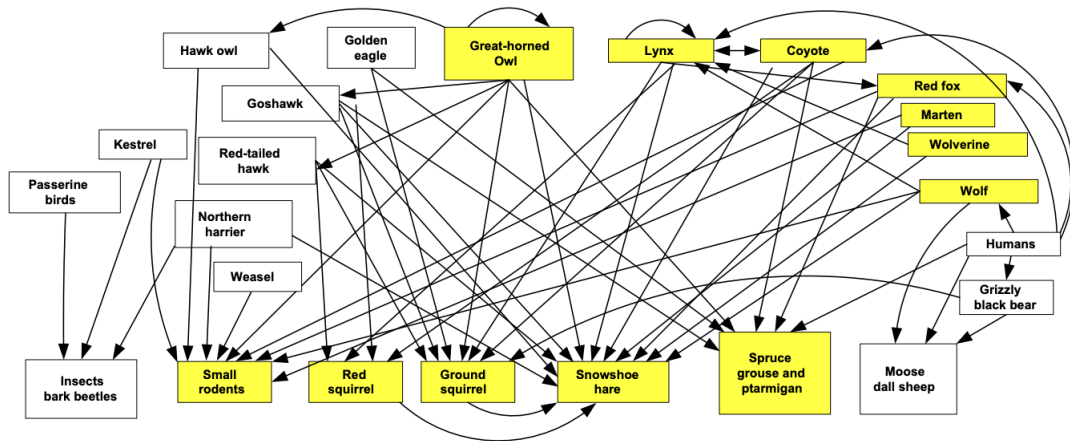


FIGURE 6 Herbivore and carnivore food web for major species in the Klane Yukon terrestrial ecosystem. Species shaded in yellow are the main species for which we have quantitative data. Occasional diet items are not shown in this diagram [Colour figure can be viewed at wileyonlinelibrary.com]

8.7.1 Adding Carrying Capacity

Making prey growth logistic instead of exponential:

$$\frac{dH}{dt} = \alpha H \left(1 - \frac{H}{K}\right) - \beta HP, \quad \frac{dP}{dt} = \lambda HP - \gamma P.$$

This produces **three** equilibria: both extinct $(0, 0)$; predator extinct, prey at capacity $(K, 0)$; and coexistence,

$$H^* = \frac{\gamma}{\lambda}, \quad P^* = \frac{\alpha}{\beta} \left(1 - \frac{H^*}{K}\right).$$

Example: Damped oscillations

With $\alpha = 0.2$, $\beta = 0.005$, $\lambda = 0.001$, $\gamma = 0.6$, $K = 1500$:

$$H^* = \frac{0.6}{0.001} = 600, \quad P^* = \frac{0.2}{0.005} \left(1 - \frac{600}{1500}\right) = 40(0.6) = 24.$$

Changed Behaviour

With carrying capacity, orbits are no longer closed — they **spiral inward**. The coexistence equilibrium becomes a **stable spiral** (an attractor) rather than a centre, and oscillations **damp** toward it instead of repeating forever.

8.7.2 Australian Application: Feral Cats and Small Marsupials

Feral cats (*Felis catus*), introduced with European settlement, kill an estimated 1.4–1.5 billion native animals per year (Woinarski et al., 2017) — a leading cause of Australian mammal extinction. A simplified model for feral cats (P) preying on small marsupials such as bilbies or numbats (H):

Parameter	Value	Meaning
α (marsupial birth rate)	0.50/yr	natural reproduction
β (predation rate)	0.020/yr	per cat per prey
λ (predator efficiency)	0.002/yr	cat births per prey killed
γ (cat death rate)	0.30/yr	natural cat mortality

Coexistence: $H^* = \gamma/\lambda = 0.30/0.002 = 150$ marsupials/km²; $P^* = \alpha/\beta = 0.50/0.020 = 25$ cats/km²; efficiency $\epsilon = \lambda/\beta = 10\%$.

Example: Effect of culling

A cat-control programme raising γ from 0.30 to 0.45 (via baiting or trapping) shifts the equilibrium:

$$H_{\text{new}}^* = \frac{0.45}{0.002} = 225 \text{ marsupials/km}^2.$$

Reducing predator survival raises the equilibrium prey population — the key management insight, and notice $P^* = \alpha/\beta$ is *unaffected* by γ , since it depends only on prey birth and predation rates.

Why the Basic Model Often Fails Here

Introduced cat–marsupial interactions rarely show the model’s sustained oscillations; instead, many marsupial species have simply gone extinct, for reasons the model ignores: (1) cats are **generalist predators** that switch to rabbits or mice when marsupials decline, preventing the predator crash that would let prey recover; (2) small marsupials have **low carrying capacities**, so drought or disease can push them below viable thresholds; (3) Australian marsupials have **no evolutionary history** with cats and lack co-evolved antipredator behaviours. Understanding *when and why* a model’s predictions fail is as scientifically important as applying it when it succeeds.

Try it in Python

W8-CS10 — *Modified Model with Carrying Capacity*: add a logistic term and observe how equilibria shift.

8.8 Connection to Scientific Method

The Lotka–Volterra model is a **mathematical hypothesis** about predator–prey systems: observe that lynx and hare oscillate; hypothesise that predator–prey feedback drives it; build the model; predict periodic cycles with a specific phase relationship; test against Hudson Bay data. When predictions don’t match perfectly, we refine: add carrying capacity for damped oscillations, add randomness for irregular amplitudes, add spatial structure for travelling waves — the same iterative cycle of modelling you’ve now seen applied to fisheries (Weeks 3, 5), drug dosing (Week 4), and land rehabilitation (Week 6).

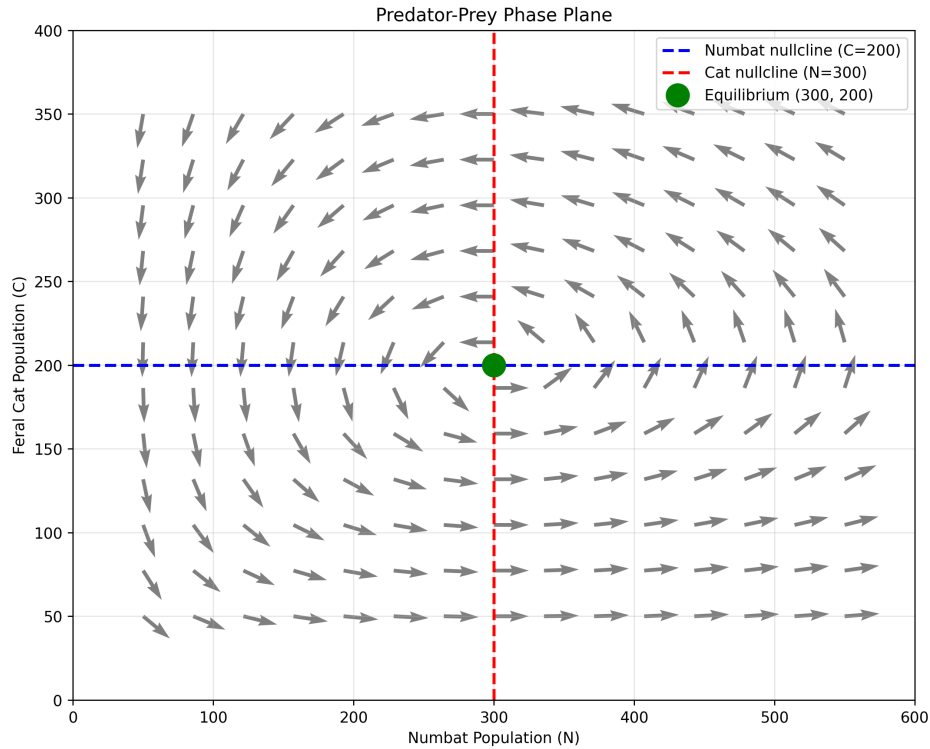
8.9 Python Lab: A Guided Walkthrough

The Week 8 lab (`Week08_Lab.STUDENT.ipynb`), “*Predator–Prey Dynamics and Differential Equations*,” models feral cats (C) threatening numbats (N) in the Western Australian wheat-belt, asking whether culling cats and relocating numbats produces lasting recovery or merely

shifts the timing of the same cycle:

$$\frac{dN}{dt} = r_N N - aNC, \quad \frac{dC}{dt} = -dC + bNC,$$

with $r_N = 0.4$, $a = 0.002$, $d = 0.3$, $b = 0.001$.



8.9.1 Part A — Equilibrium and Nullclines

Setting $dN/dt = 0$: $N(0.4 - 0.002C) = 0$, so the **numbat nullcline** is $C^* = r_N/a = 200$.
 Setting $dC/dt = 0$: $C(-0.3 + 0.001N) = 0$, so the **cat nullcline** is $N^* = d/b = 300$.
 Coexistence equilibrium: $(N^*, C^*) = (300, 200)$.

```
C_star = r_N / a    # from 0.4 - 0.002*C = 0
N_star = d / b      # from -0.3 + 0.001*N = 0
```

Activity: A — Direction and parameter ratios

1. What happens to the numbat population when $C > 200$? When $C < 200$?
2. What happens to the cat population when $N > 300$? When $N < 300$?
3. Without using the printed equilibrium values, identify which two parameter ratios determine the coexistence coordinates.

```
# 1. -----
# 2. -----
# 3. -----
```

8.9.2 Part B — Phase Plane: Turning Algebra into Movement

Placing numbats on the horizontal axis and cats on the vertical, each arrow in the vector field answers: *if the system is at this population pair, which way does it move next?* Tracing

a trajectory from a point near — but not at — coexistence, e.g. (305, 195), shows the counterclockwise closed orbit predicted by the direction-field table in §8.5. A visible spiral in an Euler-method plot is numerical drift rather than a property of the basic model.

```
N_grid = np.linspace(0, 600, 20)
C_grid = np.linspace(0, 400, 20)
NN, CC = np.meshgrid(N_grid, C_grid)

dN_dt = r_N * NN - a * NN * CC
dC_dt = -d * CC + b * NN * CC
```

Activity: B — Compare views

The phase-plane curve shows how the two populations move *relative to each other*; the time-series view shows the same trajectory with each population plotted against time. Compare the order of the peaks and troughs: which population changes first, and how does the other respond?

```
# -----
```

8.9.3 Part C — Longer Trajectories and a Management Scenario

Simulating with Euler's method (the step-by-step recipe in §8.5) over 30 years from $(N_0, C_0) = (400, 80)$ shows the full cycle amplitude, not just the small excursion near equilibrium from Part B. A second simulation tests an ongoing management intervention — removing 50 cats and adding 20 numbats each year — starting from a deliberately **stressed** initial state, $(N_0, C_0) = (200, 250)$ (numbats already scarce, cats already abundant), since the real question is not whether management preserves a healthy population but whether it can *recover* one that has already declined. This intervention *changes the model itself*, so the original coexistence equilibrium no longer necessarily describes the long-run outcome.

```
# Management scenario: remove 50 cats/year, add 20 numbats/year
# starting from a stressed state: N0, C0 = 200, 250
for i in range(steps):
    dN = (r_N * N[i] - a * N[i] * C[i] + 20) * dt
    dC = (-d * C[i] + b * N[i] * C[i] - 50) * dt
```

Activity: C — Group discussion

1. Do the populations oscillate or converge to equilibrium? How does your phase plane explain this?
2. Under the management scenario, does the numbat population recover? What is the long-run outcome?
3. What are the limitations of the Lotka–Volterra model for real conservation planning?

DISCUSSION:

```
-----
-----
-----
```

8.10 Week 8 Summary

Key Formulas

Prey equation	$dH/dt = \alpha H - \beta HP$
Predator equation	$dP/dt = \lambda HP - \gamma P$
Predator efficiency	$\epsilon = \lambda/\beta$
Extinction equilibrium	$(H^*, P^*) = (0, 0)$
Coexistence equilibrium	$(H^*, P^*) = (\gamma/\lambda, \alpha/\beta)$
Prey increasing when	$P < \alpha/\beta$
Predator increasing when	$H > \gamma/\lambda$
With carrying capacity K	interior $\left(\gamma/\lambda, \frac{\alpha}{\beta}(1 - \gamma/(\lambda K))\right)$

8.11 Before Week 9: Self-Check

Before moving on, make sure you can:

- write down the Lotka–Volterra equations and interpret each parameter and term biologically;
- find both equilibria (extinction and coexistence) and verify them by substitution;
- derive the nullclines and use them to determine the direction of flow in each of the four regions;
- explain why the basic model produces closed orbits (sustained oscillations) while the carrying-capacity version spirals inward (damped oscillations);
- read a phase portrait and a time-series plot as two views of the same trajectory, including the quarter-cycle phase lag;
- discuss why real predator–prey systems (e.g. feral cats and marsupials) can behave very differently from the model’s predictions, and why that’s still scientifically useful to know.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §8.9 — before Week 9 begins, where the course turns from deterministic dynamics to probability and uncertainty.

8.12 Exercises

These exercises mirror the topics and difficulty of the Week 8 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Model interpretation. In $dH/dt = \alpha H - \beta HP$, what does the term βHP represent, and why does it depend on both H and P ?

E2. Special case. If $H = 0$ at some instant, what does the predator equation predict happens to P over time?

E3. Equilibrium calculation. For $\alpha = 0.24$, $\beta = 0.006$, $\lambda = 0.0008$, $\gamma = 0.16$, find the coexistence equilibrium (H^*, P^*) .

E4. Verifying equilibrium. Verify your answer to E3 by substituting back into both derivatives.

E5. Predator efficiency. For $\lambda = 0.0009$ and $\beta = 0.003$, calculate the predator efficiency ϵ and interpret it in words.

E6. Direction field. For the system in E3, is the prey population increasing or decreasing at the point $(H, P) = (150, 50)$?

E7. Nullclines. For the system in E3, state the equations of both non-trivial nullclines.

E8. Carrying capacity equilibrium. Add carrying capacity $K = 800$ to the system in E3. Find the new coexistence equilibrium.

E9. Qualitative behaviour. Explain, without calculation, why adding carrying capacity changes closed orbits into inward spirals.

E10. Ecological interpretation. A conservation programme increases a predator's death rate γ . Which equilibrium value changes, and which stays the same? Explain why.

E11. Phase lag. In real lynx-hare data, lynx peaks lag hare peaks by roughly a quarter of the cycle. Explain, in terms of the model's equations, why this lag exists.

E12. Limitations. Name two assumptions of the basic Lotka-Volterra model that are violated by feral cats preying on Australian marsupials, and explain the consequence of each.

Answers

E1: the rate at which predator-prey encounters cause prey deaths; more encounters occur when both populations are larger (mass action).

E2: $dP/dt = -\gamma P$: with no prey, predators decay exponentially toward zero.

E3: $H^* = \gamma/\lambda = 0.16/0.0008 = 200$; $P^* = \alpha/\beta = 0.24/0.006 = 40$.

E4: $dH/dt = 0.24(200) - 0.006(200)(40) = 48 - 48 = 0$; $dP/dt = 0.0008(200)(40) - 0.16(40) = 6.4 - 6.4 = 0$.

E5: $\epsilon = 0.0009/0.003 = 0.3 = 30\%$: 3 new predators born per 10 prey killed.

E6: $dH/dt = 0.24(150) - 0.006(150)(50) = 36 - 45 = -9 < 0$: prey decreasing.

E7: H -nullcline: $P = \alpha/\beta = 40$; P -nullcline: $H = \gamma/\lambda = 200$.

E8: $H^* = 200$ (unchanged); $P^* = 40(1 - 200/800) = 40(0.75) = 30$.

E9: limited prey growth removes the excess energy that sustained constant-amplitude cycling, so oscillations lose energy and decay toward equilibrium.

E10: $H^* = \gamma/\lambda$ increases; $P^* = \alpha/\beta$ is unchanged, since it depends only on prey birth and predation rates.

E11: predator births (λHP) depend on current prey abundance, so predators can only increase after prey have already increased.

E12: e.g. cats are generalist (not specialist) predators, so no predator crash follows prey decline; marsupials lack co-evolved antipredator behaviour.

Chapter 9

Probability Foundations

WEEK 9

Theme: “Quantifying Uncertainty”

“Uncertainty is not ignorance — it is information about what we don’t know. Probability gives us a language to quantify the unknown with mathematical precision.”

9.1 From Deterministic to Probabilistic Thinking

Up to now, our mathematical models have been *deterministic*: given input x , output y is known with 100% certainty. But biological systems, environmental weather, and laboratory measurements are filled with inherent randomness. Probability provides the rigorous mathematical framework needed to measure, quantify, and make scientific decisions under uncertainty.

Model	Behaviour
Lotka–Volterra (Week 8)	trajectory fully determined by (H_0, P_0) and parameters
Logistic growth (Week 3)	approaches carrying capacity K exactly
Exponential (Week 2)	$N(t) = N_0 e^{rt}$ exactly

Every model so far is **deterministic**: knowing the initial state and parameters determines the future completely. Real systems involve genuine uncertainty — will a patient test positive? Will a treatment work? How many people will be infected? This week builds the mathematics for **quantifying** that uncertainty: disease diagnosis, clinical trials, epidemiology, genetics, and risk assessment all rest on it.

Kolmogorov’s Axioms

Probability was used without a rigorous definition for over 300 years. That changed in **1933**, when Russian mathematician Andrey Kolmogorov published a 62-page monograph axiomatising the entire subject with three simple rules (§9.3). They have not been revised since — every statistics textbook, medical test calculation, and AI system today rests on those same three axioms.

9.2 Random Experiments, Sample Spaces, and Events

Probability needs precise vocabulary before it needs formulas — the next few terms recur in almost every calculation for the rest of this chapter, so it’s worth being exact about what each one means.

Definition: Random Experiment — Sample Space — Event

A **random experiment** is a process whose outcome can't be predicted with certainty in advance. Its **sample space** S is the set of all possible outcomes. An **event** is a subset of S ; an **elementary event** contains exactly one outcome.

Experiment	Sample space
One coin toss	$S = \{H, T\}$
Two coin tosses	$S = \{HH, HT, TH, TT\}$
Five coin tosses	$ S = 2^5 = 32$

Sample Space Size

For n coin tosses, $|S| = 2^n$ — **not** $2n$. This is one of the most common exam errors: five tosses give $2^5 = 32$ outcomes, not $5 \times 2 = 10$.

Basic Probability

For equally likely outcomes,

$$P(A) = \frac{\text{outcomes favourable to } A}{\text{total outcomes}} = \frac{|A|}{|S|}.$$

Example: Two fair coin tosses

$S = \{HH, HT, TH, TT\}$. $P(\text{two heads}) = P(\{HH\}) = 1/4 = 0.25$.
 $P(\text{at least one head}) = P(\{HH, HT, TH\}) = 3/4 = 0.75$. $P(\text{no heads}) = P(\{TT\}) = 1/4 = 0.25$.

Try it in Python

W9-CS01 — *Library Imports*: load `numpy`, `scipy.stats`, `itertools`, and plotting tools.

W9-CS08 — *Basic Probability Calculation*: compute $P(A) = |A|/|S|$ from a sample space.

9.3 Properties and Rules of Probability

Every probability rule you will use in this chapter — complements, unions, conditional probability — can ultimately be derived from just three starting assumptions.

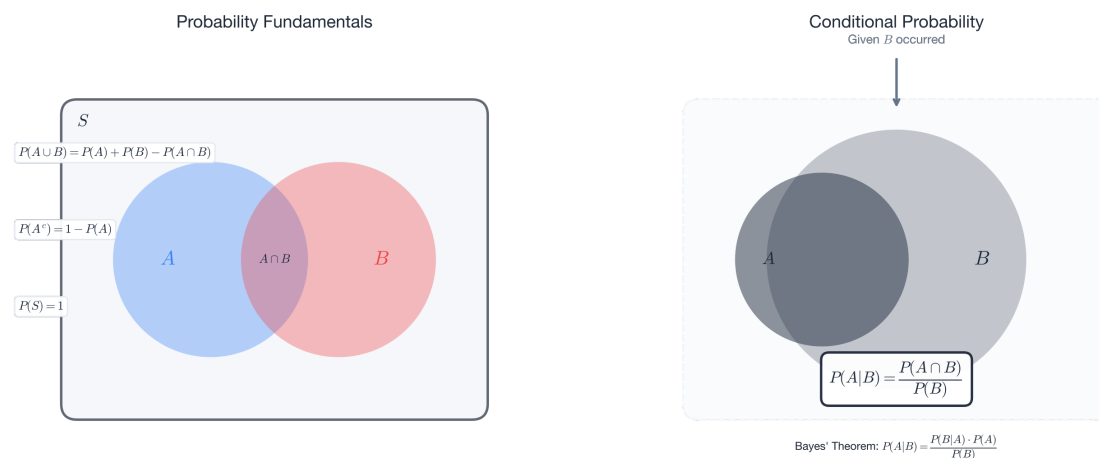
Kolmogorov's Axioms

Axiom	Statement	Interpretation
P1	$P(S) = 1$	something must happen
P2	$P(A) \geq 0$ for all A	probabilities are non-negative
P3	$A \cap B = \emptyset \Rightarrow P(A \cup B) = P(A) + P(B)$	addition for disjoint events

From these, everything else follows:

$$P(\emptyset) = 0, \quad P(A^c) = 1 - P(A), \quad 0 \leq P(A) \leq 1, \quad P(A \cup B) = P(A) + P(B) - P(A \cap B),$$

the last one because outcomes in both A and B would otherwise be counted twice. The complement rule is especially useful: to find $P(\text{at least one})$, compute $1 - P(\text{none})$.



Example: Applying the rules

Let $P(A) = 0.4$, $P(B) = 0.5$, $P(A \cap B) = 0.3$.

(a) $P(A \cup B) = 0.4 + 0.5 - 0.3 = 0.6$. (b) $P(A \cap B^c) = P(A) - P(A \cap B) = 0.4 - 0.3 = 0.1$. (c) $P((A \cup B)^c) = 1 - 0.6 = 0.4$.

Try it in Python

W9-CS02 — *Multiplication Principle*: count outcomes via product of choices.
W9-CS09 — *Enumeration Using Nested Loops*: build the sample space directly.
W9-CS10 — *Conditional Check in Enumeration*: filter outcomes satisfying an event.
W9-CS11 — *Probability Using the Combination Formula*: use $\binom{n}{k}$ to count favourable outcomes.

9.4 Exam-Style Application: Coin Tosses

To get exactly k heads in n tosses: choose which k positions are heads ($\binom{n}{k}$ ways), and each specific sequence has probability $(1/2)^n$.

Binomial Coefficient and Coin-Toss Probability

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}, \quad P(\text{exactly } k \text{ heads in } n \text{ tosses}) = \binom{n}{k} \left(\frac{1}{2}\right)^n.$$

Example: Five fair coins

$P(X = 3) = \binom{5}{3} \left(\frac{1}{2}\right)^5 = 10 \times \frac{1}{32} = \frac{10}{32} = 0.3125$ (not 0.60 — a common distractor).
 $P(X = 0) = \binom{5}{0} \left(\frac{1}{2}\right)^5 = \frac{1}{32} \approx 0.031$ (not 0.375).

Heads (k)	$\binom{5}{k}$	$P(X = k)$	Decimal
0	1	1/32	0.03125
1	5	5/32	0.15625
2	10	10/32	0.31250
3	10	10/32	0.31250
4	5	5/32	0.15625
5	1	1/32	0.03125

Symmetric around $k = 2.5$ (since $p = 0.5$); 2 or 3 heads are most likely ($\approx 31\%$ each); all-heads or all-tails is rare ($\approx 3\%$); every row sums to 1.

Try it in Python

W9-CS03 — *Factorial Calculations*: compute $n!$ and intermediate values.

W9-CS04 — *Permutation Formula*: calculate ordered arrangements.

W9-CS05/06 — *Listing Permutations/Combinations with itertools*: enumerate directly.

W9-CS07 — *Committee Assignment Problem*: apply combinations to a real selection task.

W9-CS12 — *Specific Sequence Probability*: compute $P(\text{exact sequence})$.

W9-CS22 — *Q32 Connection*: ties the sample-space idea directly back to the exam question.

9.5 Conditional Probability

Learning that event B has occurred changes what we should believe about event A — conditional probability is how that updated belief is calculated.

Conditional Probability

$$P(A | B) = \frac{P(A \cap B)}{P(B)}, \quad P(B) > 0.$$

Restrict attention to outcomes where B occurred, then ask what fraction also have A .

Example: Disease and symptoms

5% of a population has a disease (D); 90% of diseased people show symptoms (S); 10% of healthy people show symptoms too.

	D	D^c	Total
S	$0.05 \times 0.90 = 0.045$	$0.95 \times 0.10 = 0.095$	0.14
S^c	$0.05 \times 0.10 = 0.005$	$0.95 \times 0.90 = 0.855$	0.86
Total	0.05	0.95	1.00

$P(S | D) = 0.90$ is given directly. But $P(D | S) = \frac{P(D \cap S)}{P(S)} = \frac{0.045}{0.14} \approx 0.321$:

even with 90% of diseased people showing symptoms, only about 32% of *symptomatic* people actually have the disease. $P(A | B)$ and $P(B | A)$ are generally very different numbers — confusing them is one of the most common statistical errors.

Multiplication Rule

$$P(A \cap B) = P(A | B) P(B) = P(B | A) P(A).$$

9.6 Bayes' Theorem

Name the question after the bar first. Then identify the evidence, the starting prevalence, and the two pathways that could produce that evidence.

The Confusion of the Inverse: $P(A|B) \neq P(B|A)$

A critical error in science and medicine is confusing $P(A|B)$ with $P(B|A)$. For example, the probability of testing positive given that you are sick, $P(\text{Positive} | \text{Sick}) = 0.99$, is **not** the probability that you are sick given a positive test, $P(\text{Sick} | \text{Positive})$. If the disease is rare in the general population, most positive tests are actually false positives! Bayes' theorem is the exact mathematical formula that prevents this trap.

Thomas Bayes and the History of Inference

Thomas Bayes (c. 1701–1761) was a Presbyterian minister and amateur mathematician who solved this inference problem — and never published it. His key essay, written in the 1740s, was found among his papers after his death and presented to the Royal Society in 1763 by his friend Richard Price. Pierre-Simon Laplace independently re-derived the result in 1812. Alan Turing used it at Bletchley Park in 1941 to help crack the Enigma cipher (classified for decades). Today it powers medical diagnosis, spam filters, weather forecasting, and machine learning. The core intuition: *posterior belief* $\propto$ *likelihood* $\times$ *prior*.

Rearranging the multiplication rule gives $P(A | B) = \frac{P(B | A)P(A)}{P(B)}$. When A and A^c partition the sample space, the **law of total probability** gives $P(B) = P(B | A)P(A) + P(B | A^c)P(A^c)$. Combining both:

Bayes' Theorem

$$P(A | B) = \frac{P(B | A) P(A)}{P(B | A) P(A) + P(B | A^c) P(A^c)}.$$

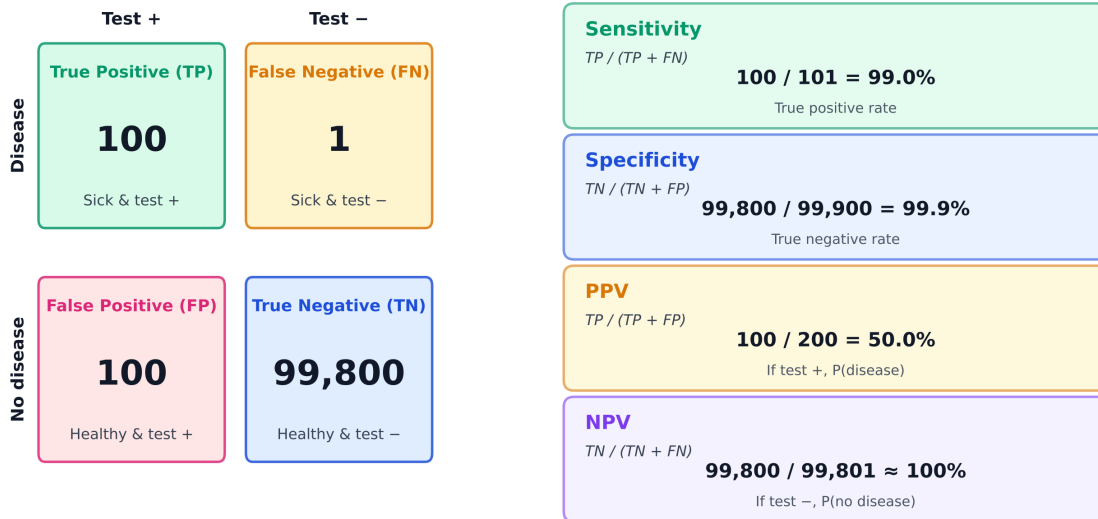
where A is the hypothesis of interest (e.g. “has the disease”), B is the observed evidence (e.g. “tested positive”), and $P(A)$ is the **prior** probability of A before seeing the evidence.

Term	Definition	Formula
Sensitivity	$P(\text{test+} \text{disease})$	true positive rate
Specificity	$P(\text{test-} \text{no disease})$	true negative rate
PPV	$P(\text{disease} \text{test+})$	positive predictive value
NPV	$P(\text{no disease} \text{test-})$	negative predictive value
Prevalence	$P(\text{disease})$	prior probability

Diagnostic testing — sensitivity, specificity, PPV, NPV

Confusion matrix per 100,000 (HIV, Fiji 2024)
prevalence 0.1% • sensitivity 99.5% • specificity 99.9%

Four diagnostic metrics



Example: HIV Testing in Fiji — 2024

Fiji's general-population HIV prevalence is approximately 0.1% (WHO/UNAIDS, 2024). A rapid test has sensitivity 99.5% and specificity 99.9%. With D = HIV positive, T^+ = positive test:

$$P(D | T^+) = \frac{0.995 \times 0.001}{0.995 \times 0.001 + 0.001 \times 0.999} = \frac{0.000995}{0.001994} \approx 0.499.$$

Despite 99.5% sensitivity and 99.9% specificity, a positive test indicates only a **$\approx 50\%$ chance** of true infection — the 100 false positives from 99,900 healthy people roughly match the 100 true positives from 100 infected people (per 100,000 tested). This is exactly the reasoning the CDC used in 1985, at a comparable $\approx 0.04\%$ general-population prevalence, to conclude that mass screening alone would produce more false than true positives — which is why confirmatory testing exists.

Targeted Testing: Prevalence Changes Everything

The same test's PPV depends entirely on the population tested: at 0.1% prevalence (general population) $PPV \approx 50\%$; at $\approx 5\%$ prevalence (a higher-risk group), PPV climbs to $\approx 98\%$. Directing testing resources toward higher-prevalence groups isn't just logistically sensible — it's *mathematically* optimal. PPV improves when (1) prevalence is higher, (2) specificity is higher, or (3) confirmatory two-stage testing is used.

9.7 Independent Events

Sometimes knowing B happened tells you nothing new about A . When that's true, the two events are independent, and probability calculations simplify considerably.

Definition: Independence

A and B are **independent** if $P(A \cap B) = P(A)P(B)$, equivalently $P(A | B) = P(A)$: knowing B occurred doesn't change A 's probability.

Independent vs. Disjoint — Not the Same Thing

Disjoint ($A \cap B = \emptyset$): if one occurs, the other cannot. **Independent** ($P(A \cap B) = P(A)P(B)$): one occurring doesn't affect the other's probability. If A, B are disjoint with $P(A), P(B) > 0$, then $P(A \cap B) = 0 \neq P(A)P(B) > 0$ — **disjoint events with positive probability are never independent.**

Example: Disease transmission

Transmission probability per contact is $p = 0.3$; 5 independent contacts occur. Using the complement:

$$P(\text{at least one transmission}) = 1 - P(\text{none}) = 1 - (0.7)^5 = 1 - 0.168 = 0.832.$$

9.8 Connection to Disease Spread

Recall $R_0 = \beta cD$ (transmissibility $\times$ contact rate $\times$ duration) from earlier lecture material: each contact is a **Bernoulli trial** with success probability β . The effective reproduction number is $R_e = sR_0$, where s is the susceptible proportion; for an epidemic to fade ($R_e < 1$), we need $s < 1/R_0$, so the immune proportion must satisfy

$$\pi > 1 - \frac{1}{R_0}.$$

Example: Herd immunity threshold

For measles, $R_0 \approx 15$: $\pi > 1 - 1/15 = 0.933$, so about **93.3%** of the population must be immune.

Disease	R_0	Herd immunity π
Measles	$\approx 12\text{--}18$	93.3%
COVID-19 (original)	$\approx 2\text{--}3$	60%
Influenza	≈ 1.5	33%
Ebola	≈ 2	50%

Higher R_0 needs a larger immune fraction to break transmission chains — measles requires near-universal immunity because each case contacts so many people, while influenza needs only about a third.

9.9 Tree Diagrams for Sequential Events

A tree diagram visualises sequential random experiments: each branch is a conditional probability given everything above it, and path probabilities multiply along the branches (the multiplication rule, applied step by step).

Example: Two-stage testing

A screening test has 95% sensitivity, 90% specificity; positives get a confirmatory PCR test with 99% sensitivity, 98% specificity. Prevalence is 2%.

1. **Diseased and confirmed:** $P(D \cap T_1^+ \cap T_2^+) = 0.02 \times 0.95 \times 0.99 = 0.01881$.
2. **Not diseased but confirmed (false-positive path):** $P(D^c \cap T_1^+ \cap T_2^+) = 0.98 \times 0.10 \times 0.02 = 0.00196$.
3. **Total confirmed positive:** $0.01881 + 0.00196 = 0.02077$.
4. **PPV of two-stage testing:** $\frac{0.01881}{0.02077} \approx 0.906$.

This is up from roughly 16% for a single screening test alone, illustrating why confirmatory testing is standard practice.

Try it in Python

W9-CS13/14 — *Manual Binomial Probability* and `scipy.stats.binom.pmf`: compute the same result two ways.

W9-CS15/16 — *Binomial Bar Plot* and *Complete Distribution Loop*: visualise and tabulate the full PMF.

W9-CS17–19 — *Infection Analysis*: compare $p = 0.45$ vs. $p = 0.60$.

W9-CS20/21 — *Comparing Distributions* and *Summary Statistics*: mean, mode, spread side by side.

W9-CS23 — *Q35 Connection*: bridges to next week's hypothesis testing.

W9-CS24 — *Complete Lab Template*: end-to-end starter for the lab notebook.

9.10 The Binomial Distribution

The **binomial distribution** models the number of successes in n independent trials, each with the same success probability p : a fixed number of trials, two outcomes per trial, constant p , and independence.

Binomial PMF

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n, \quad X \sim \text{Binomial}(n, p).$$

$$E[X] = np, \quad \text{Var}(X) = np(1 - p).$$

where n is the number of independent trials, p is the probability of success on each trial, and k is the number of successes observed.

Variance is largest at $p = 0.5$ (maximum per-trial uncertainty) and shrinks as $p \rightarrow 0$ or 1 .

Example: Disease Transmission — Generalised

With transmission probability $p = 0.3$ per contact and $n = 10$ contacts:

$$P(X = 3) = \binom{10}{3} (0.3)^3 (0.7)^7 = 120 \times 0.027 \times 0.0824 \approx 0.267, \quad E[X] = 10(0.3) = 3.$$

This is exactly the coin-toss calculation from §9.4, generalised to any repeated-trial experiment — and it's the same distribution behind the exam-style calculations throughout this chapter.

9.11 Hypothesis Testing: A Preview

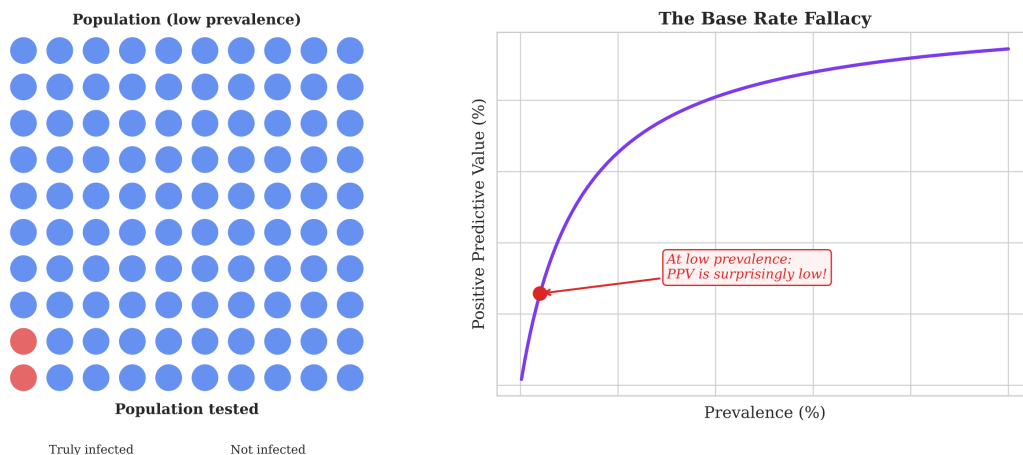
The binomial distribution lets us ask a fundamental scientific question: *is this result surprising?* Suppose a drug is claimed to cure at least 80% of patients ($H_0: p = 0.80$); in a trial of 20 patients, only 12 are cured ($H_a: p < 0.80$). If H_0 is true, $X \sim \text{Binomial}(20, 0.80)$ with $E[X] = 16$ — we observed 4 fewer cures than expected. The **p-value**, $P(X \leq 12 \mid H_0) \approx 0.032$, is the probability of a result *at least this extreme* if the drug really works as claimed: about 3.2% of trials would look this bad by chance alone, which is unlikely enough to cast doubt on H_0 .

Important

The p-value is *not* the probability that H_0 is true. It is the probability of the observed data (or more extreme), *given* that H_0 is true. Week 10 formalises the complete framework: significance levels, Type I and Type II errors, and the decision rule for rejecting H_0 .

9.12 Python Lab: A Guided Walkthrough

The Week 9 lab (Week09_Lab_STUDENT.ipynb), “Probability, Bayes’ Theorem, and Medical Testing,” evaluates a COVID-19 rapid antigen test: sensitivity 85%, specificity 95%, community prevalence 2%.



9.12.1 Part A — Bayes’ Theorem

The law of total probability gives $P(T^+) = P(T^+ \mid D)P(D) + P(T^+ \mid D^c)P(D^c)$, and Bayes’ theorem then gives $PPV = P(D \mid T^+)$.

```
P_Tplus_and_D      = sensitivity * P_D
P_Tplus_and_Dbar   = false_pos_rate * P_Dbar
P_Tplus            = P_Tplus_and_D + P_Tplus_and_Dbar
PPV = P_Tplus_and_D / P_Tplus
```

The lab then verifies this against a full 10,000-person confusion-matrix table (true/false positives and negatives computed directly from the four rates), so every intermediate count can be checked by hand.

Activity: Part A — Compute the PPV

1. Calculate $P(T^+ | D)P(D)$ and $P(T^+ | D^c)P(D^c)$ separately.
2. Add them to find $P(T^+)$, then compute $PPV = P(D | T^+)$.
3. Out of 10,000 people tested, how many are true positives and how many are false positives? Confirm your PPV matches this ratio.

```
# 1. P(T+|D)P(D) = -----
#    P(T+|Dc)P(Dc) = -----
# 2. P(T+) = -----
#    PPV = -----
# 3. True positives (10,000 people) =
#    -----
#    False positives (10,000 people) =
#    -----
```

9.12.2 Part B — A Negative Result Answers a Different Question

PPV asks whether a positive result is likely correct; **NPV** asks the parallel question for a negative result: given T^- , what's the probability of truly being disease-free? At low prevalence, most people are disease-free *before* testing, so NPV is typically very high even with an imperfect test — worth confirming numerically rather than assuming.

```
P_D_given_Tminus = ((1 - sensitivity) * P_D) / P_Tminus
NPV = 1 - P_D_given_Tminus
```

Activity: Part B — Compute the NPV

1. Find $P(T^-) = 1 - P(T^+)$ using your answer from Part A.
2. Compute $P(D | T^-)$: the probability of having the disease despite a negative test.
3. Compute $NPV = 1 - P(D | T^-)$. How does it compare with PPV, and why is the difference so large at this prevalence?

```
# 1. P(T-) = -----
# 2. P(D|T-) = -----
# 3. NPV = -----
#    Why so different from PPV:
#    -----
```

9.12.3 Part C — Sensitivity Analysis: Same Test, Different Meaning

Holding sensitivity and specificity fixed, only prevalence varies, isolating a crucial idea: a test result doesn't have one universal meaning; its interpretation depends entirely on the population tested (exactly the base-rate curve shown above).

```
def bayes_PPV(prev, sens=sensitivity, spec=specificity):
    fpr = 1 - spec
    P_pos = sens * prev + fpr * (1 - prev)
    return sens * prev / P_pos
```

Activity: Group discussion

1. Why is the PPV only $\approx 26\%$ even though the test is 85% sensitive? Where do the false positives come from?
2. At what prevalence does PPV reach 50%? Estimate from the plot, then verify with Python.
3. Should health authorities use this test for population screening at 2% prevalence? What are the consequences of a false-positive result — and of a false negative?

```
# 1. -----
# 2. -----
# 3. -----
```

9.13 Week 9 Summary**Key Formulas**

Sample space size (n coins)	$ S = 2^n$
Basic probability	$P(A) = A / S $
Complement rule	$P(A^c) = 1 - P(A)$
Addition rule	$P(A \cup B) = P(A) + P(B) - P(A \cap B)$
Conditional probability	$P(A B) = P(A \cap B)/P(B)$
Multiplication rule	$P(A \cap B) = P(A B)P(B)$
Independence	$P(A \cap B) = P(A)P(B)$
Bayes' theorem	$P(A B) = P(B A)P(A)/P(B)$
Binomial coefficient	$\binom{n}{k} = n!/(k!(n-k)!)$
Binomial probability	$P(X = k) = \binom{n}{k}p^k(1-p)^{n-k}$

9.14 Before Week 10: Self-Check

Before moving on, make sure you can:

- compute sample space sizes for repeated trials and avoid the $2n$ -vs- 2^n error;
- apply the complement, addition, and multiplication rules fluently;
- distinguish $P(A | B)$ from $P(B | A)$, and independence from disjointness;
- state and apply Bayes' theorem to a diagnostic-testing scenario, including sensitivity, specificity, PPV, and NPV;
- explain why PPV depends on prevalence, and why targeted testing is mathematically (not just practically) sensible;
- compute a binomial probability by hand using $\binom{n}{k}p^k(1-p)^{n-k}$;
- describe, at a conceptual level, what a p-value measures and what it does not.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §9.13 — before Week 10 begins, where hypothesis testing gets its full formal treatment.

9.15 Exercises

These exercises mirror the topics and difficulty of the Week 9 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

- E1. Sample space size.** How many outcomes are there in the sample space for 7 coin tosses?
- E2. Basic probability.** A fair six-sided die is rolled once. Find $P(\text{roll is at least 4})$.
- E3. Complement rule.** If $P(A) = 0.35$, find $P(A^c)$.
- E4. Addition rule.** If $P(A) = 0.3$, $P(B) = 0.45$, and $P(A \cap B) = 0.15$, find $P(A \cup B)$.
- E5. Conditional probability.** If $P(A \cap B) = 0.12$ and $P(B) = 0.4$, find $P(A | B)$.
- E6. Independence check.** If $P(A) = 0.2$, $P(B) = 0.5$, and $P(A \cap B) = 0.10$, are A and B independent?
- E7. Binomial coefficient.** Evaluate $\binom{7}{2}$.
- E8. Binomial probability.** For $X \sim \text{Binomial}(6, 0.4)$, find $P(X = 2)$.
- E9. Bayes' theorem.** A disease has prevalence 2%. A test has sensitivity 92% and specificity 96%. Find the PPV.
- E10. Independent trials.** A mosquito bite transmits a pathogen with probability 0.15. For 6 independent bites, find the probability of at least one transmission.
- E11. Herd immunity.** A disease has $R_0 = 4$. Find the herd immunity threshold π .
- E12. Two-stage testing.** A screening test (90% sensitivity, 85% specificity) is followed by a confirmatory test (98% sensitivity, 97% specificity) for those who screen positive, in a population with 3% prevalence. Find the PPV after confirmatory testing.

Answers

- E1:** $2^7 = 128$.
- E2:** rolls 4, 5, 6 favourable: $P = 3/6 = 0.5$.
- E3:** $P(A^c) = 1 - 0.35 = 0.65$.
- E4:** $P(A \cup B) = 0.3 + 0.45 - 0.15 = 0.6$.
- E5:** $P(A | B) = 0.12/0.4 = 0.3$.
- E6:** $P(A)P(B) = 0.2 \times 0.5 = 0.10 = P(A \cap B)$: yes, independent.
- E7:** $\binom{7}{2} = \frac{7!}{2!5!} = 21$.
- E8:** $P(X = 2) = \binom{6}{2}(0.4)^2(0.6)^4 = 15 \times 0.16 \times 0.1296 \approx 0.311$.
- E9:** $P(T^+) = 0.92(0.02) + 0.04(0.98) = 0.0184 + 0.0392 = 0.0576$; PPV = $0.0184/0.0576 \approx 0.319$.
- E10:** $1 - (0.85)^6 \approx 1 - 0.377 = 0.623$.
- E11:** $\pi > 1 - 1/4 = 0.75$.
- E12:** Diseased & confirmed: $0.03 \times 0.90 \times 0.98 = 0.02646$; not diseased & confirmed: $0.97 \times 0.15 \times 0.03 = 0.004365$; PPV = $0.02646/(0.02646 + 0.004365) \approx 0.858$.

Chapter 10

Random Variables and Hypothesis Testing

WEEK 10

Theme: “Making Decisions Under Uncertainty”

“The question is not whether disease spreads randomly, but whether the evidence allows us to distinguish a new threat from statistical noise. Hypothesis testing gives science its teeth.”

10.1 Random Variables: Building the Idea

What is a Random Variable?

A random variable X is simply a mathematical rule that assigns a numerical value to every possible outcome of a random process. Instead of working with descriptive outcomes like “sunny day” or “rainy day”, X converts real-world events into numbers (such as X = daily rainfall in mm) so we can compute expected averages, variances, and probabilities.

A natural first answer to “what is a random variable?” is *a number that comes from a random experiment*. That leaves three questions open.

(1) What counts as a number? Outcomes are often non-numerical — heads or tails, infected or not. We need an *explicit rule* converting each outcome to a number; without it, no averages or variances are possible. **(2) What makes it random?** The value is genuinely undetermined until the experiment runs — not merely unmeasured. **(3) Which number maps to which outcome?** Different rules give different random variables from the *same* experiment: mapping heads $\mapsto$ 1, tails $\mapsto$ 0 is one variable; heads $\mapsto$ 5, tails $\mapsto$ -3 is a completely different one, with a different distribution.

Definition: Random variable

A **random variable** X is a function assigning a specific numerical value to every possible outcome of a random experiment.

Type	Description	Examples
Discrete	countable values	infected-cell count, coin-flip heads
Continuous	any value in an interval	body temperature, concentration (mg/L)

We focus mainly on discrete random variables this week, with the Normal distribution as our one continuous example.

Probability Mass Function

For a discrete X , the **PMF** $p(x) = P(X = x)$ satisfies $\sum_x p(x) = 1$ and $p(x) \geq 0$. (For continuous X , the analogous object is the **probability density function**, where probabilities are areas under a curve rather than sums — the Normal distribution in §10.4.)

Example: Two fair coins

Let X = number of heads.

x	Outcome	$P(X = x)$
0	TT	1/4
1	HT or TH	1/2
2	HH	1/4

Check: $\frac{1}{4} + \frac{1}{2} + \frac{1}{4} = 1$. ✓

10.2 Expected Value and Variance

A random variable's expected value is its long-run average outcome; its variance measures how far individual outcomes typically stray from that average.

Expected Value and Variance

$$E[X] = \sum_x x p(x), \quad \text{Var}(X) = E[(X - \mu)^2] = \sum_x (x - \mu)^2 p(x) = E[X^2] - (E[X])^2,$$

$$\sigma = \sqrt{\text{Var}(X)}.$$

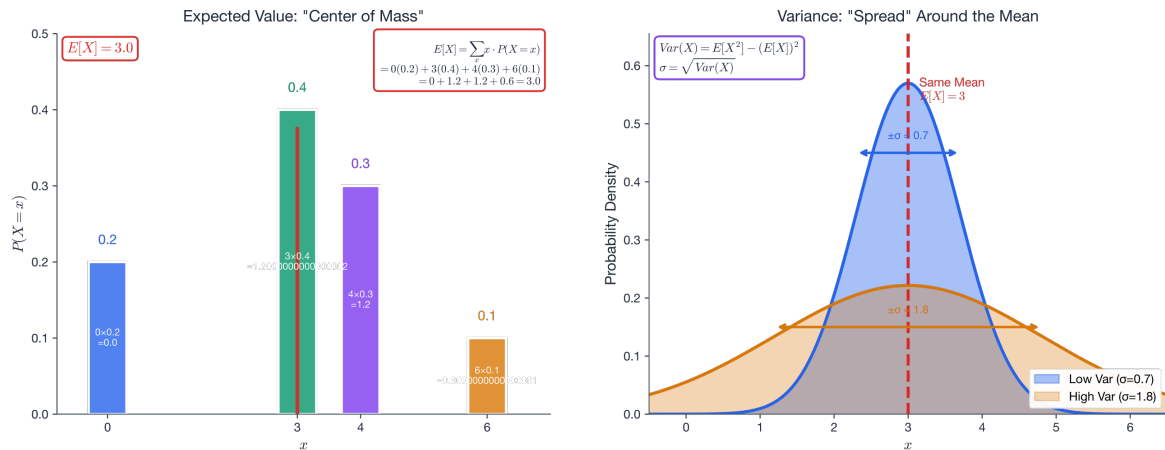
where $p(x)$ is the probability mass function of X and $\mu = E[X]$. $E[X]$ is the long-run average — not necessarily a value X can actually take. The shortcut $E[X^2] - (E[X])^2$ is usually faster than working with $(X - \mu)^2$ directly.

Example: Mutations per cell division

A lab records X = mutations per division with PMF:

x	1	2	3	4
$p(x)$	0.2	0.3	0.3	0.2

(1) $E[X] = 1(0.2) + 2(0.3) + 3(0.3) + 4(0.2) = 0.2 + 0.6 + 0.9 + 0.8 = 2.5$. (2) $E[X^2] = 1(0.2) + 4(0.3) + 9(0.3) + 16(0.2) = 0.2 + 1.2 + 2.7 + 3.2 = 7.3$. (3) $\text{Var}(X) = 7.3 - 2.5^2 = 7.3 - 6.25 = 1.05$. (4) $\sigma = \sqrt{1.05} \approx 1.025$ mutations.



Try it in Python

W10-CS01 — *Setup and Imports*: load `numpy`, `scipy.stats`, plotting tools.

W10-CS02 — *Expected Value Calculation*: compute $E[X]$ from a PMF.

10.3 Named Discrete Distributions

A handful of probability patterns — a single yes/no trial, a fixed number of repeated trials — recur so often across science that they've earned their own names and ready-made formulas.

Bernoulli

A single trial with two outcomes — success (probability p) or failure ($1 - p$). $X \sim \text{Bernoulli}(p)$: $E[X] = p$, $\text{Var}(X) = p(1 - p)$.

Example: Seed germination

A seed germinates under drought with probability $p = 0.4$. $\text{Var}(X) = 0.4 \times 0.6 = 0.24$.

Binomial

n independent Bernoulli trials, each with success probability p : $X \sim \text{Binomial}(n, p)$,

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, \dots, n,$$

$$E[X] = np, \quad \text{Var}(X) = np(1 - p).$$

Example: Ten Coins — Eight or More Heads

$n = 10$, $p = 0.5$. $P(X \geq 8) = P(X=8) + P(X=9) + P(X=10)$:

$$P(X=8) = \binom{10}{8} (0.5)^{10} = \frac{45}{1024}, \quad P(X=9) = \frac{10}{1024}, \quad P(X=10) = \frac{1}{1024},$$

so $P(X \geq 8) = \frac{56}{1024} \approx 0.0547$ — about a 5.5% chance, unusual but not impossible. (This is exactly the kind of tail probability a hypothesis test evaluates — see §10.5.)

Try it in Python

W10-CS03 — *Bernoulli Distribution*: single-trial success/failure model.

W10-CS04 — *Binomial PMF Calculation*: compute $P(X = k)$.

W10-CS05 — *Visualizing Binomial Distribution*: bar plot of the PMF.

10.4 The Normal Distribution and Z-Scores

From counts to measurements. Every random variable so far has been a *count* — heads, infected plants, successes in n trials — so its distribution was a list of probabilities, one per whole number, and questions were answered by *adding* PMF values. Rainfall, blood pressure and body mass are not counts: any value in an interval is possible, and the probability of hitting one exact value is zero. For such **continuous** variables the PMF is replaced by a smooth **density** curve, and probability becomes the *area* under that curve between two values — the sum of Week 9 has turned into the integral of Week 7. The most important density in science is the normal (bell) curve.

A z -score is a translation device: it changes a measurement in its original units into “how many standard deviations from typical?”

$X \sim N(\mu, \sigma^2)$ has a bell-curve density, symmetric about μ , with σ controlling spread; the percentages in the empirical rule below are areas under this curve.

Empirical Rule (68–95–99.7)

Interval	Approx. probability
$\mu \pm \sigma$	68%
$\mu \pm 2\sigma$	95%
$\mu \pm 3\sigma$	99.7%

Z-Score

$$Z = \frac{X - \mu}{\sigma}$$

where X is the observed value, μ is the population mean, and σ is the population standard deviation (all in the same units as X ; Z itself is unitless). Z follows the standard normal $N(0, 1)$. Key critical value: $z_{0.025} = 1.96$, i.e., $P(Z > 1.96) = 0.025$.

Example: Blood pressure

$X \sim N(100, 15^2)$ (mmHg). What fraction have $X > 130$?

$$Z = \frac{130 - 100}{15} = 2, \quad P(X > 130) = P(Z > 2) \approx 0.0228 \text{ (2.28\%)}.$$

The empirical rule gives $\mu + 2\sigma = 130$, so $\approx 2.5\%$ above — close to, but not exactly, the true value of 2.28%.

10.5 Hypothesis Testing

Hypothesis Testing as a Scientific Courtroom

Hypothesis testing is the scientific equivalent of a court trial. We begin by assuming the defendant is innocent — this is the **Null Hypothesis** H_0 (assuming no real effect, just random chance). We collect data, and only if the probability of seeing such extreme data by pure chance (the **p-value**) is smaller than our significance threshold ($\alpha = 0.05$), do we “reject the null hypothesis” and declare a significant discovery.

We test whether observed data are consistent with a **null hypothesis** H_0 or provide evidence for an **alternative** H_a .

The Four-Step Procedure

1. State H_0 and H_a ; choose α (typically 0.05).
2. Collect data; compute the test statistic.
3. Compute (or bound) the **p-value**: the probability of a result *at least as extreme* as the data, *assuming* H_0 is true.
4. If p-value $< \alpha$: **reject** H_0 ; otherwise: **fail to reject**.

One-tailed: $H_a : \theta > \theta_0$ or $H_a : \theta < \theta_0$ (direction specified *before* data collection). **Two-tailed:** $H_a : \theta \neq \theta_0$ (any deviation matters).

Rule

Decide the direction of H_a *before* looking at the data. Choosing the direction afterward inflates the false-positive rate — this is one of the most common ways hypothesis tests go wrong in practice.

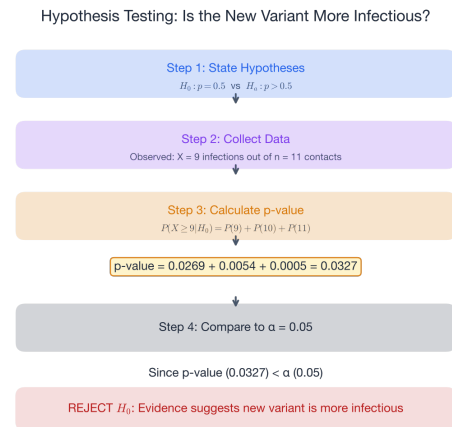
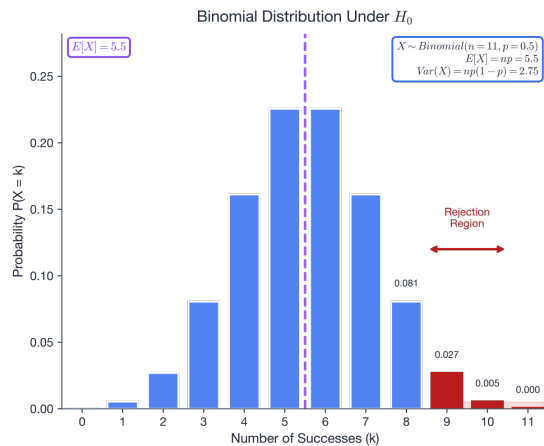
Which distribution does step 3 use? The p -value is a probability, so it has to come from *some* distribution — the one the data would follow *if* H_0 were true. When the data are a count of successes in n independent trials, that distribution is the binomial of §10.3 with p set to the value claimed by H_0 , and the p -value is a tail sum of its PMF; this is what the example below does. When the data are a measurement, or n is large, the same tail probability is read as an area under a normal curve using the z -score of §10.4, which is why $z_{0.025} = 1.96$ matters so much (§10.7). The four steps never change — only the distribution plugged into step 3.

Example: Pesticide trial

A pesticide is claimed to reduce infection probability below 40%. In a trial, 9 of 12 plants are infected. Test at $\alpha = 0.05$: is the rate *higher* than 40%?

(1) $H_0 : p \leq 0.40$ vs. $H_a : p > 0.40$ (one-tailed, upper). (2) $X = 9$ of $n = 12$; assume H_0 true, $p = 0.40$. (3) p-value = $P(X \geq 9 \mid \text{Bin}(12, 0.40)) = \sum_{k=9}^{12} \binom{12}{k} (0.4)^k (0.6)^{12-k} \approx 0.015$. (4) $0.015 < 0.05$, so **reject** H_0 .

Conclusion: significant evidence ($p = 0.015$) that the infection rate exceeds 40%, suggesting the pesticide is ineffective or even harmful.



Try it in Python

- W10-CS06** — *p-Value via `binom.pmf` Loop*: sum tail probabilities manually.
- W10-CS07** — *p-Value via `binomtest`*: use the built-in SciPy function.
- W10-CS09** — *Visualizing the p-Value*: highlight the critical region on the PMF.
- W10-CS10** — *Critical Values via `binom.ppf`*: find rejection thresholds.
- W10-CS11** — *One-Tailed vs. Two-Tailed Tests*: compare both approaches side by side.

10.6 Type I and Type II Errors

Read the rows as the decision we make and the columns as the hidden truth. Errors occur where decision and truth disagree.

	H_0 True	H_0 False
Reject H_0	Type I error (α)	Correct ✓
Fail to reject H_0	Correct ✓	Type II error (β)

Type I rate = α = significance level (false positive). **Type II** rate = β (false negative). **Power** = $1 - \beta$ = probability of correctly rejecting a false H_0 .

Where “Type I” and “Type II” Actually Came From

Ronald Fisher, the dominant statistician of the 1920s, popularised the significance test built around a single question: is this result surprising enough, under H_0 , to reject it? His famous illustration was the “Lady Tasting Tea” — a colleague at Rothamsted Experimental Station claimed she could tell, by taste, whether milk or tea was poured into the cup first, and Fisher designed a randomised experiment to test her claim rigorously rather than anecdotally. But Fisher’s framework only ever spoke of rejecting or not rejecting H_0 ; it had no language for the *alternative* hypothesis, and therefore no way to quantify the risk of missing a real effect. **Jerzy Neyman** and **Egon Pearson**, working separately in the late 1920s and 1930s, argued this was a serious gap: a decision procedure needs to weigh *two* kinds of mistake, not one. They formalised the pair we now call **Type I error** (rejecting a true H_0) and **Type II error** (failing to reject a false H_0), along with the modern idea of statistical **power**. Fisher never accepted this reframing, and the two camps argued in print for decades over which philosophy of testing was correct. Nearly every hypothesis test run today is, in prac-

tice, a hybrid of both — Fisher’s p -value language layered onto the Neyman–Pearson decision framework — inherited from a dispute neither side considered settled.

The Trade-off

Decreasing α (a stricter threshold) reduces Type I errors but *increases* Type II errors — a real effect becomes harder to detect. At a fixed significance level α , increasing sample size n reduces Type II error and increases power; it does not change the nominal Type I error rate.

10.7 Statistical Power and Sample Size

Returning to the Week 9 drug example, $H_0 : p = 0.80$, observed $\hat{p} = 12/20 = 0.60$: how reliably would we detect this departure at different sample sizes?

n	Approximate power
20	$\approx 68\%$
100	$\approx 99.7\%$

Normal Approximation

The exact sampling distribution of $\hat{p}$ is Binomial (discrete bars spaced $1/n$ apart). The smooth curves used in power calculations use $\hat{p} \approx N(p, p(1-p)/n)$, which improves as n grows: at $n = 20$ bars are 0.05 apart; at $n = 100$ they’re 0.01 apart and nearly continuous.

Example: Large-sample test

With $n = 100$ patients (same $H_0 : p = 0.80$), observing 60% success:

$$Z = \frac{0.60 - 0.80}{\sqrt{0.80 \times 0.20/100}} = \frac{-0.20}{0.040} = -5.0 \Rightarrow p\text{-value} \approx 0.000003.$$

The same 60% observed rate with five times as many patients gives overwhelming evidence — exactly why larger studies are far more reliable.

Two Equivalent Z-Score Forms

$$Z = \frac{\hat{p} - p_0}{\sqrt{p_0(1-p_0)/n}} = \frac{k - np_0}{\sqrt{np_0(1-p_0)}}.$$

The second form uses raw counts: observed successes k minus the Binomial mean np_0 , scaled by the Binomial standard deviation. For $n = 100$: $Z = (60 - 80)/\sqrt{16} = -5.0$ — identical. For the original Week 9 trial ($n = 20$, $k = 12$): $Z = (12 - 16)/\sqrt{3.2} \approx -2.24$, consistent with the exact binomial p -value of 0.032 found there (that trial used the exact Binomial rather than this Normal approximation, since $n = 20$ is small).

Key Insight

Statistical power answers: *if there really is an effect, how likely are we to detect it?*
More data $\rightarrow$ higher power $\rightarrow$ fewer missed discoveries.

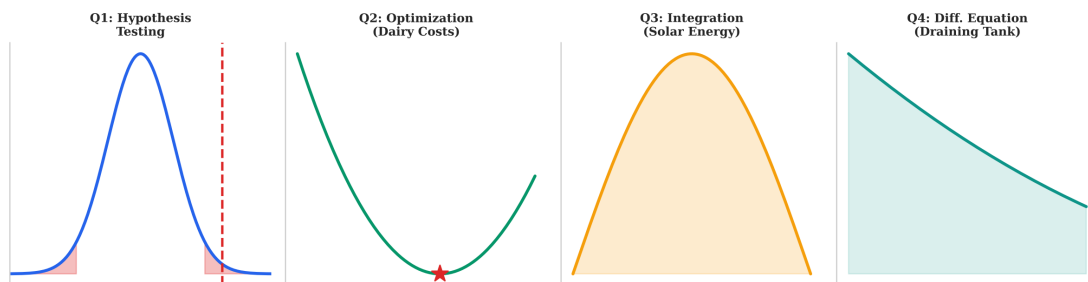
Try it in Python

W10-CS08 — *Q35 Exam Solution*: walk through the exam-style question step by step.

W10-CS12 — *Disease Transmission Analysis*: end-to-end real-world application.

10.8 Python Lab: A Guided Walkthrough

The Week 10 lab (`Week10_Lab_STUDENT.ipynb`) is a *mock exam workshop*: three exam-style questions spanning the semester, worked individually under time pressure, then debriefed as a group — plus an optional separable-ODE extension.



10.8.1 Q1 — Hypothesis Testing

A pharmaceutical company claims its antibiotic is effective in at least 80% of cases. A hospital audit of $n = 20$ patients finds it effective in only $x = 11$. Test at $\alpha = 0.05$.

Under H_0 , $X \sim \text{Bin}(20, 0.80)$ — the boundary value $p = 0.80$ is used because a p-value is always computed *at* the null hypothesis, the most conservative assumption consistent with H_0 . Building the lower tail from these components,

$$P(X=11) \approx 0.0074, \quad P(X=10) \approx 0.0020, \quad P(X=9) \approx 0.0005, \quad P(X \leq 8) \approx 0.0001,$$

gives $P(X \leq 11) \approx 0.0100$.

```
from math import comb

def binomial_pmf(k, n, p):
    return comb(n, k) * p**k * (1-p)**(n-k)

p_value = sum(binomial_pmf(k, 20, 0.80) for k in range(11 + 1))
```

Activity: Q1 — State your conclusion

1. State H_0 and H_a , and identify the test as left-, right-, or two-tailed.
2. Explain why $X \sim \text{Bin}(20, 0.80)$ is the distribution used under H_0 .
3. Compute $P(X \leq 11)$ from the components above, and compare it with your code's output.
4. Compare the p-value with $\alpha = 0.05$. State the statistical decision and a one-sentence conclusion for a hospital administrator.

```
# a. H0: _____ Ha: _____
#      Tail: _____
```

```
# b. -----
# c. P(X <= 11) =
# -----
# d. Decision:
# -----
# Conclusion:
# -----
```

10.8.2 Q2 — Optimisation (Review, Week 5)

A dairy cooperative has daily cost $C(x) = 2x^2 - 80x + 1200$ (output x , thousand litres) and sells milk at \$1200 per thousand litres, so profit is $\Pi(x) = 1200x - C(x)$.

```
def C(x): return 2*x**2 - 80*x + 1200
def Revenue(x): return 1200 * x
def Pi(x): return Revenue(x) - C(x)
def dPi(x): return 1200 - (4*x - 80)
```

Activity: Q2 — Work the optimisation

1. Write $\Pi(x)$ explicitly and find $\Pi'(x)$.
2. Set $\Pi'(x) = 0$ and solve for the profit-maximising output x^* .
3. Use the second derivative test to verify x^* is a maximum.
4. Calculate maximum daily profit $\Pi(x^*)$.
5. If a cost increase changes $C(x)$ to $2x^2 - 80x + 1500$, how does x^* change? Explain why, without recomputing from scratch.

```
# a. Pi(x) = ----- Pi'(x) =
# -----
# b. x* = -----
# c. Pi''(x) = ----- (sign confirms:
# -----)
# d. Pi(x*) = -----
# e. -----
```

10.8.3 Q3 — Integration (Review, Week 7)

A solar array generates $P(t) = 500\sin(\pi t/12)$ watts over $0 \leq t \leq 12$ hours. (You'll meet the derivative and integral of $\sin$ formally in Week 11; for now, take the antiderivative $-\frac{12}{\pi}\cos(\pi t/12)$ as given and evaluate it at the limits, exactly as in any other definite integral.)

```
def P(t): return 500 * np.sin(np.pi * t / 12)

t_fine = np.linspace(0, 12, 10000)
energy_Wh_numeric = np.trapz(P(t_fine), t_fine)
```

Activity: Q3 — Work the integral

1. What is the peak power, and at what time t does it occur?
2. Set up the integral for total energy generated (watt-hours) over the full day.
3. Evaluate $\int_0^{12} 500 \sin\left(\frac{\pi t}{12}\right) dt$ analytically using the given antiderivative.
4. Convert your answer to kWh. If electricity sells at 30 cents/kWh, what is daily revenue?
5. Compare your analytic answer to the code's numerical result.

```
# a. Peak power = _____ W at t = _____
# b. Energy = integral of _____ from t
    =0 to 12
# c. Energy (Wh) =
    _____
# d. Energy (kWh) = _____ Revenue =
    _____
# e. _____
```

10.8.4 Bonus: Separable Differential Equations (Optional)

Not part of the mock exam, but a genuinely new technique worth a look. A water tank drains by Torricelli's law, $\frac{dV}{dt} = -0.5\sqrt{V}$, $V(0) = 100$ L. **Separating variables** means gathering all V -terms on one side and all t -terms on the other, then integrating each side independently:

$$\frac{dV}{\sqrt{V}} = -0.5 dt \implies \int V^{-1/2} dV = \int -0.5 dt.$$

This is the standard first method for solving ODEs that aren't already in the $dy/dx = f(x)$ form used in Weeks 6–7: isolate each variable on its own side, *then* integrate.

Activity: Bonus — Solve the draining tank

1. Integrate both sides and apply $V(0) = 100$ to find the constant of integration.
2. Write the explicit formula $V(t)$.
3. How long until the tank is half empty ($V = 50$ L)? Completely empty ($V = 0$ L)?

```
# a. Integrated: _____ C = _____
# b. V(t) =
    _____
# c. Half empty at t = _____ min; Empty at t =
    _____ min
```

10.9 Week 10 Summary

Key Formulas

Expected value	$E[X] = \sum_x x p(x)$
Variance (shortcut)	$\text{Var}(X) = E[X^2] - (E[X])^2$
Standard deviation	$\sigma = \sqrt{\text{Var}(X)}$
Bernoulli	$E[X] = p; \quad \text{Var}(X) = p(1-p)$
Binomial PMF	$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$
Binomial moments	$E[X] = np; \quad \text{Var}(X) = np(1-p)$
Z-score	$Z = (X - \mu)/\sigma$
Empirical rule	68/95/99.7% within 1/2/3 σ
Decision rule	reject H_0 if p-value $< \alpha$
Power	$1 - \beta$

10.10 Before Week 11: Self-Check

Before moving on, make sure you can:

- explain what makes a variable “random” and why the outcome-to-number mapping must be stated explicitly;
- compute $E[X]$ and $\text{Var}(X)$ from a PMF, using the shortcut variance formula;
- apply the Bernoulli and Binomial PMFs, and state their means and variances;
- convert a Normal random variable to a Z-score and apply the empirical rule;
- carry out all four steps of a hypothesis test, including stating H_0/H_a correctly *before* looking at the data;
- distinguish Type I from Type II error, and explain the α - β trade-off;
- explain why increasing sample size increases statistical power.

If any of these feel shaky, revisit the relevant section above — or work back through the mock-exam questions in §10.8 — before Week 11 begins, where periodic functions complete the toolkit this chapter’s solar-power question got a preview of.

10.11 Exercises

These exercises mirror the topics and difficulty of the Week 10 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. PMF check. For X with $P(X=1) = 0.15$, $P(X=2) = 0.35$, $P(X=3) = k$, $P(X=4) = 0.20$, find k .

E2. Expected value. For the distribution in E1 (with your value of k), find $E[X]$.

E3. Variance shortcut. For the distribution in E1, find $\text{Var}(X)$ and σ .

E4. Bernoulli. A vaccine is effective with probability $p = 0.75$. Find $E[X]$ and $\text{Var}(X)$.

E5. Binomial probability. For $X \sim \text{Binomial}(8, 0.3)$, find $P(X = 3)$.

E6. Binomial moments. For $X \sim \text{Binomial}(15, 0.4)$, find $E[X]$ and $\text{Var}(X)$.

E7. Z-score. $X \sim N(50, 8^2)$. Find $P(X > 62)$.

E8. Empirical rule. $X \sim N(200, 10^2)$. Using the empirical rule, estimate $P(180 < X < 220)$.

E9. Hypothesis setup. A company claims a process defect rate of at most 5%. A sample suggests the true rate is higher. State H_0 and H_a , and identify the test as one- or two-tailed.

E10. p-value decision. A test gives p-value = 0.041 at $\alpha = 0.05$. State the decision and a one-sentence interpretation.

E11. Type I/II errors. A court trial treats H_0 as “innocent.” Describe, in words, what a Type I and a Type II error mean in this context.

E12. Power and sample size. Explain, in one or two sentences, why doubling the sample size (with α fixed) increases statistical power.

Answers

E1: probabilities sum to 1: $0.15 + 0.35 + k + 0.20 = 1 \Rightarrow k = 0.30$.

E2: $E[X] = 1(0.15) + 2(0.35) + 3(0.30) + 4(0.20) = 0.15 + 0.70 + 0.90 + 0.80 = 2.55$.

E3: $E[X^2] = 1(0.15) + 4(0.35) + 9(0.30) + 16(0.20) = 0.15 + 1.40 + 2.70 + 3.20 = 7.45$; $\text{Var}(X) = 7.45 - 2.55^2 = 0.9475$; $\sigma \approx 0.973$.

E4: $E[X] = 0.75$; $\text{Var}(X) = 0.75 \times 0.25 = 0.1875$.

E5: $P(X=3) = \binom{8}{3}(0.3)^3(0.7)^5 \approx 0.2541$.

E6: $E[X] = 15(0.4) = 6$; $\text{Var}(X) = 15(0.4)(0.6) = 3.6$.

E7: $Z = (62 - 50)/8 = 1.5$; $P(X > 62) = P(Z > 1.5) \approx 0.0668$.

E8: $180 = \mu - 2\sigma$, $220 = \mu + 2\sigma$: empirical rule gives $\approx 95\%$.

E9: $H_0 : p \leq 0.05$; $H_a : p > 0.05$; one-tailed (upper).

E10: $0.041 < 0.05$: reject H_0 ; there is statistically significant evidence against H_0 at the 5% level.

E11: Type I: convicting an innocent person (rejecting H_0 when it's true). Type II: acquitting a guilty person (failing to reject H_0 when it's false).

E12: a larger sample shrinks the standard error of the sampling distribution, making it easier to distinguish a real effect from chance variation, so a true effect is more likely to be detected.

Part IV

Making Optimal Decisions

Chapter 11

Trigonometric Functions and Periodic Models

WEEK 11

Theme: “Modeling Cycles and Rhythms”

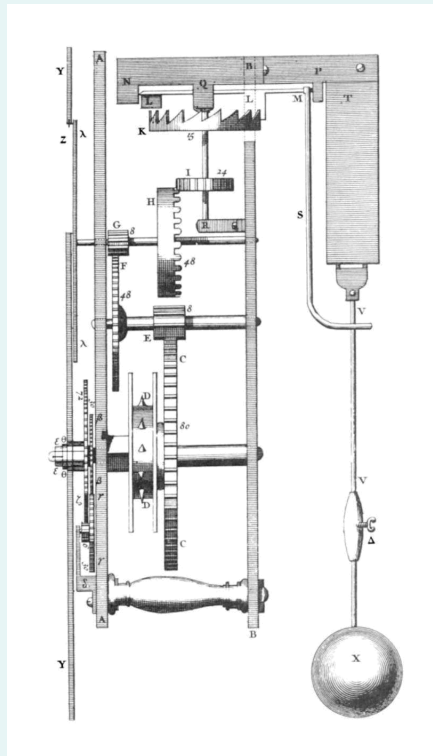
“The cosmos is full of repeating patterns — days, seasons, heartbeats, brain waves. Understanding periodicity is understanding life itself.”

11.1 Why Trigonometry for Scientists?

Welcome to Act IV (Making Optimal Decisions)! Nature is dominated by cycles: day and night, ocean tides, seasonal temperature variations, wildlife migration pulses, and cardiac rhythms. Linear and exponential models head off toward infinity; trigonometric functions repeat smoothly in periodic cycles, making them the natural mathematical language for modeling repeating phenomena.

Cycles are everywhere: circadian rhythms in hormone levels and body temperature (Week 10’s Q3 solar-power question already previewed this), seasonal patterns in temperature and migration, physiological oscillations in heart rate and breathing, and repetitive motor movements. These phenomena share one mathematical structure — **periodic functions** — and sine and cosine give us a parameter-rich model for capturing exactly how each one behaves: how big the swing is, how fast it repeats, and when it peaks.

Periodic Motion Before Calculus



In 1656, Christiaan Huygens invented the pendulum clock, the first timekeeping device accurate enough for science — built entirely on the periodicity of a swinging pendulum. Nine years later, confined to bed and watching two of his pendulum clocks mounted on the same wall, Huygens noticed something strange: no matter how he set them, the pendulums gradually swung into perfect opposite synchrony. Vibrations traveling through the shared wall were enough to lock the two periods together — the first documented observation of **synchronisation** between coupled oscillators, a phenomenon now central to everything from circadian entrainment by light to the synchronised flashing of fireflies.

11.2 The Geometry of Circles: Degrees and Radians

Why Radians Instead of Degrees?

Degrees (360° in a full turn) are an arbitrary human convention originating from ancient calendars. Radians measure angles directly by the arc length along the unit circle (2π radians = 360°). In calculus, the elegant derivative rules $\frac{d}{dx}[\sin x] = \cos x$ are **only true when x is measured in radians!**

Definition: Radian

One **radian** is the angle subtended at the centre of a circle by an arc equal in length to the radius. Since circumference = $2\pi r$, a full circle contains 2π radians:

$$2\pi \text{ radians} = 360^\circ.$$

Conversion Formulas

$$\theta_{\text{rad}} = \theta_{\text{deg}} \times \frac{\pi}{180}, \quad \theta_{\text{deg}} = \theta_{\text{rad}} \times \frac{180}{\pi}.$$

Degrees	0°	30°	45°	60°	90°	180°	270°	360°
Radians	0	$\pi/6$	$\pi/4$	$\pi/3$	$\pi/2$	π	$3\pi/2$	2π

Example: Converting 90° to radians

$$90^\circ \times \frac{\pi}{180} = \frac{90\pi}{180} = \frac{\pi}{2}.$$

Try it in Python

W11-CS01 — *Trig Imports & Setup*: bring in `numpy`, plotting tools.

W11-CS02 — *Degrees ↔ Radians*: convert angles in either direction.

11.3 The Unit Circle: Defining Trigonometric Functions

The right-triangle definitions still apply, but the unit circle removes the “acute angle only” restriction. The coordinates themselves now provide cosine and sine.

For a right triangle with angle θ , opposite side O , adjacent side A , hypotenuse H : $\sin \theta = O/H$, $\cos \theta = A/H$, $\tan \theta = O/A = \sin \theta / \cos \theta$.

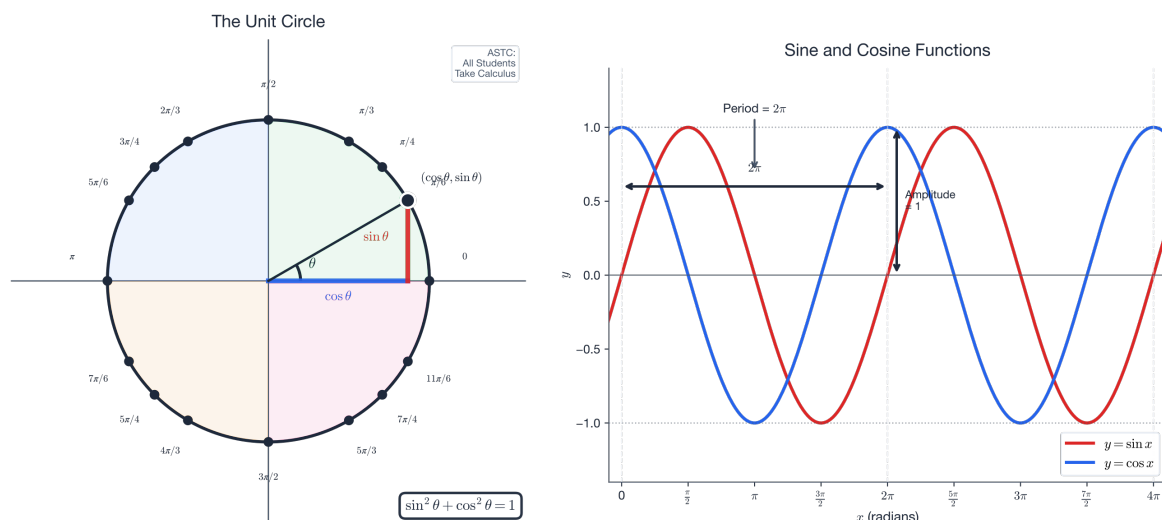
Definition: Unit circle definition

The **unit circle** has radius 1, centred at the origin. For any angle θ measured counter-clockwise from the positive x -axis, the point on the circle has coordinates $(\cos \theta, \sin \theta)$. This extends trigonometry beyond acute angles to all real numbers.

Fundamental Identity

$$\sin^2 \theta + \cos^2 \theta = 1$$

— immediate from the Pythagorean theorem applied to $x^2 + y^2 = 1$ on the unit circle.



θ	0	$\pi/6$	$\pi/4$	$\pi/3$	$\pi/2$
$\sin \theta$	0	$1/2$	$\sqrt{2}/2$	$\sqrt{3}/2$	1
$\cos \theta$	1	$\sqrt{3}/2$	$\sqrt{2}/2$	$1/2$	0
$\tan \theta$	0	$1/\sqrt{3}$	1	$\sqrt{3}$	undef

Memory aid for sine at $0, \pi/6, \pi/4, \pi/3, \pi/2$: $\frac{\sqrt{0}}{2}, \frac{\sqrt{1}}{2}, \frac{\sqrt{2}}{2}, \frac{\sqrt{3}}{2}, \frac{\sqrt{4}}{2}$.

Try it in Python

W11-CS03 — *Exact Values Table*: tabulate sin, cos, tan at key angles.

11.4 Signs in Quadrants: The ASTC Rule

Quad.	Angle range	sin	cos	tan	Memory
I	$0 \leq \theta < \pi/2$	+	+	+	All
II	$\pi/2 \leq \theta < \pi$	+	−	−	Sin
III	$\pi \leq \theta < 3\pi/2$	−	−	+	Tan
IV	$3\pi/2 \leq \theta < 2\pi$	−	+	−	Cos

ASTC = “All Students Take Calculus” (or “All Stations To Central”).

Symmetry relations (from unit-circle geometry): $\sin(\pi - \theta) = \sin \theta$, $\cos(\pi - \theta) = -\cos \theta$, $\sin(-\theta) = -\sin \theta$ (odd), $\cos(-\theta) = \cos \theta$ (even).

11.5 Graphs of Trigonometric Functions

$y = \sin x$ and $y = \cos x$ are smooth, wave-like curves with amplitude 1 and period 2π ; $\cos x = \sin(x + \pi/2)$, so cosine leads sine by $\pi/2$.

The General Sinusoidal Function

$$y = A \sin(B(x + C)) + D \quad \text{or} \quad y = A \cos(B(x + C)) + D.$$

Param.	Effect	Formula
A	amplitude	Amplitude = $ A $
B	angular frequency (cycle speed)	Period = $2\pi/ B $
C	phase shift (horizontal translation)	shift = $-C$ (left if $C > 0$)
D	vertical shift	midline at $y = D$

Frequency and Period

$$T = \frac{2\pi}{|B|}, \quad f = \frac{1}{T} = \frac{|B|}{2\pi}, \quad \omega = B = 2\pi f.$$

Example: Period and frequency

For $x(t) = \frac{7}{2} \cos(\pi t)$: period $T = 2\pi/\pi = 2$ s, frequency $f = 1/2 = 0.5$ Hz.

Try it in Python

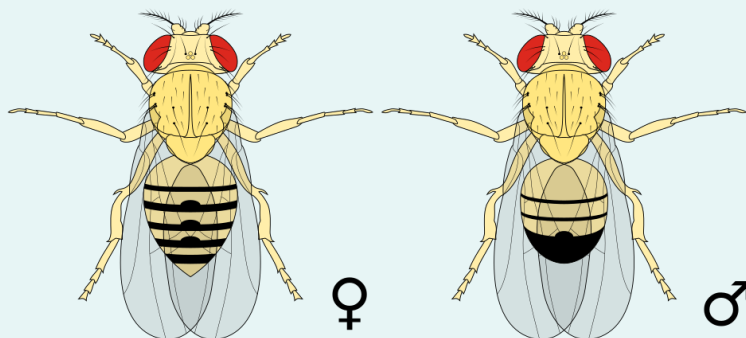
W11-CS04 — *Sine and Cosine Graphs*: basic plots over one period.

W11-CS05 — *Effect of A, B, C, D* : visualise amplitude, period, phase, vertical shift.

11.6 Modelling Periodic Phenomena: Parameter Fitting

Do not guess all four parameters at once. Read the vertical range first for A and D , then the repeat time for B , and only then use a known peak or trough for the phase shift.

The Biological Clock



Circadian rhythms are ≈ 24 -hour cycles driven by an internal molecular clock and synchronised by light. Body temperature typically bottoms out around 36.5°C near 4–5 AM and peaks near 37.2°C in late afternoon — a pattern affecting alertness, cognitive performance, and drug metabolism. The clock itself is genetic: *Drosophila* (fruit fly) mutants lacking a functional *period* gene lose their circadian rhythm entirely. Jeffrey Hall, Michael Rosbash, and Michael Young shared the 2017 Nobel Prize in Physiology or Medicine for discovering the molecular feedback loop — built from the same clock genes found in humans — that keeps this rhythm running. The molecular mechanism is biochemical, but its output is the same four-parameter wave you’re learning to fit this week.

Step-by-Step Model Fitting

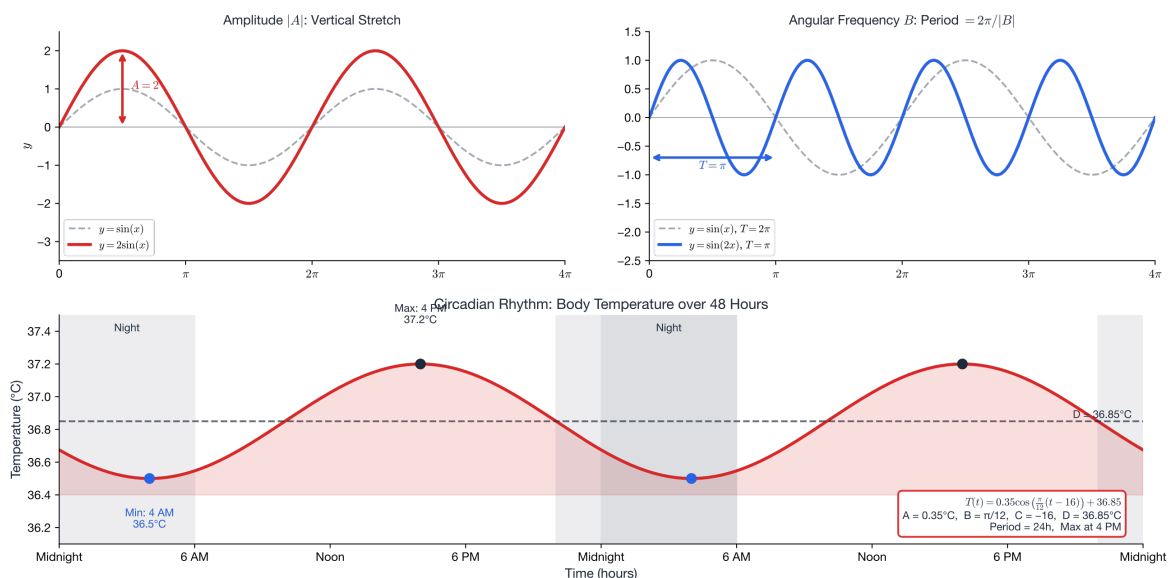
1. **Amplitude:** $A = \frac{\max - \min}{2}$.
2. **Vertical shift:** $D = \frac{\max + \min}{2} = \min + A$.
3. **Angular frequency:** $B = \frac{2\pi}{\text{period}}$.
4. **Phase shift:** requires knowing when a max or min occurs. For **cosine**, the maximum is at argument 0, so if the max occurs at $x = x_{\max}$, then $C = -x_{\max}$. For **sine**, the maximum is at argument $\pi/2$: solve $B(x_{\max} + C) = \pi/2$ for C .

Example: Pencil movement

A pencil moves between 2 cm and 12 cm, one round-trip every 2 s, and is at $x = 2$ cm (the minimum) at $t = 0$. Model: $x(t) = A \cos(B(t + C)) + D$.

- (1) $A = (12 - 2)/2 = 5$ cm. (2) $D = (12 + 2)/2 = 7$ cm. (3) $B = 2\pi/2 = \pi$ rad/s.
 (4) Cosine's minimum is at argument π : $B(0 + C) = \pi \Rightarrow \pi C = \pi \Rightarrow C = 1$.

Model: $x(t) = 5 \cos(\pi(t + 1)) + 7$. **Check:** $x(0) = 5 \cos(\pi) + 7 = 5(-1) + 7 = 2$. ✓



Try it in Python

W11-CS06 — *Fitting Sinusoidal Models*: recover A, B, C, D from data.

W11-CS07 — *Pencil Movement Example*: animate simple harmonic motion.

W11-CS10 — *Interactive Calculator*: tweak parameters and see the response.

11.7 Seasonal Temperature Modelling

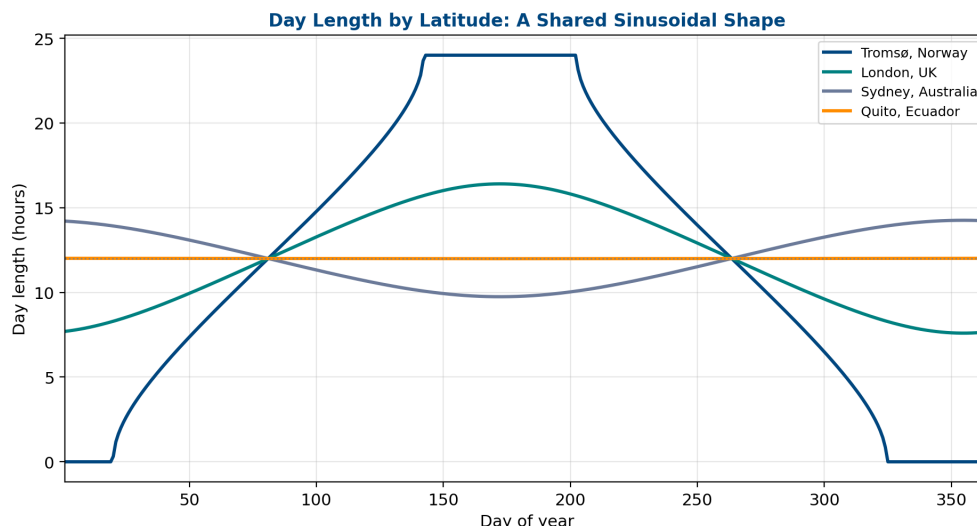
Annual temperature variation, driven by Earth's axial tilt, is nearly sinusoidal away from the equator. In the **Southern Hemisphere** (Perth, Wellington), July is coldest and January hottest; in the **Northern Hemisphere** (London, Edmonton), the pattern is reversed.

Example: Wellington Airport temperature

Data: max 17.9°C (Feb), min 9.5°C (Jul).

(1) $A = (17.9 - 9.5)/2 = 4.2^{\circ}\text{C}$. (2) $D = (17.9 + 9.5)/2 = 13.7^{\circ}\text{C}$. (3) $B = 2\pi/12 = \pi/6$ per month. (4) Coldest month July ($m = 7$); cosine's minimum is at argument π : $\frac{\pi}{6}(7 + C) = \pi \Rightarrow 7 + C = 6 \Rightarrow C = -1$.

Model: $T(m) = 4.2 \cos\left(\frac{\pi}{6}(m - 1)\right) + 13.7$.



The same underlying shape, $L(d) = A \sin\left(\frac{2\pi}{365}d + C\right) + 12$, describes day length by latitude, with amplitude A set entirely by how far a city sits from the equator. Quito (on the equator) has $A \approx 0$: 12 hours of daylight essentially every day. Tromsø, well inside the Arctic Circle, has $A \approx 12$: its curve swings from 24-hour *midnight sun* to complete *polar night*, visibly clipping flat at the top and bottom of the plot — a reminder that a pure sinusoid is an idealisation, and real systems can saturate at a physical limit the model doesn't capture. Sydney's curve has the same amplitude as London's but is shifted by half a period (C shifted by π): opposite hemisphere, opposite season.

Try it in Python

W11-CS08 — *Circadian Body Temperature*: daily temperature rhythm.

W11-CS09 — *Northern vs. Southern Hemisphere*: seasonal sinusoidal comparison.

11.8 Trigonometric Identities

The unit-circle definitions of sine and cosine have several algebraic consequences worth memorising outright, since they let you rewrite one trig expression as another without redrawing the triangle every time.

Fundamental Identities

$$\sin^2 \theta + \cos^2 \theta = 1, \quad \tan \theta = \frac{\sin \theta}{\cos \theta},$$

$$\sec \theta = \frac{1}{\cos \theta}, \quad \csc \theta = \frac{1}{\sin \theta}, \quad \cot \theta = \frac{1}{\tan \theta}.$$

Double-angle formulas (reference): $\sin(2\theta) = 2 \sin \theta \cos \theta$; $\cos(2\theta) = \cos^2 \theta - \sin^2 \theta = 2 \cos^2 \theta - 1 = 1 - 2 \sin^2 \theta$.

Example: Using the Pythagorean identity

If $\sin \theta = 0.6$ and θ is in Quadrant I, find $\cos \theta$. $\cos^2 \theta = 1 - 0.36 = 0.64$, so $\cos \theta = \pm 0.8$; since Quadrant I gives $\cos \theta > 0$, $\cos \theta = 0.8$.

Co-function Identities

Sine and cosine are the same wave, shifted by $\pi/2$:

$$\sin \theta = \cos\left(\theta - \frac{\pi}{2}\right), \quad \cos \theta = \sin\left(\theta + \frac{\pi}{2}\right).$$

When rewriting $A \sin(B(t - C)) + D$ as an equivalent cosine model (or vice versa), A , B , D are unchanged — only the phase shift C adjusts:

$$\sin \rightarrow \cos : C' = C + \frac{\pi}{2B}, \quad \cos \rightarrow \sin : C' = C - \frac{\pi}{2B}.$$

Example: Converting sine to cosine

Rewrite $2 \sin(\pi t - \frac{\pi}{3})$ as a cosine model. Here $B = \pi$, and $C = \frac{1}{3}$ since $B(t - C) = \pi(t - \frac{1}{3})$. Then $C' = \frac{1}{3} + \frac{\pi}{2\pi} = \frac{1}{3} + \frac{1}{2} = \frac{5}{6}$, so $2 \sin(\pi t - \frac{\pi}{3}) = 2 \cos(\pi t - \frac{5\pi}{6})$.

11.9 From One Wave to Many: Fourier's Insight

From Sine Waves to Tide Tables



Joseph Fourier
(1768–1830)



Kelvin's tide-predicting machine (1872)

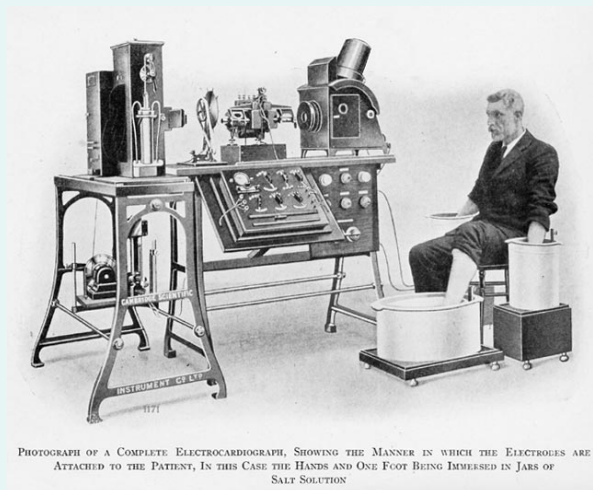
While studying heat conduction in 1822, Joseph Fourier made a startling claim: *any* periodic signal, however complex, can be decomposed into a sum of pure sine and cosine waves of different frequencies — $f(x) = \frac{a_0}{2} + \sum_{n=1}^{\infty} [a_n \cos(nx) + b_n \sin(nx)]$. His contemporaries were sceptical (Lagrange reportedly called it impossible); history proved Fourier right, and Fourier analysis now underlies MP3 compression, MRI imaging, seismology, and climate modelling.

Fifty years later, Lord Kelvin put the idea to work: tides result from the Moon's pull

($T \approx 12.4$ h), the Sun's pull ($T = 12$ h), and dozens of smaller harmonics superposed. Kelvin built a mechanical analogue computer — a machine of pulleys and gears, each turning at its own harmonic's frequency — that summed up to ten sinusoidal components simultaneously, producing accurate tide tables years in advance. A descendant of that machine helped plan the Allied landings at Normandy on 6 June 1944: the low-tide window at dawn, critical for clearing beach obstacles, was predicted from exactly this kind of periodic model. Every function you fit this week — one sine wave — is the first term of a Fourier series; real signals are usually a superposition of several.

11.10 Application: Reading the Heart's Rhythm

The ECG: A Physiological Waveform



In 1903, **Willem Einthoven** used a string galvanometer to record the heart's electrical activity, producing the first electrocardiogram (ECG) — work that earned him the 1924 Nobel Prize. Each heartbeat traces a characteristic waveform, the **PQRST complex**, and deviations from its normal shape are diagnostic: a prolonged QT interval signals arrhythmia risk, ST elevation can indicate a heart attack, and a missing P wave suggests atrial fibrillation. The same idea extends to the brain: Hans Berger recorded the first EEG in 1924, and brain states are classified by the frequency band of their electrical oscillations ($\delta < 4$ Hz, θ 4–8 Hz, α 8–13 Hz, β 13–30 Hz, $\gamma > 30$ Hz). In both cases, a periodic waveform — not just its average level — carries the diagnostic information, which is exactly why amplitude, period, and phase all matter, not just the midline D .

Activity: Part 1 — Fit the Perth model

1. Read off the maximum and minimum temperatures, and the month each occurs in.
2. Compute A , D , and B (period = 12 months).
3. Use the coldest month to find the phase shift C for $T(t) = A \cos(B(t - C)) + D$.
4. Write the complete model and check it against one data point.

```
# 1. Max = _____ (month _____)   Min = _____ (month
    _____)
# 2. A = _____   D = _____   B = _____
# 3. C = _____
# 4. T(t) = _____
# _____
#   Check at t = _____: T(_____) = _____ (matches data?
    _____)
```

11.12.2 Part 2 — Solve Your Assigned Scenario Card

Each group is assigned one of seven scenario cards — tidal heights, average temperature, daylight hours, heart rate during exercise, indoor CO₂, ocean wave height, or blood-oxygen monitoring — each giving two extreme values, a period, and the time of one extreme. Two cards, for a sense of the range:

Card	Data
Heart rate during exercise	peak 165 bpm, trough 95 bpm; full cycle every 5 min; peak at $t = 1.25$ min
Ocean wave height	max 2.4 m, min 0.4 m; period 10 s; first crest at $t = 2.5$ s

Activity: Part 2 — Solve your card

1. Identify the quantity, time variable, and units.
2. Derive A , D , the period $P = 2\pi/B$, and C .
3. Write the complete model (sine or cosine accepted).
4. Answer both applied questions on your card, showing working.

```
Card: _____
A = _____   B = _____   C = _____   D = _____
Model: f(t) = _____
Q1: _____
Q2: _____
```

11.12.3 Part 3 — Verify the Next Group's Work

Open the next group's notebook and their card. For every part, check whether A , B , C , D , the model, both answers, and the graph are correct — and record each as *confirmed* or *corrected*, with a reason.

Activity: Part 3 — Verification record

Reviewed group: _____ Card: _____

A, B, C, D: _____

Model and units: _____

First question: _____

Second question: _____

Code and graph: _____

Activity: Part 4 — Debrief

Return your verification record to the group you checked, resolve any disagreements by referring to the card, and record the discussion.

One modelling insight from today: _____

One issue to avoid next time: _____

11.13 Week 11 Summary**Key Formulas**

Degree–radian conversion	$\theta_{\text{rad}} = \theta_{\text{deg}} \times \pi/180$
Pythagorean identity	$\sin^2 \theta + \cos^2 \theta = 1$
Tangent	$\tan \theta = \sin \theta / \cos \theta$
General sinusoid	$y = A \sin(B(x + C)) + D$
Amplitude	$ A = (\text{max} - \text{min})/2$
Period	$T = 2\pi/ B $
Frequency	$f = 1/T = B /2\pi$
Vertical shift	$D = (\text{max} + \text{min})/2$
Phase shift	solved from a boundary condition
Co-function identities	$\sin \theta = \cos(\theta - \pi/2)$; $\cos \theta = \sin(\theta + \pi/2)$
Sin→Cos phase	$C' = C + \pi/(2B)$
Cos→Sin phase	$C' = C - \pi/(2B)$

11.14 Before Week 12: Self-Check

Before moving on, make sure you can:

- convert fluently between degrees and radians;
- state the unit-circle definitions of sine and cosine, and recite the exact values at the five standard angles 0 through $\pi/2$;

- apply the ASTC rule to determine the sign of a trig function in any quadrant;
- identify amplitude, period, frequency, phase shift, and vertical shift from a sinusoidal formula or a graph;
- fit a full sinusoidal model A, B, C, D to real periodic data, given two extremes, a period, and the timing of one extreme;
- apply the Pythagorean and co-function identities, including converting between sine and cosine forms.

If any of these feel shaky, revisit the relevant section above — or the corresponding activity in §11.10 — before Week 12 begins, where the course turns from single periodic models to optimising decisions under multiple simultaneous constraints.

11.15 Exercises

These exercises mirror the topics and difficulty of the Week 11 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

- E1. Radian conversion.** Convert 150° to radians, and $\frac{5\pi}{4}$ to degrees.
- E2. Exact values.** Find $\sin(\pi/3)$, $\cos(\pi/4)$, and $\tan(\pi/6)$ exactly.
- E3. Quadrant signs.** In which quadrant is θ if $\sin \theta < 0$ and $\cos \theta > 0$?
- E4. Pythagorean identity.** If $\cos \theta = -0.6$ and θ is in Quadrant II, find $\sin \theta$.
- E5. Amplitude and period.** For $y = 3 \sin(4x)$, state the amplitude and period.
- E6. Frequency.** For $y(t) = 2 \cos(6\pi t)$, find the period and frequency in Hz.
- E7. Vertical and phase shift.** For $y = 5 \cos(2(x - 3)) + 8$, state the midline and phase shift.
- E8. Model fitting (cosine).** A quantity oscillates between 4 and 20, period 8, with a maximum at $x = 2$. Find A, B, C, D for $y = A \cos(B(x + C)) + D$.
- E9. Model fitting (temperature).** A city has max temperature 24°C in January (month 1) and min 8°C in July (month 7). Find the cosine model $T(m) = A \cos(B(m + C)) + D$.
- E10. Co-function conversion.** Rewrite $3 \sin(2t - \frac{\pi}{4})$ as an equivalent cosine model.
- E11. Tidal application.** Tide height follows $h(t) = 1.2 \sin(\frac{\pi}{6}t) + 2.0$ metres, t in hours. Find the height at $t = 3$ and the period.
- E12. Heart rate.** Heart rate oscillates between 65 and 145 bpm over a 4-minute cycle, with a minimum at $t = 0$. Find the cosine model $H(t)$.

Answers

E1: $150^\circ = 5\pi/6$; $5\pi/4 = 225^\circ$.

E2: $\sin(\pi/3) = \sqrt{3}/2$; $\cos(\pi/4) = \sqrt{2}/2$; $\tan(\pi/6) = 1/\sqrt{3}$.

E3: Quadrant IV.

E4: $\sin^2 \theta = 1 - 0.36 = 0.64 \Rightarrow \sin \theta = \pm 0.8$; Quadrant II gives $\sin \theta > 0$, so $\sin \theta = 0.8$.

E5: amplitude 3; period $2\pi/4 = \pi/2$.

E6: period $2\pi/6\pi = 1/3$; frequency $= 1/(1/3) = 3$ Hz.

E7: midline $y = 8$; phase shift $= 3$ (right, since $C = -3$ in $A \cos(B(x + C)) + D$ form).

E8: $A = (20 - 4)/2 = 8$; $D = (20 + 4)/2 = 12$; $B = 2\pi/8 = \pi/4$; max at $x = 2 \Rightarrow C = -2$.

E9: $A = (24 - 8)/2 = 8$; $D = (24 + 8)/2 = 16$; $B = 2\pi/12 = \pi/6$; min at $m = 7$: $\frac{\pi}{6}(7 + C) = \pi \Rightarrow C = -1$.

E10: $B = 2$, $C = \pi/8$ (since $2t - \pi/4 = 2(t - \pi/8)$); $C' = \pi/8 + \pi/4 = 3\pi/8$; so $3 \sin(2t - \pi/4) = 3 \cos(2t - 3\pi/4)$.

E11: $h(3) = 1.2 \sin(\pi/2) + 2.0 = 1.2 + 2.0 = 3.2$ m; period $= 2\pi/(\pi/6) = 12$ h.

E12: $A = (145 - 65)/2 = 40$; $D = (145 + 65)/2 = 105$; $B = 2\pi/4 = \pi/2$; min at $t = 0$: $\frac{\pi}{2}(0 + C) = \pi \Rightarrow C = 2$; $H(t) = 40 \cos(\frac{\pi}{2}(t + 2)) + 105$.

Chapter 12

Simultaneous Equations and Linear Programming

WEEK 12

We conclude our semester journey with the ultimate goal of quantitative science: making optimal management decisions under strict real-world constraints. Whether allocating land between conservation and agriculture, or balancing nutrient feeds under budget constraints, **Linear Programming** translates goals and limitations into geometric regions to find the single best choice. *Theme: “Optimising with Constraints”*

“The real world rarely offers unlimited resources. Optimisation under constraints is the mathematician’s answer to scarcity — and the scientist’s tool for making the best possible decision.”

12.1 Why Optimisation Under Constraints?

Every week so far has built a piece of mathematical machinery for *describing* a system: linear and exponential trends (Weeks 1–2), rates of change and critical points (Weeks 3–5), accumulation (Weeks 6–7), interacting populations and uncertainty (Weeks 8–10), and periodic cycles (Week 11). This final week turns description into **decision**: given a system with limited resources, what is the *best* choice available?

Constrained optimisation appears across every scientific discipline:

Domain	Decision problem	Constraints
Nutrition	design a cost-effective diet	minimum nutrient requirements
Agriculture	allocate land to crops	available land, water, labour
Healthcare	assign staff to shifts	coverage requirements, labour rules
Ecology	allocate conservation funds	budget, species priorities
Economics	maximise profit	production capacity, demand
Engineering	design a structure	material strength, weight limits

The mathematical framework we develop — **linear programming (LP)** — was pioneered for military logistics during the Second World War and has since underpinned operations research, economics, and computational science.

From Airlift Schedules to the Simplex Method

In 1947, American mathematician George Dantzig, working for the U.S. Air Force, formalised the problem of allocating scarce resources — fuel, aircraft, personnel — among competing activities as a system of linear inequalities with a linear objective, and invented the **simplex method** to solve it. He initially called the technique

“programming in a linear structure”; *programming* here means *planning* or *scheduling*, decades before the word came to mean writing computer code. Dantzig’s method scales to problems with millions of variables and now runs quietly behind airline crew rostering, oil-refinery blending, telecommunications routing, and — as you will see this week — the humble problem of designing an affordable, low-carbon meal.

12.2 Simultaneous Linear Equations

Many scientific problems require values that satisfy several conditions *at once*. This means solving a **system of linear equations**.

Definition: System of linear equations

A system of linear equations is a collection of two or more linear equations in the same variables. Its **solution** is the set of values satisfying every equation simultaneously.

For two equations in two unknowns,

$$a_1x + b_1y = c_1, \quad a_2x + b_2y = c_2,$$

each equation is a line; geometrically the solution is where the lines **intersect**. Three outcomes are possible: a unique solution (lines cross once — the usual case), no solution (parallel lines — an *inconsistent* system), or infinitely many solutions (identical lines — a *dependent* system).

Two solution methods:

- **Substitution:** solve one equation for one variable, substitute into the other, solve, then back-substitute.
- **Elimination:** scale the equations so one variable’s coefficients match in magnitude, then add or subtract to cancel it.

Example: Solving by substitution and elimination

Solve $x + 2y = 7$ and $3x - y = 11$.

1. **Substitution:** from the first equation, $x = 7 - 2y$. Substituting: $3(7 - 2y) - y = 11 \Rightarrow 21 - 7y = 11 \Rightarrow y = \frac{10}{7}$, so $x = 7 - 2(\frac{10}{7}) = \frac{29}{7}$.

2. **Elimination:** double the second equation to $6x - 2y = 22$; adding to the first gives $7x = 29 \Rightarrow x = \frac{29}{7}$, then $y = \frac{10}{7}$ from either original equation.

Solution: $(\frac{29}{7}, \frac{10}{7}) \approx (4.14, 1.43)$ — both methods agree, as they must. ✓

Try it in Python

W12-CS01 — *Simultaneous Equations with NumPy:* solve $A\mathbf{x} = \mathbf{b}$ with `np.linalg.solve`.

W12-CS02 — *Two Lines and Their Intersection:* plot both lines and mark the solution point.

12.3 Market Equilibrium: Supply and Demand

A natural home for simultaneous equations is the competitive market: **demand** falls as price rises, **supply** rises as price rises, and **equilibrium** is the one price at which the quantity consumers want to buy equals the quantity producers want to sell.

Linear Demand and Supply

$$Q_d = a - bP \quad (\text{demand, } b > 0), \quad Q_s = -c + dP \quad (\text{supply, } d > 0),$$

where Q is quantity and P is price. **Equilibrium** solves $Q_d = Q_s$ for (P^*, Q^*) .

Example: Finding equilibrium

$Q_d = 100 - 2P$, $Q_s = -100 + 3P$. Setting $Q_d = Q_s$: $100 - 2P = -100 + 3P \Rightarrow 200 = 5P \Rightarrow P^* = 40$, and $Q^* = 100 - 2(40) = 20$.

The **maximum willingness to pay** is the price at which $Q_d = 0$: $P = 50$. The **minimum acceptable price** is where $Q_s = 0$: $P = 100/3 \approx 33.33$. Both bracket the equilibrium price, as they must for a market to clear.

Try it in Python

W12-CS03 — *Market Equilibrium*: solve $Q_d = Q_s$ and plot both curves with the equilibrium point marked.

12.4 Consumer and Producer Surplus (Revisited)

Week 7 introduced **consumer surplus (CS)** and **producer surplus (PS)** as integrals under demand and supply curves; market equilibrium now supplies the missing piece — the point (P^*, Q^*) that fixes the limits of integration.

Surplus at Equilibrium

With inverse demand $P = D(Q)$ and inverse supply $P = S(Q)$:

$$CS = \int_0^{Q^*} D(Q) \, dQ - P^* Q^*, \quad PS = P^* Q^* - \int_0^{Q^*} S(Q) \, dQ.$$

For **linear** demand and supply, both areas are triangles:

$$CS = \frac{1}{2} Q^* (P_{\max} - P^*), \quad PS = \frac{1}{2} Q^* (P^* - P_{\min}).$$

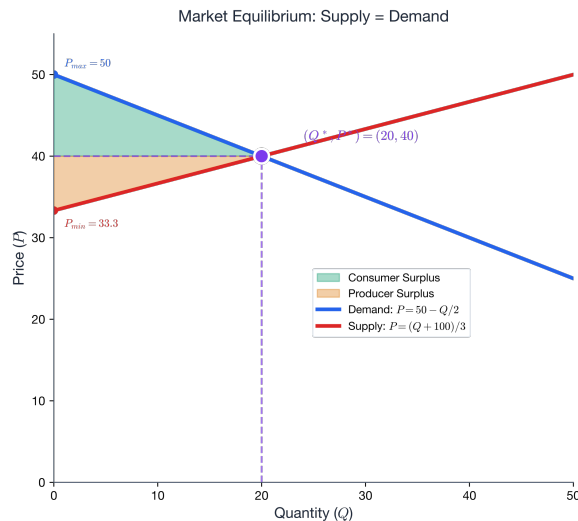
Example: Surplus for the equilibrium above

Continuing $Q_d = 100 - 2P$, $Q_s = -100 + 3P$, with $P^* = 40$, $Q^* = 20$. Inverting: $D(Q) = 50 - \frac{Q}{2}$ and $S(Q) = \frac{Q}{3} + \frac{100}{3}$.

$$CS = \int_0^{20} \left(50 - \frac{Q}{2}\right) \, dQ - 40(20) = \left[50Q - \frac{Q^2}{4}\right]_0^{20} - 800 = 900 - 800 = 100.$$

$$PS = 40(20) - \int_0^{20} \left(\frac{Q}{3} + \frac{100}{3}\right) \, dQ = 800 - \left[\frac{Q^2}{6} + \frac{100Q}{3}\right]_0^{20} = 800 - \frac{2200}{3} \approx 66.67.$$

Triangle-shortcut check: $CS = \frac{1}{2}(20)(50 - 40) = 100 \checkmark$, $PS = \frac{1}{2}(20)(40 - \frac{100}{3}) = \frac{200}{3} \approx 66.67 \checkmark$.



Economic Surplus Calculations

Consumer Surplus (CS)

Money saved by consumers

$$CS = \int_0^{Q^*} D(Q) dQ - P^* \cdot Q^*$$

$$= \int_0^{20} (50 - \frac{Q}{2}) dQ - 40 \times 20 = 900 - 800 = 100$$

Producer Surplus (PS)

Extra profit above minimum price

$$PS = P^* \cdot Q^* - \int_0^{Q^*} S(Q) dQ$$

$$= 40 \times 20 - \int_0^{20} (\frac{Q+100}{3}) dQ = 800 - 733.3 = 66.7$$

Total Economic Surplus

$$TS = CS + PS = 100 + 66.7 = 166.7$$

Maximized at competitive equilibrium!

Try it in Python

W12-CS04 — *CS/PS via Triangle Formulas*: the quick area shortcut.**W12-CS05** — *CS/PS via Integration*: the general method using `scipy.integrate.quad`.

12.5 Linear Programming: The Framework

Translate the story in three passes: choices become variables, the goal becomes the objective, and every limited resource or requirement becomes a constraint.

Market equilibrium finds one balance point. **Linear programming** goes further: it finds the *best* point among infinitely many that satisfy a whole system of linear requirements at once.

Definition: LP problem — standard form

$$\text{Maximise (or minimise) } Z = c_1x + c_2y$$

subject to linear constraints $a_{i1}x + a_{i2}y \{\leq, =, \geq\} b_i$ for $i = 1, \dots, m$, together with the non-negativity conditions $x \geq 0, y \geq 0$. Here x, y are **decision variables**, Z is the **objective function**, and each inequality is a **constraint**.

Why Algebra — Not Calculus

The objective function is linear, so it has constant slope everywhere — its derivative is a constant, never zero, so ordinary critical-point calculus finds nothing in the interior of the feasible region. The optimum is instead forced onto the *boundary* of the region, and among boundary points it is forced further, onto a *corner*: moving along any edge, Z changes monotonically, so the best value on that edge is always found at one of its two endpoints. This is the **Corner Point Theorem**, and it is why two-variable LP is solved with algebra (intersecting lines) rather than differentiation.

Corner Point Theorem

If a linear programming problem has an optimal solution, it occurs at a *vertex* (corner point) of the feasible region.

Solution procedure:

1. Define the decision variables, with units.
2. Write the objective function (maximise or minimise?).
3. Write every constraint as a linear inequality, including non-negativity.
4. Graph the constraints and shade the feasible side of each to identify the **feasible region**.
5. Find all corner points by solving pairs of boundary equations simultaneously.
6. Evaluate Z at every corner point.
7. Select the optimal corner and interpret the result in context.

12.6 Solving LP Graphically: Sustainable Campus Dining

Keep two jobs separate: first find points that are feasible, then compare their objective values. A low-cost point that breaks a requirement is not a candidate solution.

Scenario

A **university dining hall** wants to design a minimum-cost, low-carbon bowl by combining two base ingredients: a **plant-based** option and a **chicken** option. Let x = serves of the plant-based option and y = serves of the chicken option.

	Plant-based (x)	Chicken (y)	Requirement
Cost (\$/serve)	2.50	4.00	minimise
Protein (g/serve)	18	30	≥ 350
Energy (kcal/serve)	220	280	≥ 3500
Iron (mg/serve)	3	1	≥ 25
Carbon footprint (kg CO ₂ -e/serve)	0.5	2.5	≤ 20

Step 1 — Formulate.

$$\text{Minimise } Z = 2.5x + 4y$$

$$18x + 30y \geq 350 \text{ (protein), } 220x + 280y \geq 3500 \text{ (calories), } 3x + y \geq 25 \text{ (iron),}$$

$$0.5x + 2.5y \leq 20 \text{ (carbon), } x \geq 0, y \geq 0.$$

Step 2 — Graph and find corner points. Converting each boundary to an equation and solving pairs simultaneously, three points bound the feasible region — every other intersection either falls outside a constraint or is dominated:

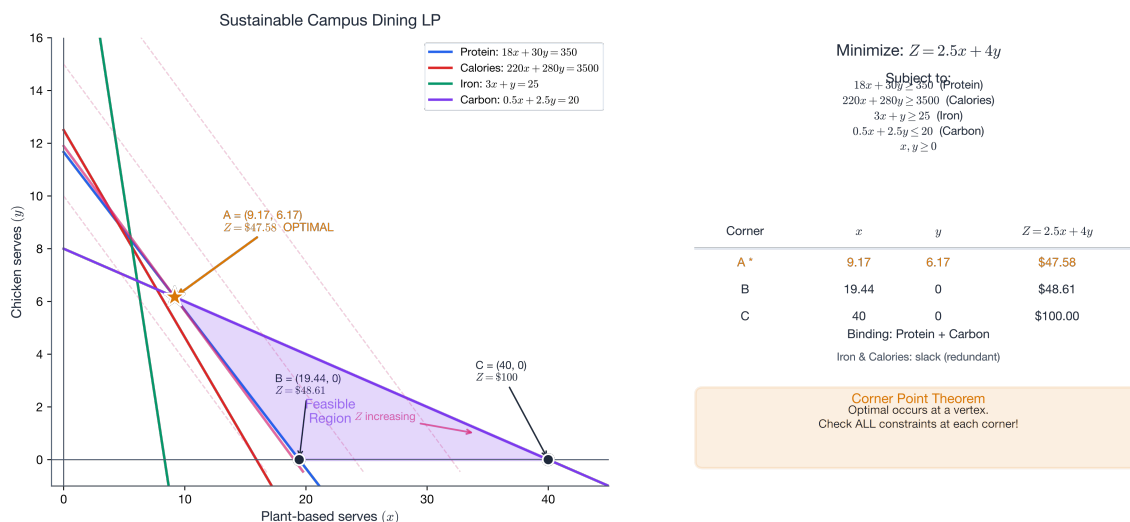
Corner	x	y	Binding constraints
A	9.17	6.17	protein $\cap$ carbon
B	19.44	0	protein $\cap$ ($y = 0$)
C	40	0	carbon $\cap$ ($y = 0$)

The iron and calorie constraints pass *outside* this triangle — they are satisfied everywhere the protein and carbon constraints already hold, so they never bind. This is a common and important finding in real LP problems: not every stated requirement ends up shaping the answer, and part of the solution process is discovering which constraints are actually doing the work.

Step 3 — Evaluate the objective.

Corner	x	y	$Z = 2.5x + 4y$
A	9.17	6.17	\$47.58
B	19.44	0	\$48.61
C	40	0	\$100.00

Step 4 — Optimal solution. The minimum is $Z^* = \$47.58$ at corner A: **9.17 plant-based serves and 6.17 chicken serves per patron per day**, with the protein and carbon-footprint constraints binding exactly and the calorie and iron requirements comfortably exceeded.

**Sensitivity: What if the Carbon Cap Tightens?**

Suppose campus sustainability policy tightens the carbon cap from 20 to 12 kg CO₂-e. The carbon constraint line $0.5x + 2.5y = 12$ now cuts further into the feasible region, moving corner A to the new intersection of protein and carbon: solving $18x + 30y = 350$ and $0.5x + 2.5y = 12$ simultaneously gives $x \approx 17.17$, $y \approx 1.37$, with $Z^* \approx \$48.38$. The composition shifts toward relatively *more* of the lower-carbon plant-based option — exactly the qualitative direction sustainability policy intends — and the cost of doing so is a higher minimum price, a direct, quantified trade-off between environmental and budget constraints. This is **sensitivity analysis**: re-solving the same LP after one parameter changes, to see how the optimal decision and its cost respond.

Try it in Python

W12-CS06 — *LP Feasible Region*: shade the constraint set for a diet-style problem.

W12-CS07 — *Corner Points Evaluation*: test every candidate vertex against all constraints.

W12-CS08 — *Sensitivity Analysis*: visualise how the optimum shifts as a parameter changes.

W12-CS09 — *LP with `scipy.optimize.linprog`*: verify the graphical solution with a production-grade solver.

12.7 LP Solution Algorithm and Special Cases

Step	Action
1	Define decision variables
2	Write the objective function (max or min?)
3	Write constraints as linear inequalities
4	Graph constraints and identify the feasible region
5	Find corner points by solving simultaneous equations
6	Evaluate the objective at each corner
7	Select the optimal corner and interpret

Three things can go wrong with this recipe, and each is diagnostic rather than a dead end:

- **Unbounded solution:** the feasible region stretches to infinity in the direction that improves Z — no finite optimum exists. This usually signals a missing constraint.
- **Multiple optimal solutions:** the objective function's gradient is parallel to a binding constraint edge — every point on that whole edge, not just one vertex, achieves the optimal Z .
- **Infeasible problem:** the constraints contradict one another and no feasible region exists at all — e.g. a minimum requirement that exceeds what a budget constraint permits.

12.8 Applications Beyond Diet Optimisation

The same seven-step recipe applies unchanged across disciplines — only the story behind x , y , and the constraints changes.

Example: Agricultural land allocation

A farmer has 100 ha and 400 labour-hours. Wheat returns \$300/ha profit and needs 3 hours/ha; corn returns \$400/ha and needs 5 hours/ha. Let x, y = hectares of wheat, corn.

$$\text{Maximise } Z = 300x + 400y \quad \text{s.t.} \quad x + y \leq 100, \quad 3x + 5y \leq 400, \quad x, y \geq 0.$$

Example: Conservation budgeting

An agency must protect at least 200 km² of habitat using land acquisition (\$50/km², 80% effective) and conservation easements (\$20/km², 40% effective), on a \$6000 budget. Let x, y = km² acquired, under easement.

$$\text{Maximise } Z = 0.8x + 0.4y \quad \text{s.t.} \quad 50x + 20y \leq 6000, \quad x + y \geq 200, \quad x, y \geq 0.$$

Healthcare staffing follows the identical structure: minimise wage cost subject to minimum shift-coverage, acuity, and skill-mix constraints — precisely the shape of one of this week's lab scenarios (§12.10.4).

12.9 Connection to Previous Weeks

Week	Concept	Connection to Week 12
1	Linear functions	Every objective and constraint is linear
5	Optimisation	LP extends unconstrained (calculus) optimisation to the constrained case
7	Definite integrals; simultaneous equations	CS/PS via integration; finding vertices of the feasible region
11	Periodic models	Seasonal or time-of-day constraints (harvest windows, energy pricing) feed directly into LP models

As §12.5 explained, LP is solved with **algebra** rather than **calculus**: a linear objective has no stationary points, so the corners of the feasible region are where the action is. Genuinely *nonlinear* constrained optimisation (Lagrange multipliers, and beyond that convex and non-convex programming) is the natural next step for more advanced courses — and for the multi-tool, cross-disciplinary modelling many of you will do in research.

12.10 Python Lab: A Guided Walkthrough

The Week 12 lab (`Week12_Lab_STUDENT.ipynb`), “*Linear Programming*,” runs in four stages, timed at roughly 35, 45, 15, and 5 minutes: a shared warmup scenario solved by the whole class (itself worked in three parts — formulate, graph, solve), an assigned scenario card worked through in depth by each group, a **whole-class discussion** in which the demonstrator confirms each group’s solution (there is no peer hand-off this week — LP write-ups benefit from expert confirmation rather than peer cross-checking), and a short debrief.

12.10.1 Part 1–3 — Warmup Scenario (Everyone)

Every group works the *Sustainable Campus Dining* problem of §12.7 together: formulate the LP, graph the feasible region, and apply the Corner Point Theorem to find the optimal bowl.

Activity: Warmup — Formulate and Solve

1. Write the objective function and all four constraints, including non-negativity.
2. Convert each constraint to an equation and find its two axis intercepts.
3. Identify which pairs of constraints intersect *inside* the feasible region — these are the corner points.
4. Evaluate Z at each corner and report the optimal composition and cost.

Objective: $Z =$ _____
 Constraints: _____
 Corner points **and** Z values: _____
 Optimal solution: $(x^*, y^*) =$ _____ $Z^* =$ _____

12.10.2 Part 4 — Your Assigned LP Problem

Each group is assigned one of three scenario cards — crop allocation, hospital staffing, or biodiversity restoration — and works through the full LP workflow independently, including a stated sensitivity change.

Card	Scenario
B — Crop allocation	100 ha for wheat (x) and chickpeas (y); water and labour caps; an agronomic minimum chickpea-to-wheat ratio; a contracted minimum wheat area. Maximise gross margin ($\$400x + \$300y$); sensitivity: water licence cut from 240 to 210 ML.
C — Hospital staffing	Nurse (x) and technician (y) shifts; minimum total coverage, patient-acuity, and technical-task requirements; a supervision ratio cap; a contracted maximum on nurse shifts. Minimise wage-index cost ($8x + 6y$); sensitivity: acuity requirement raised by a flu outbreak.
D — Biodiversity corridor	Wetland (x) and forest (y) restoration on up to 80 ha; a shared budget and water allowance; an ecological minimum forest-to-wetland ratio; a minimum wetland area for a threatened water-bird. Maximise biodiversity score ($5x + 3y$); sensitivity: restoration budget cut by a third.

Activity: Part 4 — Solve your card

1. Define decision variables (with units) and the objective function.
2. Write all constraints, including non-negativity.
3. Graph the feasible region and list every feasible corner point.
4. Evaluate the objective at each corner; state the optimal solution and which constraints bind.
5. Apply the card's sensitivity change and re-solve.
6. Write a 2–3 sentence recommendation based on both results.

```

Card: _____ Variables: x = _____ y = _____
Objective: Z = _____
Constraints: 1. _____ 2. _____ 3. _____ 4. _____
Optimal: (x*, y*) = _____ Z* = _____ Binding: _____
Sensitivity result: _____
Recommendation: _____

```

12.10.3 Part 5 — Class Discussion and Instructor Confirmation

There is no circular peer-verification this week. Instead, the demonstrator calls on groups to present their assigned card — decision variables, objective, constraints, optimal solution and binding constraints, sensitivity result, and recommendation — and confirms each answer, flagging common errors (a frequent one: forgetting to check that a candidate “corner” actually satisfies *every* constraint, not just the two used to find it).

12.10.4 Debrief

Activity: Debrief

```

LP feature we understood well:
-----
Correction or check that mattered:
-----
One feasibility/sensitivity issue to avoid next time:
-----

```

Try it in Python

W12-CS10 — *Complete Market/LP Dashboard*: an interactive end-to-end visualisation bringing equilibrium, surplus, and LP together.

12.11 Week 12 Summary

Key Formulas

Market equilibrium	solve $Q_d(P) = Q_s(P)$ for P^* , then Q^*
Consumer surplus	$CS = \int_0^{Q^*} D(Q) dQ - P^*Q^*$
Producer surplus	$PS = P^*Q^* - \int_0^{Q^*} S(Q) dQ$
CS/PS shortcut (linear)	$CS = \frac{1}{2}Q^*(P_{\max} - P^*)$; $PS = \frac{1}{2}Q^*(P^* - P_{\min})$
LP standard form	$Z = c_1x + c_2y$, s.t. linear constraints and $x, y \geq 0$
Corner Point Theorem	optimal Z occurs at a vertex of the feasible region
Unbounded LP	feasible region extends to infinity in the improving direction
Infeasible LP	constraints admit no common region

12.12 Before the Exam: Self-Check

This is the final content week of SCIE1500. Before moving to revision, make sure you can:

- solve a 2×2 system of linear equations by both substitution and elimination, and interpret geometrically why a system may have one, none, or infinitely many solutions;
- set up linear demand and supply functions, solve for market equilibrium (P^*, Q^*) , and find the maximum willingness to pay and minimum acceptable price;
- compute consumer and producer surplus by integration *and* by the linear-case triangle shortcut, and confirm the two agree;
- formulate a linear programming problem from a word description — decision variables, objective, and constraints, including non-negativity;
- graph a feasible region, identify every corner point (discarding intersections that violate another constraint), and apply the Corner Point Theorem to find the optimum;
- recognise and explain unbounded, multiply-optimal, and infeasible LP outcomes;
- carry out a simple sensitivity analysis by re-solving an LP after one parameter changes, and interpret the result in context.

Optimisation under constraints closes the course's four-act arc from observation to decision: having learned to describe systems, measure accumulation, and predict interactions, you now

have the tools to choose — rigorously, and with an explicit account of what is traded off — among the options a constrained world actually allows.

12.13 Exercises

These exercises mirror the topics and difficulty of the Week 12 diagnostic quiz but use fresh numbers, so you can use one to learn and the other to self-test.

E1. Simultaneous equations (substitution). Solve $x + y = 10$ and $2x - y = 5$.

E2. Simultaneous equations (elimination). Solve $3x + 2y = 12$ and $x - 2y = 4$.

E3. Market equilibrium. Given $Q_d = 80 - 4P$ and $Q_s = -20 + 2P$, find the equilibrium price and quantity.

E4. Willingness to pay. Given $Q_d = 100 - 2P$, find the maximum price any consumer would pay.

E5. Minimum acceptable price. Given $Q_s = -100 + 3P$, find the minimum price at which producers would supply.

E6. Surplus by integration. Evaluate $\int_0^{20} \left(50 - \frac{Q}{2}\right) dQ$.

E7. Feasible-region shading. For the constraint $2x + 3y \geq 12$, which side of the line $2x + 3y = 12$ is feasible?

E8. Finding a vertex. Find the intersection of $4x + 5y = 100$ and $y = 5$.

E9. Evaluating the objective. Evaluate $Z = 2x + 3y$ at the point $(10, 12)$.

E10. LP formulation. A diet uses x units of Food I (\$2/unit, 4 g protein/unit) and y units of Food II (\$3/unit, 5 g protein/unit); the minimum protein requirement is 100 g. Write the cost objective and the protein constraint.

E11. Diet problem corners. For the LP Minimise $Z = 2x + 3y$ subject to $4x + 5y \geq 100$, $2x + y \geq 25$, $y \geq 5$, $x, y \geq 0$, the feasible region's corner points are $B(18.75, 5)$, $C(25/6, 50/3)$, and $E(0, 25)$. Evaluate Z at each and state the optimal diet.

E12. Sensitivity. For the LP in E11, if Food I's unit cost rises from \$2 to \$5 (objective becomes $Z = 5x + 3y$), does the optimal corner point change? Does the minimum cost change?

Answers

E1: Adding the equations after doubling the first: $2x + 2y = 20$, $2x - y = 5$; subtracting gives $3y = 15 \Rightarrow y = 5$, $x = 5$. Solution $(5, 5)$.

E2: Adding directly eliminates y : $4x = 16 \Rightarrow x = 4$; then $3(4) + 2y = 12 \Rightarrow y = 0$. Solution $(4, 0)$.

E3: $80 - 4P = -20 + 2P \Rightarrow 100 = 6P \Rightarrow P^* = 16.67$; $Q^* = 80 - 4(16.67) = 13.33$.

E4: $100 - 2P = 0 \Rightarrow P = 50$.

E5: $-100 + 3P = 0 \Rightarrow P = 33.33$.

E6: $\left[50Q - \frac{Q^2}{4}\right]_0^{20} = 1000 - 100 = 900$.

E7: The side *away* from the origin (the side containing points like $(10, 10)$, where $2(10) + 3(10) = 50 \geq 12$).

E8: $4x + 25 = 100 \Rightarrow x = 18.75$. Point $(18.75, 5)$.

E9: $Z = 2(10) + 3(12) = 56$.

E10: Minimise $Z = 2x + 3y$ subject to $4x + 5y \geq 100$ (plus $x, y \geq 0$).

E11: $Z(B) = 2(18.75) + 3(5) = 52.50$; $Z(C) = 2(25/6) + 3(50/3) = 58.33$; $Z(E) = 2(0) + 3(25) = 75$. Minimum $Z = 52.50$ at $B(18.75, 5)$: 18.75 units of Food I, 5 units of Food II.

E12: $Z(B) = 5(18.75) + 3(5) = 108.75$; $Z(C) = 5(25/6) + 3(50/3) = 70.83$; $Z(E) = 5(0) + 3(25) = 75$. The optimal corner changes to $C(25/6, 50/3)$ — the feasible-region geometry is unchanged, but the new objective changes which corner is cheapest — and the minimum cost rises from \$52.50 to \$70.83.

Appendix A

Getting Started with Python and Jupyter Notebook

Atakelty Hailu, University of Western Australia

This appendix is a standalone reference for installing and using Python throughout SCIE1500. Unlike the Week 0 walkthrough (§0.7), which teaches the language hands-on, this appendix covers *software setup* and serves as a quick-lookup reference — including a Python↔R cross-walk — for the rest of the semester.

A.1 Introduction

This unit uses **Python**, a general-purpose programming language widely adopted in science, engineering, and data analysis. Unlike specialised statistical software, Python combines readability with powerful libraries for numerical computation (**numpy**), data manipulation (**pandas**), symbolic mathematics (**sympy**), and visualisation (**matplotlib**). These libraries extend Python’s base capabilities much the way add-on packages extend R.

You will write and run Python code inside **Jupyter Notebook**, an interactive environment that weaves documentation (text, equations, images), code, and results into a single `.ipynb` file — roughly what RStudio is to R.

A.2 Setting Up Your Environment

A.2.1 Two Ways to Run Jupyter Notebook

	Miniconda / Anaconda (local)	Google Colab (cloud)
What it is	A Python distribution installed on your computer. Miniconda is lightweight (~200 MB); Anaconda is the full distribution (~3 GB).	A free, browser-based Jupyter environment hosted by Google.
Pros	Works offline; full control over packages; faster for large files.	Zero installation; works on any device with a browser; easy sharing via Google Drive.
Cons	Disk space required.	Needs internet; sessions time out after inactivity.
Best for	Regular use on your own laptop.	Quick access, Chromebooks, or when you cannot install software.

Recommendation: Miniconda — lightweight, fast to install, and includes everything you need.

A.2.2 Installing Miniconda

1. Go to docs.conda.io/projects/miniconda.
2. Download the installer for your operating system.
3. Run the installer, accepting the default options.
4. Open a terminal (or **Anaconda Prompt** on Windows) and install Jupyter:

```
conda install jupyter numpy matplotlib pandas
```

5. Launch Jupyter Notebook:

```
jupyter notebook
```

Already have Anaconda? That works too — it already includes Jupyter and the packages above. With either distribution you get `conda`, a package manager that makes installing further packages easy (e.g. `conda install sympy`).

A.2.3 Using Google Colab

1. Go to colab.research.google.com.
2. Sign in with your Google account.
3. Click **New Notebook**, or upload a `.ipynb` file from your computer.
4. Colab comes with `numpy`, `matplotlib`, `pandas`, and `sympy` pre-installed.

A.2.4 Organising Your Files

A recommended folder structure keeps weekly notebooks easy to find:

```
Documents/SCIE1500/  
  Week_0/Week00_Lab_STUDENT.ipynb  
  Week_1/Week01_Lab_STUDENT.ipynb  
  Week_2/Week02_Lab_STUDENT.ipynb
```

In Jupyter (Miniconda/Anaconda), navigate to your folder via the file browser; in Colab, upload notebooks or connect to Google Drive.

A.3 The Jupyter Notebook Environment

A notebook is built from **cells**. **Code cells** contain Python instructions — press **Shift+Enter** (or click **Run**) to execute, with output appearing directly below. **Markdown cells** contain formatted text, equations (LaTeX, e.g. $f(x)=x^2$), and images. A new cell is a code cell by default; change its type from the toolbar dropdown.

The Learning Curve

Python is straightforward to learn with sustained effort, especially if you focus on the subset used in this unit. Start early, build gradually — students who feel lost in Week 1 are typically confident by mid-semester.

A.4 Quick Recap: Python Basics

The full hands-on introduction is in the Week 0 Prologue, §0.7. In brief:

- **Calculator:** + - * / **. Functions take arguments in parentheses, as $f(x)$ does in algebra.
- **Variables:** `x = 5` stores a value; names are case-sensitive.
- **Comments:** everything after `#` is ignored by Python.
- **Indentation** (not `{}` as in Java or R) defines code blocks inside `if`, `for`, `while`, and function definitions.
- **Data types:** `int`, `float`, `str`, `bool` — check with `type(x)`.
- **Operators:** arithmetic (+ - * / ** // %), comparison (`==` `!=` `>` `<` `>=` `<=`), logical (`and` `or` `not`), and compound assignment (`x += 1` is shorthand for `x = x + 1`).

A.5 Importing Modules and Key Packages

Python's built-in functions are deliberately minimal; **import** extends them with pre-built toolkits:

```
import math                # standard math library
import numpy as np         # numerical arrays
import matplotlib.pyplot as plt  # plotting
%matplotlib inline        # display plots inline in the
                           # notebook
```

Key Formulas

From Week 4 onward you will also use `import sympy as sp` for symbolic mathematics (derivatives, integrals, equation solving) — not needed in the first few weeks.

A.5.1 math: Constants and Functions

```
print(math.pi, math.e, math.sqrt(2), math.log(10), math.log10(10))
```

A.5.2 numpy: Vectorised Arrays

```
temps_F = np.array([0, 68, 97, 112, 134])
temps_C = (temps_F - 32) * (5/9)           # applied to all
                                             # elements at once
print(np.round(temps_C, 2))

x = np.linspace(0, 10, 50)                 # 50 evenly spaced values, endpoints
                                             # included
y = np.arange(0, 10)                       # integers 0..9
z = np.arange(0, 10, 2)                    # integers 0,2,4,6,8 (step size 2)
```

A.5.3 matplotlib: Plotting

```
x = np.linspace(-5, 5, 100)
y = x ** 2
plt.plot(x, y, 'r-', label='$y = x^2$')
plt.title("A Simple Parabola")
plt.xlabel("x"); plt.ylabel("y"); plt.legend(); plt.grid()
plt.show()
```

A.5.4 Finding Help

`help(np.linspace)` prints full documentation; in Jupyter, `np.linspace?` opens a quick-help pane.

A.6 Defining Your Own Functions

From Week 0 onward you will define Python functions to represent scientific models, using `def`:

```
def fahrenheit_to_celsius(F):
    """Convert Fahrenheit to Celsius."""
    return (F - 32) * (5/9)

print(fahrenheit_to_celsius(68))    # 20.0
```

Functions can take several parameters and combine directly with **numpy**, which is especially useful once your models have several free parameters (e.g. *a*, *b*, *c* below):

```
def quadratic(x, a, b, c):
    return a * x**2 + b * x + c

x = np.linspace(-5, 5, 100)
y = quadratic(x, -0.4, 0.5, 12)
plt.plot(x, y, 'b-', label='$y = -0.4x^2 + 0.5x + 12$')
plt.title("A Quadratic Model"); plt.legend(); plt.grid(); plt.show()
()
```

A.7 Working with Data: A Quick Preview

A Python **list** is general-purpose; a **numpy array** is built for numerical operations and carries convenience methods such as `.mean()` and `.std()`:

```
names = ["Tom", "Tim", "Tiger"]           # list
values = np.array([15, 22, 18, 26])       # array
print(values.mean(), values.std())
```

pandas reads tabular data (e.g. from a spreadsheet or CSV) into a **DataFrame**:

```
import pandas as pd
df = pd.read_csv("data.csv")
df.head()           # first 5 rows
df.describe()       # summary statistics
df.columns          # column names
```

A.8 Common Mistakes and How to Avoid Them

Mistake	Example	Fix
Missing parentheses	<code>print "hello"</code>	<code>print("hello")</code>
Wrong case	<code>Print("hello")</code>	<code>print("hello")</code>
Wrong exponent operator	<code>2^3</code>	<code>2**3</code> (^ is bitwise XOR in Python)
Integer-division surprise	<code>7/2</code> → 3.5 (fine in Python 3)	Use <code>7//2</code> for 3
Indentation error	Mixing tabs and spaces	Use 4 spaces consistently
Forgetting to import	<code>np.array(...)</code> with no <code>import numpy as np</code>	Add the import at the top of the notebook

A.9 Summary of Key Commands: Python vs. R

Given your background may include R, this crosswalk maps the commands you already know onto their Python equivalents:

Task	Python	R equivalent
Print a value	<code>print(x)</code>	<code>print(x)</code>
Square root	<code>math.sqrt(x)</code> / <code>np.sqrt(x)</code>	<code>sqrt(x)</code>
Natural log	<code>math.log(x)</code> / <code>np.log(x)</code>	<code>log(x)</code>
Create a vector/array	<code>np.array([1,2,3])</code>	<code>c(1,2,3)</code>
Sequence of values	<code>np.linspace(0,10,50)</code>	<code>seq(0,10,length=50)</code>
Read CSV	<code>pd.read_csv("file.csv")</code>	<code>read.csv("file.csv")</code>
Plot	<code>plt.plot(x, y)</code>	<code>plot(x, y)</code>
Define a function	<code>def f(x): return x**2</code>	<code>f <- function(x) x^2</code>
Get help	<code>help(func)</code>	<code>?func</code>
List module contents	<code>dir(module)</code>	<code>ls("package:pkg")</code>

A.10 Resources for Further Learning

- [W3Schools Python Tutorial](#)
- [NumPy for Beginners](#)
- [Matplotlib Tutorials](#)
- [SymPy Documentation](#)
- [Pandas Getting Started](#)
- [Google Colab Guide](#)
- [Miniconda Download](#) (or the full [Anaconda](#))
- [Real Python](#)
- YouTube: search “Python for beginners Jupyter Notebook” for video walkthroughs.
- [Stack Overflow](#) — search for your error messages verbatim.

The key skill is *learning how to look for answers* — that skill outlasts any single semester.

Appendix B

Formula Reference Sheet

Act I — Understanding Systems

Concept	Formula
Linear	$y = mx + c$
Quadratic vertex	$x^* = -b/(2a)$
Quadratic formula	$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$
Exponential (continuous)	$P(t) = P_0 e^{kt}$
Half-life / doubling time	$T = \ln 2/ k \approx 70/r\%$ (Rule of 70)
Logarithm rules	$\ln(AB) = \ln A + \ln B$; $\ln(A^n) = n \ln A$
Logistic	$P(t) = \frac{K}{1 + Ae^{-at}}$; $A = K/P(0) - 1$
Schaefer growth	$G(S) = gS(1 - S/K)$; MSY at $S = K/2$
Geometric sequence	$a_n = a_1 r^{n-1}$; $S_n = a_1(1 - r^n)/(1 - r)$
Arithmetic sequence	$a_n = a_1 + (n - 1)d$; $S_n = n(a_1 + a_n)/2$
Derivative (def.)	$f'(x) = \lim_{h \rightarrow 0} [f(x + h) - f(x)]/h$
Power rule	$\frac{d}{dx} x^n = nx^{n-1}$

Act II — Measuring Accumulation

Concept	Formula
Product rule	$(fg)' = f'g + fg'$
Quotient rule	$(f/g)' = (f'g - fg')/g^2$
Chain rule	$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x)$
$\frac{d}{dx} e^x$	e^x
$\frac{d}{dx} \ln x$	$1/x$
Critical point	$f'(x^*) = 0$; min if $f'' > 0$, max if $f'' < 0$
Power rule (int.)	$\int x^n dx = x^{n+1}/(n+1) + C$
FTC (Part 2)	$\int_a^b f dx = F(b) - F(a)$
Average value	$\bar{f} = \frac{1}{b-a} \int_a^b f dx$
Consumer surplus	$CS = \int_0^{Q^*} D(Q) dQ - P^* Q^*$
Producer surplus	$PS = P^* Q^* - \int_0^{Q^*} S(Q) dQ$

Act III — Predicting Interactions

Concept	Formula
L-V prey	$dH/dt = \alpha H - \beta HP$
L-V predator	$dP/dt = \lambda HP - \gamma P$
Coexistence eq.	$(H^*, P^*) = (\gamma/\lambda, \alpha/\beta)$
Predator efficiency	$\epsilon = \lambda/\beta$
Prey increasing when	$P < \alpha/\beta$
Predator increasing when	$H > \gamma/\lambda$
Bayes' Theorem	$P(A B) = P(B A)P(A)/P(B)$
Binomial PMF	$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$
$E[X]$ (discrete)	$\sum_x x P(X = x)$
$\text{Var}(X)$	$E[X^2] - (E[X])^2$
Binomial $E[X]$	np
Binomial Var	$np(1-p)$

Act IV — Making Optimal Decisions

Concept	Formula
Radian conversion	$\theta_{\text{rad}} = \theta_{\text{deg}} \cdot \pi/180$
Pythagorean identity	$\sin^2 \theta + \cos^2 \theta = 1$
Period	$T = 2\pi/ B $
Amplitude	$ A = (\max - \min)/2$
Vertical shift	$D = (\max + \min)/2$
Phase shift	solved from a boundary condition
LP objective	$Z = c_1x + c_2y$
Corner Point Thm	Optimal at vertex of feasible polygon
Market eq.	$Q_d(P) = Q_s(P)$

Appendix C

Common Mathematical Constants

These constants recur throughout mathematics and science. Keep the exact symbol in calculations where possible; round only the final numerical result to a precision justified by the data.

Constant	Approximate value	Where it appears
π (pi)	3.14159265...	Circle geometry; radians; trigonometric models.
e (Euler's number)	2.71828183...	Continuous growth and decay: $P(t) = P_0 e^{kt}$.
$\sqrt{2}$	1.41421356...	Pythagoras' theorem and right-triangle geometry.
ϕ (golden ratio)	1.61803399...	A mathematical proportion; not a required SCIE1500 formula, but often encountered in science writing.
$\ln 2$	0.69314718...	Doubling time and half-life calculations.
$\log_{10} 2$	0.30102999...	Converting between powers of 2 and scientific notation.
i	$\sqrt{-1}$	The imaginary unit, used in more advanced mathematics and engineering.
∞	not a number	Infinity: a quantity that grows without bound, used in limits and intervals.

Constants Are Not Parameters

A constant has the same value wherever it is used: for example, π is always 3.14159.... A parameter, such as γ in a predator–prey model, has a value chosen or estimated for that model. The symbol tells you very little by itself: look for the definition and units.

Index

- Antiderivative, 102
- Area between curves, 116
- Arithmetic sequence, 56
- Average value (of a function), 117

- Bayes' Theorem, 143
- Bernoulli distribution, 153
- Binomial distribution, 146

- Chain rule, 87
- Conditional probability, 142
- Consumer and producer surplus, 118
- Continuity, 73
- Corner Point Theorem, 183

- Deadweight loss, 119
- Definite integral, 113
- Derivative, 75
- Doubling time, 47

- Empirical rule, 154
- Equilibrium point, 129
- Euler's number, 43
- Even and odd functions, 33
- Event (probability), 140
- Exponential function, 41
- Exponential growth and decay, 45

- Function, 16, 28
- Function composition, 62
- Fundamental Theorem of Calculus, 114

- Geometric sequence, 47

- Independence (probability), 145
- Inverse function, 62

- Kolmogorov's axioms, 140

- Limit, 70
- Linear programming, 182
- Logistic function, 57
- Lotka-Volterra equations, 128

- Multiplication rule (probability), 143

- Natural logarithm, 46
- Normal approximation, 157
- Nullcline, 131
- Number sets, 12

- Period, 169

- Power rule, 76, 103
- Predator efficiency, 129
- Probability mass function, 152
- Product rule, 86

- Quotient rule, 87

- Radian, 166
- Random experiment, 140
- Random variable, 151
- Riemann sum, 120

- Sample space, 140
- Schaefer model, 58
- Simultaneous equations, 180

- Unit circle, 167

- Z-score, 154